

BLE 바나나 체온계를 활용한 IoT 따라잡기

들어가며

현재는 전자공학과 컴퓨터 공학이 더 널리 알려져 있어서, 일반인들도 약간의 기술 지식을 가지고 있거나 차근차근 배우면 IoT(Internet of Things) 프로젝트를 진행할 수 있는 시대입니다. 이 교재는 IoT에 흥미를 가진 일반인이나 전자, 전기, 컴퓨터와 같은 분야에 관심이 있는 학생들을 위해 제작되었습니다. 교재를 따라가면서 전체적인 개념과 프로젝트를 이해할 수 있도록 구성되어 있습니다. 조금 더 구체적으로 설명하자면, 컴퓨터나 전자공학을 전공한 학생들이 3학년까지 배운 지식을 기반으로 BLE를 통해 라즈베리파이나 AWS 클라우드에 저장하는 실습을 해 볼 수 있도록 하드웨어와 이와 관련된 Python 소스 코드를 제공합니다. 이를 통해 여러분은 실제 하드웨어와 클라우드 기술을 활용하여 데이터를 안전하게 전송하고 저장하는 방법을 배울 수 있습니다. 하드웨어는 이전에 상용화되었던 무선 체온 측정기(FEMON)과 라즈베리파이로 구성되어 있으며, 실습을 통해 이를 어떻게 활용하는지 직접 체험하게 됩니다. 또한, 교재에는 iOS와 안드로이드 용의 소스 코드도 추가로 제공되어 있어 동일한 기능을 다양한 언어로 구현해 볼 수도 있습니다. 이를 통해 여러 플랫폼에서의 프로그래밍 능력도 함께 향상시킬 수 있습니다.

정리하자면, 이 교재를 통해 독자들은 IoT의 기본 개념과 원리를 이해하고, 다양한 플랫폼에서 IoT 프로젝트를 진행할 수 있는 능력을 키울 수 있을 것입니다. 현대적인 기술 동향을 따라가며 실전적인 경험을 쌓아 나가는데 도움이 될 것입니다.

바나나 체온계 실습 키트의 구매처는 다음과 같습니다.

사이트: [사이트 링크](#)

전화번호: 070-8620-8020

E-mail: sales@lstgrp.com

제품 구성

바나나 체온계 실습 키트의 데모 영상은 아래와 같다.

Banana Thermometer Evaluation Kit Python(Raspberry Pi) Demo Video

바나나 체온계 실습 키트의 제품 구성은 아래와 같다.

이름	이미지
바나나 체온계	 The image shows the components of the Banana Thermometer Evaluation Kit. At the top is a green, curved printed circuit board (PCB) with two circular sensor modules mounted on it. Below the PCB is a black rectangular battery pack. Two pairs of wires, one red and one black, connect the battery pack to the sensor modules on the PCB.

이름

이미지

Raspberry Pi



BLE_UART



이름

이미지

Micro HDMI to HDMI



Micro SD카드



이름

이미지

Micro SD카드 리더기



C타입 충전기(5V/3A)



이름

이미지

투명박스(3L)



1. F EMON을 사용해보자

1.1. FEMON 앱

본 단락은 FEMON 어플리케이션을 사용해 통신을 하는 방법에 대해 설명한다.

FEMON 앱

바나나 체온계를 FEMON 어플리케이션과 함께 사용하는 방법은 아래와 같다

1. 패치전원 ON

- 전원버튼을 2초 정도 눌러 전원을 켜다.

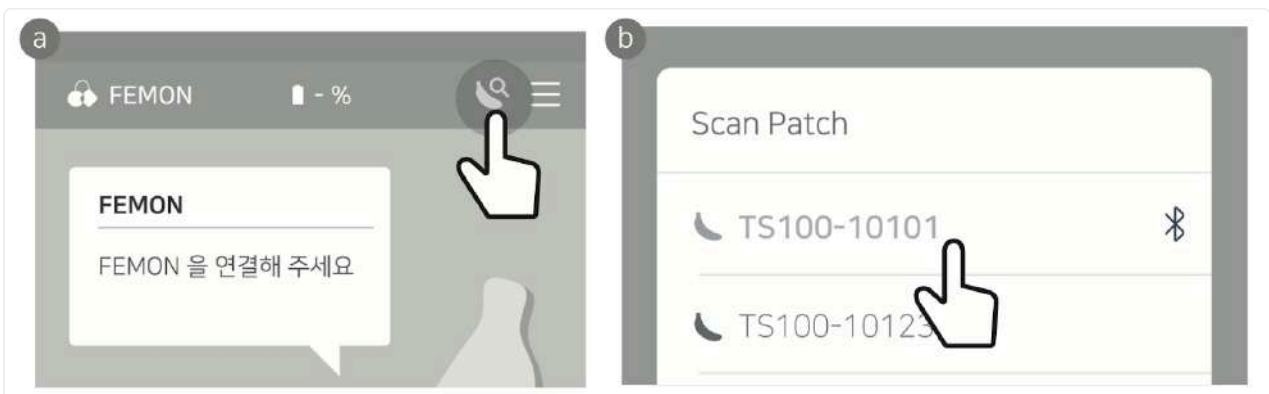
2. 앱 다운로드

- 아래 링크를 통해 FEMON 어플리케이션을 설치한다.
 - Android 설치 주소: [Android 설치 주소](#)

! “이 앱은 이전 버전의 Android를 위해 만들어진 것이기 때문에 현 기기에서 사용할 수 없습니다.” 오류가 발생한다면 [apk 설치 링크](#)를 통해 어플리케이션을 다운로드 후 설치를 진행한다.

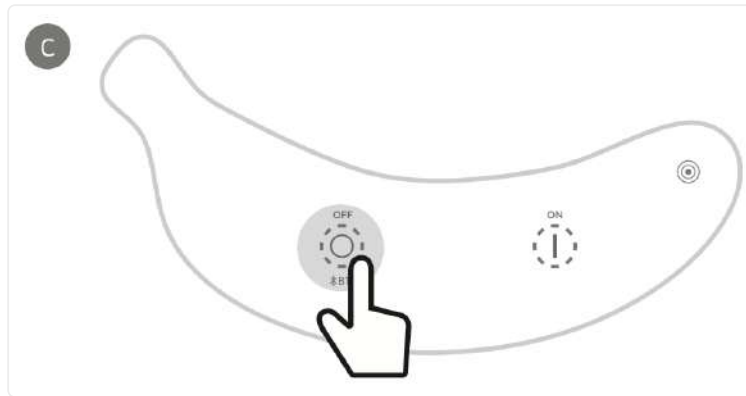
2. 앱 실행 및 패치연결

- 앱을 실행시킨 후 메인 화면의  아이콘 또는  아이콘을 터치한다.
- 블루투스 이용에 동의 후, 패치 리스트 팝업창에 연결 가능한 패치가 검색되면, 연결할 패치를 선택한다.



FEMON 블루투스 연결

- 패치가 검색 되지 않을 시, 패치의 블루투스 체크 스위치 버튼을 한번 누른 뒤, 위 과정을 다시 시도한다.



바나나 체온계 블루투스 체크 스위치 클릭

- 검색된 패치를 선택하면 블루투스 페어링 요청 확인 화면이 나타난다. 핀코드 입력란에 PIN Code 스티커에 기재되어 있는 숫자 6자리를 입력한다.

A screenshot of a mobile app interface for Bluetooth pairing. At the top, it says '블루투스 페어링 요청' (Bluetooth Pairing Request). Below that, the device name '기기 TS100-10010' is shown. There is a PIN code input field with six dots and a horizontal line below it. A checkbox is labeled 'PIN에 문자나 기호가 포함됩니다.' (Contains letters or symbols in PIN). Below the checkbox, there is a warning: '또한 다른 기기에 이 PIN을 입력해야 할 수도 있습니다. 페어링하면 연결시 주소록 및 통화 기록에 액세스 할 수 있습니다.' (You may also need to enter this PIN on other devices. If you pair, you can access contacts and call logs when connected). At the bottom, there are 'CANCEL' and 'DONE' buttons. A small circle with the letter 'd' is in the top left corner of the app window.

블루투스 페어링 PIN Code 입력

4. 패치 부착

- 체온 측정을 위한 온도 센서가 피부(겨드랑이 부위)에 완전히 밀착될 수 있도록 패치를 피부에 잘 부착한다.

5. 체온 표시

- 패치 연결 및 부착이 완료 되면, 메인화면 중앙에 측정 대상자의 체온이 표시된다.
- 센서가 정확한 온도를 감지하는데 약3~4분 정도 소요된다.

6. 패치 연결 / 해제

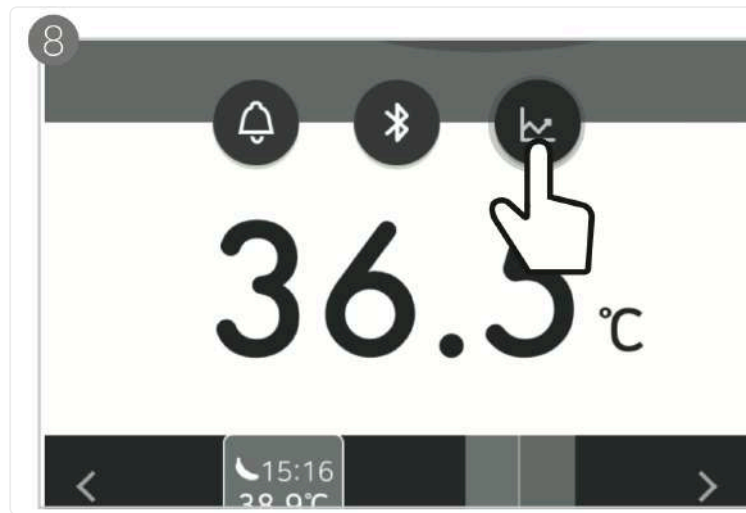
- 는 패치와 앱의 연결 상태를, 는 패치와 앱의 연결 해제 상태를 나타내며, 이 버튼을 통하여 빠르게 재연결 및 연결 해제가 가능하다.
- 단, 앱과 패치의 연결 이력이 최소 1회 이상 있어야한다.
- 재연결이 되지 않을 시에는  버튼을 누른 뒤, 패치의 블루투스 버튼을 짧게 한 번 누른다. 다른 패치를 연결하고자 할 때에는, 우측 상단의  버튼을 누른다.

7. 알림 설정 기능

- 메인화면 중앙의  버튼을 통하여, 알림 ON/OFF를 설정할 수 있으며, 알림 온도는 상단 우측 메뉴의 'Alert'에서 설정 가능하다.
- 설정 가능한 온도는 35°C ~ 42°C 이다.
- 체온계의 온도가 설정된 온도를 벗어날 시, 알림이 울리며, 체온 측정 속도가 빨라진다.
- 알림 형태> 진동, 소리, 알림창 Push, 패치의 적색 LED 점멸(안정 온도 - 녹색 LED 점멸)

8. 체온 리뷰

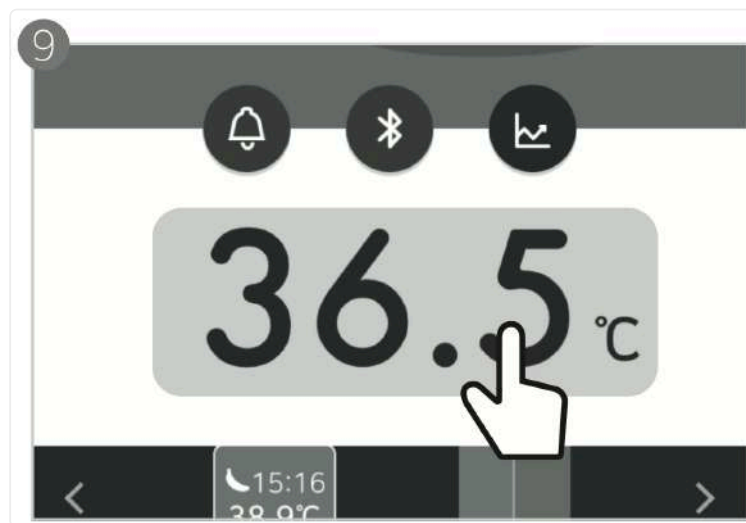
- 메인 화면 중앙의  버튼을 누르면, 시간에 따른 체온 변화 추이를 확인할 수 있다.
- 노란색 실선, 점: 패치로부터 받은 데이터
- 파란색 점: 사용자가 직접 입력한 데이터
- 빨간색 점선: 고온 알림 설정한 온도 가이드 선
- 청록색 점선: 저온 알림 설정한 온도 가이드 선
- 리뷰 페이지 상단의  버튼을 누르면, 사용자가 직접 시간과 체온을 지정하여 입력할 수 있다.



체온 리뷰

9. 집중 모니터링

체온 디스플레이를 누르면, 5초당 1회 온도 측정(기본 설정)을 20초간 1초당 1회 온도 측정으로 변경하여 대상자의 체온을 집중적으로 모니터링 할 수 있다.



집중 모니터링

10. 고온 이력 조회

- 메인화면 하단의 좌우 스크롤 바를 통하여, 최근 72시간 동안의 고온(37.0°C 이상) 이력을 확인할 수 있다. 노란색($37.0^{\circ}\text{C}\sim 37.4^{\circ}\text{C}$), 주황색($37.5^{\circ}\text{C}\sim 37.9^{\circ}\text{C}$), 빨간색(38.0°C 이상) 색상으로 표시된다. 색상 구간을 누르면 가장 높았던 기록의 온도가 말풍선으로 표시되며, 패치로부터 받은 데이터 (👤 표시) 와 사용자가 직접 입력한 데이터가(✍️ 표시) 표시된다. 자세한 온도 데이터는 📈를 통하여 확인할 수 있다.(단, 패치와 연결되어 있지 않은 시간과 저온 알림은 표시되지 않는다.)

11. 패치 절전모드

- 절전모드

- 패치의 블루투스 버튼을 3초 정도 누르고 있으면, 적색과 녹색 LED가 동시에 켜지며, 녹색 LED가 깜빡이다가 꺼지며 절전모드가 된다. 재동작을 원할 시, 패치의 블루투스 버튼을 짧게 한번 누른다.(재동작 시, LED 녹색 혹은 적색점멸)

- 전원 OFF

- 패치의 블루투스 버튼을 6초 정도 누른 상태를 유지하여 전원을 종료한다.(LED 소등 - 점멸 - 소등) 이때, 저장되었던 데이터는 저장 및 복구가 되지 않는다.
- 패치의 배터리를 전부 소모할 경우, 패치의 전원이 자동으로 종료된다. 이때, 저장되었던 데이터는 저장 및 복구가 되지 않는다.
- 패치의 전원을 OFF할 때에는, 측정 대상자로부터 패치를 떼어서 작동하여야 한다.

12. 메뉴

- 메인화면 우측 상단 버튼을 통하여 설정 및 도움말 등을 확인할 수 있다.

- 알림 설정(Alert): 저온과 고온알림의 온도를 설정할 수 있다.
- 설정(Setting): 섭씨/화씨, 위험 온도와 안정 온도일 때의 측정 주기를 설정할 수 있다.(측정 주기가 늘어날 수록, 동작 수명이 늘어난다)
- 인증(Authentication): 사용자 등록, 로그인, 로그아웃, 사용자 탈퇴를 할 수 있다.
- 법적공지(Legal Notice): 이용약관 및 개인정보처리방침을 확인할 수 있다.
- 회사정보(Company Information): 회사 정보를 확인할 수 있다.
- 앱 버전(App Version): 현재 사용하고 있는 앱의 버전을 확인할 수 있다.

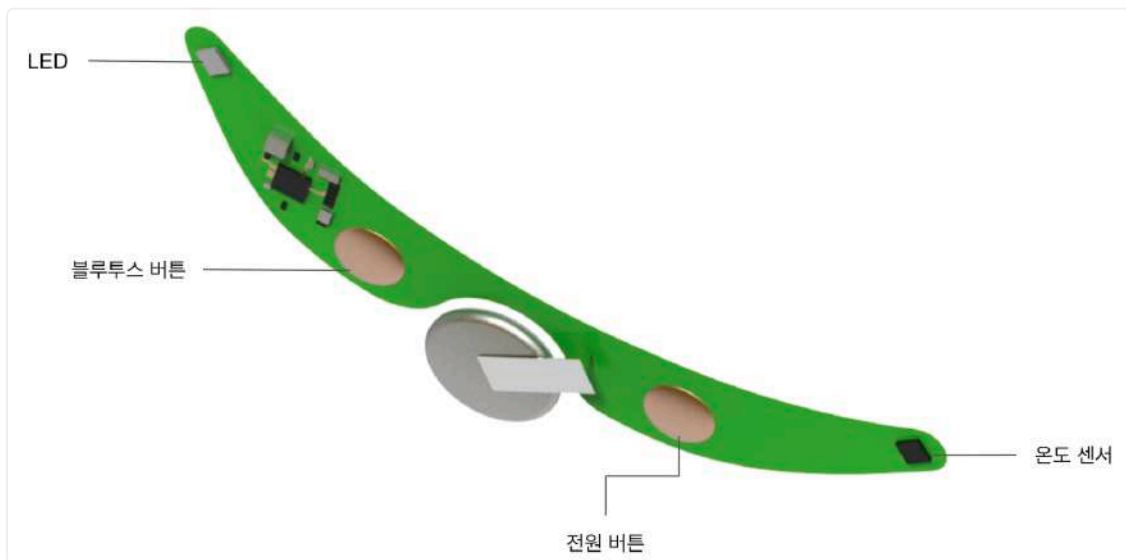
2. FEMON 구성에 대해 알아보자

2.1. 하드웨어

본 단락은 FEMON 하드웨어 구성에 대해 설명한다.

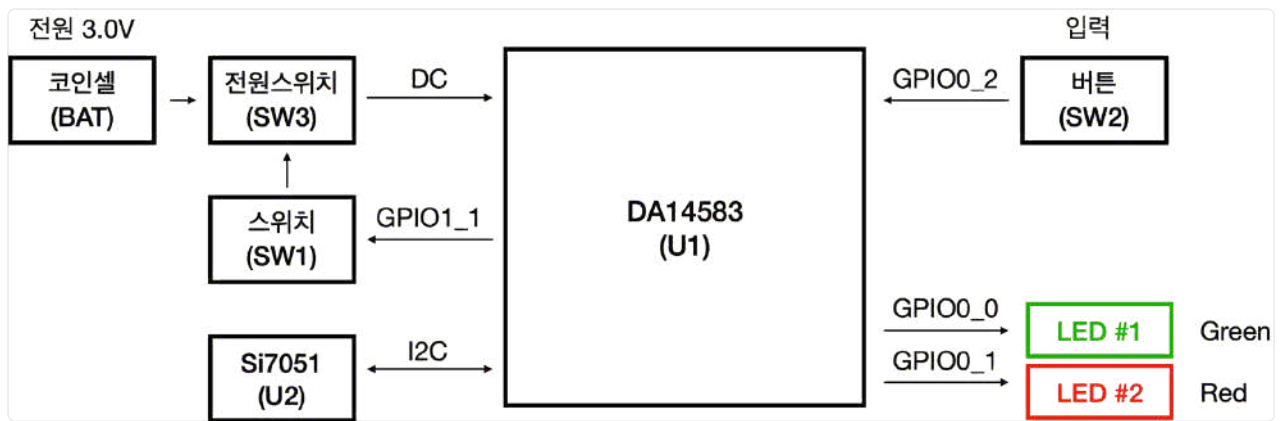
바나나 체온계의 하드웨어는 아래와 같이 전원 및 블루투스 버튼, 온도 센서, LED, BLE로 구성되며 각각의 기능은 다음과 같다. 바나나 체온계의 구매처는 [링크](#)에서 확인할 수 있다.

1. 전원 버튼: 전원 ON 모드로 전환
 2. 블루투스 버튼: 블루투스 절전모드(3초간 누름), 전원 OFF(6초간 누름)모드로 전환
 3. 온도 센서(Si7051): 사용자의 온도를 측정
 4. BLE 모듈: 바나나 체온계의 정보를 BLE를 통해 전달
 5. LED: 바나나 체온계의 현재 상태 확인 가능
- 전원 ON: 적색 LED 1초에 1회 점등
 - BLE 페어링: 적색 LED 1초에 1회 점등
 - 절전모드(블루투스 버튼 3초 누름): 버튼 눌렀을 때 적색, 녹색 LED 켜짐 - 녹색 LED 깜빡이다가 꺼짐
 - 전원 OFF(블루투스 버튼 6초 누름): LED 소등 - 적색, 녹색 LED 점멸 - 소등



바나나 체온계 하드웨어 구성도 (모식도)

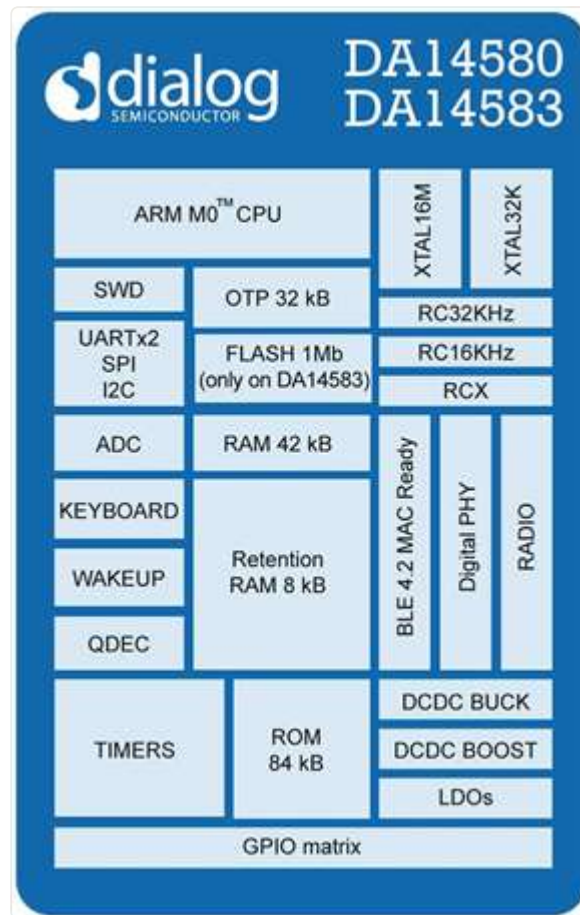
바나나 체온계의 시스템 블록도는 아래 그림과 같다.



바나나 체온계 시스템 블록도

1. 전원은 코인셀을 사용하고 있으며 전원스위치 SW3를 눌렀을 때 전원이 공급되어 MCU가 동작을 시작한다.
2. 아날로그 스위치(SW1)을 작동시켜 회로상에서 VBAT과 VCC를 연결하여 버튼(SW3)을 누르고 있는 것과 같은 효과로 전원을 지속적으로 공급한다.
3. 온도센서는 Si7051를 사용중이며 I2C로 인터페이스 되어있다.
4. LED는 2종류(Green, Red)를 사용한다.

메인 프로세서는 [DA14583](#)으로 Dialog Semiconductor(2021년 Rensas사로 인수됨)에서 제조하는 저전력 블루투스 5.2 SoC이다. 이 칩은 다양한 애플리케이션에 사용할 수 있으며, 다음과 같은 주요 기능을 제공한다.



DA14583

스펙	사양
프로세서	ARM Cortex-M0+
클럭 속도	32MHz
메모리	64KB RAM, 256KB ROM
블루투스	Bluetooth 5.2, BLE, LE Audio
인터페이스	I ² C, SPI, UART, SDIO, USB
보안	AES, SHA-256, RSA

딥 슬립 모드에서 10μA 미만의 전력 소비를 제공하여 배터리 수명을 극대화할 수 있고 I²C, SPI, UART, SDIO, USB 등 다양한 인터페이스를 지원하며, AES, SHA-256, RSA 등 다양한 보안 기능을 지원하여 데이터를 보호할 수 있다.

아래 그림은 바나나 체온계의 전체 도면이다. [링크](#)를 통해 PDF 파일을 다운로드할 수 있다.

3. BLE에 대해 알아보자

3.1. BLE란(저전력용 Bluetooth 기술)

본 단락은 저전력 블루투스 기술인 BLE에 대해 설명한다.

BLE(Bluetooth Low Energy)은 저전력 무선 통신 기술로, 주로 에너지 효율적인 짧은 거리 통신에 사용된다. 데이터 전송 속도에 초점을 맞추고 있는 기존의 블루투스와는 달리 BLE는 전력 소모를 줄이는 것에 초점을 맞추고 있어 스마트 홈, 헬스케어, 웨어러블 디바이스 등 다양한 분야에서 활용된다.

이 장에서는 저전력 블루투스 기술인 BLE에 대해서 간단히 다룬다.

BLE(Bluetooth Low Energy)는 2010년에 도입된 혁신적인 저전력 무선 통신 기술로, 기존의 블루투스 기술을 발전시켜 에너지 효율성을 크게 향상시켰다. 이 기술은 주로 짧은 거리에서의 간헐적인 데이터 전송에 최적화되어 있으며, 특히 배터리 수명이 중요한 기기들에서 널리 사용된다.

BLE의 주요 특징은 매우 낮은 전력 소비에 있다. 기존의 블루투스 기술이 지속적인 연결과 높은 데이터 전송률에 중점을 두었다면, BLE는 필요할 때만 깨어나 데이터를 전송하고 다시 절전 모드로 돌아가는 방식을 채택하고 있다. 이러한 방식으로 BLE는 일반 블루투스에 비해 전력 소비를 최대 100분의 1까지 줄일 수 있어, 코인 셀 배터리로도 몇 달에서 몇 년까지 작동할 수 있는 기기를 만들 수 있게 되었다.

BLE의 응용 분야는 매우 다양하다. 스마트 홈 분야에서는 도어락, 온도 조절기, 조명 제어 시스템 등에 사용되어 편리하고 에너지 효율적인 가정 환경을 만드는 데 기여한다. 헬스케어 분야에서는 혈당 모니터, 심박수 모니터, 체온계 등의 의료기기에 적용되어 환자의 건강 상태를 실시간으로 모니터링하고 데이터를 의료진과 공유하는 데 활용된다. 웨어러블 디바이스 시장에서는 스마트워치, 피트니스 트래커 등에 폭넓게 사용되어 사용자의 활동량, 수면 패턴 등을 추적하고 스마트폰과 연동하여 정보를 제공한다.

또한 BLE는 비콘 기술과 결합하여 실내 위치 추적, 근접 마케팅 등의 새로운 서비스를 가능하게 했다. 소매점에서는 고객이 특정 제품 근처에 접근했을 때 관련 정보나 할인 쿠폰을 제공하는 등의 서비스를 구현할 수 있게 되었다. 산업 현장에서도 BLE 센서를 이용한 장비 모니터링, 자산 추적 등에 활용되어 생산성과 효율성을 높이는 데 기여하고 있다.

BLE의 이러한 특성과 광범위한 응용 가능성으로 인해, IoT(Internet of Things) 시대의 핵심 기술로 자리잡고 있으며, 앞으로도 더 많은 혁신적인 서비스와 제품을 가능하게 할 것으로 예상된다.

3.2. Central, Peripheral

BLE에서 역할은 크게 Central, Peripheral 두가지로 나눌 수 있다.

이 두 역할은 BLE 통신의 기본 구조를 형성하며, 각각 다음과 같은 특징을 가지고 있다:

Central

Central은 주로 스마트폰, 태블릿, 노트북 등과 같은 기기를 의미한다. Central device는 주변의 BLE device들을 탐지하기 위해 스캔을 수행한다. 이 스캔 결과로 주변 Peripheral device들의 광고를 수신하며, 이를 통해 어떤 device와 연결할지 결정한다. 연결이 확립되면 Central은 선택한 Peripheral과 데이터를 주고받을 수 있다. 데이터 요청을 보내거나 주기적으로 데이터를 수신하여 필요한 정보를 얻거나 제어 명령을 전송할 수 있다.

Peripheral

Peripheral은 주로 센서, 작은 장치, 비콘 등을 의미한다. Peripheral device는 자신의 존재와 서비스를 알리기 위해 광고를 주변에 전송한다. Central의 연결 요청을 받아들이고 연결을 설정한 후에는 Central과 데이터를 주고받을 수 있다. 센서 데이터를 전송하거나 Central로부터의 명령을 수행하는 역할을 수행한다. 이 모든 동작을 에너지 효율적으로 수행하면서 중요한 정보를 전달하거나 제어 명령을 수신한다.

BLE(Bluetooth Low Energy) 통신에서 Central과 Peripheral은 두 가지 핵심적인 역할을 담당한다. 이 두 역할은 BLE 생태계의 기본 구조를 형성하며, 각각 고유한 특성과 기능을 가지고 있다.

Central 장치는 BLE 네트워크의 중심 역할을 한다. 이는 주로 스마트폰, 태블릿, 컴퓨터와 같은 처리 능력이 높고 전력 공급이 안정적인 기기들이 맡는다. Central의 주요 임무는 주변의 Peripheral 장치들을 탐색하고, 이들과 연결을 시작하는 것이다. 이 과정에서 Central은 주기적으로 주변을 스캔하여 가용한 Peripheral 장치들의 광고 신호를 감지한다. 연결이 성립되면, Central은 Peripheral로부터 데이터를 요청하거나 받아들이는 주체가 된다. Central의 중요한 특징 중 하나는 동시에 여러 Peripheral 장치들과 연결을 유지할 수 있다는 점이다. 이로 인해 Central은 다양한 센서나 기기로부터 동시에 정보를 수집하고 관리할 수 있다. 하지만 이러한 기능으로 인해 Central은 상대적으로 더 많은 전력을 소비하는 경향이 있다.

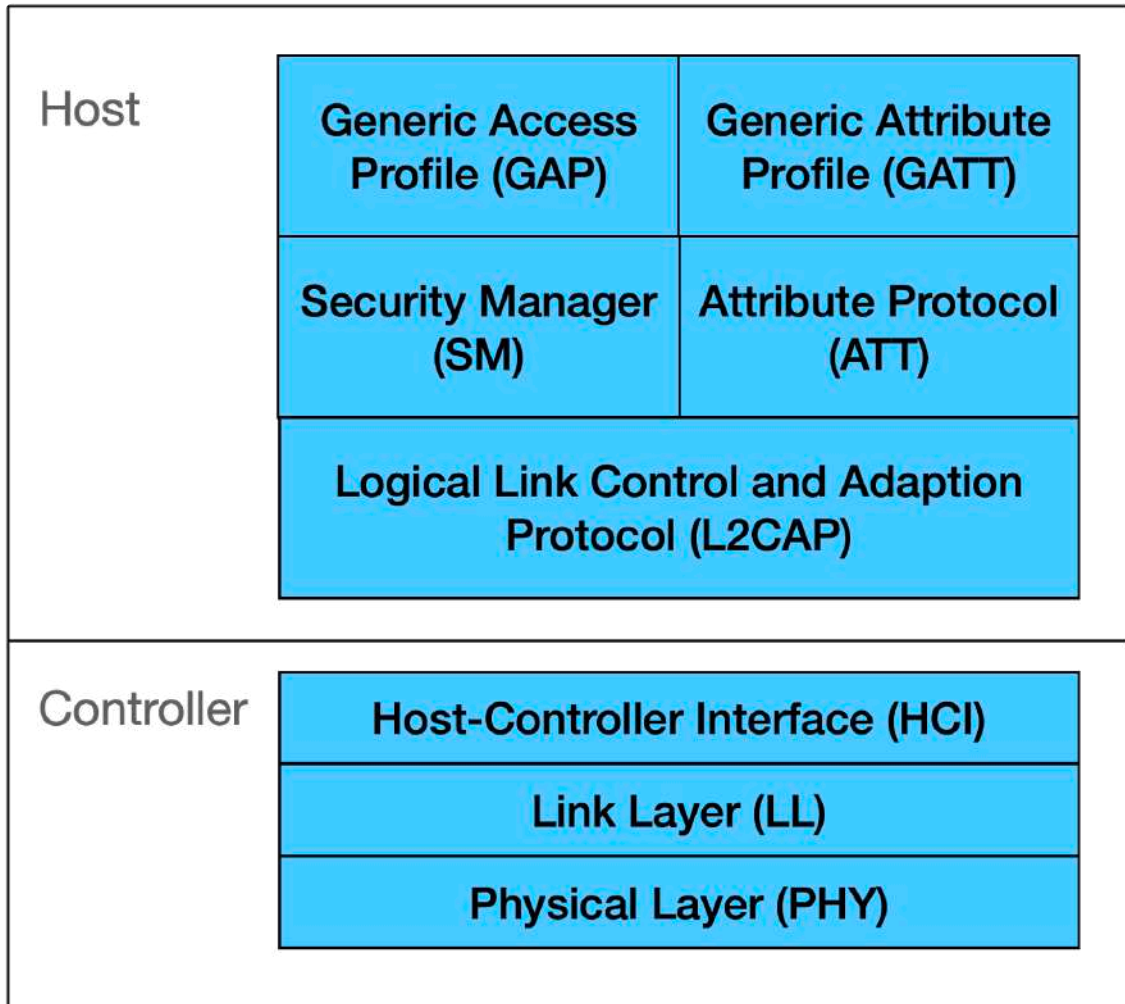
반면, Peripheral 장치는 BLE 네트워크의 주변부에 위치하며, 주로 센서, 웨어러블 기기, 비콘과 같은 소형, 저전력 장치들이 이 역할을 담당한다. Peripheral의 주요 기능은 자신의 존재를 알리는 것으로, 이를 위해 주기적으로 광고(Advertising) 신호를 broadcast한다. 이 광고 신호에는 장치의 이름, 제공하는 서비스 등의 기본 정보가 포함되어 있어, Central 장치가 연결 여부를 결정하는 데 도움을 준다. Peripheral은 한 번에 오직 하나의 Central 장치와만 연결될 수 있다는 제한이 있다. 이는 Peripheral의 리소스를 효율적으로 관리하고 전력 소비를 최소화하기 위한 설계이다. Peripheral은 주로 센서 데이터를 제공하거나 간단한 동작을 수행하는 데 특화되어 있다. 예를 들어, 심박수 모니터, 온도 센서, 스마트 전구 등이 전형적인 Peripheral 장치들이다. 이들은 대부분 배터리로 작동하기 때문에, 저전력 소비를 위해 최적화되어 있다. Peripheral은 대부분의 시간을 절전 모드로 보내다가, Central의 요청이 있을 때만 깨어나 데이터를 전송하는 방식으로 작동하여 배터리 수명을 연장한다.

이러한 Central과 Peripheral의 역할 구분은 BLE의 효율적인 저전력 통신을 가능케 하는 핵심 요소다. 이 구조를 통해 BLE는 전력 소비를 최소화하면서도 필요한 데이터 교환을 효과적으로 수행할 수 있다.

그 외에도 Observer와 Broadcaster도 있다. 아래는 앞서 언급한 역할을 개요식으로 간단히 정리해 보았다.

3.3. BLE 프로토콜 스택

BLE 스택은 크게 Host와 Controller 영역으로 구성되어 있고 각 영역은 여러 레이어로 이루어져 있다. BLE 기기는 각 레이어에서 특정 state 혹은 역할을 가지고 동작한다. BLE 프로젝트를 시작하기 전 BLE 스택의 각 요소가 어떤 역할을 수행하는지 먼저 설명한다.



BLE 프로토콜 스택

Controller

Physical Layer (PHY)

Physical Layer는 무선 전파 신호를 송수신하기 위한 하드웨어로 구성되어 있다. BLE 기기는 2.4GHz 스펙트럼에서 작동하며, 각각 2MHz로 분리된 40개의 영역으로 나뉘어져 있다. 각각의 영

역을 채널이라고 부른다. 채널 중 37, 38, 39번의 경우 디바이스를 광고(Advertising) 하거나 찾는 데(Scanning) 사용되고, 나머지 채널의 경우 데이터를 송수신하는데 사용된다.

Link Layer (LL)

Link Layer는 Physical Layer와 상호작용하는 레이어이고, 일반적으로 하드웨어와 소프트웨어의 조합으로 구현된다. 하드웨어는 소프트웨어 중앙처리장치의 과부하를 막기 위해 일반적으로 계산 비용이 많이 들고 쉽게 자동화되는 기능을 처리한다. 소프트웨어는 BLE 디바이스의 연결 상태인 Link State를 관리한다. Link State는 Standby, Initiating, Scanning, Advertising, Connection 다섯가지의 상태로 이루어져 있다.

Host-Controller Interface (HCI)

Host-Controller Interface 레이어는 Host와 Controller가 분리되어 있는 경우 두 모듈 사이의 상호작용을 담당하는 레이어이다. Host에서 Controller로 명령을 전달하고, Controller에서 Host로 이벤트를 전송하는 역할을 수행한다.

Host

Logical Link Control and Adaptation Protocol (L2CAP)

Logical Link Control and Adaptation Protocol 레이어는 상위 레이어와 하위 레이어 (Controller) 사이의 상호작용을 담당하는 레이어이다. Link Layer에서 만들어진 BLE 패킷에 실제 어플리케이션에 사용되는 데이터를 포함시키거나 추출하는 역할을 수행한다.

Security Manager (SM)

Security Manager 레이어는 자주 연결하는 device 간의 보안 코드를 관리하고 배포하는 역할을 수행한다.

Generic Access Profile (GAP)

Generic Access Profile 레이어는 Connecting 및 Advertising 동작을 관리하는 최상위 레이어로 각 device의 Link Layer State를 결정하고, 서로다른 BLE device 사이의 상호작용을 관리하는 모듈이다. BLE device는 GAP에 대해 Broadcaster, Observer, Peripheral, Central 중 하나의 역할로 동작한다.

Attribute Protocol (ATT)

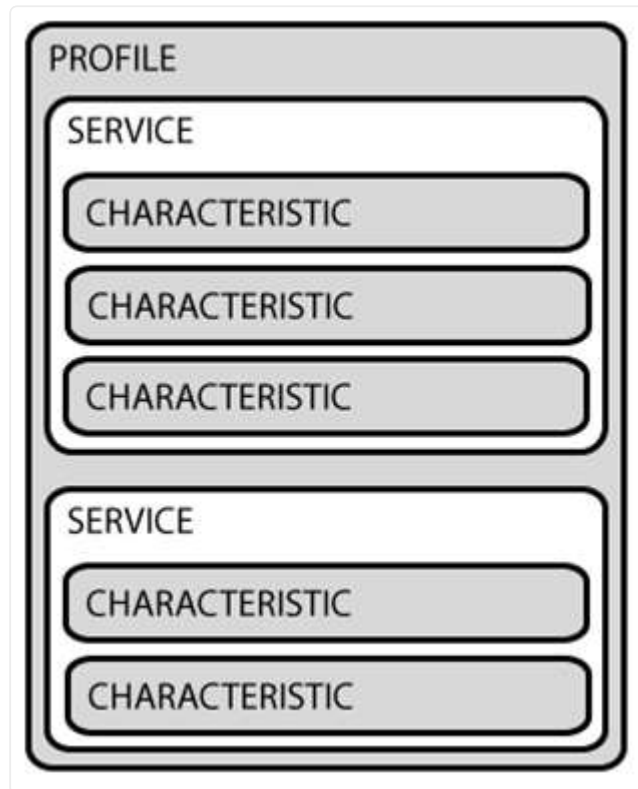
Attribute Protocol 레이어는 두 BLE 장치간에 주고받는 데이터 교환에 대한 규칙으로, 무선으로 연결된 BLE device 사이의 데이터 교환은 ATT에 의해 이뤄진다. 이때, 데이터를 가지고 있는 device를 GATT Server라고 하고, 데이터를 요청하는 device를 GATT Client라고 한다.

Generic Attribute Profile (GATT)

Application 레이어에서 데이터 교환은 ATT를 기반으로 이루어지고, 각각의 데이터구조는 Generic Attribute Profile에 의해 정의되는 데이터 구조를 사용한다. GATT에 의해 정의되는 데이터 구조는 profile, service와 characteristic으로 표현된다.

3.4. Service, Characteristic

BLE에서 사용하는 GATT 기반 동작 구조는 Profile, Service, Characteristic에 기초한다. 아래 이미지와 같이 수직 구조를 이루게 된다.



GATT 구조

Profile이란 정의된 Service의 묶음이다.

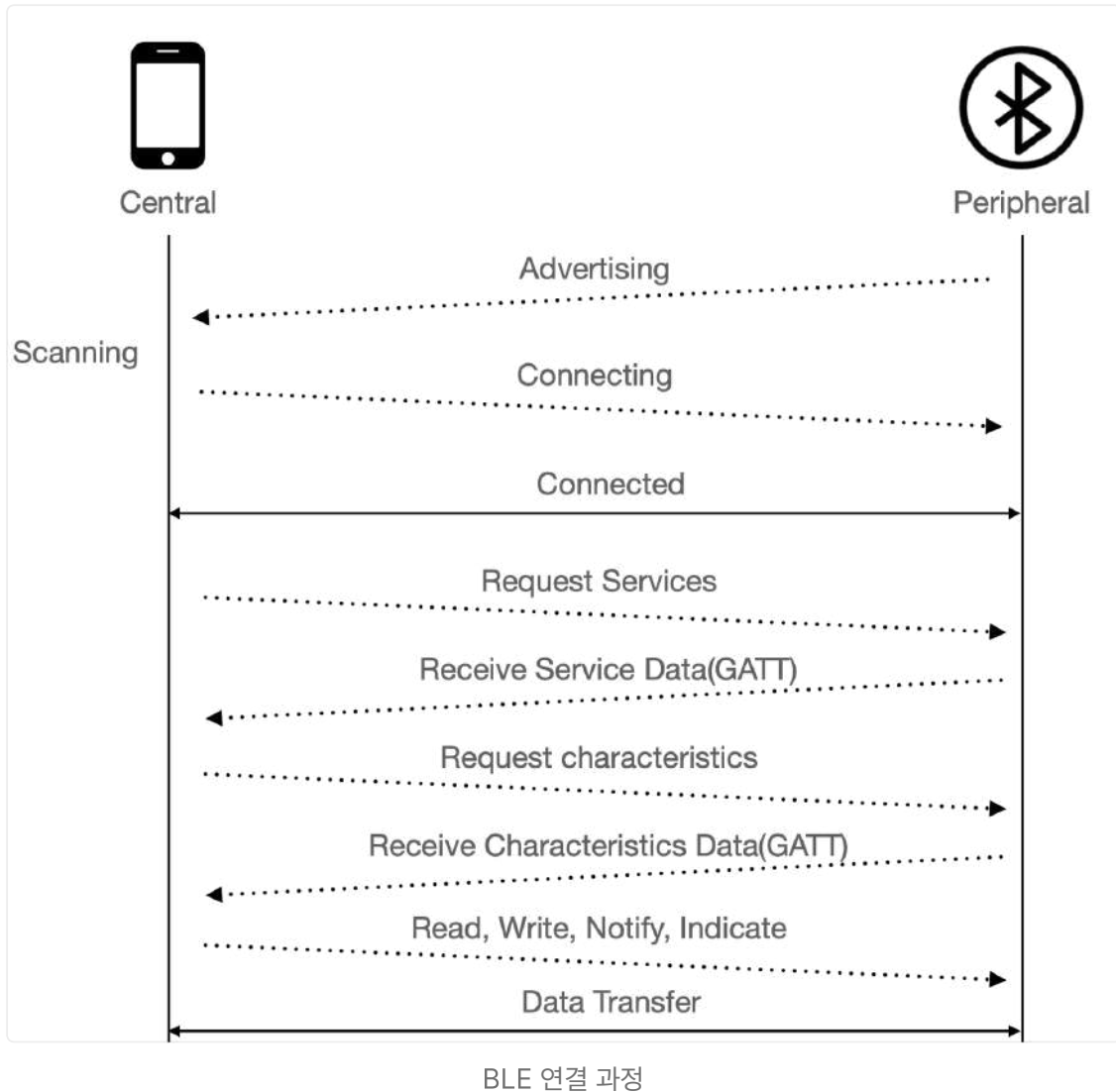
Service는 데이터를 논리적인 단위로 나누는 역할을 수행하며 characteristic이라 불리는 데이터 단위를 하나 이상 포함하고 있다. 각 service는 UUID 구분자를 가지고 있다. Bluetooth SIG에서는 자주 사용되는 service UUID를 사전에 정의해두고 있다.

Characteristic은 GATT 기반 동작구조의 가장 하위 단위이며, 단 하나의 데이터만을 포함한다. Bluetooth SIG에서는 자주 사용되는 characteristic UUID를 사전에 정의해두고 있다.

Bluetooth SIG에서 정의된 service UUID, characteristic UUID값은 [링크](#)에서 확인할 수 있다.

3.5. BLE 연결 과정

BLE 연결 과정은 아래 그림과 같은 순서로 진행된다.



위 연결과정에 대한 설명은 아래와 같다.

1. Advertising: Peripheral device에서 advertising 동작을 수행한다. 해당 device의 정보를 Central로 동작하는 근처 디바이스에 광고를 진행한다.
2. Scanning: Central의 경우 BLE 무선연결을 위해 근처에 연결 가능한 디바이스가 있는지 스캔한다.
3. Connecting: Central이 연결하고자 하는 Peripheral device로 연결 요청 신호를 전송한다. Peripheral이 연결 요청 신호를 수신하면, 두 디바이스간의 무선 연결이 시작된다.
4. Request Service: Central이 연결된 Peripheral에게 Service를 요청한다.

5. Receive Service Data: Peripheral이 Central에게 Service를 전달한다.
6. Request Characteristic: Central이 연결된 Peripheral에게 Characteristic를 요청한다.
7. Receive Characteristic Data: Peripheral이 Central에게 Characteristic를 전달한다.
8. Request: Central이 read, write, notify, indicate 네 가지 동작 중 원하는 동작이 필요한 Characteristic을 Peripheral에게 요청한다.

4. 바나나 체온계로부터 온도정보를 받아오자

4.1. Raspberry Pi(Python)

본 단락은 Raspberry Pi를 사용해 바나나 체온계와 통신하여 온도정보를 저장, 그래프를 그리는 과정을 설명한다.

4.1.1. 개발환경 구축

~~개발에 사용한 사양은 다음과 같다.~~

- Raspberry Pi: Raspberry Pi 4
- Raspberry Pi OS: Raspbian GNU/Linux 12
- Visual Studio Code: 1.83.0
- Python: 3.11.2

이 문서는 특정 개발 환경을 기준으로 작성되었다.

처음 이 분야를 배우는 분들은 가능한 아래에 명시된 환경과 동일한 보드와 소프트웨어 버전을 사용하는 것이 좋다. 왜냐하면, 문서의 내용을 가장 정확하게 따라할 수 있기 때문이다.

만약 다른 보드를 가지고 있거나 최신 버전의 소프트웨어를 사용 중이라면, 일부 내용이나 화면이 이 문서와 다를 수 있다. 이런 경우 약간의 차이점을 감안하고 진행한다. 처음에는 이 문서에 명시된 환경으로 학습하고, 기본 개념을 이해한 후에 최신 버전이나 다른 보드로 시도해보는 것이 좋다. 차이점이 발견되더라도 당황하지 않는다. 기본 원리는 대부분 동일하므로, 조금만 응용하면 충분히 따라할 수 있다.

Raspberry Pi 설치

Raspberry Pi를 사용하기 위해 OS를 설치한다. [아래 링크](#)에 접속하여 Raspberry Pi Imager를 다운로드한다. 각자 자신이 사용 중인 운영 체제에 맞는 것을 선택한다. 본 문서는 macOS를 사용했기 때문에 Download for macOS버전을 다운로드한다.

<https://www.raspberrypi.com/software/>

Install Raspberry Pi OS using Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install Raspberry Pi OS and other operating systems to a microSD card, ready to use with your Raspberry Pi. [Watch our 45-second video](#) to learn how to install an operating system using Raspberry Pi Imager.

Download and install Raspberry Pi Imager to a computer with an SD card reader. Put the SD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

Download for macOS

[Download for Windows](#)

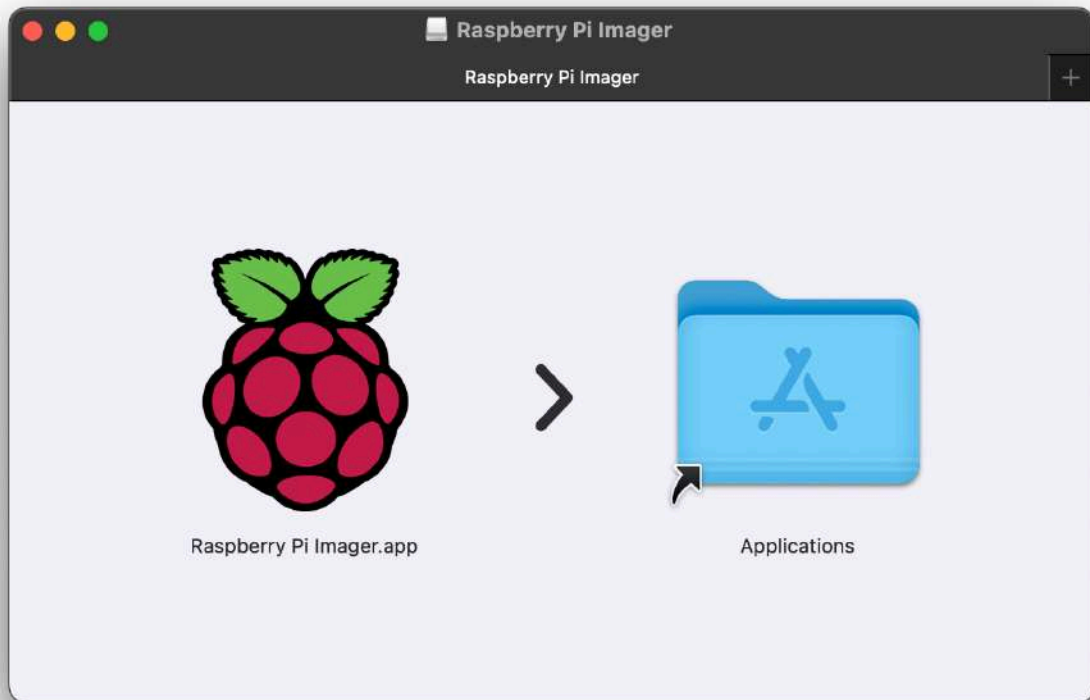
[Download for Ubuntu for x86](#)

To install on **Raspberry Pi OS**, type
`sudo apt install rpi-imager`
in a Terminal window.



Raspberry Pi 설치 프로그램 다운로드

설치 프로그램을 실행하면 아래 화면이 나타난다. 좌측에 Raspberry Pi Imager.app을 드래그해 Applications로 이동해 놓아 설치를 진행한다.



Raspberry Pi 설치 방법

설치가 완료되면 Raspberry Pi Imager를 실행한다. 좌측의 운영체제 선택을 클릭한다.

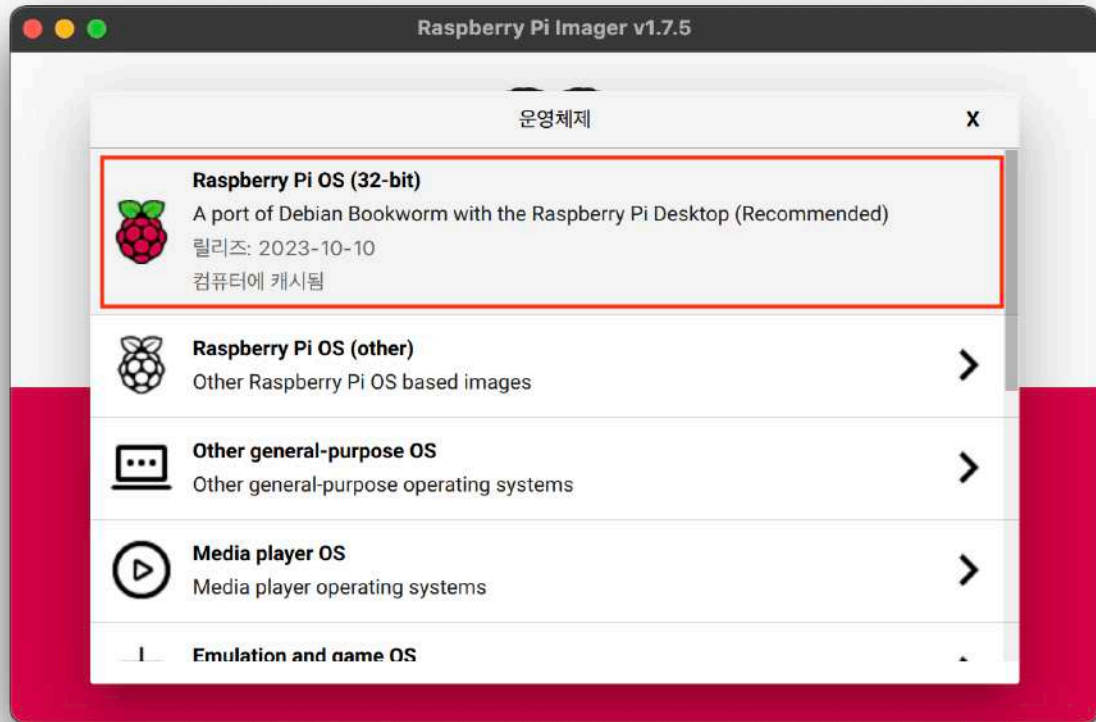


Raspberry Pi Image 실행

상단의 Raspberry Pi를 클릭하여 설치 운영체제를 선택한다.

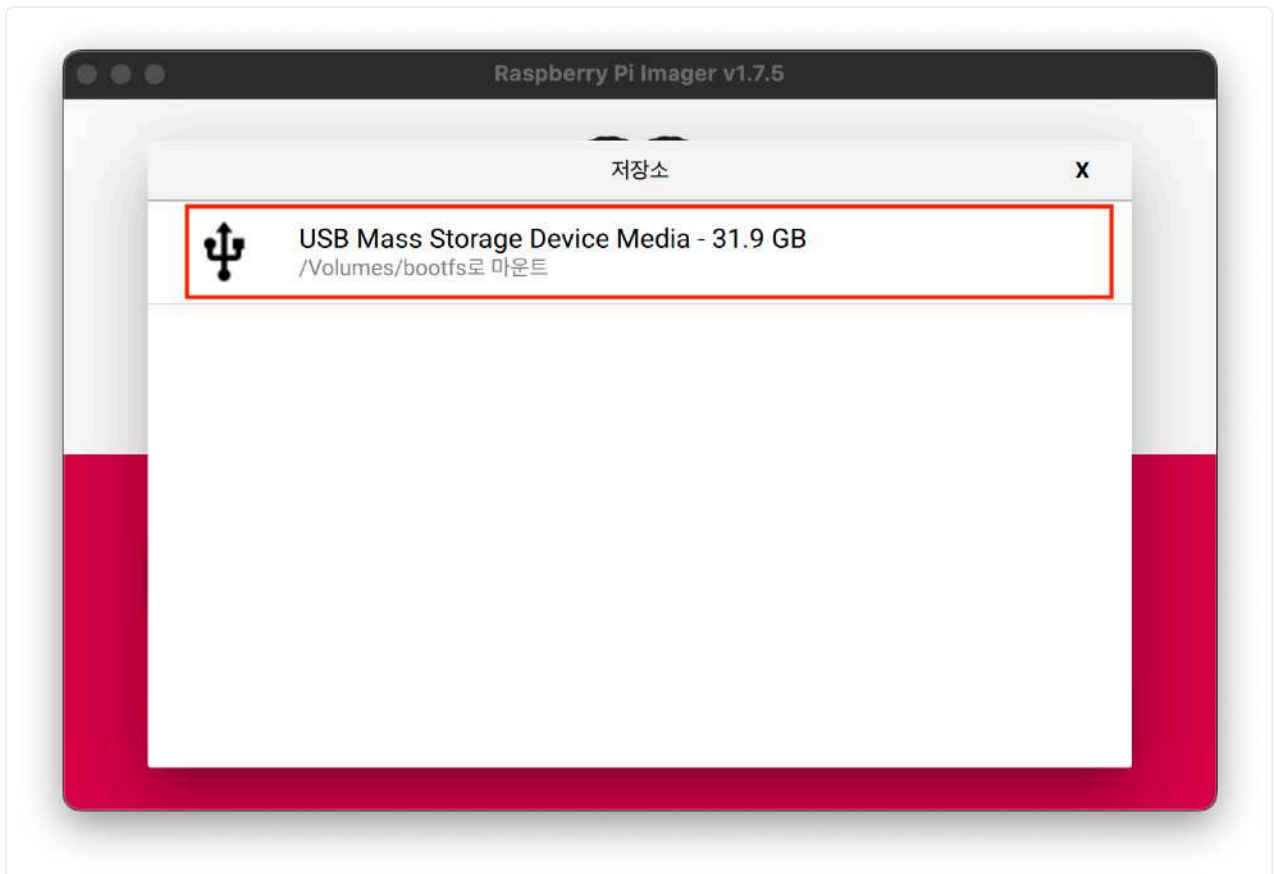
@노건호 여기에 소개하는 운영체제에 대한 설명도 간략하게 해 주세요.

다양한 운영체제 중에서 xx한 이유때문에 맨 위의 Raspberry Pi OS (32bit)를 설치한다라고 마무리.



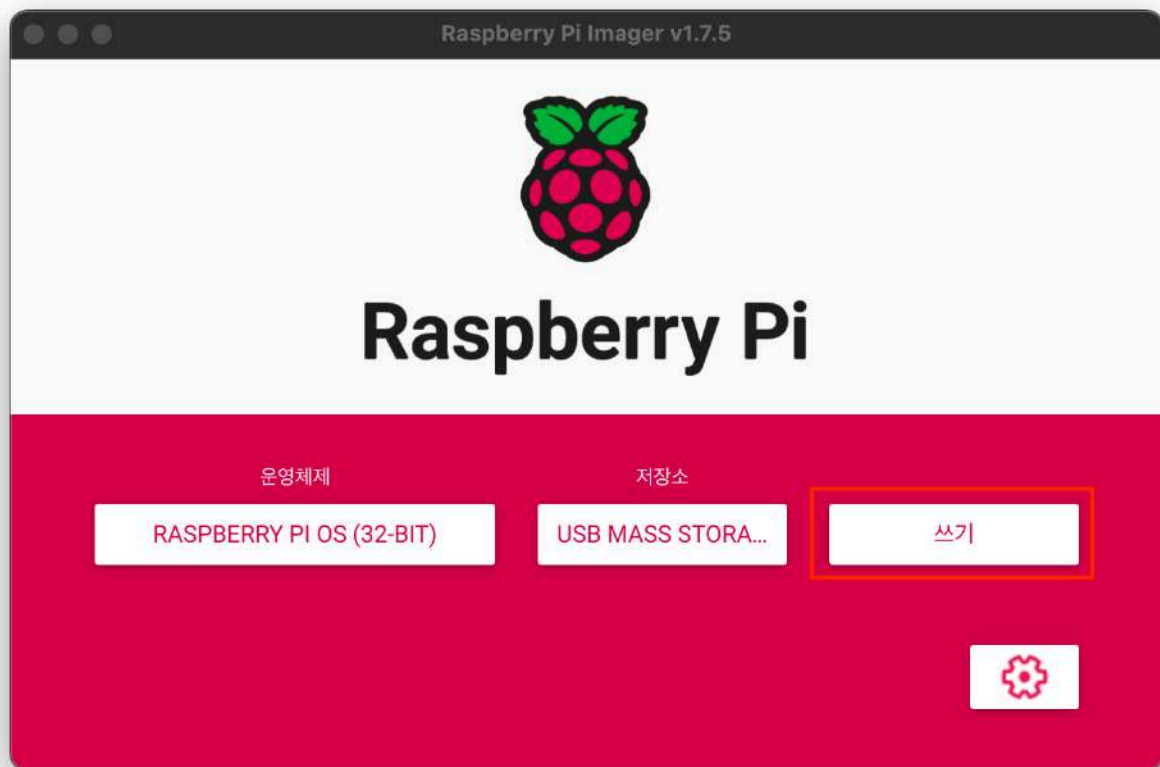
설치 운영체제 선택

SD카드를 컴퓨터에 삽입한 후 저장소를 선택한다.



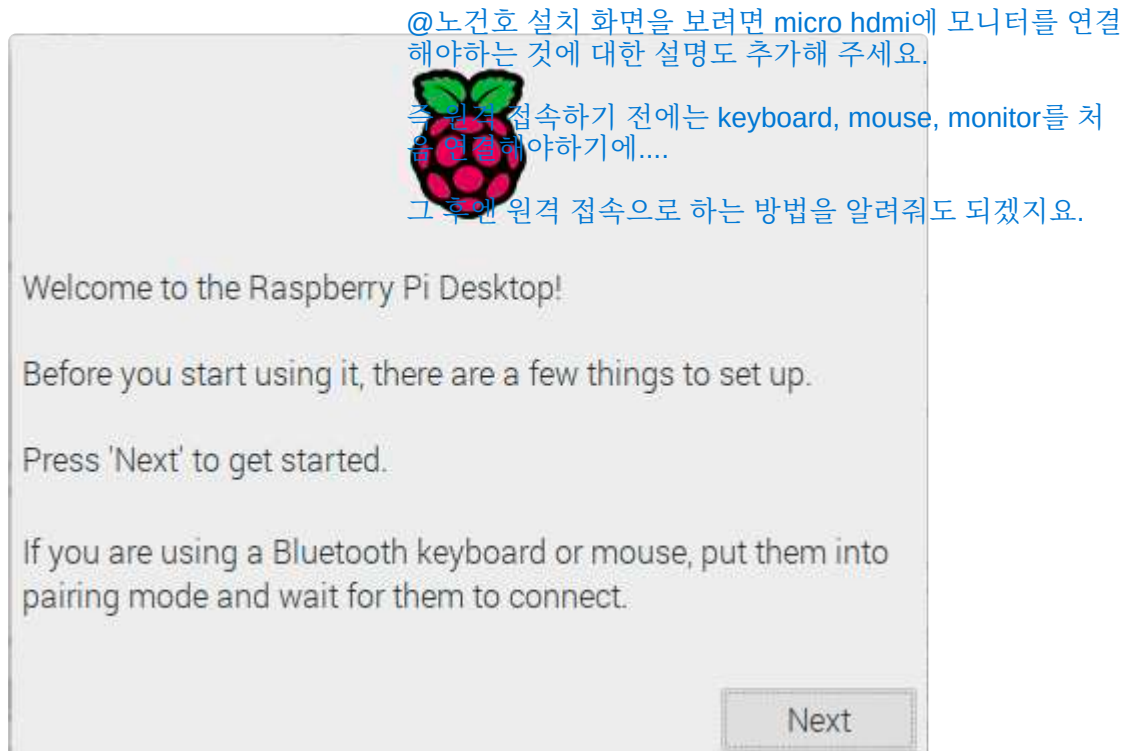
설치 저장소 선택

설치 운영체제와 저장소 선택 후 우측의 쓰기 버튼을 클릭해 운영체제를 설치한다.



Raspberry Pi 설치버튼 클릭

설치완료 후 SD카드를 Raspberry Pi에 삽입 후 USB-C 타입의 전원을 연결하면 Raspberry Pi 설정 화면이 나타난다. Next 버튼을 클릭한다.



Raspberry Pi 설정 시작

국가 설정 화면으로 전환되면 Country를 South Korea로 변경하고, Use English language 옵션을 체크한 후 Next 버튼을 클릭한다.

A screenshot of the 'Set Country' screen. At the top, the title 'Set Country' is in bold. Below it, a paragraph explains: 'Enter the details of your location. This is used to set the language, time zone, keyboard and other international settings.' There are three dropdown menus: 'Country' with 'United Kingdom' selected, 'Language' with 'British English' selected, and 'Timezone' with 'Belfast' selected. Below these are two checkboxes: 'Use English language' and 'Use US keyboard', both of which are unchecked. A line of text says 'Press 'Next' when you have made your selection.' At the bottom are two buttons: 'Back' on the left and 'Next' on the right.

Set Country

Enter the details of your location. This is used to set the language, time zone, keyboard and other international settings.

Country: United Kingdom ▼

Language: British English ▼

Timezone: Belfast ▼

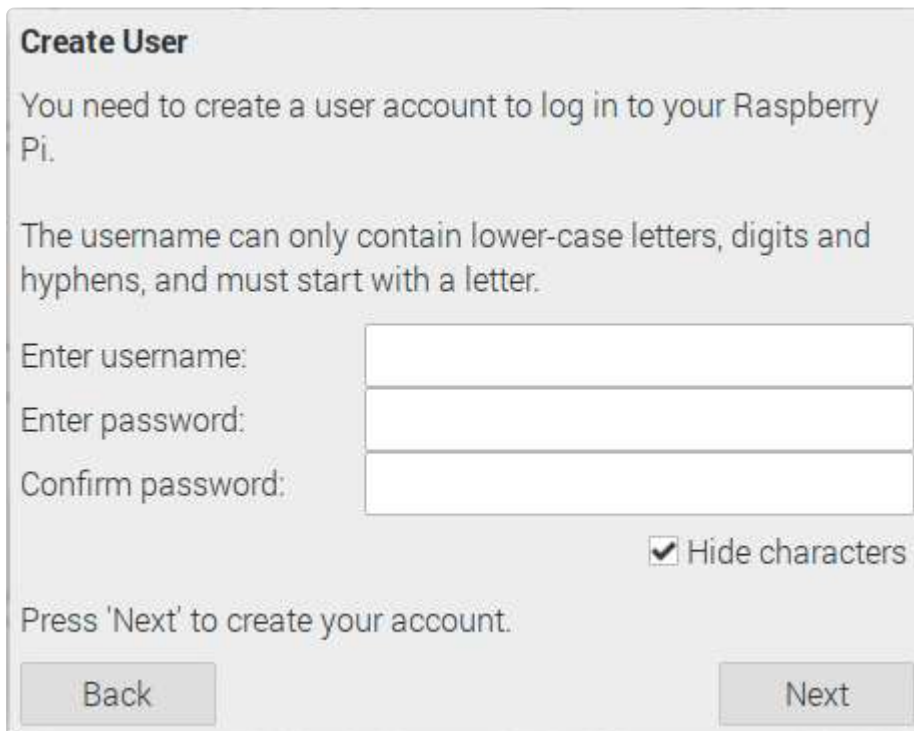
☐ Use English language ☐ Use US keyboard

Press 'Next' when you have made your selection.

Back Next

국가 선택

사용자명 설정 화면으로 전환되면 사용자 이름과 비밀번호를 입력 후 Next 버튼을 클릭한다.

A screenshot of the 'Create User' screen. At the top, the title 'Create User' is in bold. Below it, a paragraph explains: 'You need to create a user account to log in to your Raspberry Pi.' Another paragraph explains: 'The username can only contain lower-case letters, digits and hyphens, and must start with a letter.' There are three input fields: 'Enter username:', 'Enter password:', and 'Confirm password:'. To the right of the password fields is a checkbox labeled 'Hide characters' which is checked. A line of text says 'Press 'Next' to create your account.' At the bottom are two buttons: 'Back' on the left and 'Next' on the right.

Create User

You need to create a user account to log in to your Raspberry Pi.

The username can only contain lower-case letters, digits and hyphens, and must start with a letter.

Enter username:

Enter password:

Confirm password:

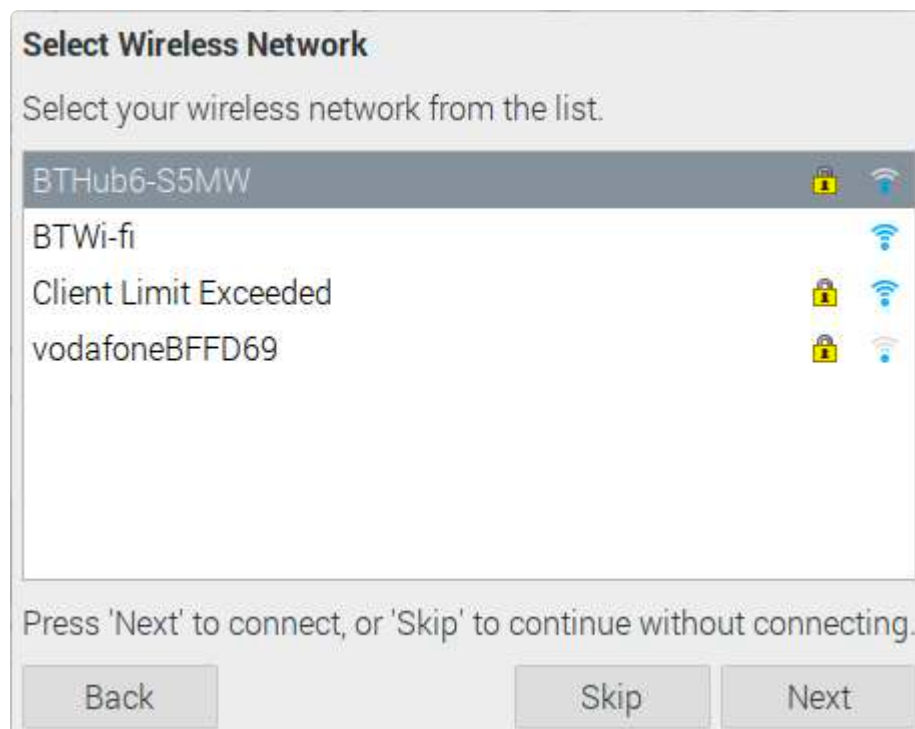
☒ Hide characters

Press 'Next' to create your account.

Back Next

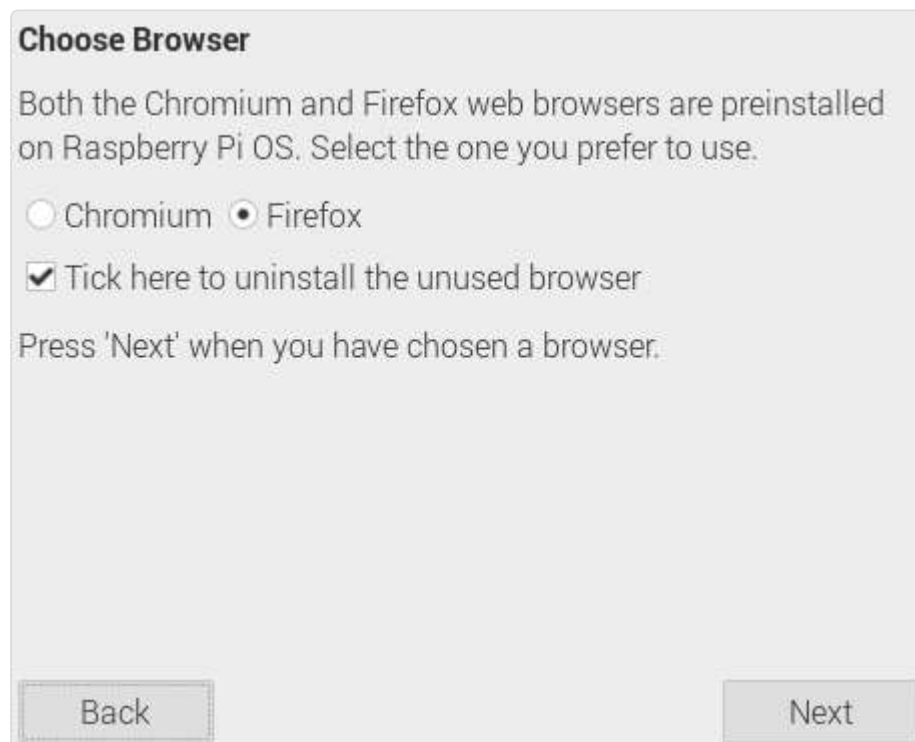
사용자명, 패스워드 생성

WiFi 설정 화면으로 전환되면 사용 가능한 WiFi 선택 후 Next 버튼을 클릭한다.



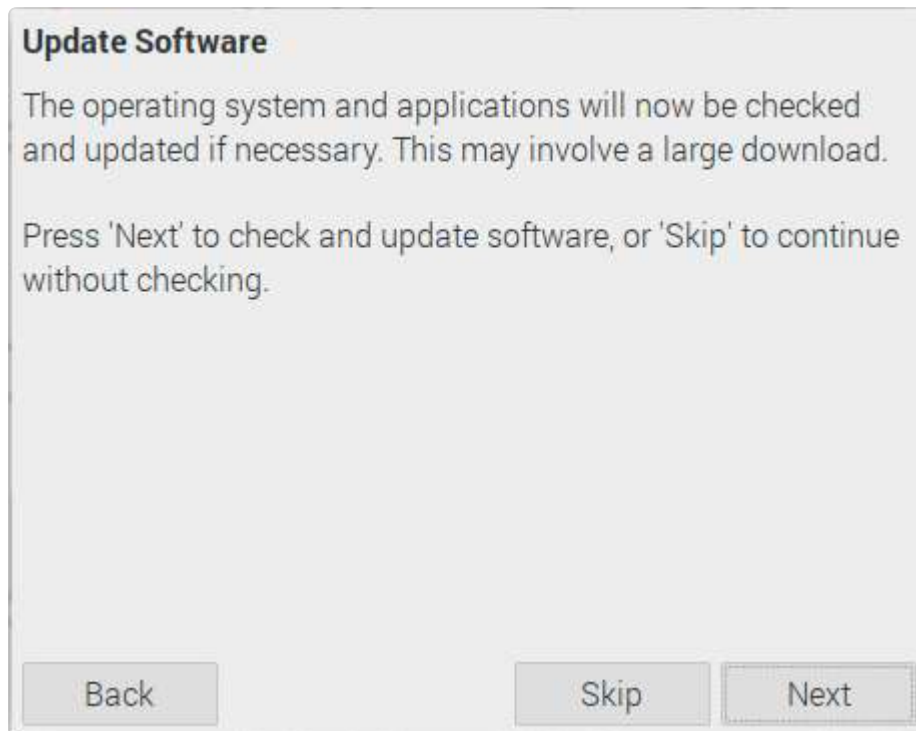
WiFi 설정

브라우저 선택 화면으로 전환되면 사용할 브라우저 선택 후 Next 버튼을 클릭한다. 본 문서는 Chromium으로 선택 후 진행한다.



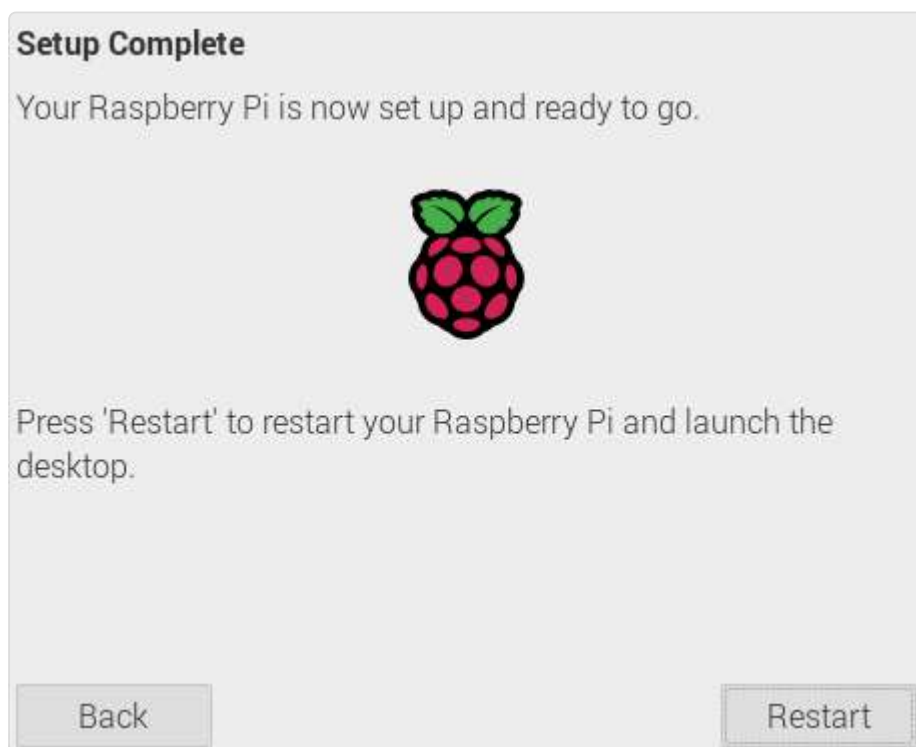
브라우저 선택

소프트웨어 업데이트 화면으로 전환되면 Next 버튼을 클릭해 업데이트를 진행한다.



소프트웨어 업데이트

설정완료 화면으로 전환되면 Restart 버튼을 클릭해 Raspberry Pi를 재시작한다.



라즈베리파이 설정 완료

라즈베리파이를 재시작하면 아래 화면과 같은 화면이 나타난다.



라즈베리파이 실행

Visual Studio Code 설치

본 문서는 개발 에디터인 Visual Studio Code(이하 VSCode)를 사용해 개발을 진행한다.
Terminal을 실행해 아래 명령어를 순차적으로 입력한다.

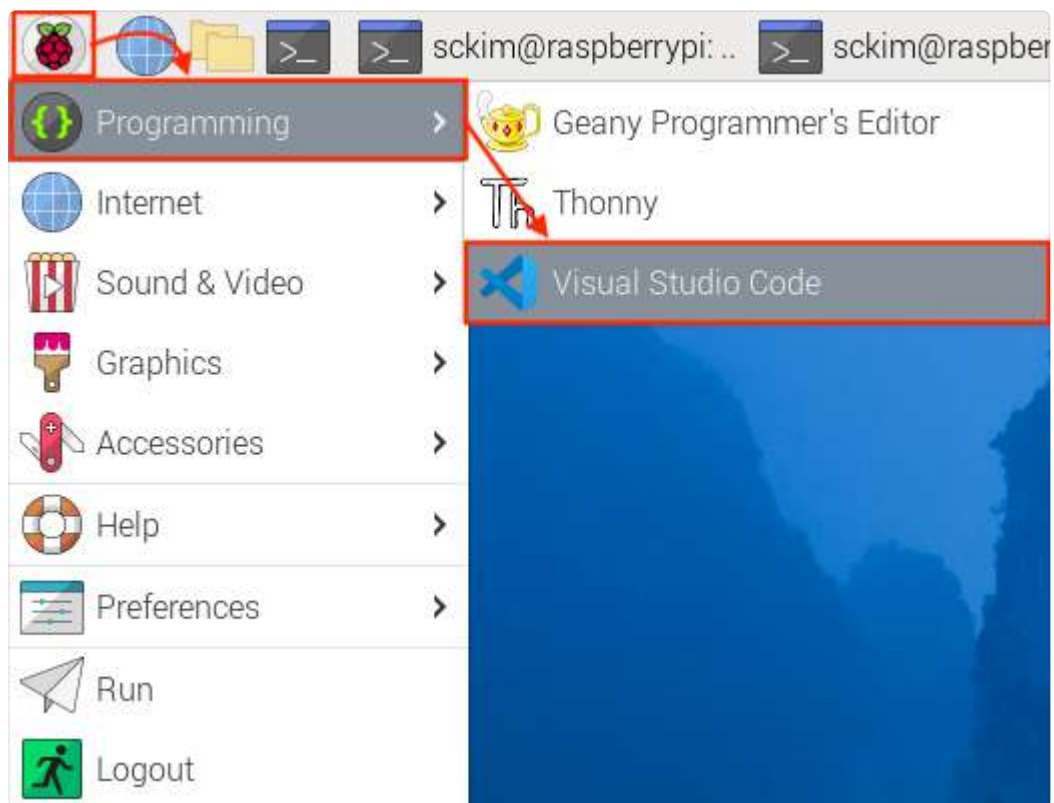
```
// 설치 가능한 패키지 리스트 최신화  
sudo apt update  
sudo apt upgrade
```

```
// Visual Studio Code 설치  
sudo apt install code
```

```
sckim@raspberrypi: ~  
File Edit Tabs Help  
sckim@raspberrypi:~ $ sudo apt update  
Hit:1 http://archive.raspberrypi.com/debian bookworm InRelease  
Hit:2 http://raspbian.raspberrypi.com/raspbian bookworm InRelease  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
27 packages can be upgraded. Run 'apt list --upgradable' to see them.  
W: http://raspbian.raspberrypi.com/raspbian/dists/bookworm/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details  
sckim@raspberrypi:~ $ sudo apt install code  
Reading package lists... Done
```

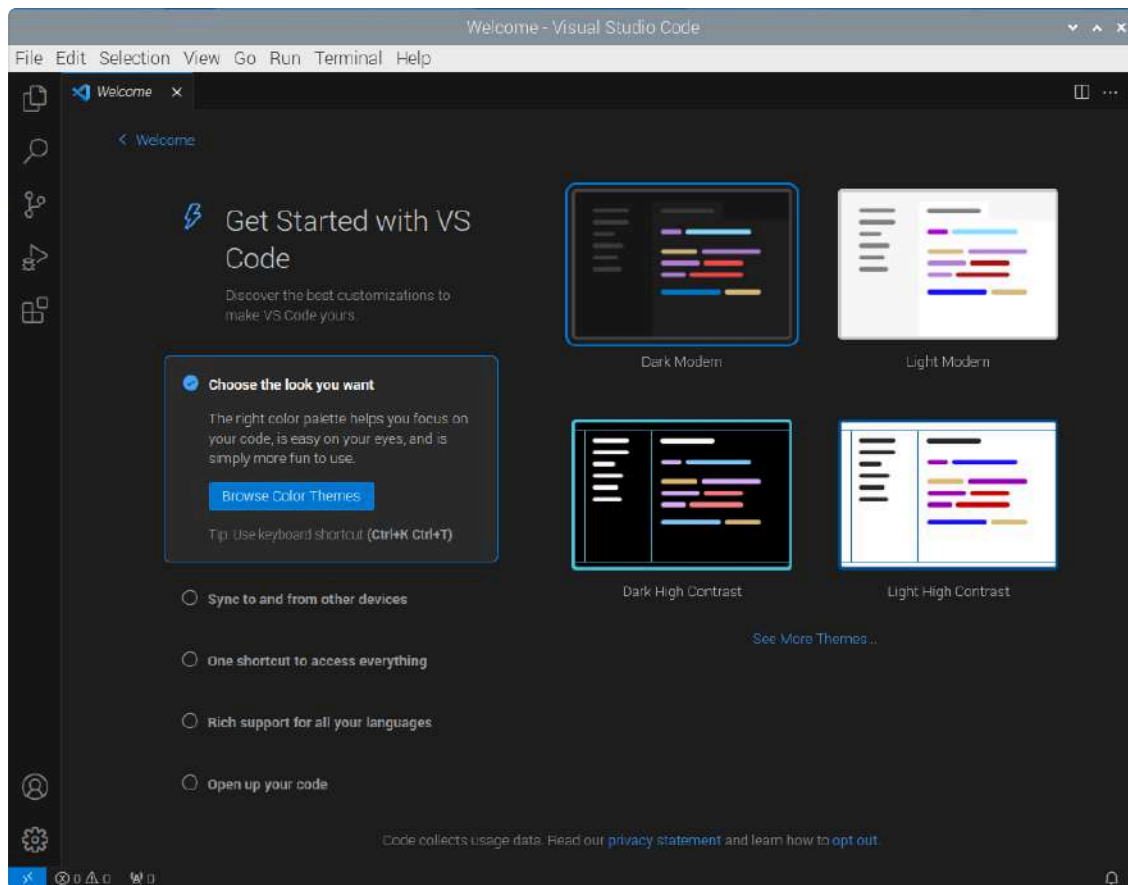
VSCode 설치

VSCode 설치 이후 아래 그림과 같이 클릭하면 실행된다.



Visual Studio Code 실행방법

VSCode를 실행하면 아래 화면이 나타나는것을 확인할 수 있다.

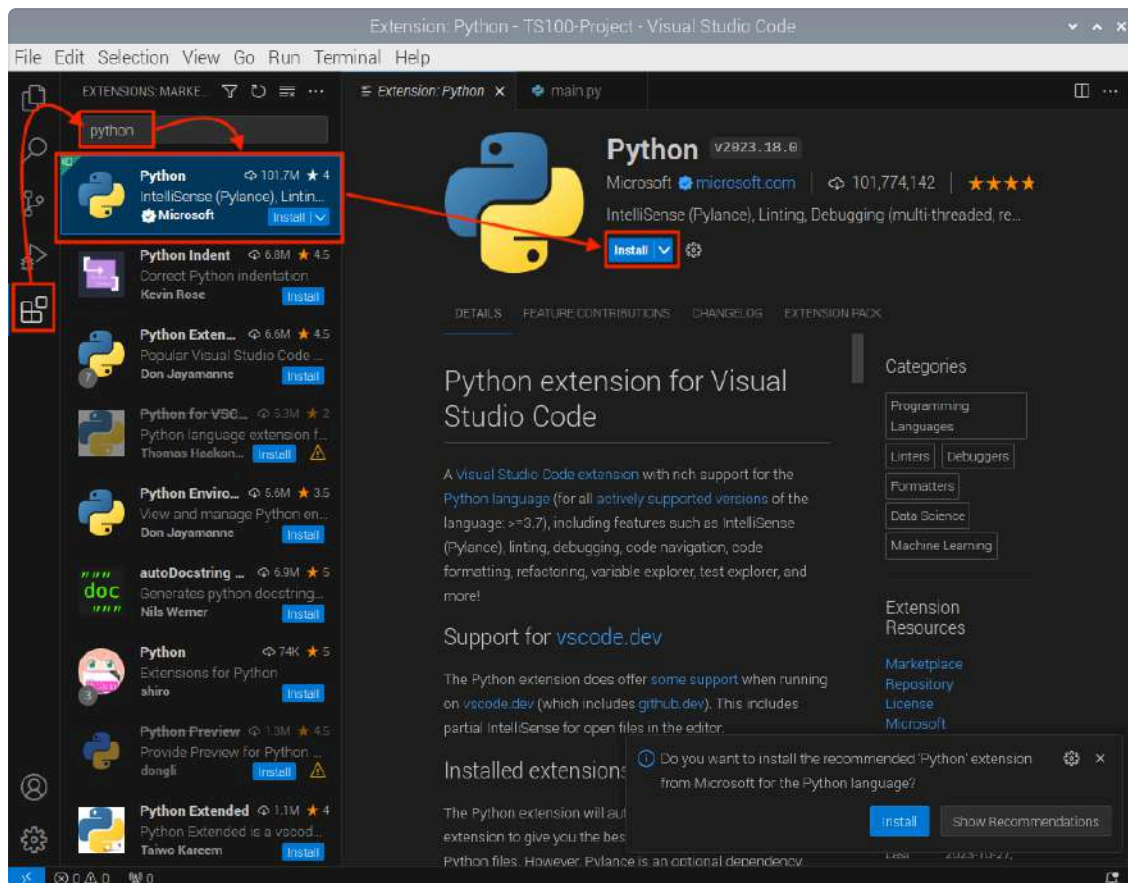


Visual Studio Code 실행

Python 확장 패키지 설치

VSCoide화면에서 Extension 버튼 클릭 후 Python을 검색해 Python 확장을 설치한다.

VSCoide(Visual Studio Code)는 강력한 텍스트 에디터이다. 하지만 기본 상태의 VSCoide는 범용 텍스트 편집기일 뿐, 특정 프로그래밍 언어를 위한 기능은 제한적이다. 이를 보완하기 위해 플러그인 개념의 '확장 모듈'을 제공한다. 각 프로그래밍 언어나 작업에 특화된 기능을 추가할 수 있다. 여기서는 Python을 활용할 예정이므로 Python 확장 모듈을 설치한다.



Python Extension 설치

관련 모듈 설치

바나나 체온계와 BLE 통신을 수행하고, 그 결과를 시각화하기 위해 그래프로 보여주기 위한 추가 모듈이 필요하다. 이 추가모듈은 VSCode의 확장 모듈이 아니라 Python의 확장 모듈임을 혼동하지 말아야 한다. 파이썬의 모듈은 C 언어에서 라이브러리와 유사한 개념이다. 다음 방법을 따라 설치해 보자. Terminal을 실행해.... 이후 동일.

바나나 체온계와 BLE 통신을 수행하고, 그래프로 나타내기 위해 관련 모듈을 설치해야 한다.

Terminal을 실행해 아래 명령어를 차례대로 입력한다. sudo reboot를 입력하면 Raspberry Pi가 재실행된다.

```
// 설치 가능한 패키지 리스트 최신화
sudo apt update
sudo apt upgrade

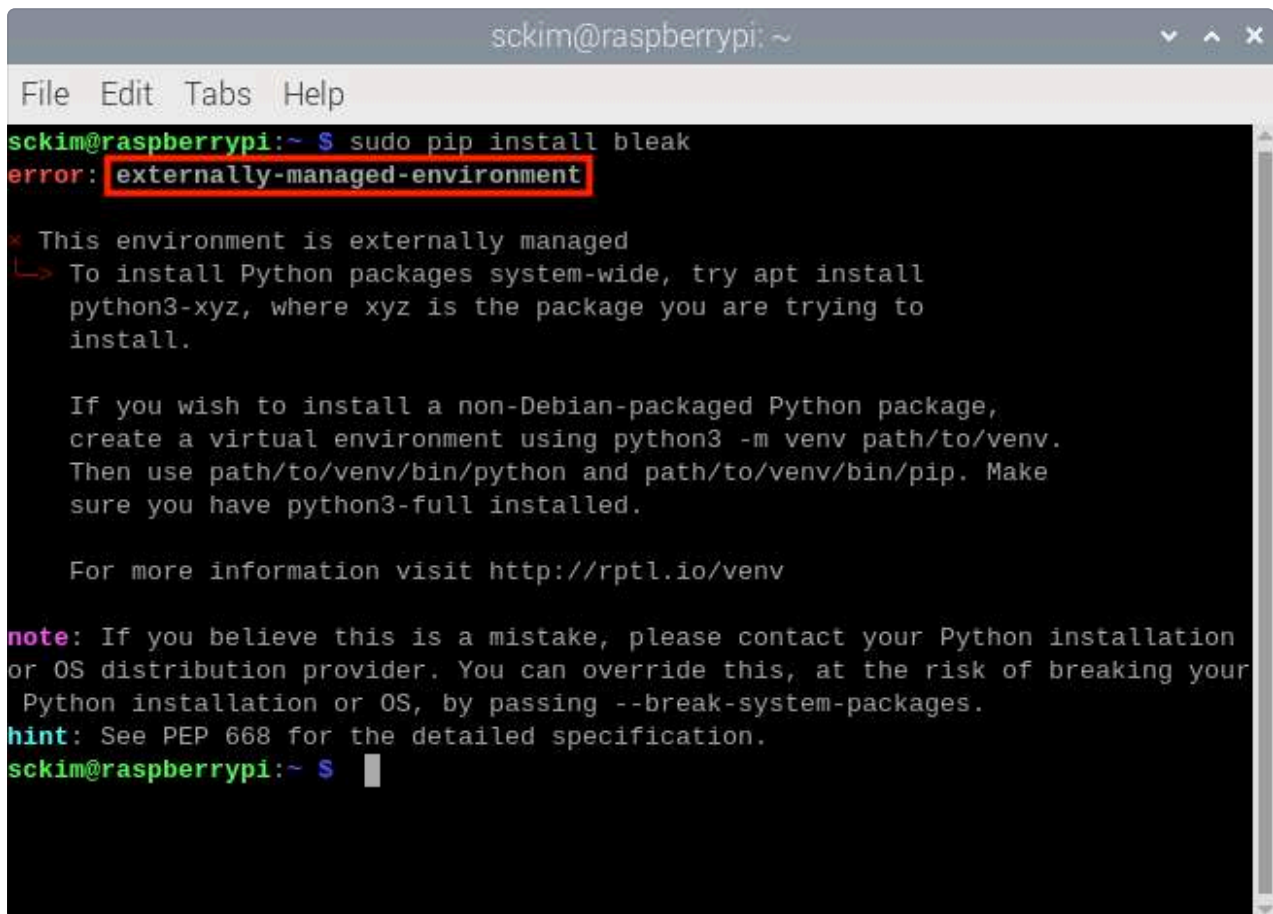
// 블루투스 관련 모듈 설치
sudo apt install bluetooth pi-bluetooth bluez blueman

// 재실행
sudo reboot
```

블루투스 관련 모듈 설치가 완료된 후 블루투스 기능을 사용하기 위해 Bleak 라이브러리를 설치한다. Terminal을 실행해 아래 명령어를 입력한다.

```
// Bleak 라이브러리 설치
sudo pip install bleak
```

위 명령어를 실행했을 때 ~~아래 그림과 같이~~ externally-managed-environment 오류가 발생할 수 있다. 이 오류는 Python 3.11 이상 버전에서 apt 관리 패키지와 pip 설치 패키지가 혼합되는것을 방지하는 [PEP 668](#) 보호 매커니즘이 도입되어 발생하는 오류이다.

A terminal window titled 'sckim@raspberrypi: ~' with a menu bar (File, Edit, Tabs, Help). The command 'sudo pip install bleak' is entered. The output shows an error: 'error: externally-managed-environment'. Below this, a message explains that the environment is externally managed and suggests using 'apt install python3-xyz' for system-wide packages or creating a virtual environment with 'python3 -m venv path/to/venv'. It also provides a note about contacting the Python installation provider and a hint to see PEP 668 for details. The prompt returns to 'sckim@raspberrypi:~\$'.

externally-managed-environment 오류

externally-managed-environment 오류가 발생하면 ~~아래 명령어를 입력해~~ bleak 설치한다.

```
// Bleak 라이브러리 설치
sudo apt install python3-bleak
```

@노건호 아래 그림, 아래 명령어은 부사를 언급하지 않더라도 명확하게 아는 내용이면 삭제합니다.

바나나 체온계로부터 수신된 온도정보를 그래프로 출력하기 위해 그래프 관련 모듈을 설치한다. Terminal을 실행해 아래 명령어를 차례대로 입력한다.

```
// 그래프 관련 모듈 설치
sudo pip install matplotlib

// externally-managed-environment 오류 발생시 아래 명령어 입력해 설치
sudo apt install python3-matplotlib
```

@노건호 지금까지 설치한 모듈을 한번에 확인할 수 있도록 여기에 정리한번 해줘요.
예,

지금까지 Python의 모듈을 설치하였다. 다음과 같은 모듈이 있는지 다시 한번 확인해 보자.
.....

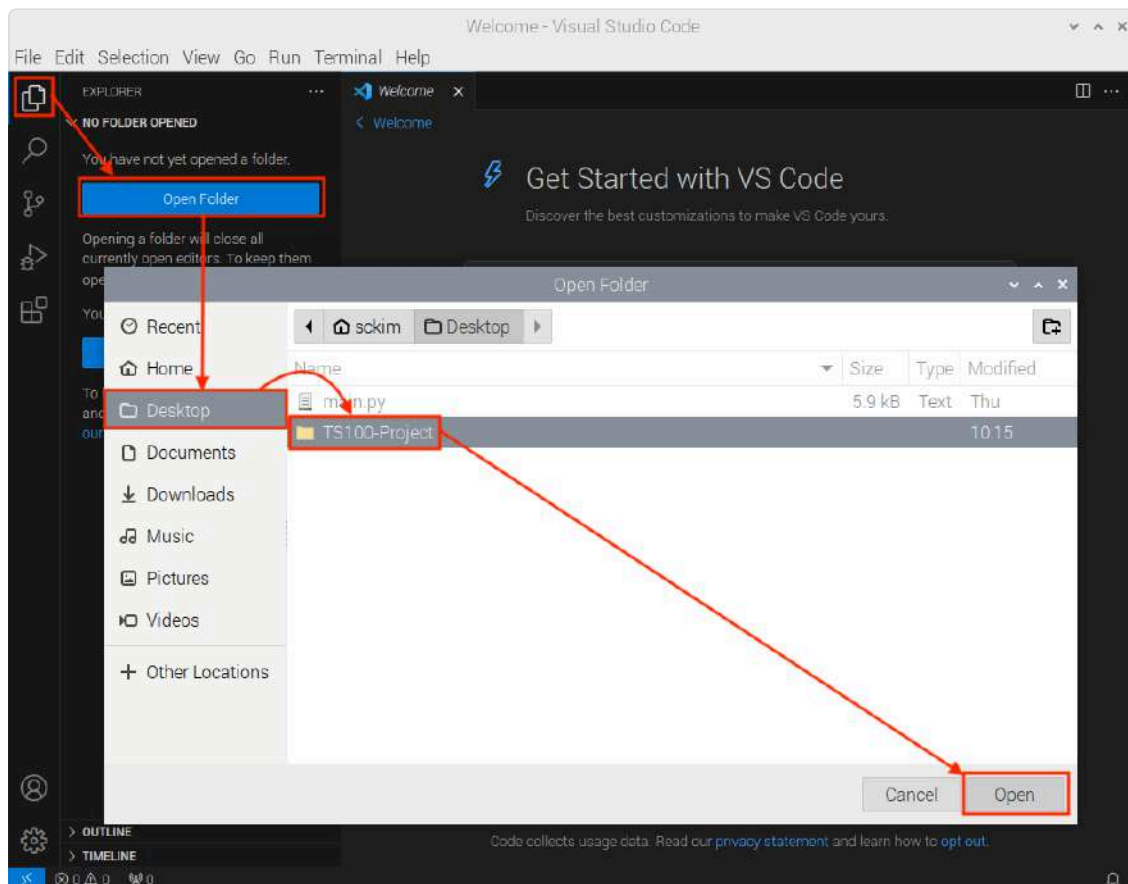
4.1.2. 프로젝트 생성

이제 코드를 작성할 시점이다. 하나의 소스 코드로 작업할 것이 아니기 때문에 코드를 저장할 폴더를 만든다. 프로젝트에서 사용될 파일을 생성하기 위해 홈화면(라즈베리파이의 바탕 화면)에서 우클릭 -> New Folder 버튼을 클릭해 “TS100-Project” 폴더를 생성한다.



Python 프로젝트 폴더 생성

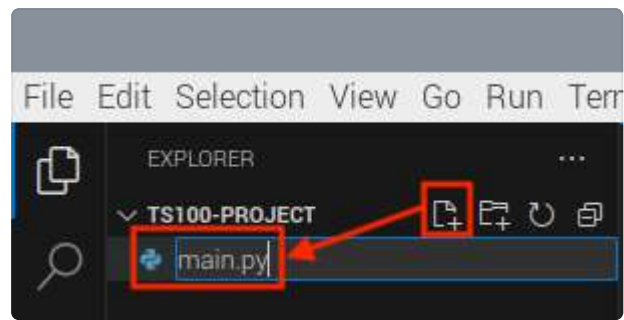
이후 VSCode를 열고 Open Folder 버튼을 클릭해 기존에 만들었던 TS100-Project 폴더 선택 후 Open 버튼을 클릭해 파일을 연다.



Visual Studio Code 폴더 열기

이후 아래 그림과 같이 main.py 파일을 생성한다.

드디어 파이썬의 메일 코드인 main.py를 아래 그림과 같이 새 파일 아이콘을 클릭하여 생성한다.



main.py 파일 생성 방법

4.1.3. BLE 연결

블루투스 기기 Scan

Python에서 바나나 체온계를 스캔하고 연결하기 위해 필요한 모듈들을 import해야 한다. main.py 파일에 다음과 같이 모듈들을 import 한다.

Python에서 바나나 체온계를 Scan하기 위해서는 아래 모듈을 import 해야한다. main.py 파일에 아래와 같이 모듈을 import한다.

```
import asyncio # 비동기화 모듈  
from typing import Optional # 타입 힌트를 위한 모듈  
from bleak import BleakScanner, BleakClient # Bluetooth Low Energy (BLE) 통신을 위한 모듈  
  
from bleak import BleakScanner, BleakClient # BLE 관련 모듈
```

이 모듈들은 각각 특정 기능을 제공한다.

BLE 관련 모듈 import 후 main 모듈을 추가한다. if 내 프로그램이 메인 모듈이 된다.

asyncio:

비동기 프로그래밍을 지원하는 모듈이다.

BLE 장치와의 통신은 시간이 걸릴 수 있으므로, 비동기 처리를 통해 프로그램의 효율성을 높인다.

```
if __name__ == '__main__':  
    # 메인 프로그램 실행
```

typing (Optional):

이후 비동기 처리를 위해 현재 실행 중인 이벤트를 루프를 가져온다. 아래와 같이 코드를 추가한다. 코드의 가독성을 높이고 잠재적인 오류를 미리 발견할 수 있게 도와준다.

```
if __name__ == '__main__':  
    # 비동기 이벤트 루프 생성  
    loop = asyncio.get_event_loop()  
    bleak = BleakScanner, BleakClient):  
    # Bluetooth Low Energy (BLE) 장치와 통신하기 위한 라이브러리이다.  
    BleakScanner: BLE 장치를 검색(스캔)하는 데 사용된다.  
    BleakClient: 검색된 BLE 장치에 연결하고 데이터를 주고받는 데 사용된다.
```

이후 전체 기기를 Scan하기 위해 scan 함수를 생성한다. scan하는 함수는 비동기 처리를 위해

async 함수로 구현된다.

모듈을 import한 후, 프로그램의 메인 부분을 작성한다. Python에서는 스크립트가 직접 실행될 때 특별한 동작을 수행하도록 하기 위해 다음과 같은 조건문을 사용한다.

이 조건문은 '이 파일이 직접 실행되는 경우에만 아래 코드를 실행하라'는 의미이다. 이렇게 함으로써 이 파일이 다른 프로그램에 의해 모듈로 import될 때는 실행되지 않고, 직접 실행될 때만 원하는 동작을 수행하게 할 수 있습니다.

정의한다.

```

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

```

~~이후 main 함수에 scan을 호출하는 코드를 추가한다. scan 함수가 종료될 때까지 기다려야하기 때문에 함수가 종료될 때까지 기다린다.~~

main 함수에서 비동기 이벤트 루프를 사용해서 scan 함수를 호출하여 scan 함수가 종료될 때 까지 기다린다.

```

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan 함수 실행
    scan_devices = loop.run_until_complete(scan())

```

@노건호 asyncio의 사용법에 대한 설명도 조금 자세히 넣었으면 합니다.
get_event_loop 와 run_until_complete 만이라도.

이후 코드를 실행하면 아래 그림과 같이 콘솔에 scan된 바나나 체온계 기기를 확인할 수 있다.

```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943

```

Scan된 바나나 체온계 콘솔 출력

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan 함수 실행
    scan_devices = loop.run_until_complete(scan())

```

블루투스 기기 선택

스캔된 BLE(Bluetooth Low Energy) 기기의 정보를 사용자에게 시각적으로 보여주기 위해, 그래픽 사용자 인터페이스(GUI)를 만들 필요가 있다. 이를 위해 Python의 표준 GUI(그래픽 사용자 인터페이스) 라이브러리인 tkinter를 사용한다.

Scan된 BLE 기기 정보를 GUI로 보여주기 위해 tkinter를 import한다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # 추가된 부분(GUI 관련 라이브러리)

```

tkinter는 Python에 기본으로 포함된 GUI 킷이다. 별도의 설치 없이 바로 사용할 수 있다. 'tk'라는 별칭으로 import하는 것이 일반적이다. ttk(themed tk)는 tkinter의 확장 모듈로, 더 현대적이고 미려한 위젯을 제공한다. 주요 기능으로는 윈도우를 생성하여 애플리케이션의 메인 창을 만들 수 있고, 버튼, 레이블, 텍스트 박스, 리스트 박스 등 다양한 UI 요소를 제공한다. 이 뿐만 아니라, 사용자의 마우스 클릭, 키보드 입력 등의 이벤트를 처리할 수 있고, 위젯들을 원하는 위치에 배치할 수 있는 다양한 레이아웃 관리 기능도 제공한다.

이렇게 tkinter를 사용하여 코드를 실행하는 터미널에서 테스트 코드를 실행하는 것이 아니라 시각적으로 더 이해할

Scan된 device를 확인할 수 있는 show_scan_devices() 함수를 추가한다. Scan된 device가 화면에 버튼 형태로 나타나고, 버튼을 클릭했을 때 button_click() 함수가 호출되어 화면이 사라지는 형태로 구현되었다.

```
# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device: button_click(device, window))
        button.pack()
        buttons.append(button)

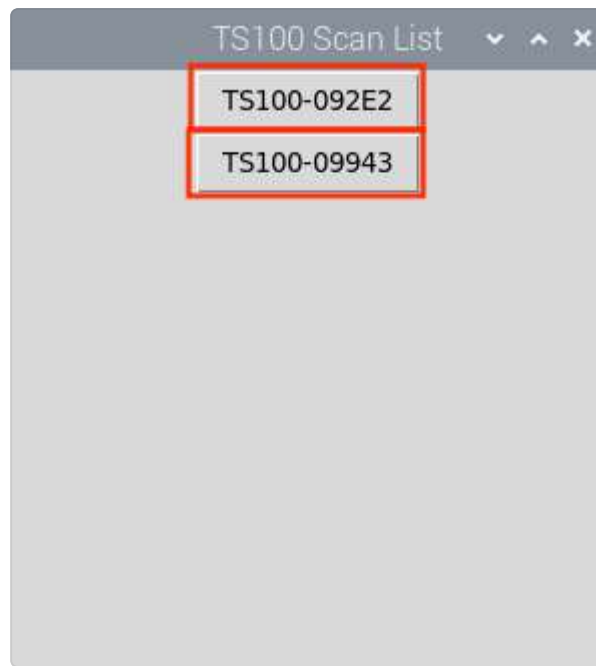
    # 윈도우 종료까지 실행
    window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    print("Select Device is " + device.name)

    # GUI 제거
    window.quit()

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)
```

위 코드를 실행하면 아래 그림처럼 화면에 Scan된 기기를 확인할 수 있고, 연결하고자 하는 Device를 선택하면 화면이 종료된다.



Scan된 Device GUI 확인

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈


import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    print("Select Device is " + device.name)

    # GUI 제거
    window.quit()

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

```

블루투스 기기 연결

Scan된 device를 담는 전역변수 selected_device를 추가한다.

```
selected_device = None
```

button_click() 함수에 연결하고자 하는 바나나 체온계를 selected_device 변수에 담는 코드를 추가한다.

```

def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

```

블루투스 기기를 연결하기 위해 connect 함수를 추가한다. 비동기 처리를 위해 async 함수로 구현한다.

```

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

```

main 함수에서 연결요청을 수행할 device를 선택했을 때 connect 과정을 수행하는 코드를 추가한다.

```

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

```

위 코드를 실행하면 아래와 같은 콘솔(Terminal)에 다음과 같이 출력된다.이 출력되는것을 확인할 수 있다.

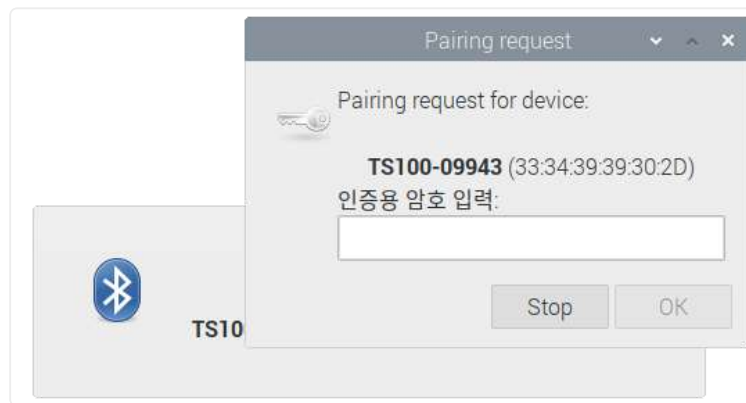
```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943
TS100-092E2
connect start
connected

```

connect된 바나나 체온계 콘솔 출력

처음 페어링을 진행하는 경우 아래 그림과 같이 PIN Code를 입력해야한다.



PIN Code 입력 화면

바나나 체온계 뒷면의 PIN Code 6자리를 입력하면 된다.



바나나 체온계 PIN Code 위치

지금까지의 내용이 반영된 코드이다. [✓]다음은 위 과정을 모두 수행한 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈


import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

selected_device = None

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

```

코드가 잘린것 같습니다. 확인해 주세요.

Service, Characteristic 요청

블루투스 연결이 완료 후, 바나나 체온계에서 제공하는 전체 Service와 Characteristic을 확인하기 위해 바나나 체온계에 Service와 Characteristic을 요청한다.

Service 요청을 위해 ~~아래 함수~~를 추가한다.

→ 위한 `get_service_and_characterisc` 함수를 추가한다.

```
# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(device):
    # Service 확인
    for service in device.services:
        print(service)
    # Characteristic 확인
    for characteristic in service.characteristics:
        print(characteristic)

    print()
```

`get_service_and_characteristic` 함수를 ~~호출하는 코드~~를 `main`함수에 추가한다.

에서 호출한다.

```
if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    if selected_device is not None:
        connected_device = loop.run_until_complete(connect(selected_device))
        print("device is " + selected_device.name)

    # 바나나 체온계와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))
```

온전한 코드가 보이도록.

위 코드를 실행하면 아래 그림과 같이 전체 Service와 Characteristic을 확인할 수 있다.

위 코드를 실행하여 바나나 체온계에서 제공되는 Service와 Characteristic을 살펴볼 수 있다.

```
00001801-0000-1000-8000-00805f9b34fb (Handle: 12): Generic Attribute Profile
00002a05-0000-1000-8000-00805f9b34fb (Handle: 13): Service Changed

00001809-0000-1000-8000-00805f9b34fb (Handle: 47): Health Thermometer
0000fbfb-0000-1000-8000-00805f9b34fb (Handle: 48): Vendor specific
00002a08-0000-1000-8000-00805f9b34fb (Handle: 70): Date Time
0000f9f9-0000-1000-8000-00805f9b34fb (Handle: 62): Vendor specific
0000fdfd-0000-1000-8000-00805f9b34fb (Handle: 66): Vendor specific
0000fafa-0000-1000-8000-00805f9b34fb (Handle: 51): Vendor specific
0000fcfc-0000-1000-8000-00805f9b34fb (Handle: 58): Vendor specific
00002a1c-0000-1000-8000-00805f9b34fb (Handle: 54): Temperature Measurement

0000180a-0000-1000-8000-00805f9b34fb (Handle: 20): Device Information
00002a50-0000-1000-8000-00805f9b34fb (Handle: 31): PnP ID
00002a24-0000-1000-8000-00805f9b34fb (Handle: 23): Model Number String
00002a29-0000-1000-8000-00805f9b34fb (Handle: 21): Manufacturer Name String
00002a27-0000-1000-8000-00805f9b34fb (Handle: 25): Hardware Revision String
00002a23-0000-1000-8000-00805f9b34fb (Handle: 29): System ID
00002a26-0000-1000-8000-00805f9b34fb (Handle: 27): Firmware Revision String

0000fef5-0000-1000-8000-00805f9b34fb (Handle: 33): Dialog Semiconductor GmbH
724249f0-5ec3-4b5f-8804-42345af08651 (Handle: 36): Unknown
9d84b9a3-000c-49d8-9183-855b673fda31 (Handle: 40): Unknown
5f78df94-798c-46f5-990a-b3eb6a065c88 (Handle: 44): Unknown
8082caa8-41a6-4021-91c6-56f9b954cc34 (Handle: 34): Unknown
457871e8-d516-4ca1-9116-57d0b17b9cb2 (Handle: 42): Unknown
6c53db25-47a1-45fe-a022-7c92fb334fd4 (Handle: 38): Unknown

0000180f-0000-1000-8000-00805f9b34fb (Handle: 16): Battery Service
00002a19-0000-1000-8000-00805f9b34fb (Handle: 17): Battery Level
```

바나나 체온계 전체 Service, Characteristic

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device:
            print(device.name))
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인
    for service in connected_device.services:
        print(service)
        # Characteristic 확인
        for characteristic in service.characteristics:
            print(characteristic)

    print()

selected_device = None

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면

```

```

if selected_device is not None:
    # connect 요청 후 함수가 끝날때까지 기다림
    connected_device = loop.run_until_complete(connect(selected_device))
    # 바나나 체온계와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_

```

온도정보 가져오기

바나나 체온계의 여러 characteristic 중 온도정보를 받아오는 기능을 구현한다. 온도 정보는 [아래 특성](#)에서 확인할 수 있다. Characteristic

```

Service: Health Thermometer(00001809-0000-1000-8000-00805f9b34fb)
Characteristic: Temperature Measurement(00002a1c-0000-1000-8000-00805f9b34fb)

```

온도정보 수신

여러 Service 중 Health Thermometer와 관련된 service만 수신받기 위해 get_service_and_characteristic 함수를 아래와 같이 수정한다.

```

# Service UUID
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                print(characteristic)

    print()

```

이후 온도정보 characteristic을 통해 notify를 수신하기 위해 함수를 아래와 같이 수정한다.

```

# Characteristic
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

```

연결된 기기가 종료될 때까지 이벤트 루프를 종료하지 않기 위해 wait_connect 함수를 추가하고 main을 수정한다.

```

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

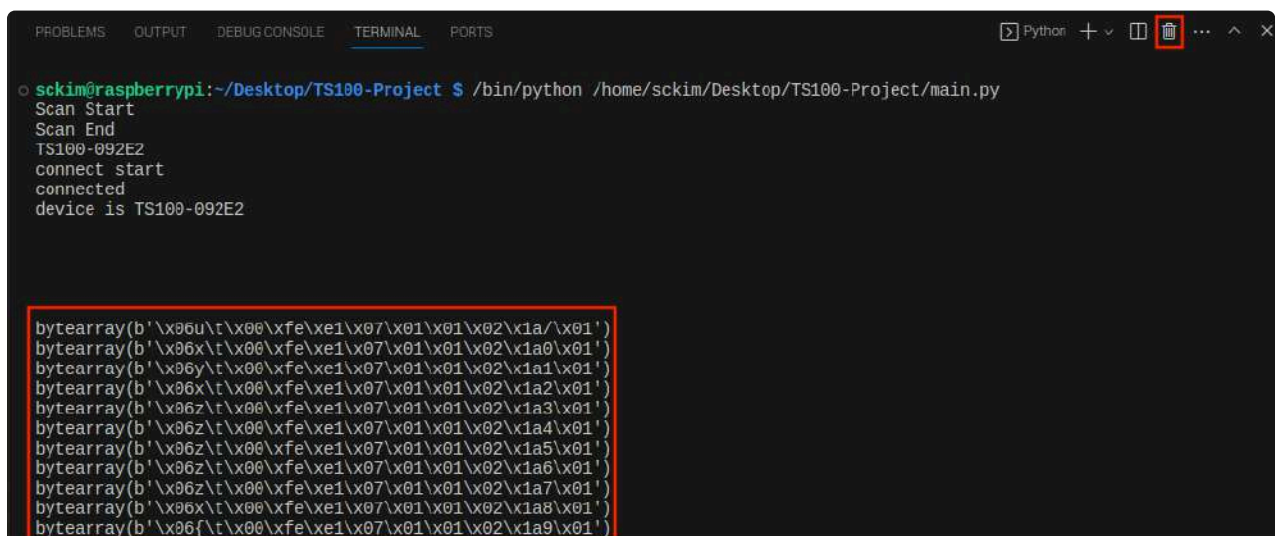
    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

    # 바나나 체온계바나나 체온계바나나 체온계바나나 체온계 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))

    # 종료 방지함수 호출
    loop.run_until_complete(wait_connect())

```

이후 코드를 실행하면 아래와 같이 온도정보가 콘솔창에 출력되는것을 확인할 수 있다. 현재 wait_connect 함수는 별도의 종료를 하지않으면 코드가 계속 실행되므로 우측 상단의 쓰레기통  아이콘을 클릭해 코드실행을 종료한다.



```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-092E2
connect start
connected
device is TS100-092E2

bytearray(b'\x06u\t\x00\xfe\xe1\x07\x01\x01\x02\x1a\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a0\x01')
bytearray(b'\x06y\t\x00\xfe\xe1\x07\x01\x01\x02\x1a1\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a2\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a3\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a4\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a5\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a6\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a7\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a8\x01')
bytearray(b'\x06{\t\x00\xfe\xe1\x07\x01\x01\x02\x1a9\x01')

```

온도정보 notify 수신 확인

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device:
            button.pack()
            buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

```

```

# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

    # 바나나 체온계와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))

    # 종료 방지함수 호출
    loop.run_until_complete(wait_connect())

```

온도정보 변환

바나나 체온계로부터 수신받은 데이터는 온도정보와 날짜정보로 이루어져 있다. 온도정보를 변환하는 함수를 아래와 같이 작성한다.

@노건호 이 부분은 숫자 한 두개의 예를 들어 t1, t2, t3, d의 값에 따라 온도값이 어떻게 되는지 서술해 주기 바랍니다.

```
# 온도정보 계산하는 함수
```

```
def temperature_calculate(t1, t2, t3, d):
```

```
# Int로 변환, 비트 시프트 후 합산
```

```
int_t1 = int(t1)
```

```
int_t2 = int(t2) << 8 # t2의 값을 왼쪽으로 8비트 shift 한다.
```

```
int_t3 = int(t3) << 16 # t3의 값을 왼쪽으로 16비트 shift 한다.
```

```
signed_value = int_t1 + int_t2 + int_t3
```

```
# 자릿수 정하는 변수
```

```
digit = int(d)
```

```
if digit > 127:
```

```
    digit -= 256
```

```
# 온도
```

```
temperature = float(signed_value) * 10 ** digit
```

```
# 소수점 1자리만 남도록 수정
```

```
temperature = round(temperature * 10) / 10
```

```
return temperature
```

위와 같이 하는 방법과

signed_vlaue = int(t1) + (int(t2)<<8)) + (int(int3)<<16))과 동일하다면, 위 코드도 같은 역할을 한다는 설명을 추가해 주기 바랍니다.

날짜정보를 변환하는 함수를 아래와 같이 작성한다.

@노건호 여기도 앞과 같이 구체적인 array 값을 예를 들어서 설명해 주세요.

```
# 날짜정보 변환하기 위해 사용하는 라이브러리
```

```
from datetime import datetime
```

```
# 날짜정보 계산하는 함수
```

```
def date_calculate(array):
```

```
    year = (array[1] & 0x0F) << 8 | (array[0] & 0x00FF)
```

```
    month = array[2]
```

```
    day = array[3]
```

```
    hour = array[4]
```

```
    minute = array[5]
```

```
    second = array[6]
```

```
    date_component = datetime(year, month, day, hour, minute, second)
```

```
    return date_component
```

기존에 작성했던 notify_callback 함수를 아래와 같이 온도정보와 날짜정보를 받아와 출력하는 함수로 변경한다.

```

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)

    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

```

표현이 어색합니다. 더 쉽게 설명해 주세요.

이제 프로그램을 이후 프로젝트를 실행하면 아래 그림과 같이 변환된 온도정보가 출력된다.되는것을 확인할 수 있다.

```

0-Project/main.py:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100
Scan Start
Scan End
TS100-092E2
TS100-09943
connect start
connected
device is TS100-09943
2017-01-01 00:00:12
33.5
2017-01-01 00:00:13
34.2
2017-01-01 00:00:14
34.6
2017-01-01 00:00:15
34.8
2017-01-01 00:00:16
35.0
2017-01-01 00:00:17
34.9
2017-01-01 00:00:18
34.6
2017-01-01 00:00:19
34.2
2017-01-01 00:00:20
33.8
2017-01-01 00:00:21
33.4

```

온도정보 변환

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 날짜정보 변환하기 위해 사용하는 라이브러리
from datetime import datetime

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device:
            button.pack()
            buttons.append(button)

# 윈도우 종료까지 실행

```

```

window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4]

    array_slice = data[5:]

```

```

date_array = list(map(int, array_slice))
# 날짜정보 변환
date = date_calculate(date_array)
# 온도, 날짜정보 출력
print(date)
print(temperature)

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

# 날짜정보 계산하는 함수
def date_calculate(array):
    year = (array[1] & 0x0F) << 8 | (array[0] & 0x00FF)
    month = array[2]
    day = array[3]
    hour = array[4]
    minute = array[5]
    second = array[6]

    date_component = datetime(year, month, day, hour, minute, second)

    return date_component

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

```

```

# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

    # 바나나 체온계와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))

    # 종료 방지함수 호출
    loop.run_until_complete(wait_connect())

```

날짜정보 수정

현재 바나나 체온계에서 수신되는 날짜정보는 현재 시간과 맞지 않는다. 올바른 시간을 확인하려면 바나나 체온계에 현재 시간을 알려주는 과정이 필요하다.

날짜와 관련된 특성은 아래와 같다.

```

Service: Health Thermometer(00001809-0000-1000-8000-00805f9b34fb)
Characteristic: Date Time(00002a08-0000-1000-8000-00805f9b34fb)

```

현재 시간을 리턴하는 함수를 아래와 같이 구현한다.

```

# 현재 시간을 리턴하는 메서드(byte 타입)
def current_date():
    current_datetime = datetime.now()

    year = current_datetime.year
    month = current_datetime.month
    day = current_datetime.day
    hour = current_datetime.hour
    minute = current_datetime.minute
    second = current_datetime.second

    result = [
        year & 0xFF,
        (year >> 8) & 0xFF,
        month & 0xFF,
        day & 0xFF,
        hour & 0xFF,
        minute & 0xFF,
        second & 0xFF
    ]

    return bytes(result)

```

Date Time Characteristic이 수신되었을 때 현재 날짜를 업로드하기 위해 get_service_and_characteristic 함수를 아래와 같이 수정한다.

```

# Service, Characteristic
date_time_characteristic = '00002a08-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 날짜정보 Update(추가된 부분)
                if characteristic.uuid == date_time_characteristic:
                    await connected_device.write_gatt_char(characteristic)

                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4]

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)

    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

```

@노건호 이 함수가 여기 처음이
라면서 본문에 설명도 추가해 주세요.

이후 코드를 실행하면 아래와 같이 올바른 날짜로 수정되어 온도정보가 수신되는것을 확인할 수 있다.

```
o sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943
TS100-092E2
connect start
connected
device is TS100-09943
2023-11-07 16:40:45
32.2
2023-11-07 16:40:46
33.3
2023-11-07 16:40:47
33.8
2023-11-07 16:40:48
34.1
2023-11-07 16:40:49
34.3
2023-11-07 16:40:50
34.4
```

온도정보 변환 확인

지금까지의 내용이 반영된 코드이다.

공간 없이 그냥 코드를 보여주면 좋겠습니다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 날짜정보 변환하기 위해 사용하는 라이브러리
from datetime import datetime

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device:
            button.pack()
            buttons.append(button)

# 윈도우 종료까지 실행

```

```

window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connection):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connection.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 날짜정보 Update(추가된 부분)
                if characteristic.uuid == date_time_characteristic:
                    await connection.write_gatt_char(characteristic)

                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connection.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환

```

```

def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)
    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

# 날짜정보 계산하는 함수
def date_calculate(array):
    year = (array[1] & 0x0F) << 8 | (array[0] & 0x00FF)
    month = array[2]
    day = array[3]
    hour = array[4]
    minute = array[5]
    second = array[6]

    date_component = datetime(year, month, day, hour, minute, second)

    return date_component

# 현재 시간을 리턴하는 메서드(byte 타입)
def current_date():

```

```

current_datetime = datetime.now()

year = current_datetime.year
month = current_datetime.month
day = current_datetime.day
hour = current_datetime.hour
minute = current_datetime.minute
second = current_datetime.second

result = [
    year & 0xFF,
    (year >> 8) & 0xFF,
    month & 0xFF,
    day & 0xFF,
    hour & 0xFF,
    minute & 0xFF,
    second & 0xFF
]

return bytes(result)

# 이벤트루프 종료를 방지하기 위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

date_time_characteristic = '00002a08-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

```

```
# 바나나 체온계와 연결되어 있다면
if connected_device is not None:
    # Service, Characteristic 출력하는 함수 호출
    loop.run_until_complete(get_service_and_characteristic(connected_

# 종료 방지함수 호출
loop.run_until_complete(wait_connect()))
```

데이터 저장

바나나 체온계로부터 수신한 온도정보를 데이터베이스를 사용하여 저장한다.

데이터베이스란 여러 사람이 공유하고 사용할 목적으로 통합 관리되는 정보의 집합이다. 몇 개의 자료 파일을 조직적으로 통합하여 자료 항목의 중복을 없애고 자료를 구조화하여 기억시켜 놓은 자료의 집합체라고 할 수 있다.



일반적으로 데이터베이스는 다음과 같은 기능을 제공한다:

데이터 저장: 데이터를 체계적으로 저장하여 나중에 쉽게 검색하고 사용할 수 있다. 예를 들어, 바나나 체온계에서 측정한 온도 데이터를 날짜와 시간별로 저장할 수 있다.

데이터 검색: 저장된 데이터 중 원하는 정보를 쉽게 찾을 수 있다. 예를 들어, 특정 날짜의 체온 데이터를 검색할 수 있다.

본 문서는 별도의 라이브러리를 설치하지 않고, 단일 파일로 저장되어 간편하게 사용할 수 있는 데이터베이스를 사용하기 위한 툴로 SQLite를 사용한다. sqlite를 import하는 코드를 아래와 같이 추가한다.

```
import sqlite3
```

이후 데이터베이스를 생성하기 위해 아래 코드를 추가한다.

레코드: 테이블의 각 행을 레코드라고 한다. 각 레코드는 하나의 데이터 항목에 대한 정보를 포함한다. 예를 들어, 특정 시간에 측정된 체온 정보가 하나의 레코드가 된다.

필드: 테이블의 각 열을 필드라고 한다. 필드는 데이터의 특정 속성을 나타낸다. 예를 들어, 체온 테이블에서 날짜, 시간, 체온은 각각 하나의 필드가 된다.

기본 키: 각 레코드를 고유하게 식별할 수 있는 필드이다. 예를 들어, 체온 데이터를 고유하게 식별하기 위해 날짜와 시간을 조합하여 기본 키로 사용할 수 있다.

데이터베이스의 장점

데이터 일관성: 데이터베이스는 데이터를 중복 없이 저장하고 관리하기 때문에 데이터의 일관성을 유지할 수 있다.

데이터 무결성: 데이터베이스는 저장된 데이터가 정확하고 신뢰할 수 있도록 다양한 제약 조건을 설정할 수 있다.

데이터 보안: 데이터베이스는 접근 권한을 설정하여 데이터를 보호할 수 있다. 예를 들어, 특정 사용자만 데이터를 조회하거나 수정할 수 있도록 제한할 수 있다.

이러한 데이터베이스의 기능과 구성 요소를 이해하면, 데이터를 효율적으로 저장하고 관리할 수 있게 된다. 초보자들도 데이터베이스를 활용하여 체계적으로 데이터를 다룰 수 있는 능력을 기를 수 있다.

이러한 장점때문에 여기서도 데이터베이스를 사용해서 데이터를 저장한다.

```

# 데이터베이스 객체 생성(temperature.db 파일 생성됨)
conn = sqlite3.connect("temperature.db")
# SQL 동작을 위해 cursor 오브젝트 생성
cursor = conn.cursor()

if __name__ == '__main__':
    # 테이블 생성(한번만 실행되도록 try-catch 이용)
    try:
        cursor.execute("CREATE TABLE TEMPERATURE(name text, temperature d
    except:
        # 이미 테이블이 실행되었다면 아래 구문 콘솔 출력
        print("database already created")
    ...

```

코드를

실행하면 좌측 상단에 temperature.db가 생성된것을 확인할 수 있다.



temperature.db 생성

이후 온도정보 notify를 수신했을 때 데이터 변환 후 데이터정보를 저장할 수 있도록 아래 코드를 변환한다.

```

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4]

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)
    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

# 데이터베이스에 저장(추가된 부분)
cursor.execute("INSERT INTO TEMPERATURE (name, temperature, date) VAL
conn.commit()

```

코드작성 후 실행하면 올바르게 데이터베이스에 온도정보가 저장된다.

저장된 결과는 다음 장인 "온도정보 확인"에서 볼 수 있다.

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 날짜정보 변환하기 위해 사용하는 라이브러리
from datetime import datetime

import sqlite3

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device:
        button.pack()
        buttons.append(button)

```

```

# 윈도우 종료까지 실행
window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connection_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connection_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 날짜정보 Update(추가된 부분)
                if characteristic.uuid == date_time_characteristic:
                    await connection_device.write_gatt_char(characteristic)

                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connection_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

```

```

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)
    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

    # 데이터베이스에 저장(추가된 부분)
    cursor.execute("INSERT INTO TEMPERATURE (name, temperature, date) VALUES (?, ?, ?)")
    conn.commit()

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

# 날짜정보 계산하는 함수
def date_calculate(array):
    year = (array[1] & 0x0F) << 8 | (array[0] & 0x00FF)
    month = array[2]
    day = array[3]
    hour = array[4]
    minute = array[5]
    second = array[6]

```

```

        date_component = datetime(year, month, day, hour, minute, second)

        return date_component

# 현재 시간을 리턴하는 메서드(byte 타입)
def current_date():
    current_datetime = datetime.now()

    year = current_datetime.year
    month = current_datetime.month
    day = current_datetime.day
    hour = current_datetime.hour
    minute = current_datetime.minute
    second = current_datetime.second

    result = [
        year & 0xFF,
        (year >> 8) & 0xFF,
        month & 0xFF,
        day & 0xFF,
        hour & 0xFF,
        minute & 0xFF,
        second & 0xFF
    ]

    return bytes(result)

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

date_time_characteristic = '00002a08-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

# 데이터베이스 객체 생성(temperature.db 파일 생성됨)
conn = sqlite3.connect("temperature.db")
# SQL 동작을 위해 cursor 오브젝트 생성
cursor = conn.cursor()

if __name__ == '__main__':
    # 테이블 생성(한번만 실행되도록 try-catch 이용)
    try:

```

```

        cursor.execute("CREATE TABLE TEMPERATURE(name text, temperature d
except:
    # 이미 테이블이 실행되었다면 아래 구문 콘솔 출력
    print("database already created")
# 비동기 이벤트 루프 생성
loop = asyncio.get_event_loop()
# 비동기 형태로 Scan함수 실행
scan_devices = loop.run_until_complete(scan())
# Scan된 기기 화면으로 출력
show_scan_devices(scan_devices)

connected_device = None
# 선택한 기기가 None이 아니라면
if selected_device is not None:
    # connect 요청 후 함수가 끝날때까지 기다림
    connected_device = loop.run_until_complete(connect(selected_devic

# 바나나 체온계바나나 체온계바나나 체온계바나나 체온계바나나 체온계바나나 체온계
if connected_device is not None:
    # Service, Characteristic 출력하는 함수 호출
    loop.run_until_complete(get_service_and_characteristic(connected_

# 종료 방지함수 호출
loop.run_until_complete(wait_connect()))

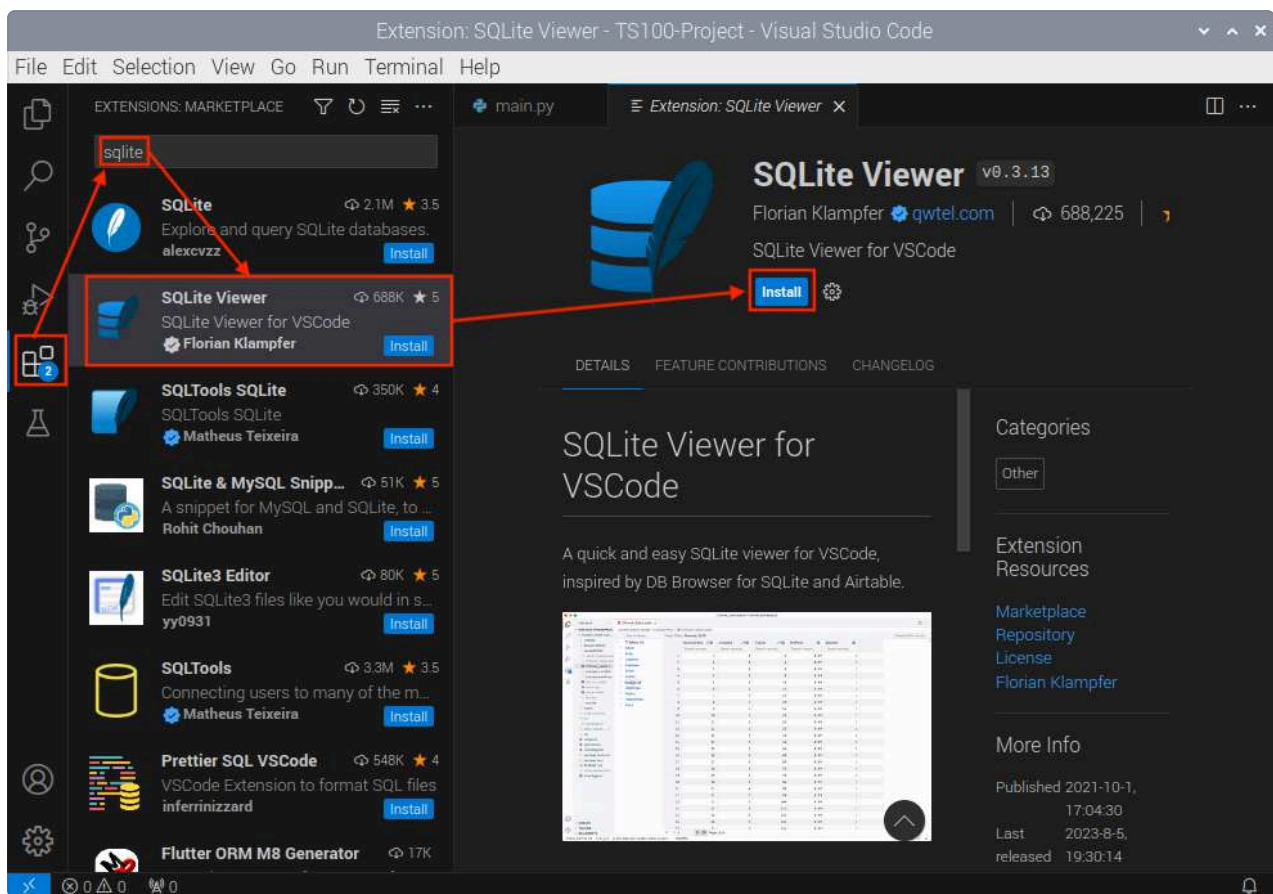
```

온도정보 확인

VSC의

온도정보 확인을 위해 ~~아래 그림과 같이~~ Extension → sqlite 검색 → SQLite Viewer를 설치한다.

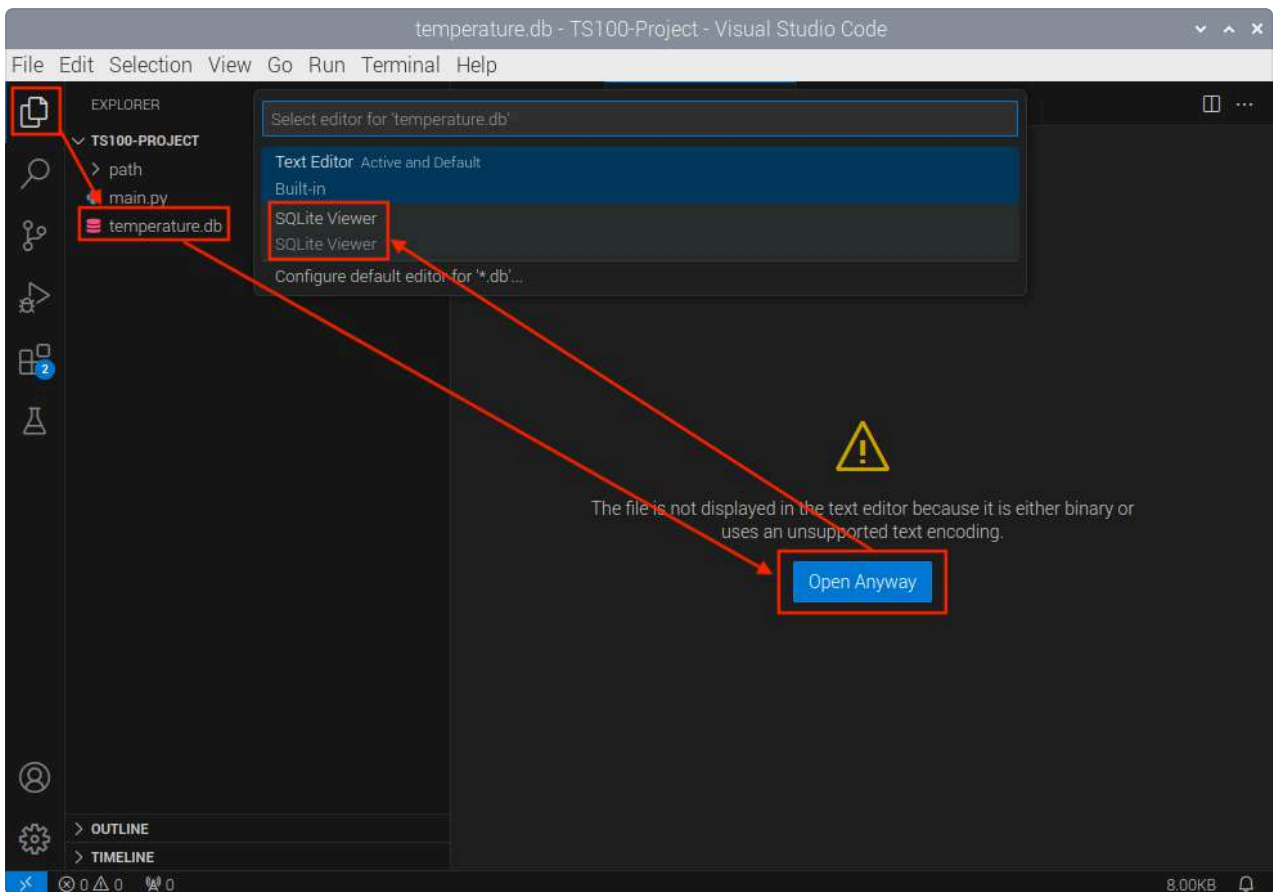
@노건호 아래 그림과 같이 보다 VSC(Visual studio code)로 언급하는 방법은 어떨
까 싶어요.



SQLite Viewer 설치방법

vsc의

설치완료 후 ~~아래 그림과 같이~~ Explorer → temperature.db 클릭 → Open Anyway 클릭 → SQLite Viewer를 순서대로 클릭한다.



데이터베이스 SQLite Viewer로 실행

temperature.db에 온도정보가 들어가있는것을 확인할 수 있다.

SQLite Viewer는 데이터가 실시간으로 업데이트되지 않으므로 데이터베이스를 새롭게 갱신하고 싶다면 좌측 상단의 Refresh 버튼을 클릭해 데이터를 갱신한다.

temperature.db - TS100-Project - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

TS100-PROJECT

- main.py
- temperature.db

temperature.db

Search table Reset Filters Records: 35 Search 35 records...

Tables (1)

TEMPERATURE

	name	date	temperature
1	TS100-142E6	2023-12-11 04:45:31	24.4
2	TS100-142E6	2023-12-11 04:45:32	24.5
3	TS100-142E6	2023-12-11 04:45:33	24.5
4	TS100-142E6	2023-12-11 04:45:34	24.5
5	TS100-142E6	2023-12-11 04:45:35	24.5
6	TS100-142E6	2023-12-11 04:45:36	24.4
7	TS100-142E6	2023-12-11 04:45:37	24.4
8	TS100-142E6	2023-12-11 04:45:38	24.4
9	TS100-142E6	2023-12-11 04:45:39	24.4
10	TS100-142E6	2023-12-11 04:45:40	24.5
11	TS100-142E6	2023-12-11 04:45:41	24.5
12	TS100-142E6	2023-12-11 04:45:42	24.5
13	TS100-142E6	2023-12-11 04:45:43	24.5
14	TS100-142E6	2023-12-11 04:45:44	24.5
15	TS100-142E6	2023-12-11 04:45:45	24.5
16	TS100-142E6	2023-12-11 04:45:46	24.5
17	TS100-142E6	2023-12-11 04:45:47	24.5
18	TS100-142E6	2023-12-11 04:45:48	24.5
19	TS100-142E6	2023-12-11 04:45:49	24.5

OUTLINE

TIMELINE

Page 1 / 1

Try SQLite Viewer Web

temperature.db 데이터 확인

데이터 그래프로 그리기

온도정보를 그래프로 그리기 위해 아래 코드를 추가해 라이브러리를 import한다.

```
import matplotlib.pyplot as plt
```

그래프에 그려질 온도정보를 담고있는 전역변수를 아래와같이 추가한다.

생성한다. ? 생성하여 초기화한다.? 뭐가 정확한 표현일까요?

```
temperature_list = []
```

그래프 설정 함수를 아래와 같이 추가한다.

```

# LineChart 변수
line_chart = None

# 그래프 설정하는 함수
def set_graph():
    global line_chart
    # Set Graph
    plt.ion() # Activate interactive mode
    fig, ax = plt.subplots()
    line = ax.plot([], [], label='Temperature', marker='o', lw=1)
    ax.set_ylim(20, 40)
    ax.set_xlim(0, 20)
    ax.set_ylabel('Temperature (°C)')
    ax.legend()

    # 전역변수에 차트정보 설정
    line_chart = line

```

✓
set_graph함수를 호출하는 코드를 main함수에 추가한다.

@노건호 함수 명과 "함수" 사이에 스페이스가 들어가도록 일괄 수정해 주세요.

```

if __name__ == '__main__':
    # Create Table(Call Once)
    try:
        cursor.execute("CREATE TABLE TEMPERATURE(name text, temperature d
    except:
        print("database already created")

    loop = asyncio.get_event_loop()

    scan_devices = loop.run_until_complete(scan())

    show_scan_devices(scan_devices)

    connected_device = None
    if selected_device is not None:
        connected_device = loop.run_until_complete(connect(selected_device
        print("device is " + selected_device.name)

    # 그래프 설정 함수(추가된 부분)
    set_graph()

    if connected_device is not None:
        # Get Service
        loop.run_until_complete(get_service_and_characteristic(connected_

    loop.run_until_complete(wait_connect())

```

parse_temperature_information 함수에 아래 그래프 그리는 부분을 추가로 작성한다.

```

# parse temperature data
def parse_temperature_information(data: bytearray):
    # get temperature information
    temperature = temperature_calculate(data[1], data[2], data[3], data[4]

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # get date information
    date = date_calculate(date_array)

    print(date)
    print(temperature)

# save database
cursor.execute("INSERT INTO TEMPERATURE (name, temperature, date) VAL
conn.commit()

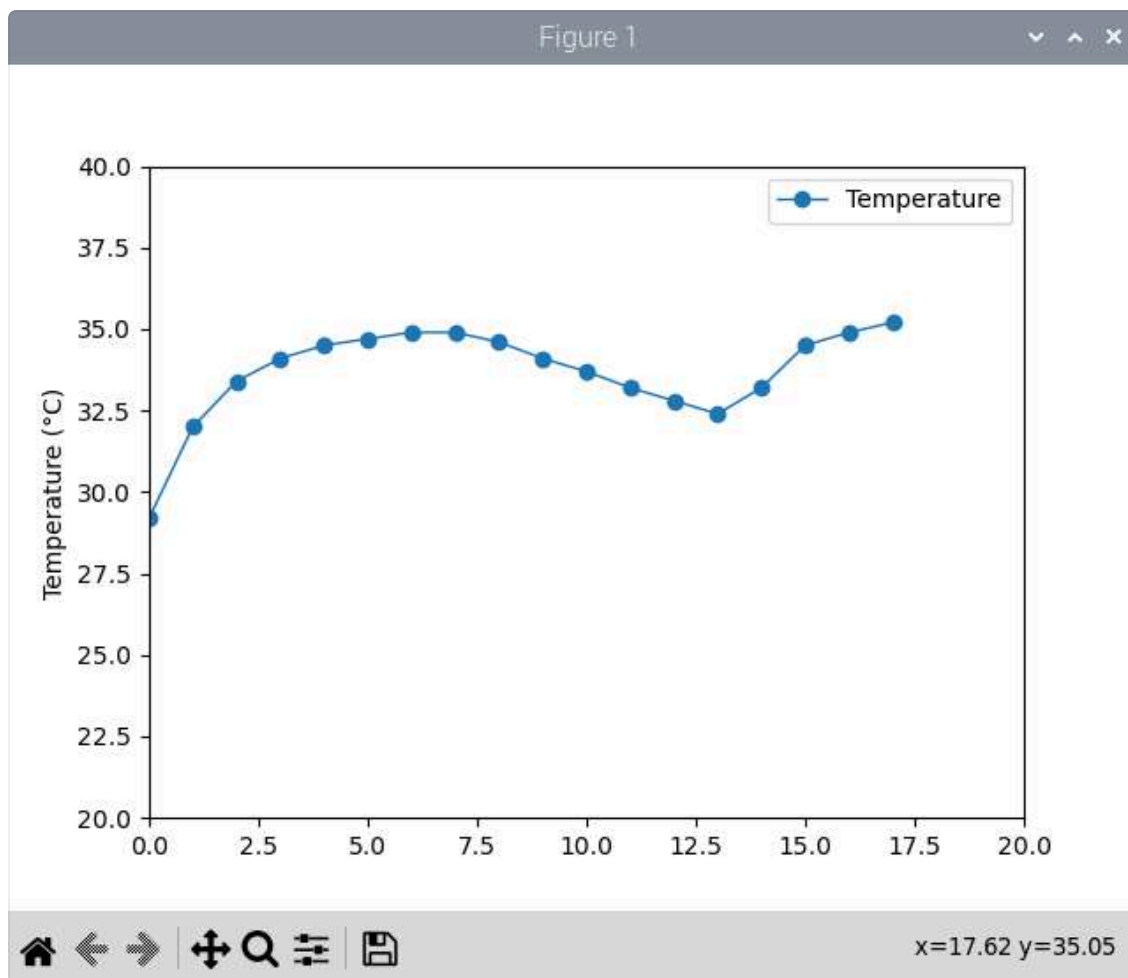
# 배열에 값 추가
temperature_list.append(temperature)

# 20개의 데이터만 graph_temperature_data 배열에 담을 수 있도록 설정
graph_temperature_data = temperature_list
if len(temperature_list) > 20:
    graph_temperature_data = temperature_list[-20:]

# 그래프 그리기
line_chart[0].set_data(range(len(graph_temperature_data)), graph_temp
plt.pause(1)

```

이후 코드를 실행하면 아래 그림과 같이 온도정보가 그래프로 나타나는것을 확인할 수 있다.



온도정보 그래프 출력

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 날짜정보 변환하기 위해 사용하는 라이브러리
from datetime import datetime

import sqlite3

# 그래프 관련 라이브러리
import matplotlib.pyplot as plt

# 비동기 형태로 BLE 장치 검색
async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 TS100을 포함하면 콘솔에 출력
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("TS100 Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)

```

```

        button.pack()
        buttons.append(button)

# 윈도우 종료까지 실행
window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 바나나 체온계에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connection_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connection_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 날짜정보 Update(추가된 부분)
                if characteristic.uuid == date_time_characteristic:
                    await connection_device.write_gatt_char(characteristic)

                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connection_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수

```

```

def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)
    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

    # 데이터베이스에 저장 (추가된 부분)
    cursor.execute("INSERT INTO TEMPERATURE (name, temperature, date) VALUES (?, ?, ?)")
    conn.commit()

    # 배열에 값 추가
    temperature_list.append(temperature)

    # 20개의 데이터만 graph_temperature_data 배열에 담을 수 있도록 설정
    graph_temperature_data = temperature_list
    if len(temperature_list) > 20:
        graph_temperature_data = temperature_list[-20:]

    # 그래프 그리기
    line_chart[0].set_data(range(len(graph_temperature_data)), graph_temperature_data)
    plt.pause(1)

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도

```

```

    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

# 날짜정보 계산하는 함수
def date_calculate(array):
    year = (array[1] & 0x0F) << 8 | (array[0] & 0x00FF)
    month = array[2]
    day = array[3]
    hour = array[4]
    minute = array[5]
    second = array[6]

    date_component = datetime(year, month, day, hour, minute, second)

    return date_component

# 현재 시간을 리턴하는 메서드(byte 타입)
def current_date():
    current_datetime = datetime.now()

    year = current_datetime.year
    month = current_datetime.month
    day = current_datetime.day
    hour = current_datetime.hour
    minute = current_datetime.minute
    second = current_datetime.second

    result = [
        year & 0xFF,
        (year >> 8) & 0xFF,
        month & 0xFF,
        day & 0xFF,
        hour & 0xFF,
        minute & 0xFF,
        second & 0xFF
    ]

    return bytes(result)

# 그래프 설정하는 함수
def set_graph():
    global line_chart
    # Set Graph
    plt.ion() # Activate interactive mode
    fig, ax = plt.subplots()

```

```

line = ax.plot([], [], label='Temperature', marker='o', lw=1)
ax.set_ylim(20, 40)
ax.set_xlim(0, 20)
ax.set_ylabel('Temperature (°C)')
ax.legend()

# 전역변수에 차트정보 설정
line_chart = line

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

date_time_characteristic = '00002a08-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

temperature_list = []

line_chart = None

# 데이터베이스 객체 생성(temperature.db 파일 생성됨)
conn = sqlite3.connect("temperature.db")
# SQL 동작을 위해 cursor 오브젝트 생성
cursor = conn.cursor()

connected_device = None

if __name__ == '__main__':
    # 테이블 생성(한번만 실행되도록 try-catch 이용)
    try:
        cursor.execute("CREATE TABLE TEMPERATURE(name text, temperature d
    except:
        # 이미 테이블이 실행되었다면 아래 구문 콘솔 출력
        print("database already created")
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    # 선택한 기기가 None이 아니라면
    .
    .
    .

```

```
if selected_device is not None:
    # connect 요청 후 함수가 끝날때까지 기다림
    connected_device = loop.run_until_complete(connect(selected_device))

# 그래프 설정 함수
set_graph()

# 바나나 체온계 연결되어 있다면
if connected_device is not None:
    # Service, Characteristic 출력하는 함수 호출
    loop.run_until_complete(get_service_and_characteristic(connected_device))

# 종료 방지함수 호출
loop.run_until_complete(wait_connect())
```

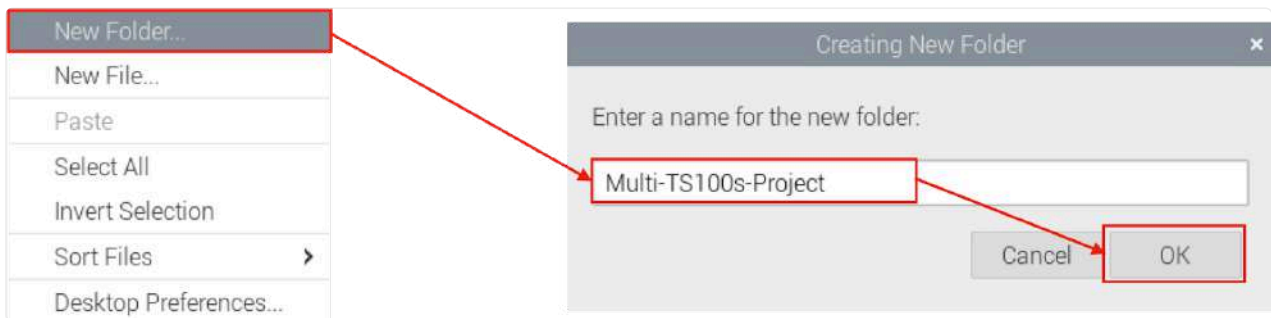
지금까지 1대의 장치로 부터 정보를 얻는 것을 살펴보았다면, 이제 보다 일반적인 목적을 위해서 여러 대의 장치에서 동시에 정보를 받는 방법을 알아보고자 한다. 다만 여기서는 새로운 디바이스를 동일한 바나나 체온계를 사용하여 보여준다.

4.1.4. 2대이상 기기 연결

본 목차는 2대 이상의 바나나 체온계에 연결하여 온도정보를 수신하는 방법에 대해 서술한다. 기존에 사용했던 코드를 활용해 진행한다.

프로젝트 생성

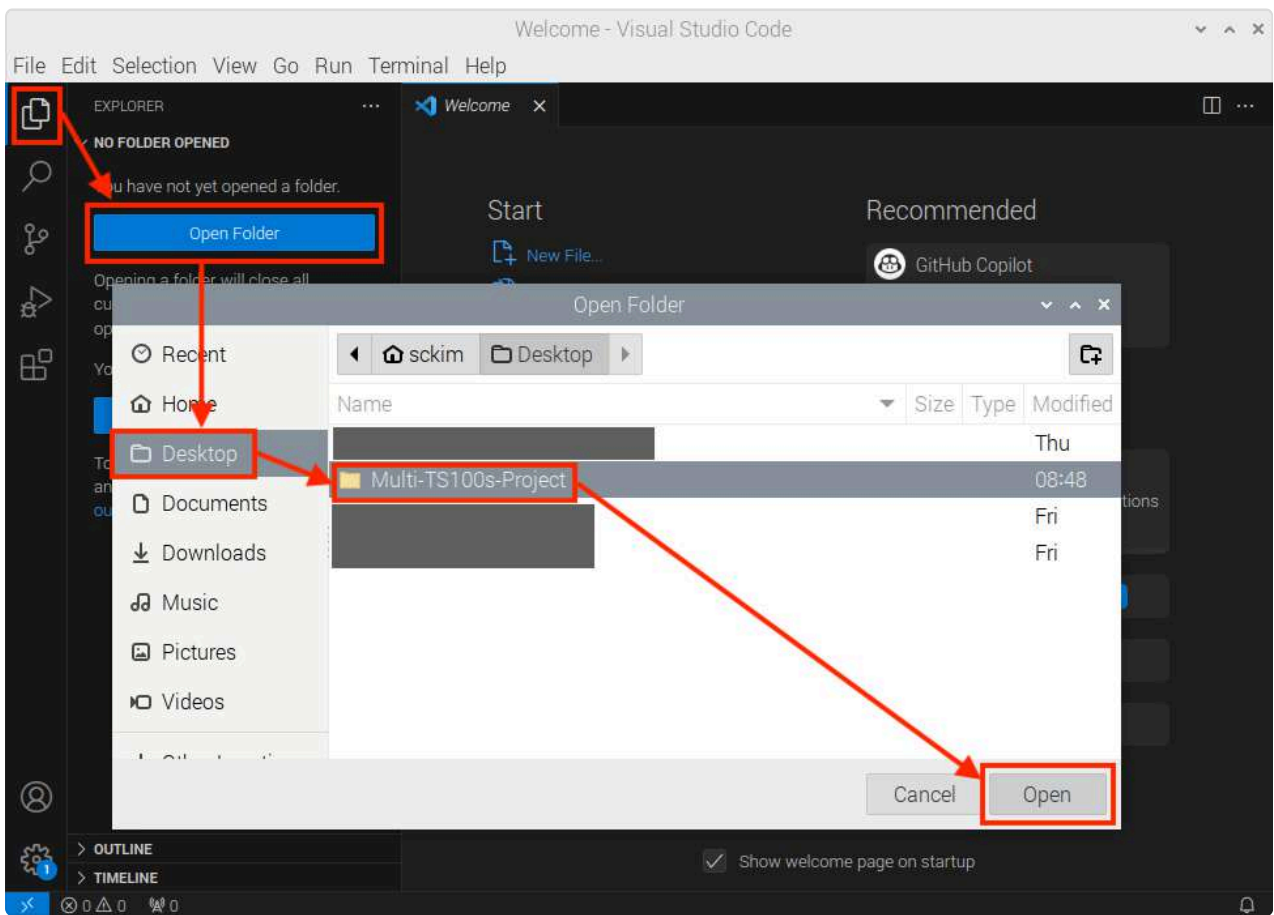
새로운 프로젝트에서 사용될 파일을 생성하기 위해 홈화면(라즈베리파이의 바탕 화면)에서 우클릭 → New Folder 버튼을 클릭해 폴더를 생성한다. 본 문서는 Multi-TS100s-Project 이름을 가진 폴더를 생성한다.



Raspberry Pi 프로젝트 폴더 생성

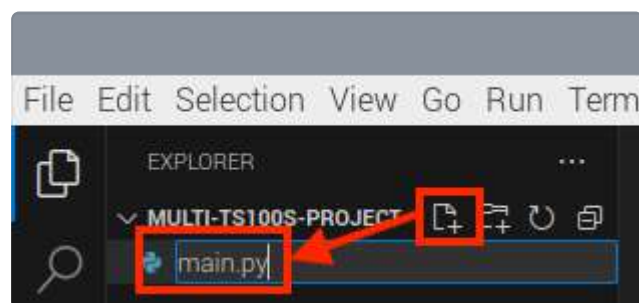
@노건호 앞에서도 말한 것 처럼 "다름 그림" 대신에 이런 표현이 좋아보입니다.

이후 Visual Studio Code를 열고 Open Folder 버튼을 클릭해 기존에 만들었던 TS100-Project 폴더 선택 후 Open 버튼을 클릭해 파일을 연다.
폴더?



Visual Studio Code 폴더 열기

이후 아래 그림 순서와 같이 main.py 파일을 생성한다.



main.py 파일 생성 방법

화면 레이아웃 설정

2대 이상 스캔된 화면을 그리기 위해 아래와 같이 코드를 작성한다.

```

import tkinter as tk
from tkinter import *

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, textvariable=scan_list)
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, textvariable=temperature_info)
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

    # Scan 버튼
    scan_button = Button(root, text="Scan", command=scan_button_click)
    scan_button.pack()

```

```

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

# Scan 버튼클릭 시 호출되는 함수
def scan_button_click(scan_frame):
    print("Scan Button Click")

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

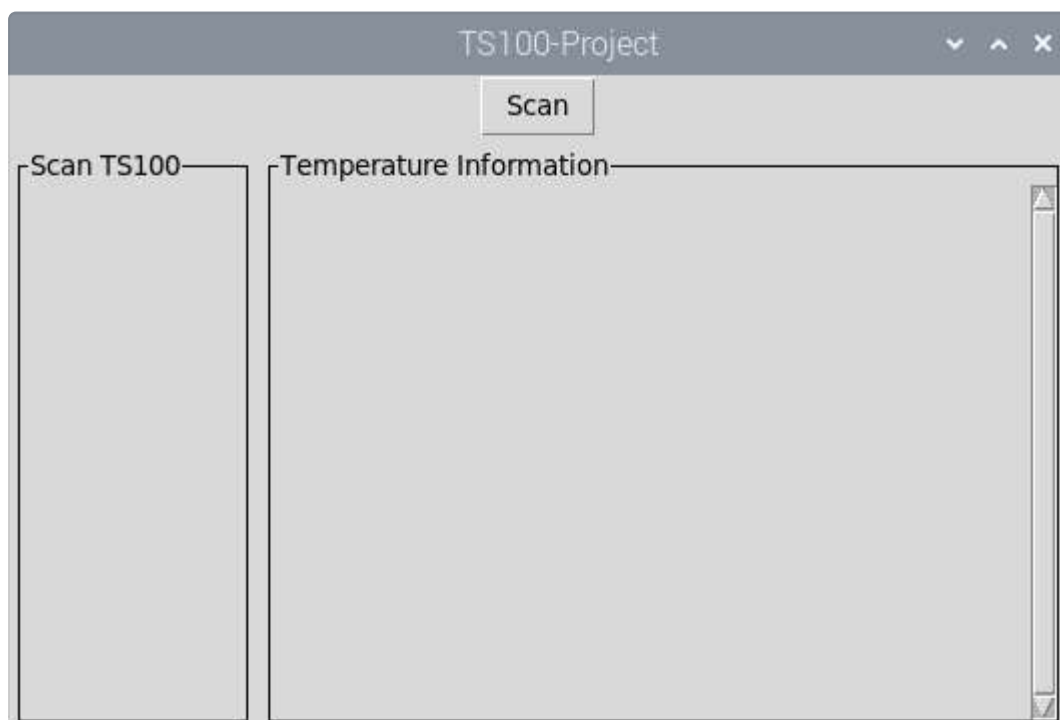
# 이벤트루프 생성
loop = asyncio.get_event_loop()

# 비동기로 UI 계속 업데이트
def asyncio_loop():
    loop.call_soon(loop.stop)
    loop.run_forever()
    root.after(100, asyncio_loop)

root.after(100, asyncio_loop)
root.mainloop()

```

이후 코드를 실행하면 아래 그림과 같이 화면이 나타나고, 상단의 Scan 버튼을 클릭하면 Scan Button Click이 콘솔에 출력되는것을 확인할 수 있다.



```
o sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Pro  
ject/main.py  
Scan Button Click
```

버튼클릭 콘솔 출력

블루투스 기기 Scan

2대의 기기를 연결하기 위해서는 `scan_button_click()`, Scan 관련 함수를 아래와 같이 수정한다.
비동기 처리를 위해 `asyncio`을 import한다.

@노건호 앞의 코드에서 수정한 코드 부분만 표시하는 방법이 있으면 수정된 부분을 알려주면 좋을 듯해요. 쉬운 방법이 있을까요?

```

import asyncio

from bleak import BleakScanner

...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button():
    # 기존에 scan했던 기기 화면에서 제거
    for button in scan_devices_list:
        root.after(0, button.destroy)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda devi
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

    # 이벤트루프 생성해 BLE 기기 scan
    loop = asyncio.get_event_loop()
    ts100_scan_list = loop.run_until_complete(scan())

    create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

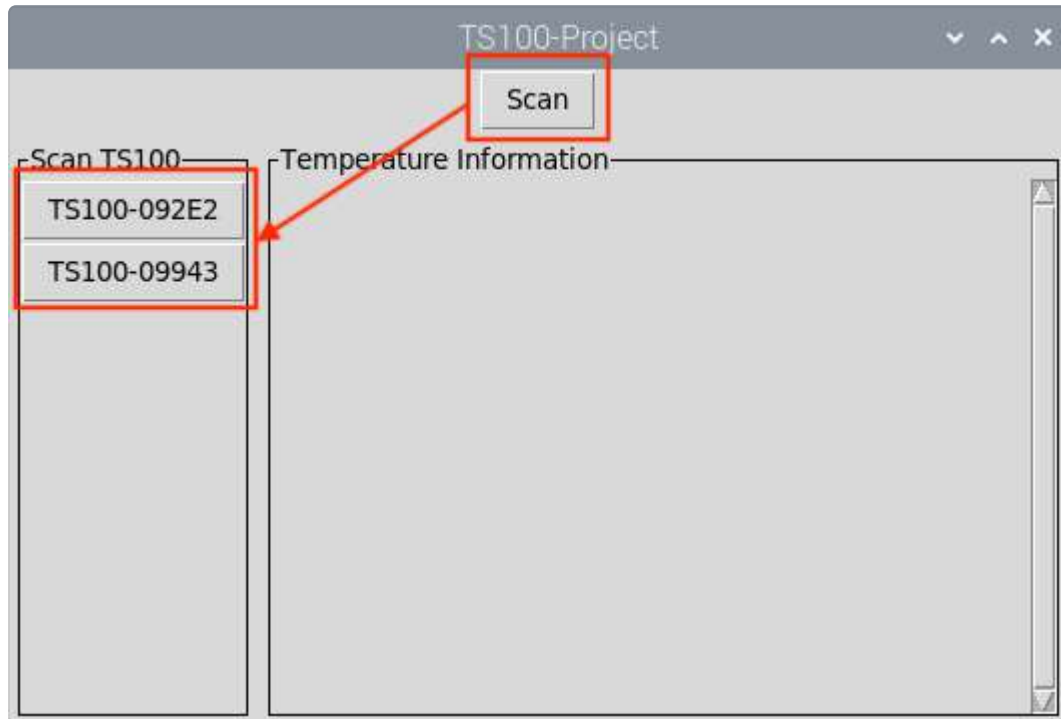
    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

```

이후 코드를 실행하면 아래와 같은 결과를 확인할 수 있다. Scan 버튼을 계속 클릭하면 Scan된 기기가 업데이트되는것을 확인할 수 있다.



바나나 체온계 Scan

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text=
scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid"
canvas = Canvas(temperature_information_frame, borderwidth=0)
scrollbar = Scrollbar(temperature_information_frame, orient="vertical",
scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sti
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

'''
    Scan
'''

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button():
    # 기존에 scan했던 기기 화면에서 제거
    for button in scan_devices_list:
        root.after(0, button.destroy)
        scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda device:
            button.pack()
            scan_devices_list.append(button)

def scan_button_click():
    remove_button()

    # 이벤트루프 생성해 BLE 기기 scan
    loop = asyncio.get_event_loop()
    ts100_scan_list = loop.run_until_complete(scan())

    create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"

```

```

        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

    # 이벤트루프 생성
    loop = asyncio.get_event_loop()

    # 비동기로 UI 계속 업데이트
    def asyncio_loop():
        loop.call_soon(loop.stop)
        loop.run_forever()
        root.after(100, asyncio_loop)

    root.after(100, asyncio_loop)
    root.mainloop()

```

블루투스 기기 연결

Scan된 Device를 연결하는 connect 함수를 아래와 같이 작성한다. 버튼을 클릭했을 때 connect 함수를 호출할 수 있도록 scan_device_button_click 함수도 아래와 같이 수정한다.

```
def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

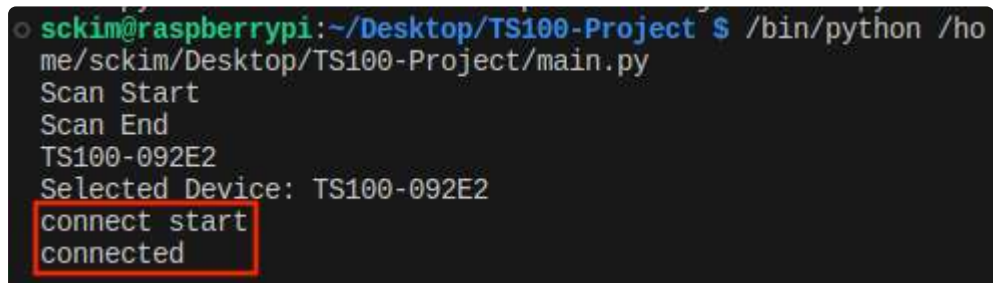
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None
```

코드수정 후 실행하면 아래와 같이 콘솔에 출력되는것을 확인할 수 있다.



```
sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-092E2
Selected Device: TS100-092E2
connect start
connected
```

Device 연결 콘솔 출력

연결된 기기를 Temperature Information 영역에 나타내는 코드를 아래와 같이 작성한다.

연결된 device를 구분하기 위해 device_info_frame_dict 딕셔너리 변수를 추가한다. dictionary란 키와 값의 쌍을 저장하는 자료구조로, 키를 통해 값을 빠르게 검색할 수 있다.

device_info_frame_dict 에는 바나나 체온계바나나 체온계바나나 체온계바나나 체온계바나나 이름이 key, 화면에 나타나는 Text Frame이 value로 이루어진 데이터를 저장한다.

@노건호 딕셔너리에 대한 설명이 없었다면 여기에 딕셔너리에 대한 설명을 간단하게나마 추가해 주세요.

```

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # 연결된 기기 화면에 생성
        root.after(0, create_device_info_frame, candidate_device, selecte

    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

# dictionary 선언(key: device_name, value: Text Frame)
device_info_frame_dict = {}

# 연결된 기기 Frame을 생성하는 함수
def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict

    # Device 영역 제거
    def remove_frame():
        print("click")

    frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
    frame.pack()

    # TitleFrame: 좌측 상단에 제목이 표시되는 프레임
    title_frame = Frame(frame, padx=5, pady=5)
    title_label = Label(title_frame, text=device.name, font=("Helvetica",
    title_label.pack(side="left")

    # NotifyStartFrame: 우측 상단에 X 버튼이 표시되는 프레임
    # NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
    remove_button_frame = Frame(frame, padx=5, pady=5)
    remove_button = Button(remove_button_frame, text="X", command=remove_
    remove_button.pack()

    # ContentFrame: 하단에 내용을 보여주는 프레임
    content_frame = Frame(frame, padx=10, pady=10)
    content_label_text = StringVar()
    content_label = Label(content_frame, textvariable=content_label_text.

```

```

content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
notify_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

# TitleFrame: 좌측 상단에 제목이 표시되는 프레임
title_frame = Frame(frame, padx=5, pady=5)
title_label = Label(title_frame, text=device.name, font=("Helvetica",
title_label.pack(side="left")

# NotifyStartFrame: 우측 상단에 X 버튼이 표시되는 프레임
# NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
remove_button_frame = Frame(frame, padx=5, pady=5)
remove_button = Button(remove_button_frame, text="X", command=remove_
remove_button.pack()

# ContentFrame: 하단에 내용을 보여주는 프레임
content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connected!")
content_label.pack(side="bottom")

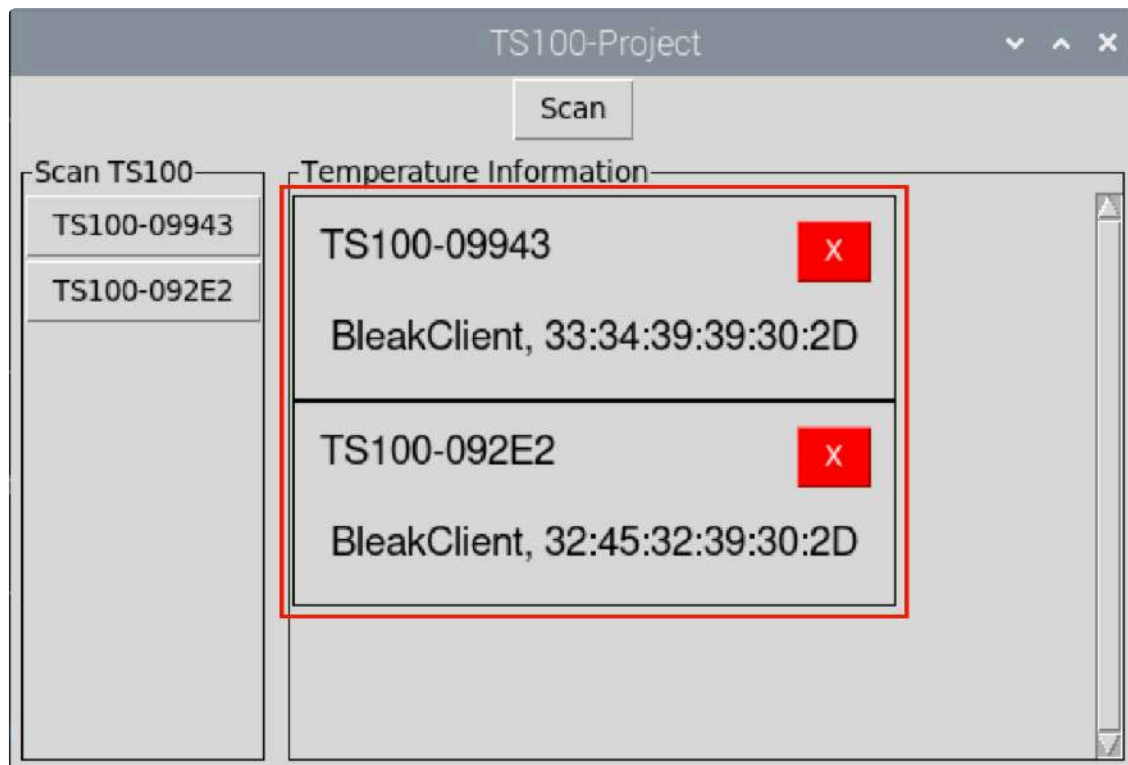
# 배치
title_frame.grid(row=0, column=0, sticky="ew")
notify_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

```

코드수정 후 실행하면 아래 그림과 같이 연결된 기기가 Temperature Information 화면에 출력되는 것을 확인할 수 있다.



연결 기기 Frame 추가

Device 연결에 성공하면 기존에 좌측화면에서 클릭했던 기기를 삭제할 수 있도록 `remove_button_click` 함수를 아래와 같이 수정한다.

```

# 연결된 기기는 버튼 리스트에서 삭제할 수 있도록 수정
def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)
...

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

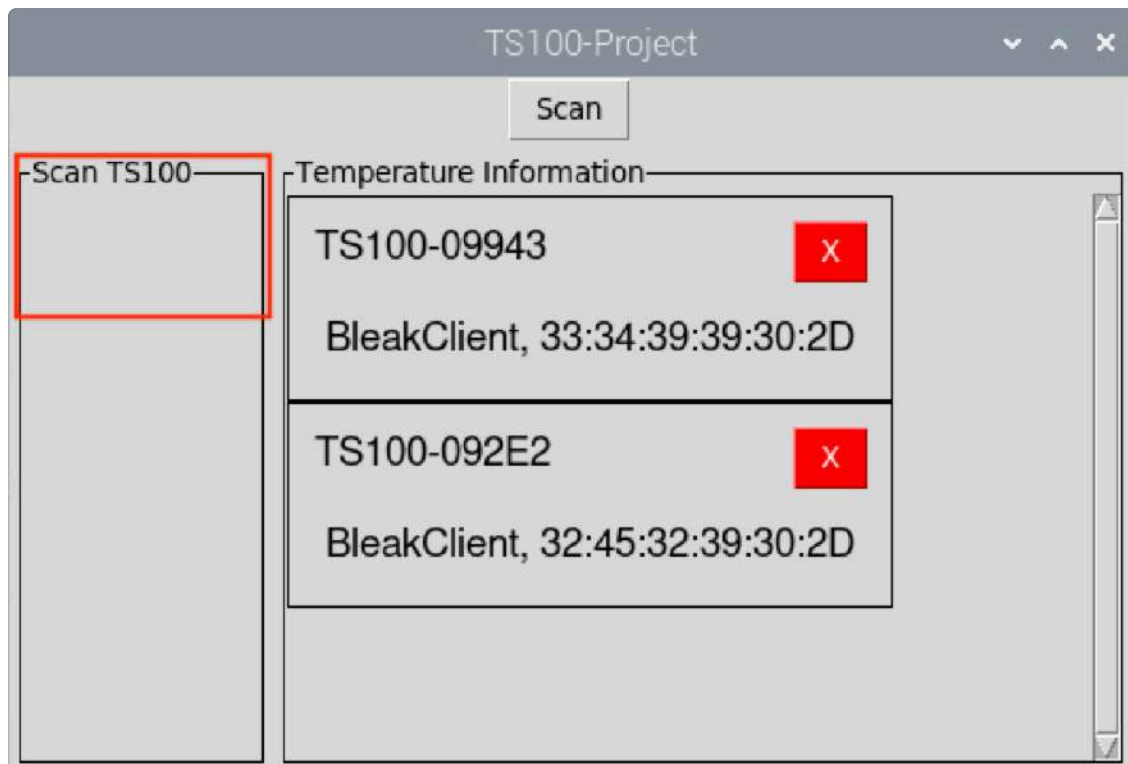
    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button_click, candidate_device.name)
        # 연결된 기기 화면에 생성
        root.after(0, create_device_info_frame, candidate_device, selecte

    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

```

코드수정 후 실행하면 아래 그림과 같이 연결한 Device는 Scan TS100 영역에서 사라진것을 확인할 수 있다.



연결된 기기 Scan TS100 List 삭제

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner, BleakClient

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Scan List")
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Temperature Information")
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

...
Scan
...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda devi
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

# 이벤트루프 생성해 BLE 기기 scan

```

```

loop = asyncio.get_event_loop()
ts100_scan_list = loop.run_until_complete(scan())

create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

...
    Connect
...

# dictionary 선언(key: device_name, value: device_text_frame)
device_info_frame_dict = {}

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button, candidate_device.name)
        # info 화면에 추가

```

```

        root.after(0, create_device_info_frame, candidate_device, selecte

except Exception as e:
    # 연결 실패시 발생
    print('error: ', e, end='')
    return None

def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

    # value값으로 dictionary 값 삭제하는 함수
    def remove_characteristic_by_value(name):
        keys_to_remove = [key for key, value in sender_dict.items() if va
        for key in keys_to_remove:
            del sender_dict[key]

    frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
    frame.pack()

    # TitleFrame: 좌측 상단에 제목이 표시되는 프레임
    title_frame = Frame(frame, padx=5, pady=5)
    title_label = Label(title_frame, text=device.name, font=("Helvetica",
    title_label.pack(side="left")

    # NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
    remove_button_frame = Frame(frame, padx=5, pady=5)
    remove_button = Button(remove_button_frame, text="X", command=remove_
    remove_button.pack()

    # ContentFrame: 하단에 내용을 보여주는 프레임
    content_frame = Frame(frame, padx=10, pady=10)
    content_label_text = StringVar()
    content_label = Label(content_frame, textvariable=content_label_text,

```

```

content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

    # 이벤트루프 생성
    loop = asyncio.get_event_loop()

    # 비동기로 UI 계속 업데이트
    def asyncio_loop():
        loop.call_soon(loop.stop)
        loop.run_forever()
        root.after(100, asyncio_loop)

    root.after(100, asyncio_loop)
    root.mainloop()

```

Service, Characteristic 요청

블루투스 연결이 완료된 후 Service, Characteristic 요청을 위해 아래 함수를 추가한다.

@노건호 1대가 아닌 2대이기에 앞의 것과 수정된 부분이 있으면 여기에 설명을 달아두면 좋겠어요.

```

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인
    for service in connected_device.services:
        print(service)
    # Characteristic 확인
    for characteristic in service.characteristics:
        print(characteristic)

    print()

```

connect 함수에서 아래 코드를 추가한 후 코드를 실행하면 전체 Service와 Characteristic을 확인할 수 있다.

```

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # Characteristic, Service 요청
        await get_service_and_characteristic(candidate_device, selected_c

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button_click, candidate_device.name)

        # 연결된 기기 화면에 생성
        root.after(0, create_device_info_frame, candidate_device, selecte

    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

```

```
00001801-0000-1000-8000-00805f9b34fb (Handle: 12): Generic Attribute Profile
00002a05-0000-1000-8000-00805f9b34fb (Handle: 13): Service Changed

00001809-0000-1000-8000-00805f9b34fb (Handle: 47): Health Thermometer
0000fbfb-0000-1000-8000-00805f9b34fb (Handle: 48): Vendor specific
00002a08-0000-1000-8000-00805f9b34fb (Handle: 70): Date Time
0000f9f9-0000-1000-8000-00805f9b34fb (Handle: 62): Vendor specific
0000fdfd-0000-1000-8000-00805f9b34fb (Handle: 66): Vendor specific
0000fafa-0000-1000-8000-00805f9b34fb (Handle: 51): Vendor specific
0000fcfc-0000-1000-8000-00805f9b34fb (Handle: 58): Vendor specific
00002a1c-0000-1000-8000-00805f9b34fb (Handle: 54): Temperature Measurement

0000180a-0000-1000-8000-00805f9b34fb (Handle: 20): Device Information
00002a50-0000-1000-8000-00805f9b34fb (Handle: 31): PnP ID
00002a24-0000-1000-8000-00805f9b34fb (Handle: 23): Model Number String
00002a29-0000-1000-8000-00805f9b34fb (Handle: 21): Manufacturer Name String
00002a27-0000-1000-8000-00805f9b34fb (Handle: 25): Hardware Revision String
00002a23-0000-1000-8000-00805f9b34fb (Handle: 29): System ID
00002a26-0000-1000-8000-00805f9b34fb (Handle: 27): Firmware Revision String

0000fef5-0000-1000-8000-00805f9b34fb (Handle: 33): Dialog Semiconductor GmbH
724249f0-5ec3-4b5f-8804-42345af08651 (Handle: 36): Unknown
9d84b9a3-000c-49d8-9183-855b673fda31 (Handle: 40): Unknown
5f78df94-798c-46f5-990a-b3eb6a065c88 (Handle: 44): Unknown
8082caa8-41a6-4021-91c6-56f9b954cc34 (Handle: 34): Unknown
457871e8-d516-4ca1-9116-57d0b17b9cb2 (Handle: 42): Unknown
6c53db25-47a1-45fe-a022-7c92fb334fd4 (Handle: 38): Unknown

0000180f-0000-1000-8000-00805f9b34fb (Handle: 16): Battery Service
00002a19-0000-1000-8000-00805f9b34fb (Handle: 17): Battery Level
```

바나나 체온계 전체 Service, Characteristic

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner, BleakClient

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Scan List")
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Temperature Information")
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

...
Scan
...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda devi
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

# 이벤트루프 생성해 BLE 기기 scan

```

```

loop = asyncio.get_event_loop()
ts100_scan_list = loop.run_until_complete(scan())

create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

...
    Connect
...

# dictionary 선언(key: device_name, value: device_text_frame)
device_info_frame_dict = {}

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # Characteristic, Service 요청
        await get_service_and_characteristic(candidate_device, selected_c

```

```

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button, candidate_device.name)
        # info 화면에 추가
        root.after(0, create_device_info_frame, candidate_device, selecte

except Exception as e:
    # 연결 실패시 발생
    print('error: ', e, end='')
    return None

def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

    # value값으로 dictionary 값 삭제하는 함수
    def remove_characteristic_by_value(name):
        keys_to_remove = [key for key, value in sender_dict.items() if va
        for key in keys_to_remove:
            del sender_dict[key]

    frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
    frame.pack()

    # TitleFrame: 좌측 상단에 제목이 표시되는 프레임
    title_frame = Frame(frame, padx=5, pady=5)
    title_label = Label(title_frame, text=device.name, font=("Helvetica",
    title_label.pack(side="left")

    # NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
    remove_button_frame = Frame(frame, padx=5, pady=5)
    remove_button = Button(remove_button_frame, text="X", command=remove_
    remove_button.pack()

    # ContentFrame: 하단에 내용을 보여주는 프레임

```

```

content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

...
    notify
...

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인
    for service in connected_device.services:
        print(service)
        # Characteristic 확인
        for characteristic in service.characteristics:
            print(characteristic)

        print()

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

    # 이벤트루프 생성
    loop = asyncio.get_event_loop()

    # 비동기로 UI 계속 업데이트
    def asyncio_loop():
        loop.call_soon(loop.stop)
        loop.run_forever()
        root.after(100, asyncio_loop)

    root.after(100, asyncio_loop)

```

```
root.mainloop()
```

온도정보 가져오기

바나나 체온계의 여러 characteristic 중 온도정보를 받아오는 기능을 구현한다. 온도 정보는 아래 특성에서 확인할 수 있다.

```
Service: Health Thermometer(00001809-0000-1000-8000-00805f9b34fb)
Characteristic: Temperature Measurement(00002a1c-0000-1000-8000-00805f9b34fb)
```

온도정보 수신

여러 Service 중 Health Thermometer와 관련된 service만 수신받기 위해 `get_service_and_characteristic` 함수를 아래와 같이 수정한다.

```
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(candidate_device, connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                print(characteristic)

    print()
```

이후 온도정보 characteristic을 통해 notify를 수신하기 위해 함수를 아래와 같이 수정한다.

```

health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(candidate_device, connected_device):
    # Service 확인
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

```

2대이상의 기기에서 수신된 온도정보를 구분하기 위해 sender_dict라는 dictionary 자료형을 추가하고, get_service_and_characteristic을 아래와 같이 수정한다. sender_dict에는 characteristic이 key, 바나나 체온계 이름이 value로 이루어진 데이터를 저장한다.

```

health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

# dictionary 선언(key: characteristic, value: device_name)
sender_dict = {}

async def get_service_and_characteristic(candidate_device, connected_device):
    # Service 확인
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    # Dictionary에 추가
                    if characteristic not in sender_dict:
                        sender_dict[characteristic] = candidate_device.name
                    await connected_device.start_notify(characteristic, notify_callback)

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

```

이후 코드실행 후 바나나 체온계와 연결하면 수신된 온도정보가 콘솔창에 출력되는것을 확인할 수 있다.

```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943
TS100-092E2
Selected Device: TS100-09943
connect start
connected
get service and characteristic
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x04;\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x00\x01')
bytearray(b'\x06\xdb\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x01\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x02\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x03\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x04\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x05\x01')
bytearray(b'\x06\xdb\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x06\x01')
bytearray(b'\x06\xdb\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x07\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x08\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\t\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\n\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x0b\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x0c\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\r\x01')
bytearray(b'\x06\xda\t\x00\xfe\xe7\x07\x0b\x14\x10\x05\x0e\x01')

```

온도정보 notify 수신 확인

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner, BleakClient

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="")
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="")
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

...
Scan
...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda devi
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

# 이벤트루프 생성해 BLE 기기 scan

```

```

loop = asyncio.get_event_loop()
ts100_scan_list = loop.run_until_complete(scan())

create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

...
Connect
...

# dictionary 선언(key: device_name, value: device_text_frame)
device_info_frame_dict = {}

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # Characteristic, Service 요청
        await get_service_and_characteristic(candidate_device, selected_c

```

```

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button, candidate_device.name)
        # info 화면에 추가
        root.after(0, create_device_info_frame, candidate_device, selecte

except Exception as e:
    # 연결 실패시 발생
    print('error: ', e, end='')
    return None

def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

    # value값으로 dictionary 값 삭제하는 함수
    def remove_characteristic_by_value(name):
        keys_to_remove = [key for key, value in sender_dict.items() if va
        for key in keys_to_remove:
            del sender_dict[key]

    frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
    frame.pack()

    # TitleFrame: 좌측 상단에 제목이 표시되는 프레임
    title_frame = Frame(frame, padx=5, pady=5)
    title_label = Label(title_frame, text=device.name, font=("Helvetica",
    title_label.pack(side="left")

    # NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
    remove_button_frame = Frame(frame, padx=5, pady=5)
    remove_button = Button(remove_button_frame, text="X", command=remove_
    remove_button.pack()

    # ContentFrame: 하단에 내용을 보여주는 프레임

```

```

content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

...
    notify
...

health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

# dictionary 선언(key: characteristic, value: device_name)
sender_dict = {}

# Service, Characteristic 요청하는 함수
async def get_service_and_characteristic(candidate_device, connected_devi
    # Service 확인
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    # Dictionary에 추가
                    if characteristic not in sender_dict:
                        sender_dict[characteristic] = candidate_device.na
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

if __name__ == "__main__":
    . . .

```

```

root = tk.Tk()

create_ui(root)

# 이벤트루프 생성
loop = asyncio.get_event_loop()

# 비동기로 UI 계속 업데이트
def asyncio_loop():
    loop.call_soon(loop.stop)
    loop.run_forever()
    root.after(100, asyncio_loop)

root.after(100, asyncio_loop)
root.mainloop()

```

온도정보 변환

바나나 체온계로부터 수신받은 데이터는 온도정보와 날짜정보로 이루어져 있다. 온도정보를 변환하는 함수를 아래와 같이 작성한다.

[@노건호](#) 앞에서 자세히 설명하였다면 중복하지 말고 그 부분을 살펴보라고 말하는게 좋을 듯해요.

```

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

```

기존에 작성했던 notify_callback 함수를 아래와 같이 온도정보를 받아와 출력하는 함수로 변경한다.

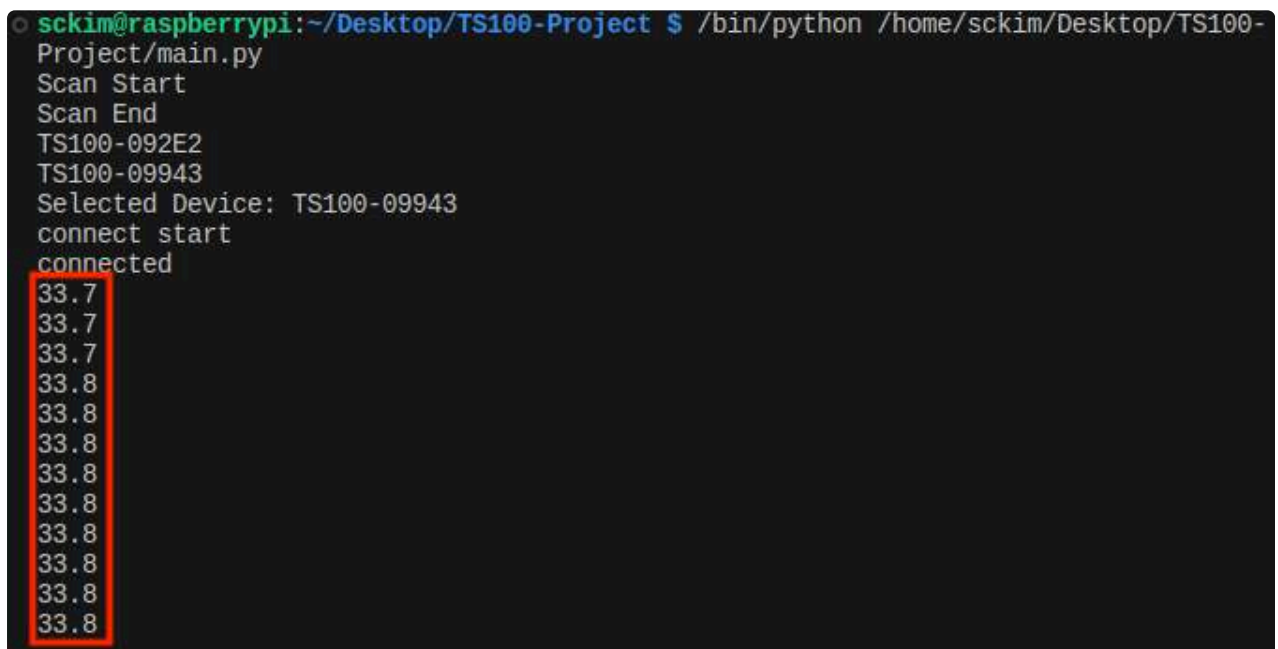
```
# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    parse_temperature_information(data)

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    # 온도정보 출력
    print(temperature)

    return temperature
```

이후 프로젝트를 실행하면 아래 그림과 같이 변환된 온도정보가 출력되는것을 확인할 수 있다.



A terminal window on a Raspberry Pi showing the execution of a Python script. The output includes device identification and connection status, followed by a series of temperature readings. The first two readings are 33.7, and the subsequent eight are 33.8. These temperature values are highlighted with a red rectangular box.

```
sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-092E2
TS100-09943
Selected Device: TS100-09943
connect start
connected
33.7
33.7
33.7
33.8
33.8
33.8
33.8
33.8
33.8
33.8
33.8
33.8
33.8
```

온도정보 수신 확인

온도정보 화면에 출력

바나나 체온계로부터 수신받은 온도정보를 화면에 출력하기 위해 notify_callback 함수를 아래와 같이 수정한다.

```

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    temperature = parse_temperature_information(data)

    # 수신된 기기 이름 확인
    device_name = sender_dict.get(handle)

    if device_name is None:
        return

    # 업데이트할 텍스트 영역 확인 후 업데이트
    string_var = device_info_frame_dict.get(device_name)

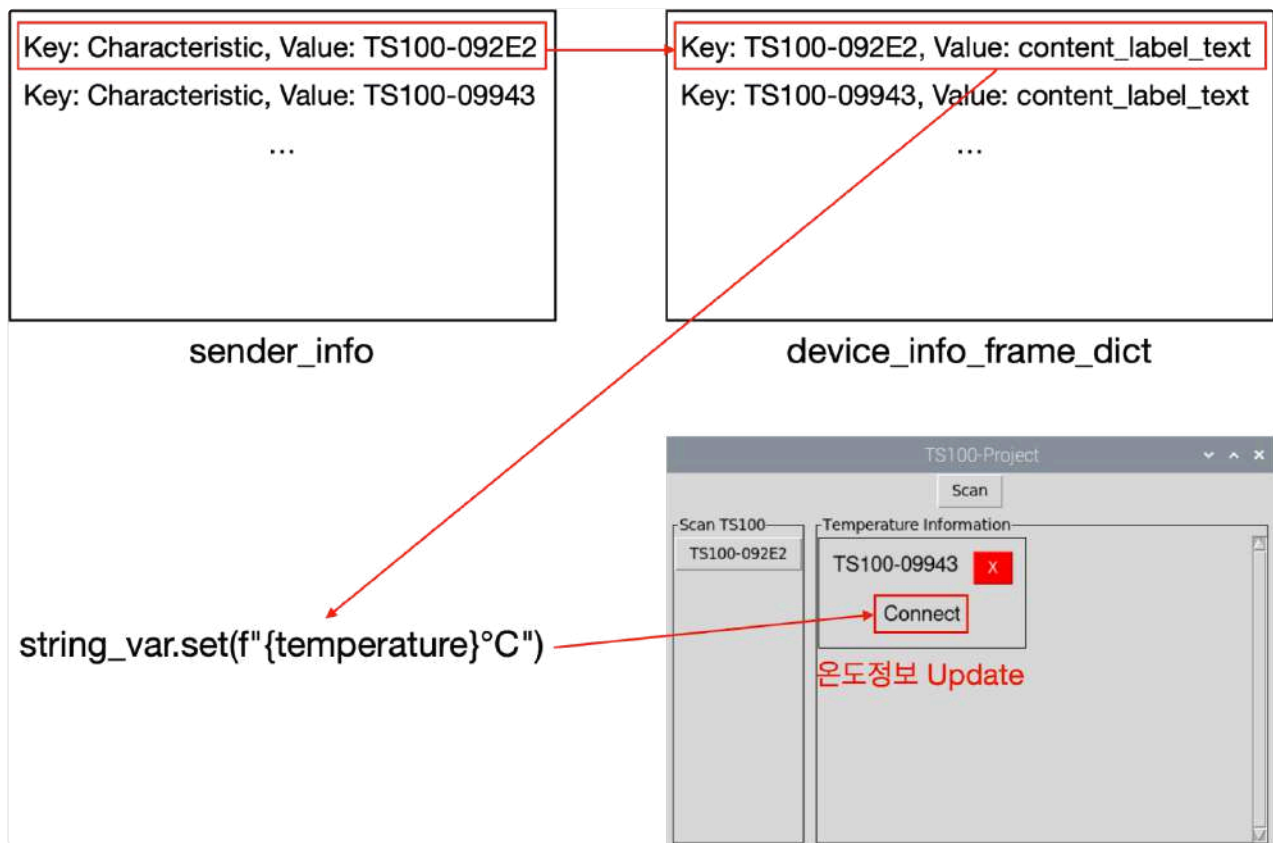
    if string_var is None:
        return

    string_var.set(f"{temperature}°C")

```

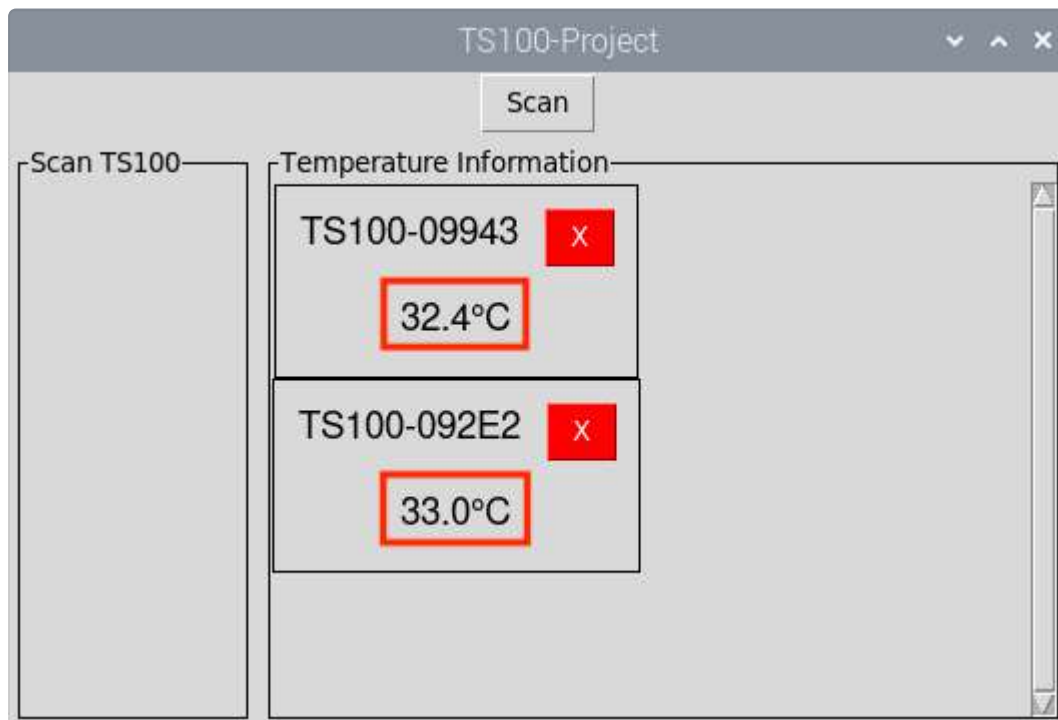
온도정보를 업데이트하는 순서는 아래 그림과 같다.

1. sender_info dictionary에서 수신된 characteristic을 가진 이름을 획득한다.
2. device_info_frame_dict에서 바나나 체온계 이름을 key로 가진 온도정보 라벨 프레임을 획득한다.
3. set() 함수를 통해 온도정보를 업데이트한다.



온도정보 업데이트 순서

코드실행 후 2대 연결과정을 수행하면 아래 그림과 같이 온도정보가 화면에 업데이트 되는것을 확인할 수 있다.



온도정보 업데이트

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner, BleakClient

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Scan List")
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="Temperature Information")
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

...
Scan
...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda device=device: remove_button(device.name))
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

# 이벤트루프 생성해 BLE 기기 scan

```

```

loop = asyncio.get_event_loop()
ts100_scan_list = loop.run_until_complete(scan())

create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

...
    Connect
...

# dictionary 선언(key: device_name, value: device_text_frame)
device_info_frame_dict = {}

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # Characteristic, Service 요청
        await get_service_and_characteristic(candidate_device, selected_c

```

```

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button, candidate_device.name)
        # info 화면에 추가
        root.after(0, create_device_info_frame, candidate_device, selecte

except Exception as e:
    # 연결 실패시 발생
    print('error: ', e, end='')
    return None

def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

    # value값으로 dictionary 값 삭제하는 함수
    def remove_characteristic_by_value(name):
        keys_to_remove = [key for key, value in sender_dict.items() if va
        for key in keys_to_remove:
            del sender_dict[key]

    frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
    frame.pack()

    # TitleFrame: 좌측 상단에 제목이 표시되는 프레임
    title_frame = Frame(frame, padx=5, pady=5)
    title_label = Label(title_frame, text=device.name, font=("Helvetica",
    title_label.pack(side="left")

    # NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
    remove_button_frame = Frame(frame, padx=5, pady=5)
    remove_button = Button(remove_button_frame, text="X", command=remove_
    remove_button.pack()

    # ContentFrame: 하단에 내용을 보여주는 프레임

```

```

content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

...
    notify
...

health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

# dictionary 선언(key: characteristic, value: device_name)
sender_dict = {}

# Service, Characteristic 요청하는 함수
async def get_service_and_characteristic(candidate_device, connected_devi
    # Service 확인
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    # Dictionary에 추가
                    if characteristic not in sender_dict:
                        sender_dict[characteristic] = candidate_device.na
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    temperature = parse_temperature_information(data)

# 수신된 기기 이름 확인

```

```

device_name = sender_dict.get(handle)

if device_name is None:
    return

# 업데이트할 텍스트 영역 확인 후 업데이트
string_var = device_info_frame_dict.get(device_name)

if string_var is None:
    return

string_var.set(f"{temperature}°C")

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    # 온도정보 출력
    print(temperature)

    return temperature

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

```

```

# 이벤트루프 생성
loop = asyncio.get_event_loop()

# 비동기로 UI 계속 업데이트
def asyncio_loop():
    loop.call_soon(loop.stop)
    loop.run_forever()
    root.after(100, asyncio_loop)

root.after(100, asyncio_loop)
root.mainloop()

```

블루투스 기기 연결 해제

Temperature Information 영역의 X 버튼을 클릭하면 BLE연결을 해제하는 기능을 구현하기 위해 create_device_info_frame 함수 내 remove_frame, remove_characteristic_by_value 함수를 아래와 같이 수정한다.

아래 코드는 UI에서 삭제, BLE연결 해제, dictionary 값 삭제 기능을 수행한다.

```

# 연결된 기기 Frame을 생성하는 함수
def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

# value값으로 dictionary 값 삭제하는 함수
def remove_characteristic_by_value(name):
    keys_to_remove = [key for key, value in sender_dict.items() if va
    for key in keys_to_remove:
        del sender_dict[key]

frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
frame.pack()

# TitleFrame: 좌측 상단에 제목이 표시되는 프레임
title_frame = Frame(frame, padx=5, pady=5)
title_label = Label(title_frame, text=device.name, font=("Helvetica",
title_label.pack(side="left")

# NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
remove_button_frame = Frame(frame, padx=5, pady=5)
remove_button = Button(remove_button_frame, text="X", command=remove_
remove_button.pack()

# ContentFrame: 하단에 내용을 보여주는 프레임
content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove button frame.grid(row=0, column=1, stickv="e")

```

```

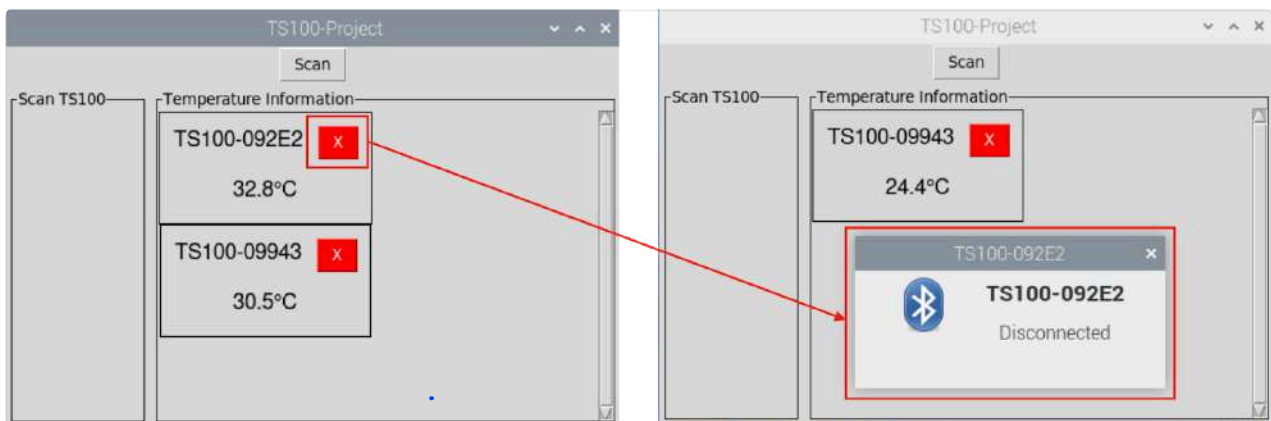
content_frame.grid(row=1, column=0, columnspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

```

이후 코드를 실행해 X 버튼을 클릭하면 화면에서 사라지고, BLE 연결이 해제되는것을 확인할 수 있다.



블루투스 기기 연결 해제

지금까지의 내용이 반영된 코드이다.

```

import tkinter as tk
from tkinter import *

import asyncio

from bleak import BleakScanner, BleakClient

scan_frame = None
scroll_frame = None

def create_ui(root):
    global scan_frame
    global scroll_frame

    # UI 제목
    root.title("TS100-Project")

    # Bottom 프레임, Scan TS100, Temperature Information을 포함함
    bottom_frame = Frame(root)

    # Scan리스트 Frame
    scan_ts100_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="")
    scan_ts100_frame.grid(row=0, column=0, padx=5, pady=5, sticky="nsew")

    # 전역변수 할당
    scan_frame = scan_ts100_frame

    # 온도정보 Frame
    temperature_information_frame = LabelFrame(bottom_frame, relief="solid", bd=1, text="")
    canvas = Canvas(temperature_information_frame, borderwidth=0)
    scrollbar = Scrollbar(temperature_information_frame, orient="vertical")
    scrollable_frame = Frame(canvas)

    # 전역변수 할당
    scroll_frame = scrollable_frame

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    temperature_information_frame.grid(row=0, column=1, padx=5, pady=5, sticky="nsew")
    canvas.pack(side="left", fill="both", expand=True)
    scrollbar.pack(side="right", fill="y", expand=True)

```

```

# Scan 버튼
scan_button = Button(root, text="Scan", command=scan_button_click)
scan_button.pack()

# 하위 프레임
bottom_frame.pack(side="bottom", fill="both", expand=True)

...
Scan
...

# 스캔된 버튼을 담고있는 배열
scan_devices_list = []

def remove_button(button_name=None):
    if button_name is not None:
        if not scan_devices_list:
            return

        remove_button = None
        for device in scan_devices_list:
            if device.cget("text") == button_name:
                remove_button = device
                break

        if remove_button:
            root.after(0, remove_button.destroy)
            scan_devices_list.remove(remove_button)
        return

# 기존에 scan했던 기기 화면에서 제거
for button in scan_devices_list:
    root.after(0, button.destroy)
    scan_devices_list.remove(button)

def create_scan_devices(ts100_scan_list):
    global scan_devices_list

    # Scan했던 Device 버튼 생성
    for device in ts100_scan_list:
        button = Button(scan_frame, text=device.name, command=lambda device=device: remove_button(device.name))
        button.pack()
        scan_devices_list.append(button)

def scan_button_click():
    remove_button()

# 이벤트루프 생성해 BLE 기기 scan

```

```

loop = asyncio.get_event_loop()
ts100_scan_list = loop.run_until_complete(scan())

create_scan_devices(ts100_scan_list)

async def scan():
    ts100_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    for device in scan_devices:
        device_name = device.name if device.name else "Unknown Device"
        if "TS100" in device_name:
            ts100_device.append(device)
            print(device_name)

    return ts100_device

def scan_device_button_click(device):
    print("Selected Device: " + device.name)

    loop = asyncio.get_event_loop()
    loop.run_until_complete(connect(device))

...
Connect
...

# dictionary 선언(key: device_name, value: device_text_frame)
device_info_frame_dict = {}

async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')

        # Characteristic, Service 요청
        await get_service_and_characteristic(candidate_device, selected_c

```

```

        # scan_ts100에서 연결된 기기 삭제
        root.after(0, remove_button, candidate_device.name)
        # info 화면에 추가
        root.after(0, create_device_info_frame, candidate_device, selecte

except Exception as e:
    # 연결 실패시 발생
    print('error: ', e, end='')
    return None

# 연결된 기기 Frame을 생성하는 함수
def create_device_info_frame(device, ble_client):
    global scroll_frame
    global device_info_frame_dict
    global sender_dict

    # Device 영역 제거
    def remove_frame():
        # 프레임에서 삭제
        frame.destroy()

    # BLE 연결 해제
    loop = asyncio.get_event_loop()
    loop.run_until_complete(ble_client.disconnect())

    # dictionary 삭제
    del device_info_frame_dict[device.name]
    remove_characteristic_by_value(device.name)

# value값으로 dictionary 값 삭제하는 함수
def remove_characteristic_by_value(name):
    keys_to_remove = [key for key, value in sender_dict.items() if va
    for key in keys_to_remove:
        del sender_dict[key]

frame = Frame(scroll_frame, relief="solid", bd=1, padx=5, pady=5)
frame.pack()

# TitleFrame: 좌측 상단에 제목이 표시되는 프레임
title_frame = Frame(frame, padx=5, pady=5)
title_label = Label(title_frame, text=device.name, font=("Helvetica",
title_label.pack(side="left")

# NotifyStartFrame: 우측 상단에 notify를 시작하는 버튼이 표시되는 프레임
remove_button_frame = Frame(frame, padx=5, pady=5)
remove_button = Button(remove_button_frame, text="X", command=remove_
remove_button.pack()

```

```

# ContentFrame: 하단에 내용을 보여주는 프레임
content_frame = Frame(frame, padx=10, pady=10)
content_label_text = StringVar()
content_label = Label(content_frame, textvariable=content_label_text,
content_label_text.set("Connect")
content_label.pack(side="bottom")

# 배치
title_frame.grid(row=0, column=0, sticky="ew")
remove_button_frame.grid(row=0, column=1, sticky="e")
content_frame.grid(row=1, column=0, colspan=2, sticky="nsew")

# 그리드 가중치 설정 (윈도우 크기 조절에 대응하기 위해)
scroll_frame.grid_rowconfigure(1, weight=1)
scroll_frame.grid_columnconfigure(0, weight=1)

# dictionary에 추가
device_info_frame_dict[device.name] = content_label_text

...
    notify
...

health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

# dictionary 선언(key: characteristic, value: device_name)
sender_dict = {}

# Service, Characteristic 요청하는 함수
async def get_service_and_characteristic(candidate_device, connected_device):
    # Service 확인
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    # Dictionary에 추가
                    if characteristic not in sender_dict:
                        sender_dict[characteristic] = candidate_device.name
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    temperature = parse_temperature_information(data)

```

```

# 수신된 기기 이름 확인
device_name = sender_dict.get(handle)

if device_name is None:
    return

# 업데이트할 텍스트 영역 확인 후 업데이트
string_var = device_info_frame_dict.get(device_name)

if string_var is None:
    return

string_var.set(f"{temperature}°C")

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    # 온도정보 출력
    print(temperature)

    return temperature

# 온도정보 계산하는 함수
def temperature_calculate(t1, t2, t3, d):
    # Int로 변환, 비트 시프트 후 합산
    int_t1 = int(t1)
    int_t2 = int(t2) << 8
    int_t3 = int(t3) << 16

    signed_value = int_t1 + int_t2 + int_t3

    # 자릿수 정하는 변수
    digit = int(d)
    if digit > 127:
        digit -= 256

    # 온도
    temperature = float(signed_value) * 10 ** digit
    # 소수점 1자리만 남도록 수정
    temperature = round(temperature * 10) / 10

    return temperature

if __name__ == "__main__":
    root = tk.Tk()

    create_ui(root)

```

```
create_ui(1000)

# 이벤트루프 생성
loop = asyncio.get_event_loop()

# 비동기로 UI 계속 업데이트
def asyncio_loop():
    loop.call_soon(loop.stop)
    loop.run_forever()
    root.after(100, asyncio_loop)

root.after(100, asyncio_loop)
root.mainloop()
```

5. 클라우드로 온도 정보를 전송해보자

5.1. 회원기

계정 생성

바나나 체온계로부터 받은 온도 데이터를 클라우드에 데이터를 저장하는 클라우드 데이터 저장의 장점 클라우드 저장소는 다양한 장점을 늘리거나 줄일 수 있다. 이는 초감작스러운 데이터 증가에도 두

또한, 접근성도 중요한 장점이며 용이하게 하며, 데이터의 활용도로 작업을 공유할 수 있다.

비용 절감도 클라우드 저장소의 다. 이는 특히 중소기업이나 스

보안 역시 클라우드 서비스 제 다양한 보안 기능을 제공한다.

자동 백업 및 복구 기능을 통해 구할 수 있는 기능을 제공한다.

마지막으로, 관리 효율성을 들 리에만 집중할 수 있다. 이는 IT 최신 동향

클라우드 기술의 발전에 따라 여러 최신 동향이 나타나고 있다. 먼저, 멀티 클라우드 전략이 많은 기업에서 채택되고 있다. 이는 여러 드 서비스를 혼합해 사용하는 전략으로, 특정 클라우드 제공업체에 대한 종속성을 줄이고 각 클라우드의 장점을 최대한 활용하기 위 다.

또한, 하이브리드 클라우드가 인기를 끌고 있다. 하이브리드 클라우드는 온프레미스(자체 서버)와 클라우드 서비스를 결합한 방식으 한 데이터는 온프레미스에, 덜 민감한 데이터는 클라우드에 저장하는 형태를 취한다. 이는 데이터 보안과 효율성을 동시에 높이는 방

클라우드 네이티브 기술도 주목받고 있다. 클라우드 환경에서 최적화된 컨테이너, 마이크로서비스, 서버리스 아키텍처 등이 이에 하 이러한 기술들은 애플리케이션의 유연성, 확장성, 효율성을 높여준다. 예를 들어, 컨테이너 기술을 통해 애플리케이션을 일관된 환경 행할 수 있으며, 서버리스 아키텍처를 통해 서버 관리 부담을 줄일 수 있다.

인공지능(AI) 및 머신러닝(ML) 통합도 클라우드 플랫폼에서 활발히 이루어지고 있다. 클라우드 서비스 제공업체들은 AI 및 ML 서 공하여 데이터 분석, 예측 모델링 등을 쉽게 수행할 수 있도록 지원한다. 이는 기업들이 데이터에서 인사이트를 도출하고, 비즈니스 정을 개선하는 데 큰 도움이 된다.

마지막으로, 보안 강화가 중요한 트렌드로 자리 잡고 있다. 데이터 프라이버시와 보안에 대한 관심이 높아지면서 클라우드 서비스 제 들은 더 강력한 보안 기능과 규제 준수 지원을 제공하고 있다. 이는 데이터 보호와 규제 준수를 동시에 달성하려는 기업들에게 중요한 된다.

널리 사용되는 클라우드 서버 소개

현대의 많은 기업들은 다양한 클라우드 서비스를 이용하고 있다. 그 중에서도 **Amazon Web Services (AWS)**는 가장 널리 사 클라우드 서비스로, 다양한 서비스와 도구를 제공하여 기업들이 필요로 하는 거의 모든 컴퓨팅 리소스를 지원한다. AWS는 전 세계가 큰 인프라를 갖추고 있어, 글로벌 서비스를 제공하기에 적합하다.

Microsoft Azure는 Microsoft의 클라우드 서비스로, Windows 환경과의 높은 호환성과 다양한 엔터프라이즈 솔루션을 제공한다. 는 특히 기존의 Microsoft 제품과 통합되어 사용하기 쉬운 점이 강점이다.

Google Cloud Platform (GCP)는 Google의 클라우드 서비스로, 데이터 분석, 머신러닝, 인공지능 등의 강력한 도구를 제공하 GCP는 특히 대규모 데이터 처리를 위한 도구와 서비스가 잘 갖추어져 있다.

바나나 체온계로부터 받은 온도 정보를 클라우드에 저장하는 방법에 대해 설명한다. 클라우드에 데이

클라우드 데이터 저장의 장점

확장성: 클라우드 저장소는 사용자의 필요에 따라 저장 용량을 쉽게 늘리거나 줄일 수 있다. 초기 비용 는 방식이 가능하다.

접근성: 인터넷만 연결되어 있다면 언제 어디서나 데이터를 접근하고 사용할 수 있다. 여러 장소에서

비용 절감: 자체 서버를 운영하는 비용과 비교해 초기 투자 비용이 낮고, 유지 보수 비용이 적게 든다.

보안: 대부분의 클라우드 서비스 제공업체는 데이터 암호화, 백업, 접근 제어 등 다양한 보안 기능을

자동 백업 및 복구: 데이터 손실 위험을 줄이기 위해 자동으로 데이터를 백업하고, 필요 시 손쉽게 복

관리 효율성: 클라우드 서비스 제공업체가 서버 유지 보수, 업데이트 등을 관리해 주기 때문에 사용

최신 동향

멀티 클라우드 전략: 많은 기업이 여러 클라우드 서비스를 혼합해 사용하는 멀티 클라우드 전략을 채 체에 대한 종속성을 줄이고, 각 클라우드의 장점을 최대한 활용하기 위함이다.

하이브리드 클라우드: 온프레미스(자체 서버)와 클라우드 서비스를 결합한 하이브리드 클라우드가 인 레미스에, 덜 민감한 데이터는 클라우드에 저장하는 방식이다.

클라우드 네이티브 기술: 클라우드 환경에서 최적화된 컨테이너, 마이크로서비스, 서버리스 아키텍처 선의 유연성, 확장성, 효율성을 높인다.

인공지능(AI) 및 머신러닝(ML) 통합: 클라우드 플랫폼에서 AI 및 ML 서비스를 제공하여 데이터 분석 게 한다.

보안 강화: 데이터 프라이버시와 보안에 대한 관심이 높아지면서 클라우드 서비스 제공업체들은 더 강 공하고 있다.

클라우드 데이터 저장은 이러한 장점과 최신 기술을 통해 데이터 관리의 효율성과 유연성을 크게 향상 비스를 활용하면 더 쉽게 데이터를 저장하고 관리할 수 있다.

클라우드 데이터 저장의 장점

확장성: 클라우드 저장소는 사용자의 필요에 따라 저장 용량을 쉽게 늘리거나 줄일 수 있다. 초기 비용 다.

접근성: 인터넷만 연결되어 있다면 언제 어디서나 데이터를 접근하고 사용할 수 있다. 여러 장소에서

비용 절감: 자체 서버를 운영하는 비용과 비교해 초기 투자 비용이 낮고, 유지 보수 비용이 적게 든다.

보안: 대부분의 클라우드 서비스 제공업체는 데이터 암호화, 백업, 접근 제어 등 다양한 보안 기능을

자동 백업 및 복구: 데이터 손실 위험을 줄이기 위해 자동으로 데이터를 백업하고, 필요 시 손쉽게 복

관리 효율성: 클라우드 서비스 제공업체가 서버 유지 보수, 업데이트 등을 관리해 주기 때문에 사용

최신 동향

멀티 클라우드 전략: 많은 기업이 여러 클라우드 서비스를 혼합해 사용하는 멀티 클라우드 전략을 채

하이브리드 클라우드: 온프레미스(자체 서버)와 클라우드 서비스를 결합한 하이브리드 클라우드가 인

클라우드 네이티브 기술: 클라우드 환경에서 최적화된 컨테이너, 마이크로서비스, 서버리스 아키텍처

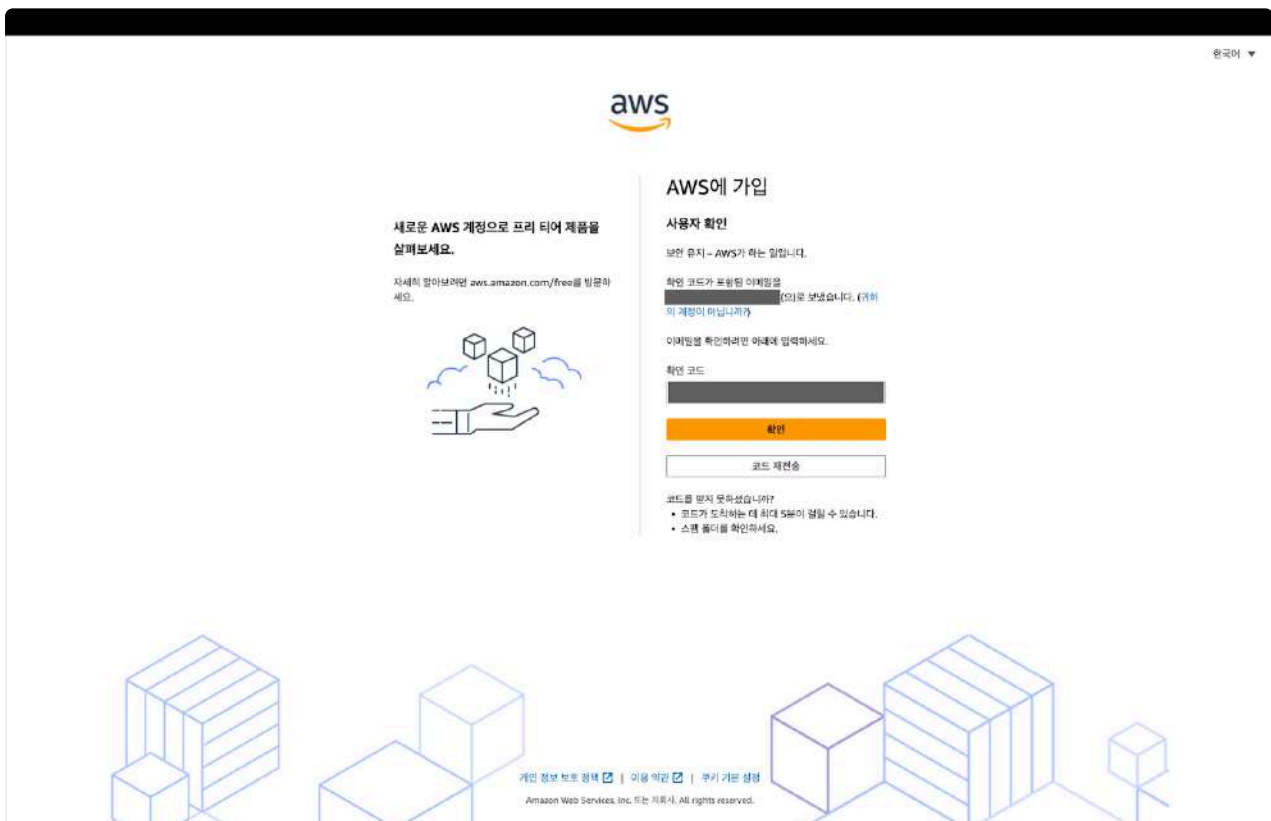
인공지능(AI) 및 머신러닝(ML) 통합: 클라우드 플랫폼에서 AI 및 ML 서비스를 제공하여 데이터 분석

보안 강화: 데이터 프라이버시와 보안에 대한 관심이 높아지면서 클라우드 서비스 제공업체들은 더 강



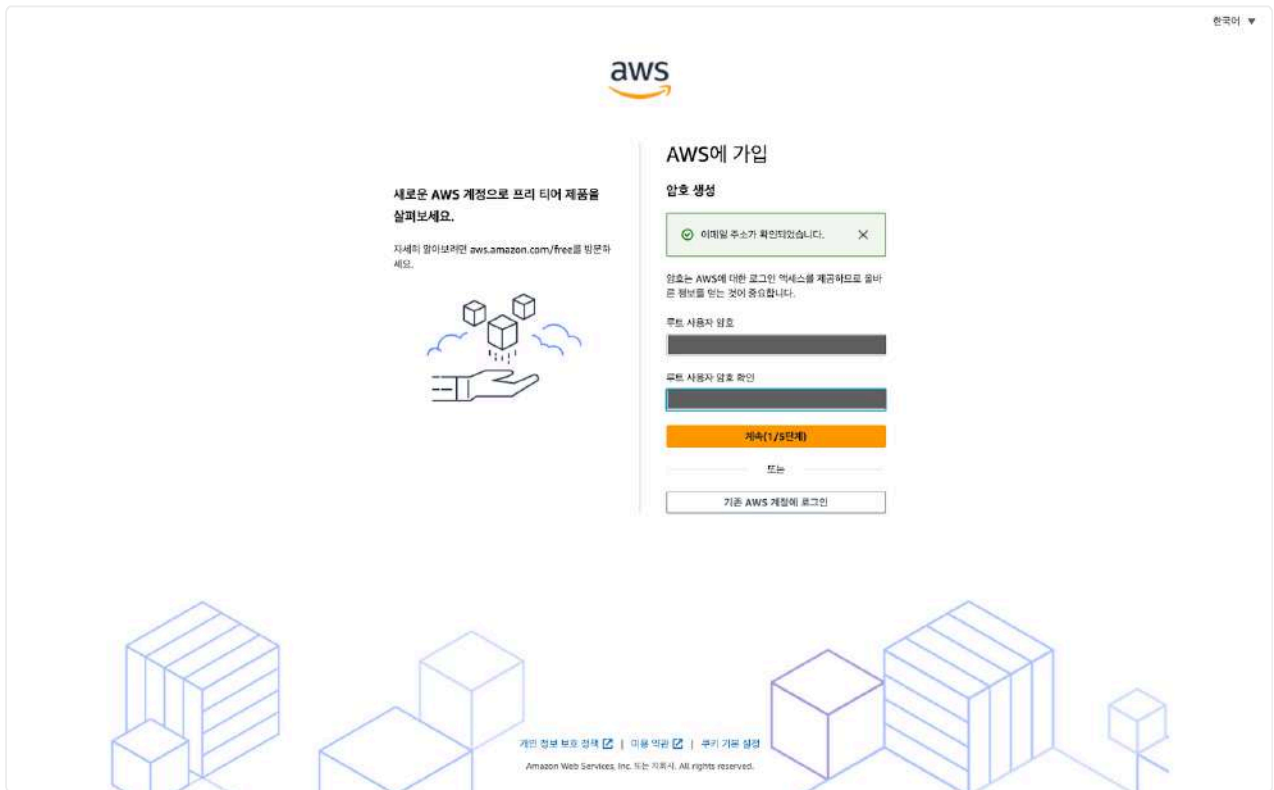
이메일 주소 확인 메일

다시 웹페이지로 돌아와서 이메일에서 복사했던 확인코드를 확인코드 입력창에 붙여넣은 후 확인버튼을 클릭한다.



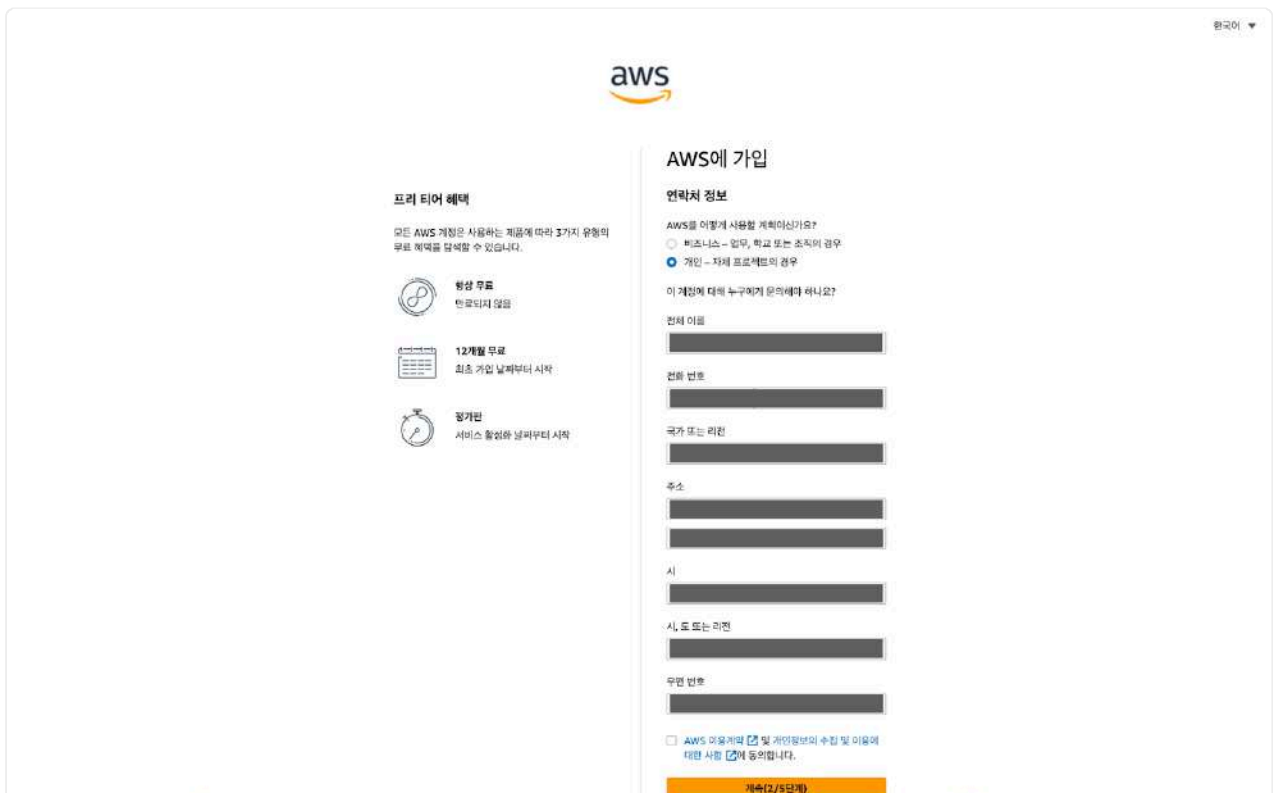
사용자 확인

확인코드를 입력하면 루트 사용자 암호 입력 화면으로 전환된다. 비밀번호 양식에 맞춰 암호를 입력한 후 계속 버튼을 클릭한다.



암호 생성

아래 그림과 같이 연락처 정보 입력화면으로 전환되면 전화번호와 개인정보 입력 후 계속 버튼을 클릭한다. 주소의 경우 영어 또는 다른 라틴 문자 기반 언어로 입력해야한다.



연락처 정보 입력

결제 정보 입력 화면으로 전환되면 결제정보 입력 후 확인 및 계속 버튼을 클릭한다. 결제입력정보를 전부 입력하면 100원이 결제되었다가 바로 취소처리된다.

한국어 ▼

aws

보안 확인

① AWS 프리 티어 한도보다 낮은 사용량에 대해서는 요금이 부과되지 않습니다. AWS에서는 자격 증명용 확인하기 위해 3~5일 동안 1 USD(또는 그에 상응하는 현지 통화 금액)를 대기 중인 트랜잭션으로 일시 보류합니다.

AWS에 가입

결제 정보

신용카드 번호

VISA

만료 날짜

카드 소유자 이름

참구지 주소

☒ 내 연락처 주소 사용

☐ 새 주소 사용

이메일 주소

이메일 주소는 AWS와의 거래를 위해 VAT 영수증을 발송하는 데 사용됩니다.

확인 및 계속(3/5단계)

확인 요금을 승인하기 위해 은행의 웹 사이트로 리디렉션될 수 있습니다.

결제 정보 입력

자격 증명 확인 화면으로 전환되면 휴대전화 번호와 보안 검사 문자 입력 후 SMS 전송 버튼을 클릭한다.

AWS에 가입

자격 증명 확인

AWS 계정을 사용하려면 먼저 전화번호를 확인해야 합니다. 계속하면 AWS 자동 시스템이 확인 코드 전송을 위해 연락합니다.

확인 코드를 어떻게 보내 드릴까요?

☒ 문자 메시지(SMS)
 ☐ 음성 통화

국가 또는 리전 코드

대한민국 (+82)

휴대전화 번호

보안 검사

위에 보이는 문자를 입력하세요.

SMS 전송(4/5단계)

[개인 정보 보호 정책](#) | [이용 약관](#) | [우키 기본 설정](#) | [로그아웃](#)

자격 증명 확인

자격 증명 확인 화면으로 전환되면 이전에 입력했던 휴대전화 번호로 수신된 메시지의 인증코드 입력 후 계속 버튼을 클릭한다.

AWS에 가입

자격 증명 확인

코드 확인

계속(4/5단계)

문제가 있으신가요? 때로는 확인 코드를 가져오는 데 최대 10분이 걸립니다. 이보다 오래 걸리면 [이전 페이지](#)로 돌아가서 다시 시도하세요.

[개인 정보 보호 정책](#) | [이용 약관](#) | [우키 기본 설정](#) | [로그아웃](#)

자격 증명 확인

Support 플랜 선택 화면으로 전환되면 “기본 지원 - 무료” 플랜 선택 후 가입완료 버튼을 클릭한다.

한국어 ▼



AWS에 가입

Support 플랜 선택

비즈니스 또는 개인 계정에 대한 Support 플랜을 선택합니다. [플랜 및 요금 예시를 비교](#)해 보세요. 언제든지 AWS Management Console에서 플랜을 변경할 수 있습니다.

☒ 기본 지원 - 무료

- AWS를 처음 시작하는 신규 사용자에게 권장
- AWS 리소스에 대한 인증 무료 24시간 셀프 서비스 액세스
- 계정 및 청구 문제 전용
- Personal Health Dashboard 및 Trusted Advisor에 대한 액세스



☐ 개발자 지원 - 시작가는 29 USD/월

- AWS를 체험해보는 개발자에게 권장
- 일부 시간 중 AWS Support에 대한 이메일 액세스
- 12시간(업무 시간 기준) 이내의 응답 시간



☐ 비즈니스 지원 - 시작가는 100 USD/월

- AWS 기반 프로덕션 워크로드 실행에 추천
- 이메일, 전화 및 채팅을 통한 전용 24시간 기술 지원
- 1시간 이내의 응답 시간
- Trusted Advisor 모범 사례 권장 사항 전체 세트



 엔터프라이즈 수준의 지원이 필요한가요?

최저 월 15,000 USD로 15분 이내에 응답을 받을 수 있으며 기술 지원 관리자가 배정된 컨시어지 스태프의 서비스를 이용할 수 있습니다. [자세히 알아보기](#)

가입 완료

Support 플랜 선택

AWS 가입완료 화면으로 전환되면 AWS Management Console로 이동 버튼을 클릭한다.

한국어 ▼





축하합니다.

AWS에 가입해 주셔서 감사합니다.

계정을 활성화하는 중입니다. 이 작업은 몇 분 밖에 걸리지 않습니다. 이 작업이 완료되면 이메일을 받게 됩니다.

AWS Management Console로 이동

[다른 계정에 가입](#) or [영업 팀에 문의](#)하세요.

AWS 회원 가입 완료

5.2. 로그인

로그인 화면으로 전환되면 루트 사용자 이메일 주소를 입력하고 다음 버튼을 클릭한다.



The screenshot shows the AWS login page. On the left, there's a login form with the AWS logo at the top. Below the logo is the title '로그인' (Login). There are two radio button options: '루트 사용자' (Root user) which is selected, and 'IAM 사용자' (IAM user). The 'Root user' option has a note about unrestricted access. Below these is a text input field for the '루트 사용자 이메일 주소' (Root user email address). A blue '다음' (Next) button is below the input field. A disclaimer about AWS terms and cookies is present. At the bottom of the form is a button 'AWS 계정 새로 만들기' (Create new AWS account). On the right, there's a promotional banner for 'AWS SKILL BUILDER' with the text 'AWS 실습을 통한 AWS 학습' (Learn AWS through hands-on practice) and 'AWS를 배우는 새로운 방법' (A new way to learn AWS). It includes a button '지금 구독하기 »' (Subscribe now »). At the very bottom, there's a copyright notice '© 2023, Amazon Web Services, Inc. 또는 자회사. All rights reserved.' and a language dropdown menu set to '한국어' (Korean).

aws

로그인

☒ 루트 사용자
무제한 액세스 권한이 필요한 작업을 수행하는 계정 소유
자입니다. 자세히 알아보기

☐ IAM 사용자
일일 작업을 수행하는 계정 내 사용자입니다. 자세히 알아
보기

루트 사용자 이메일 주소

다음

계속 진행하는 경우 AWS 고객 계약 또는 AWS 서비스에 대한 기
타 계약 및 개인 정보 보호 정책에 동의하게 됩니다. 이 사이트는 필
수 쿠키를 사용합니다. 자세한 내용은 쿠키 고지를 참조하세요.

AWS를 처음 사용하십니까?

AWS 계정 새로 만들기

aws

AWS SKILL BUILDER

AWS 실습을 통한
AWS 학습

AWS를 배우는 새로운 방법

지금 구독하기 »

© 2023, Amazon Web Services, Inc. 또는 자회사. All rights reserved.

한국어 ▼

AWS 로그인

비밀번호 입력 화면으로 전환되면 비밀번호 입력 후 로그인 버튼을 클릭한다.



루트 사용자 로그인 ⓘ

이메일:

비밀번호 [비밀번호 찾기](#)

[로그인](#)

[다른 계정으로 로그인](#)

[AWS 계정 새로 만들기](#)



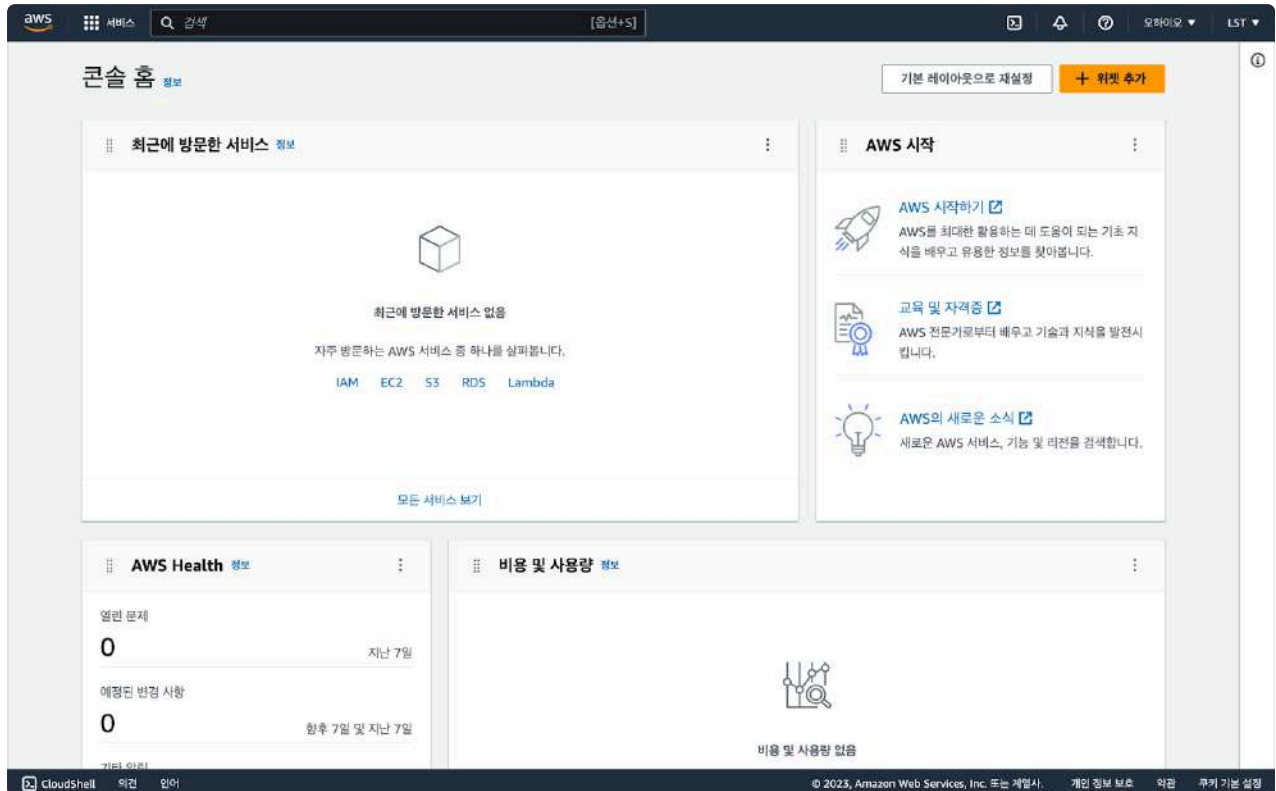
© 2023, Amazon Web Services, Inc. 또는 자회사. All rights reserved.

한국어 ▼

AWS 비밀번호 입력

5.3. AWS 기본 화면

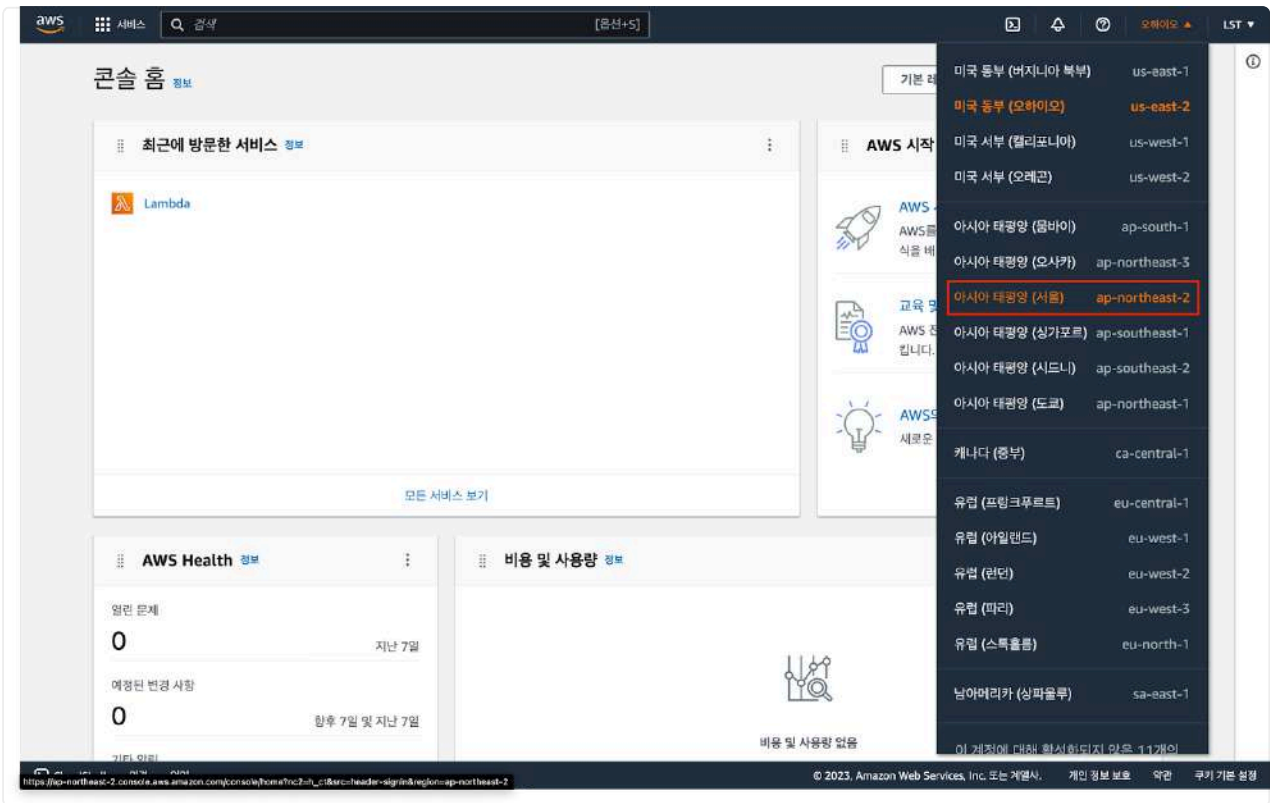
로그인을 성공하면 아래와 같은 화면이 나타난다.



AWS 콘솔 홈

지역?

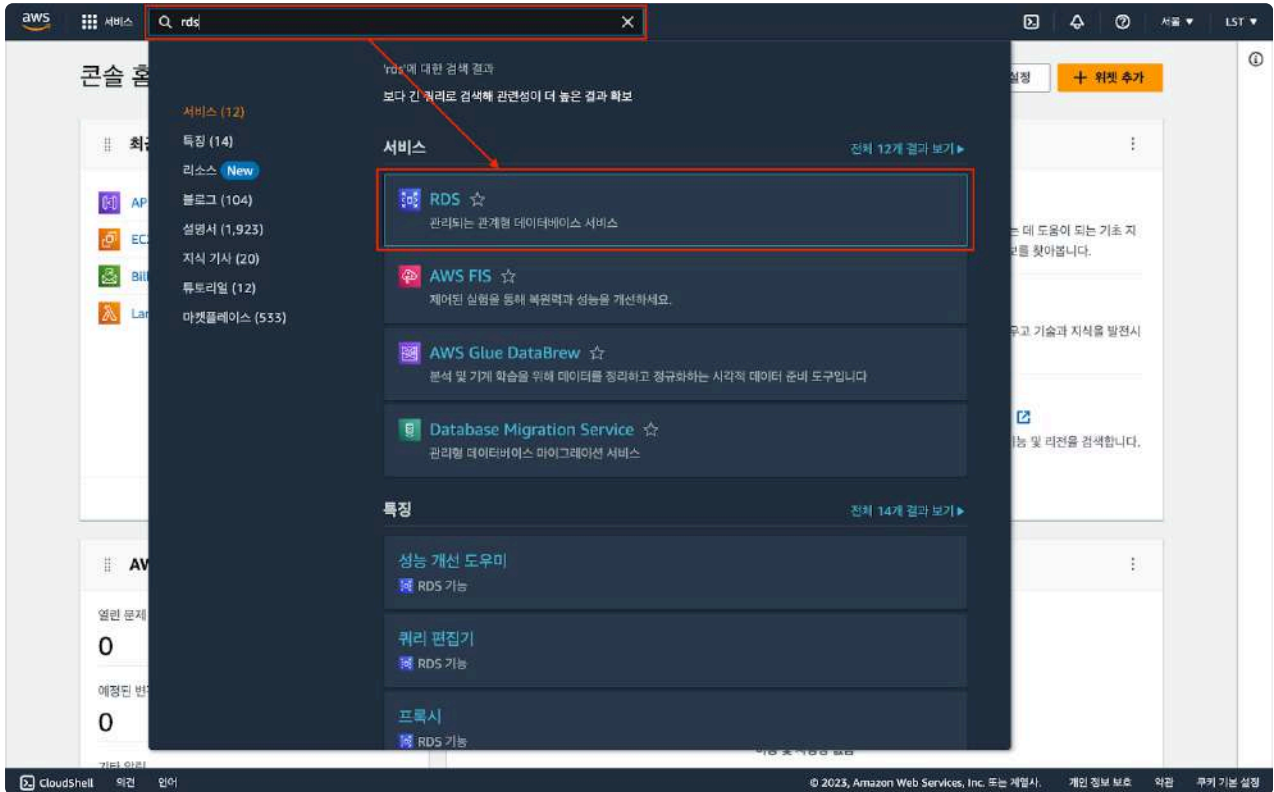
우측 상단에 **리전**을 아시아 태평양(서울)로 변경한다. 리전이란 AWS 서버의 위치를 말하며 현재 위치와 멀어질수록 느려지기 때문에 서울로 변경한다.



러전 변경
지역?

5.4. 클라우드 데이터베이스 생성

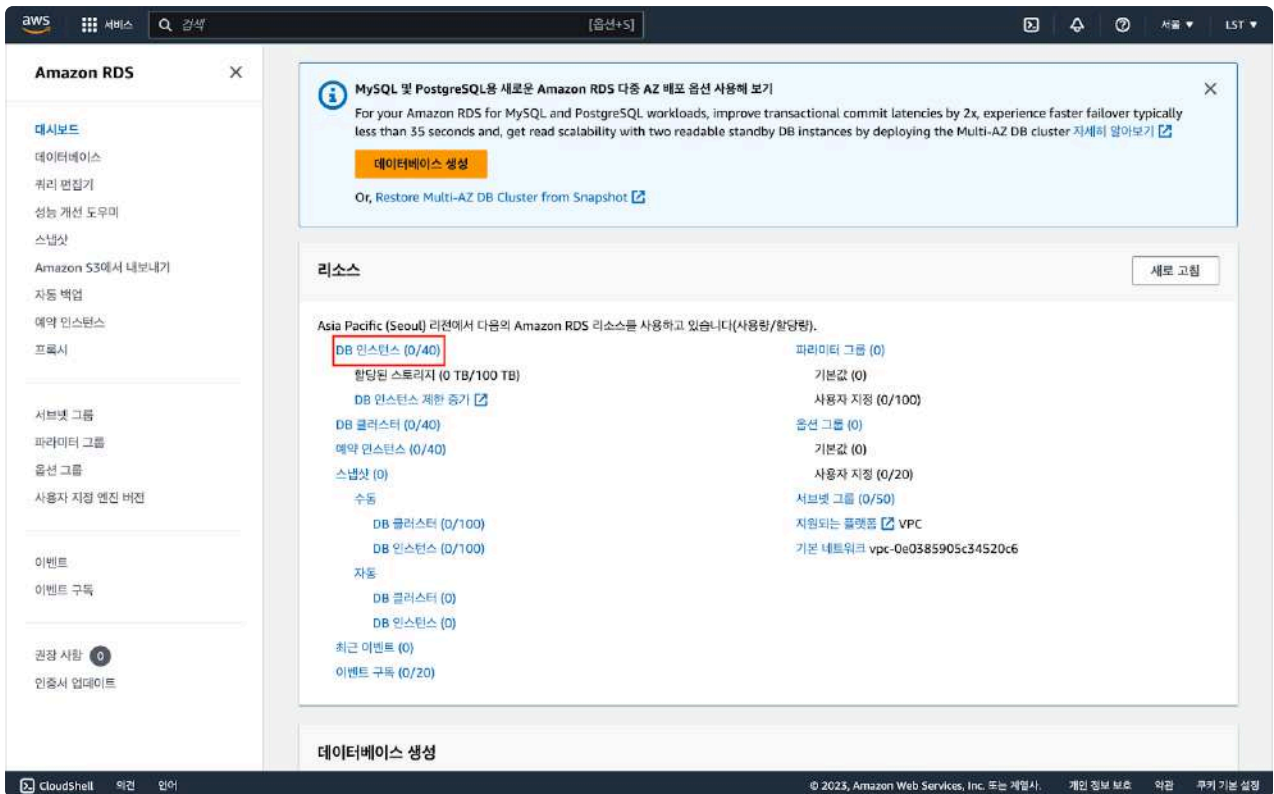
클라우드에 온도정보를 저장하기 위해서는 데이터베이스를 생성한다. AWS 콘솔 홈 화면에서 상단 검색 영역 선택 후 rds 입력하고, 검색결과 중 RDS를 클릭한다.



RDS 검색 방법

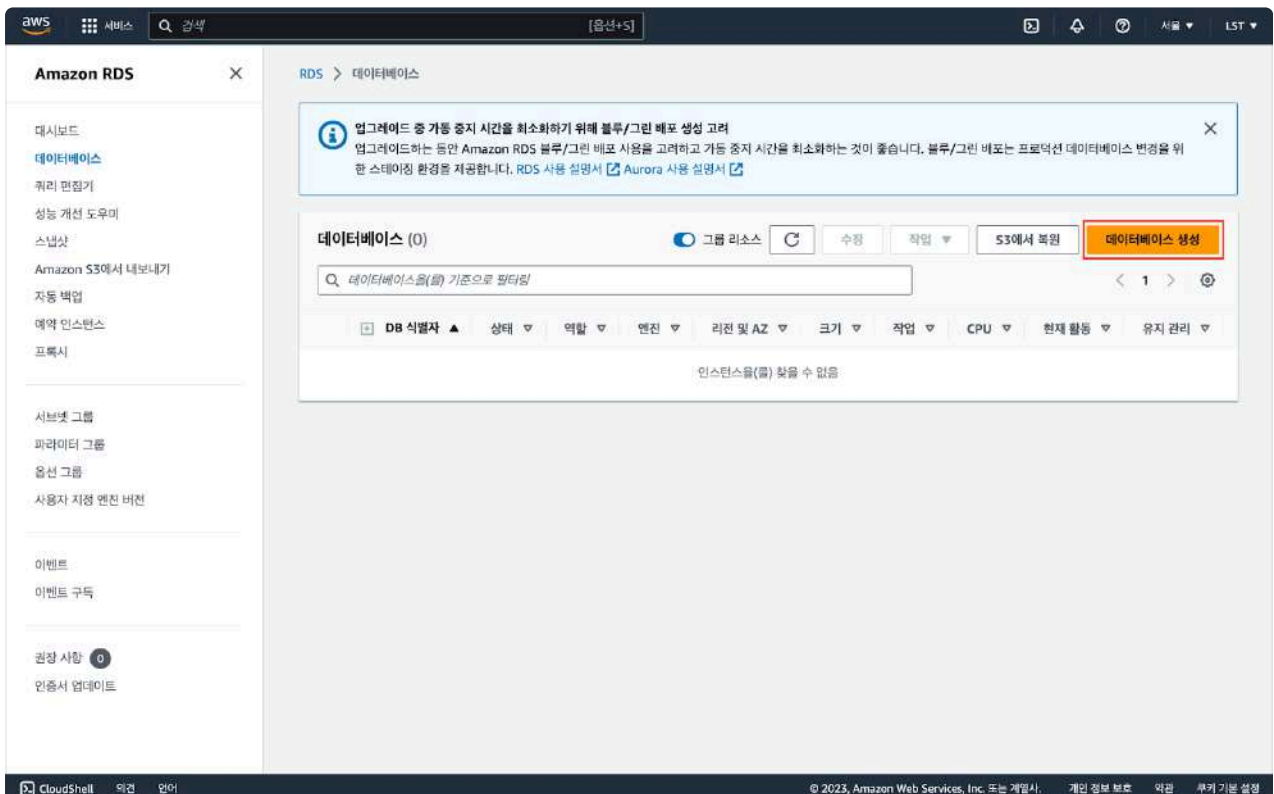
RDS 화면으로 변경되면 리소스 영역의 DB 인스턴스를 클릭한다.

@고건호 RDS에 대한 설명도 한페이지 정도로..추가해 줘요.



AWS RDS 콘솔

데이터베이스 화면에서 우측의 데이터베이스 생성 버튼을 클릭하여 데이터베이스 생성을 시작한다.



AWS 데이터베이스 콘솔

데이터베이스 생성 방식은 표준 생성 방식을 선택한다.

데이터베이스 생성 방식 선택

☒ 표준 생성

가용성, 보안, 백업 및 유지 관리에 대한 옵션을 포함하여 모든 구성 옵션을 설정합니다.

☐ 손쉬운 생성

권장 모범 사례 구성을 사용합니다. 일부 구성 옵션은 데이터베이스를 생성한 후 변경할 수 있습니다.

데이터베이스 생성 방식 선택

이후 엔진 옵션에서 설치하고싶은 데이터베이스를 선택한다. ~~본 문서~~에서는 PostgreSQL로 선택한다. 아래 엔진 버전은 별도로 설정하지 않는다.

본 문서 -> 여기서는

엔진 옵션

엔진 유형

☐ Aurora (MySQL Compatible)



☐ Aurora (PostgreSQL Compatible)



☐ MySQL



☐ MariaDB



☒ PostgreSQL



☐ Oracle



☐ Microsoft SQL Server



필터 숨기기

☐ 다중 AZ DB 클러스터를 지원하는 버전 표시

기본 DB 인스턴스 1개와 읽기 가능한 대기 DB 인스턴스 2개로 다중 AZ DB 클러스터를 생성합니다. 다중 AZ DB 클러스터는 최대 2배 빠른 트랜잭션 커밋 지연 시간과 일반적으로 35초 미만의 자동 장애 조치를 제공합니다.

엔진 버전

PostgreSQL 15.3-R2

엔진 옵션 선택

템플릿은 프리 티어로 설정한다. AWS 프리티어는 1년동안 매월 RDS와 사용할 수 있는 db.t2.micro, db.t3.micro 및 db.t4g.micro 인스턴스 750시간, 스토리지 20GB 및 백업용 스토리지 20GB를 사용할 수 있으며 프리티어 범위를 넘어선다면 과금이 발생한다. 템플릿을 프리티어로 선택하면 아래 가용성 및 내구성 영역은 비활성화된다.

템플릿

해당 사용 사례를 충족하는 샘플 템플릿을 선택하세요.

☐ 프로덕션
고가용성 및 빠르고 일관된 성능을 위해 기본값을 사용하세요.

☐ 개발/테스트
이 인스턴스는 프로덕션 환경 외부에서 개발 용도로 마련되었습니다.

☒ 프리 티어
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.
[정보](#)

가용성 및 내구성

배포 옵션 정보
아래의 배포 옵션은 위에서 선택한 엔진에서 지원하는 배포 옵션으로 제한됩니다.

- ☒ 다중 AZ DB 클러스터 - 신규
기본 DB 인스턴스와 읽기 가능한 예비 DB 인스턴스 2개가 있는 DB 클러스터를 생성합니다. 각 DB 인스턴스는 서로 다른 가용 영역(AZ)에 있습니다. 고가용성, 데이터 이중화를 제공하고 읽기 워크로드를 처리하기 위한 용량을 늘립니다.
- ☐ 다중 AZ DB 인스턴스(다중 AZ DB 클러스터 스냅샷에는 지원되지 않음)
다른 AZ에 기본 DB 인스턴스와 예비 DB 인스턴스를 생성합니다. 고가용성 및 데이터 이중화를 제공하지만 예비 DB 인스턴스는 읽기 워크로드에 대한 연결을 지원하지 않습니다.
- ☐ 단일 DB 인스턴스(다중 AZ DB 클러스터 스냅샷에는 지원되지 않음)
예비 DB 인스턴스가 없는 단일 DB 인스턴스를 생성합니다.

템플릿 선택

설정 영역에서 마스터 사용자 이름과 비밀번호를 입력한다. 사용자 이름과 비밀번호는 데이터베이스에 연결할 때 사용된다. DB 인스턴스 식별자는 별도로 설정하지 않는다.

인스턴스 클래스는 데이터베이스 템플릿을 프리티어로 설정했기 때문에 db.t3.micro, db.t4g.micro만 선택할 수 있다. 각 인스턴스에 대한 설명은 [링크](#)에서 확인할 수 있다. 본 문서에서 db.t3.micro 클래스를 선택한다.

인스턴스 구성

아래의 DB 인스턴스 구성 옵션은 위에서 선택한 엔진에서 지원하는 옵션으로 제한됩니다.

DB 인스턴스 클래스 정보

스탠다드 클래스(m 클래스 포함)

메모리 최적화 클래스(r 및 x 클래스 포함)

버스터블 클래스(t 클래스 포함)

db.t3.micro

2 vCPUs 1 GiB RAM 네트워크: 2,085Mbps

이전 세대 클래스 포함

인스턴스 구성

스토리지 영역에서 스토리지 자동 조정 활성화를 해제한다. 프리티어 최대 사용량이 20GB이기 때문에 체크를 해제하여 스토리지가 자동으로 늘어나지 않도록 설정한다. 스토리지 유형은 별도로 수정하지 않는다. 스토리지 유형은 [링크](#)에서 확인할 수 있다.

스토리지

스토리지 유형 정보

범용 SSD(gp2)

블록 크기에 따라 기준 성능 결정

할당된 스토리지 정보

20

GiB

최솟값은 20GiB이고, 최댓값은 6,144GiB입니다.

After you modify the storage for a DB instance, the status of the DB instance will be in storage-optimization. Your instance will remain available as the storage-optimization operation completes.

자세히 알아보기

▼ 스토리지 자동 조정

스토리지 자동 조정 정보

애플리케이션의 필요에 따라 데이터베이스 스토리지의 동적 조정 지원을 제공합니다.

스토리지 자동 조정 활성화

이 기능을 활성화하면 지정한 임계값 초과 후 스토리지를 늘릴 수 있습니다.

스토리지 설정

연결 설정 영역의 퍼블릭 액세스 설정을 “예” 로 변경한다.

VPC 보안 그룹(방화벽) 정보
 데이터베이스에 대한 액세스를 허용할 VPC 보안 그룹을 하나 이상 선택합니다. 보안 그룹 규칙이 적절한 수신 트래픽을 허용하는지 확인합니다.

☒ **기존 항목 선택**
 기존 VPC 보안 그룹 선택

☐ **새로 생성**
 새 VPC 보안 그룹 생성

기존 VPC 보안 그룹

Choose one or more options ▼

default ✕

가용 영역 정보

기본 설정 없음 ▼

RDS 프록시
 RDS 프록시는 애플리케이션 확장성, 복원력 및 보안을 개선하는 완전관리형고가용성 데이터베이스 프록시입니다.

☐ **RDS 프록시 생성 정보**
 RDS는 프록시에 대한 IAM 역할과 Secrets Manager 보안 암호를 자동으로 생성합니다. RDS 프록시에 대한 추가 비용이 있습니다. 자세한 내용은 다음을 참조하세요. [Amazon RDS 프록시 요금](#)

인증 기관 - 선택 사항 정보
 서버 인증서를 사용하면 Amazon 데이터베이스에 대한 연결이 이루어지고 있는지 검증하여 추가 보안 계층을 제공합니다. 프로비저닝하는 모든 데이터베이스에 자동으로 설치되는 서버 인증서를 확인하여 이를 수행합니다.

rds-ca-2019 (기본값) ▼

인증 기관을 선택하지 않으면 RDS에서 대신 인증 기관을 선택합니다.

▼ 추가 구성

데이터베이스 포트 정보
 데이터베이스가 애플리케이션 연결에 사용할 TCP/IP 포트입니다.

5432

연결 설정 (2/2)

데이터베이스 인증 옵션은 기본 설정으로 되어있는 암호인증을 선택한다.

데이터베이스 인증

데이터베이스 인증 옵션 정보

☒ **암호 인증**
 데이터베이스 암호를 사용하여 인증합니다.

☐ **암호 및 IAM 데이터베이스 인증**
 AWS IAM 사용자 및 역할을 통해 데이터베이스 암호와 사용자 자격 증명을 사용하여 인증합니다.

☐ **암호 및 Kerberos 인증**
 권한이 부여된 사용자가 Kerberos 인증을 사용하여 이 DB 인스턴스에서 인증하도록 허용하려는 디렉터리를 선택합니다

데이터베이스 인증 옵션

모니터링 옵션의 경우 기존 설정과 동일하게 성능 인사이트 켜기를 활성화한다.

모니터링

☒ 성능 인사이트 켜기

성능 인사이트의 보존 기간 [정보](#)

7 일(프리 티어) ▼

AWS KMS 키 [정보](#)

(default) aws/rds ▼

계정

933598862266

KMS 키 ID

alias/aws/rds

⚠ 성능 개선 도우미를 활성화한 후에는 KMS 키를 변경할 수 없습니다.

☐ DevOps Guru 켜기 [정보](#)

DevOps Guru for RDS는 DB 인스턴스의 성능 이상을 자동으로 감지하고 권장 사항을 제공합니다.

▼ 추가 구성

항상된 모니터링

모니터링

☐ Enhanced 모니터링 활성화

Enhanced 모니터링 지표를 활성화하면 다른 프로세스 또는 스레드에서 CPU를 사용하는 방법을 확인하려는 경우에 유용합니다.

모니터링 옵션

추가 구성 영역에서 자동 백업 활성화 버튼을 비활성화한다. 만약 이미지 파일이 20기가를 넘어간다면 프리티어 구간을 넘어서 요금이 청구될 수 있다.

▼ 추가 구성

데이터베이스 옵션, 암호화 켜짐, 백업 꺼짐, 역추적 꺼짐, 유지 관리, CloudWatch Logs, 삭제 방지 꺼짐.

데이터베이스 옵션

초기 데이터베이스 이름 [정보](#)

데이터베이스 이름을 지정하지 않으면 Amazon RDS에서 데이터베이스를 생성하지 않습니다.

DB 파라미터 그룹 [정보](#)

default.postgres15

옵션 그룹 [정보](#)

default:postgres-15

백업

☐ 자동 백업을 활성화합니다.

데이터베이스의 특정 시점 스냅샷을 생성합니다.

암호화

☒ 암호화 활성화

지정된 인스턴스를 암호화하려면 이 옵션을 선택합니다. AWS Key Management Service 콘솔을 사용하여 마스터 키 ID와 별칭이 생성된 후 해당 항목이 목록에 표시됩니다. [정보](#)

추가 구성(1/2)

추가 구성 나머지 옵션은 별도로 수정하지 않는다.

AWS KMS 키 [정보](#)

(default) aws/rds

계정

933598862266

KMS 키 ID

alias/aws/rds

로그 내보내기

Amazon CloudWatch Logs로 게시할 로그 유형 선택

☐ PostgreSQL 로그

☐ 업그레이드 로그

IAM 역할

다음 서비스 연결 역할은 로그를 CloudWatch Logs로 게시하기 위해 사용됩니다.

RDS 서비스 연결 역할

유지 관리

자동 마이너 버전 업그레이드 [정보](#)

☒ 마이너 버전 자동 업그레이드 사용

마이너 버전 자동 업그레이드를 설정하면 새 마이너 버전이 출시되는 즉시 업그레이드됩니다. 자동 업그레이드는 데이터베이스의 유지 관리 기간 동안 수행됩니다.

유지 관리 기간 [정보](#)

보류 중인 수정 사항 또는 Amazon RDS가 데이터베이스에 적용한 유지 관리를 사용하려는 기간을 선택합니다.

☐ 기간 선택

☒ 기본 설정 없음

삭제 방지

☐ 삭제 방지 활성화

데이터베이스가 실수로 삭제되는 것을 방지합니다. 이 옵션을 활성화하면 데이터베이스를 삭제할 수 없습니다.

추가 구성(2/2)

가장 하단 영역은 월별 추정 요금이 나타난다. 프리티어를 사용한다면 어떤 범위까지 무료로 사용할 수 있는지 확인할 수 있다. 모든 영역을 확인했다면 데이터베이스 생성 버튼을 클릭해 데이터베이스를 생성한다. 데이터베이스를 생성하는 데 약 2~3분 정도 소요된다.

된다.

Estimated Monthly costs

DB 인스턴스	20.44 USD
스토리지	2.62 USD
합계	23.06 USD

This billing estimate is based on on-demand usage as described in [Amazon RDS 요금](#). Estimate does not include costs for backup storage, IOs (if applicable), or data transfer.

다음을 사용하여 DB 인스턴스의 월별 청구액을 추산할 수 있습니다. [AWS 월 사용량 계산기](#).

월별 추정 요금

Amazon RDS 프리 티어는 12개월 동안 사용할 수 있습니다. 매월 프리 티어를 통해 아래 나열된 Amazon RDS 리소스를 무료로 사용할 수 있습니다.

- 단일 AZ db.t2.micro, db.t3.micro 또는 db.t4g.micro 인스턴스에서 Amazon RDS를 750시간 사용.
- 20GB의 범용 스토리지(SSD).
- 20GB의 자동 백업 스토리지 및 사용자가 시작한 모든 DB 스냅샷.

[Learn more about AWS Free Tier.](#)

When your free usage expires or if your application use exceeds the free usage tiers, you simply pay standard, pay-as-you-go service rates as described in the [Amazon RDS Pricing page](#).

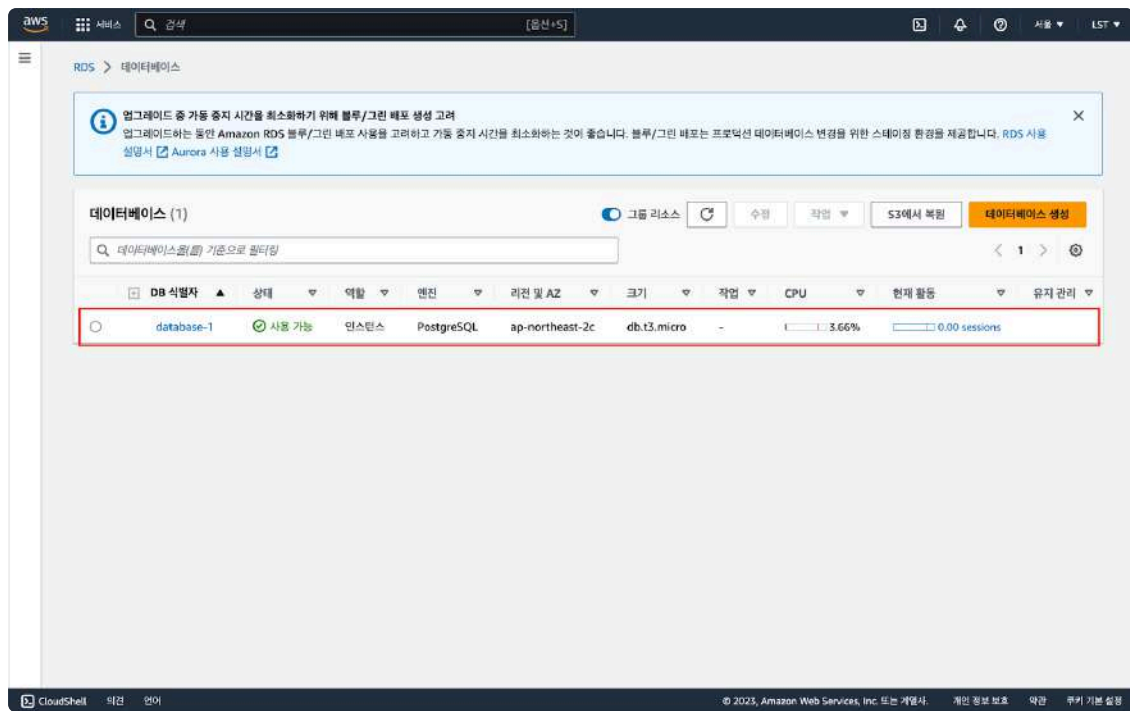
❗ 귀하는 AWS 서비스와 함께 사용하는 타사 제품 또는 서비스 일체에 대해 필요한 모든 권리를 보유할 책임이 있습니다.

취소

데이터베이스 생성

월별 추정 요금

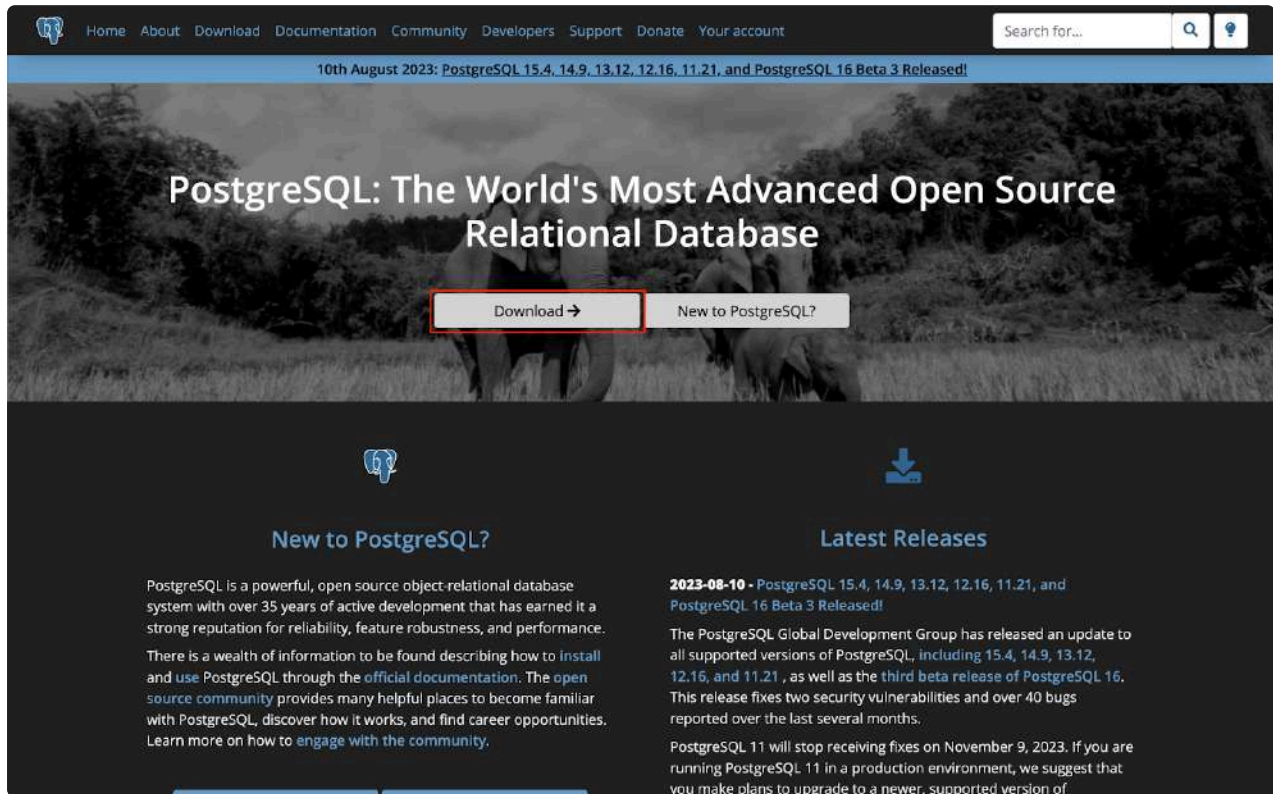
데이터베이스가 올바르게 생성된다면 아래 그림과 같이 데이터베이스가 추가된것을 확인할 수 있다.



데이터베이스 콘솔

5.5. PostgreSQL 설치

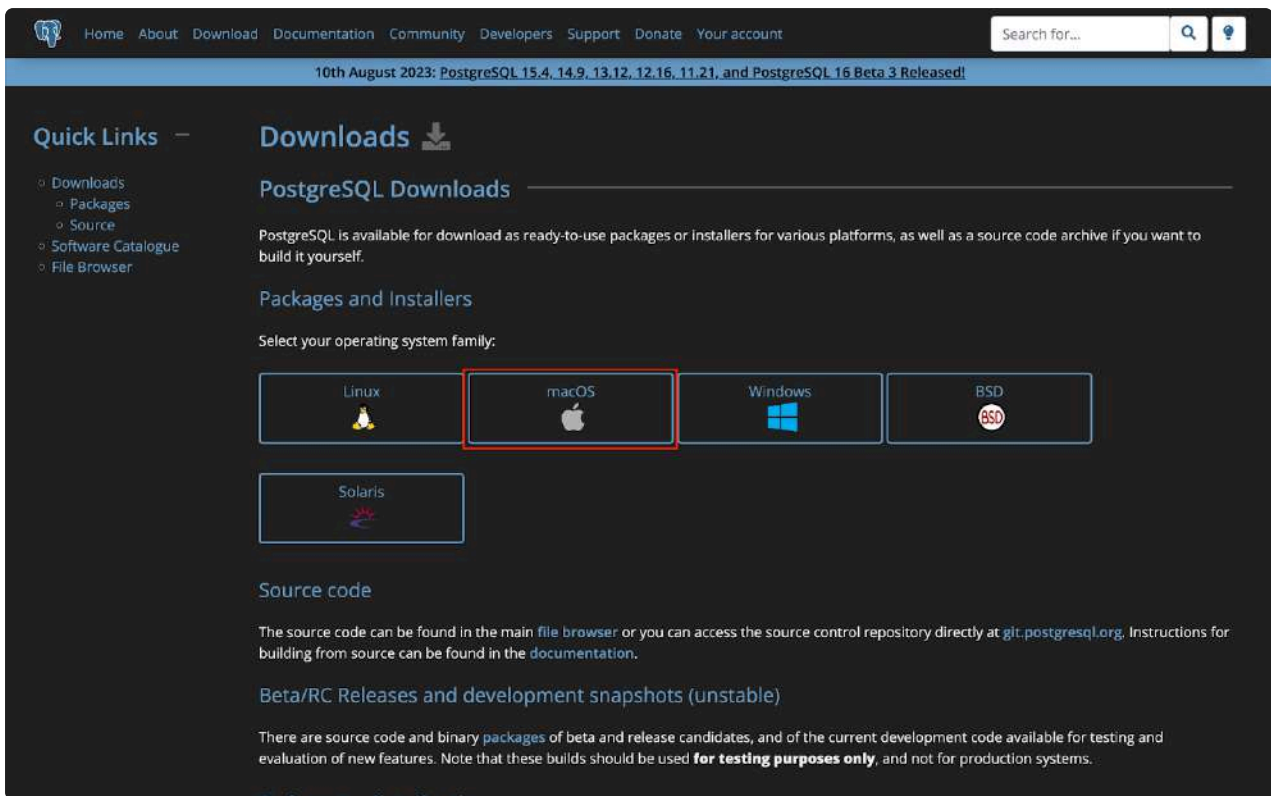
@노건호 postgresQL 이외에도 있을테니...몇가지 종류를 언급하고 이것을 선택한 이유로 추가해 줘요.
데이터베이스에 연결하기 위해 postgresQL 프로그램을 설치한다. [링크](#)에 접속해 중앙에 있는 Download 버튼을 클릭한다.



PostgreSQL 홈페이지

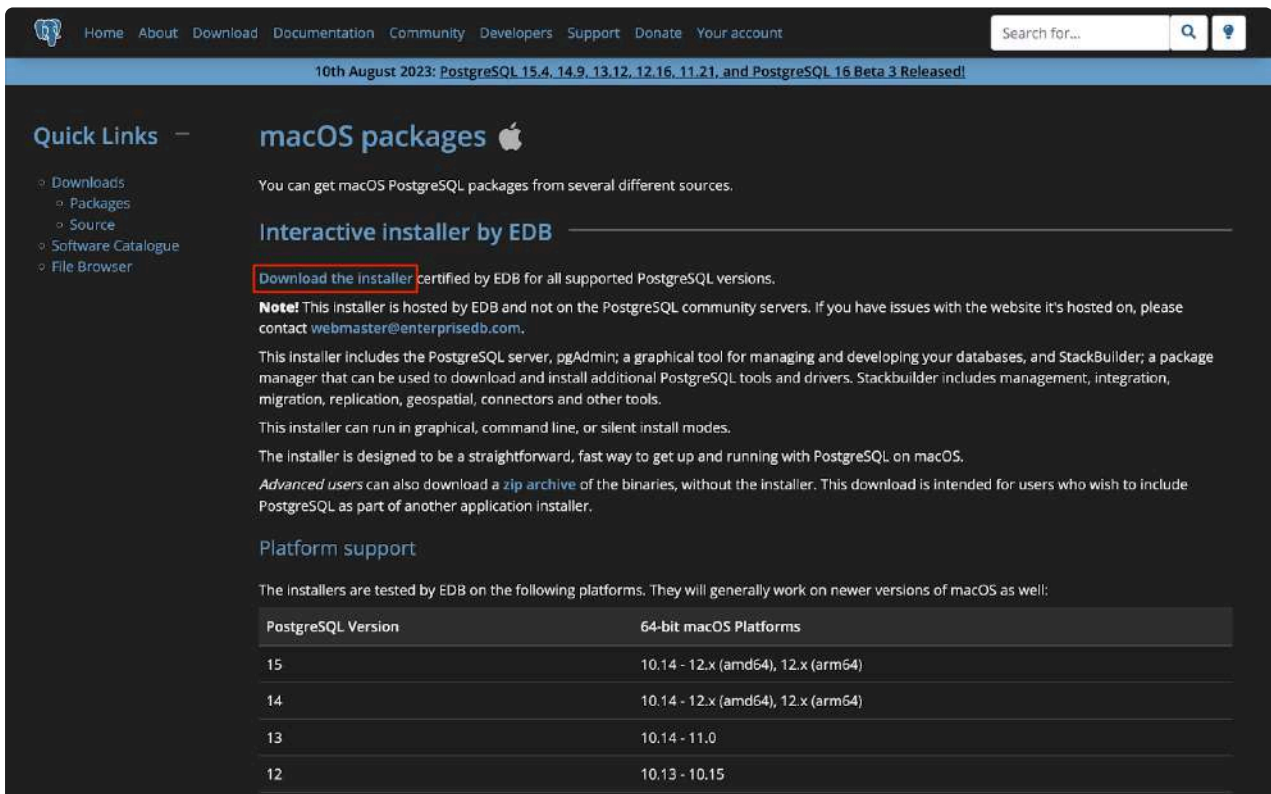
Downloads 화면에서 원하는 OS를 선택한다. 본 문서는 macOS를 선택해 진행한다.

여기서는 iMac(?) 에서 작업을 하고 있으므로 macOS를 선택한다.



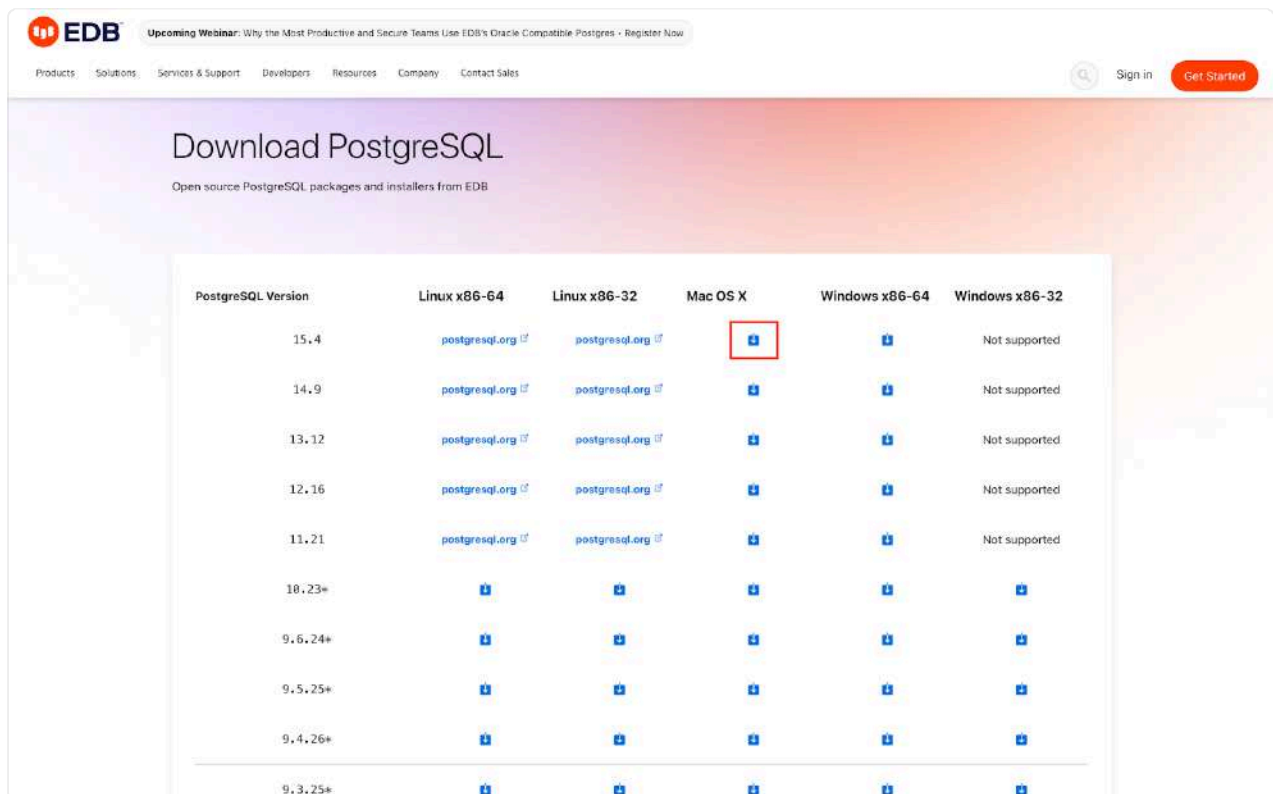
PostgreSQL 설치 OS 선택

macOS packages 화면에서 상단의 Download the installer 화면을 클릭한다.



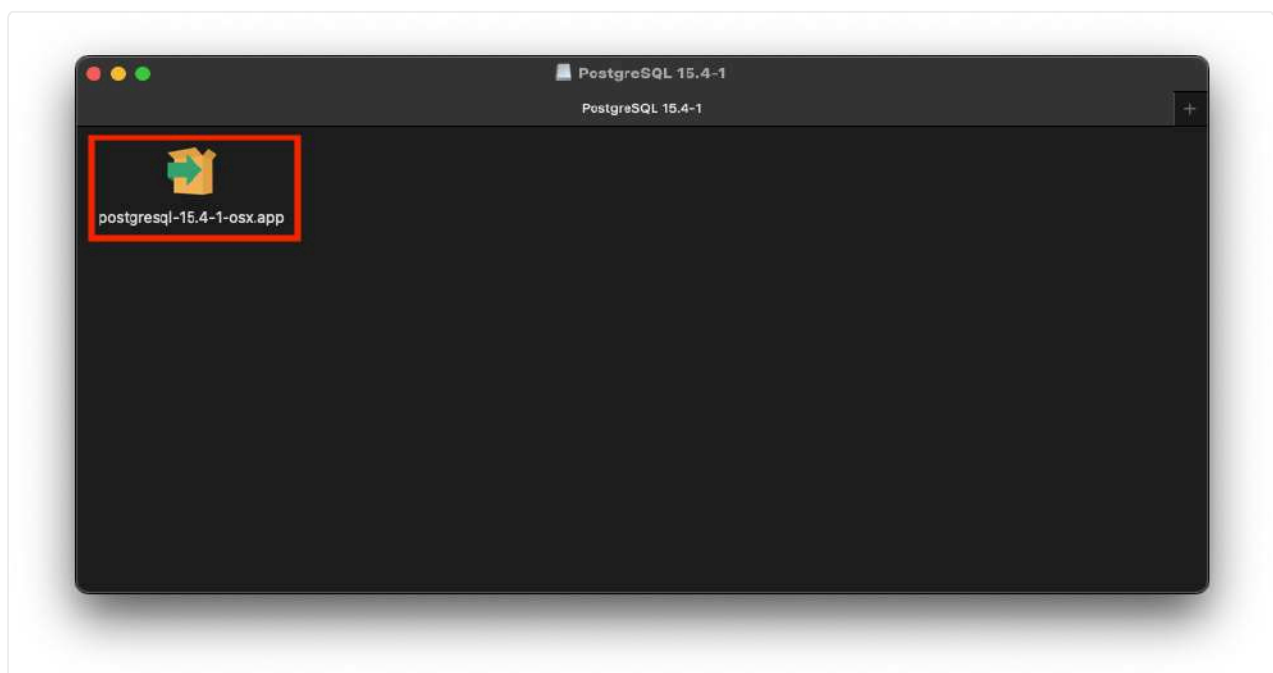
PostgreSQL macOS 다운로드 선택

아래 화면에서 원하는 버전의 설치파일을 다운로드한다. 이 문서에서는 15.4 버전의 Mac OS X 버전을 설치한다.



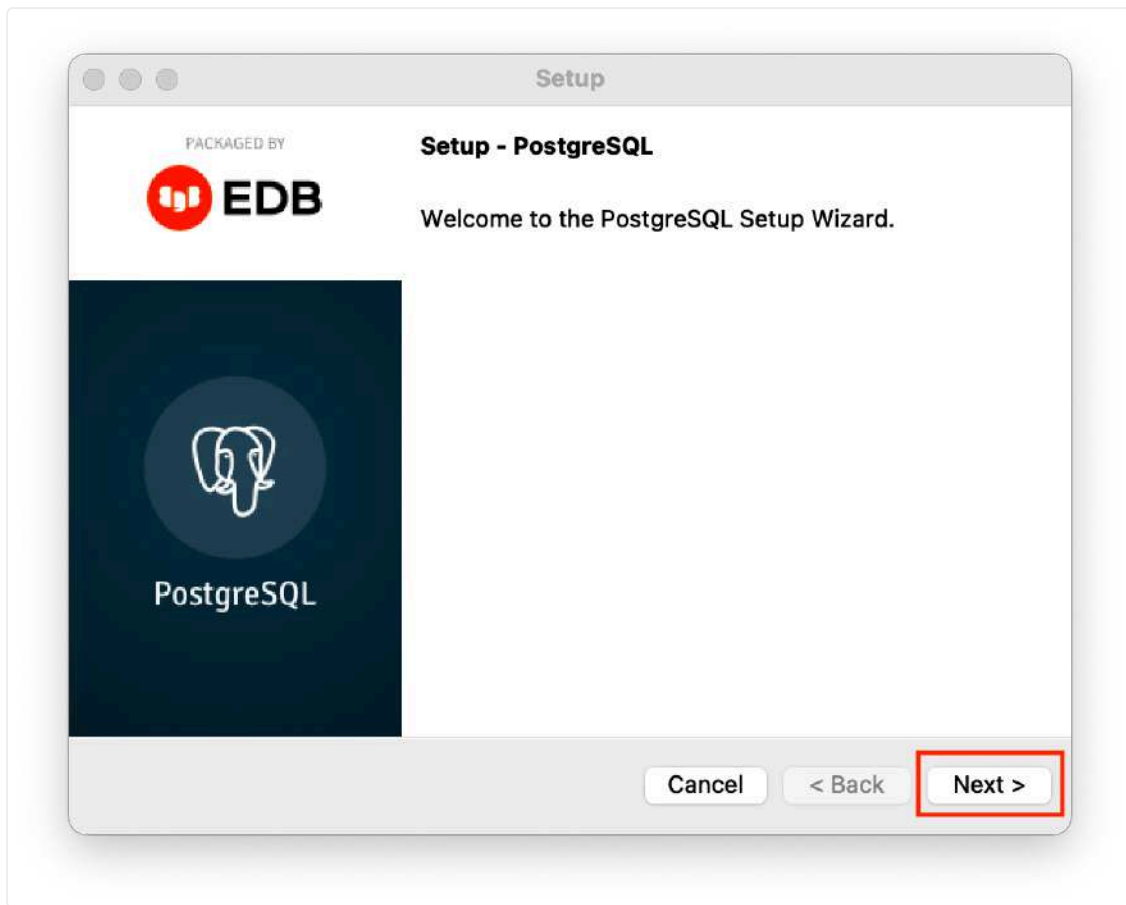
PostgreSQL 설치버전 선택 화면

설치파일 실행 후 아래 그림에서 postgresql-15.4.1-osx.app 어플리케이션을 두번 클릭해 어플리케이션 설치를 시작한다.



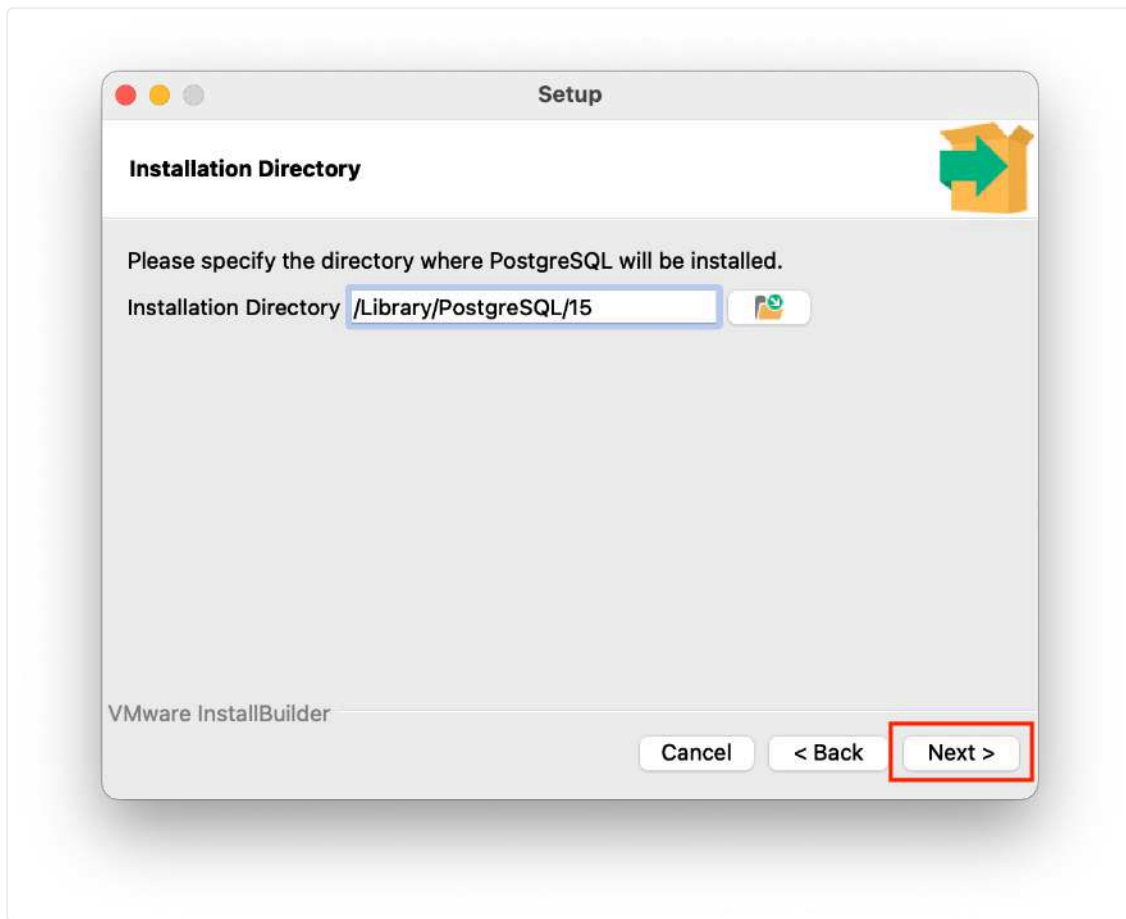
PostgreSQL 설치파일

Setup 화면에서 Next 버튼을 클릭한다.



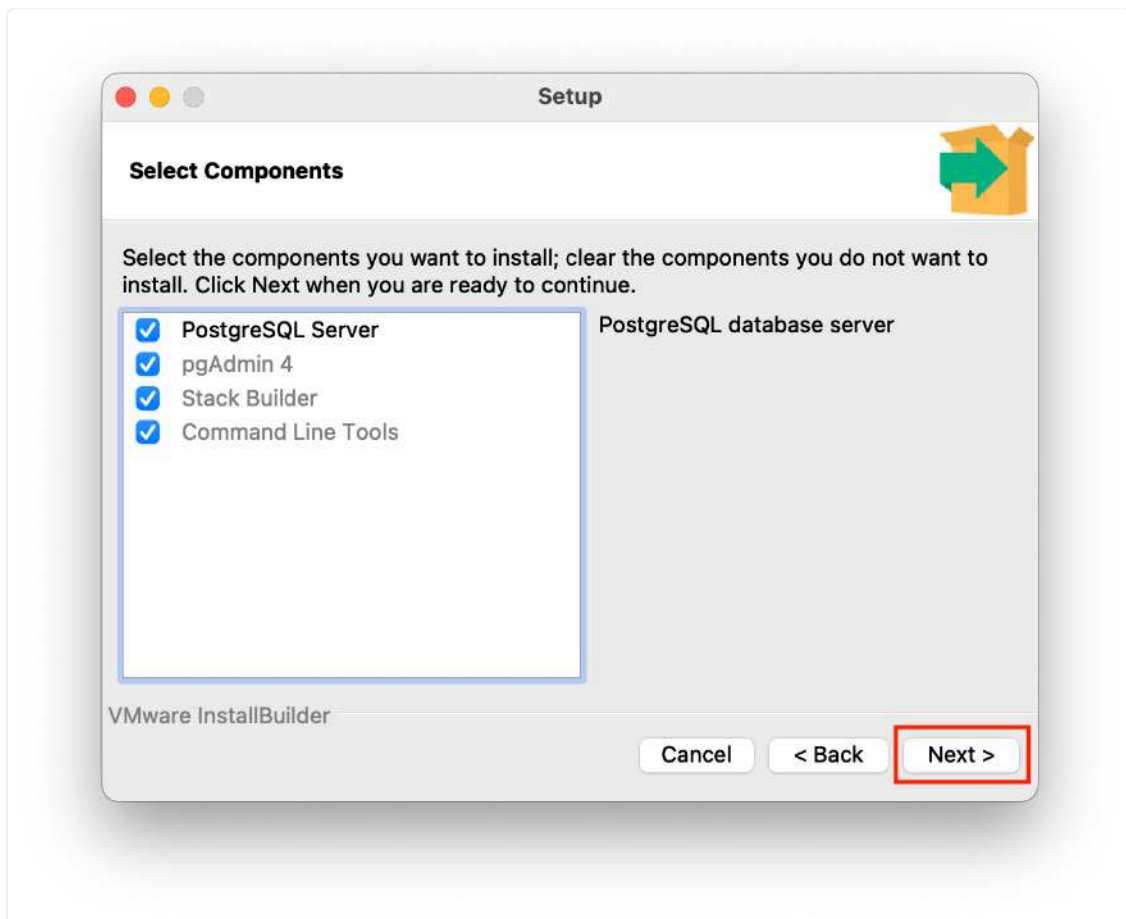
PostgreSQL Setup

Installation Directory 화면에서 설치위치를 지정 후 Next 버튼을 클릭한다.



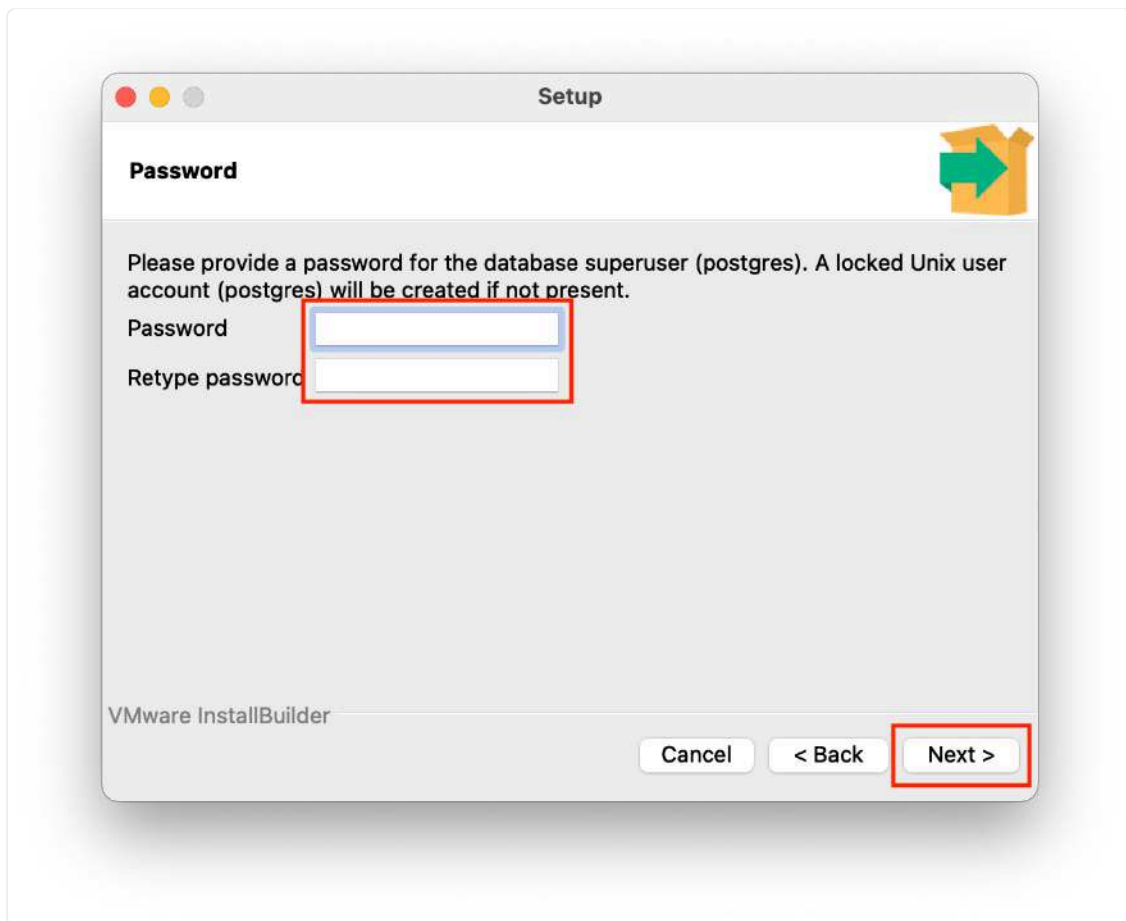
설치위치 설정

설치 컴포넌트를 선택 후 Next 버튼을 클릭한다. 본 문서는 전체 파일 선택 후 설치를 진행한다.



설치 Components 선택

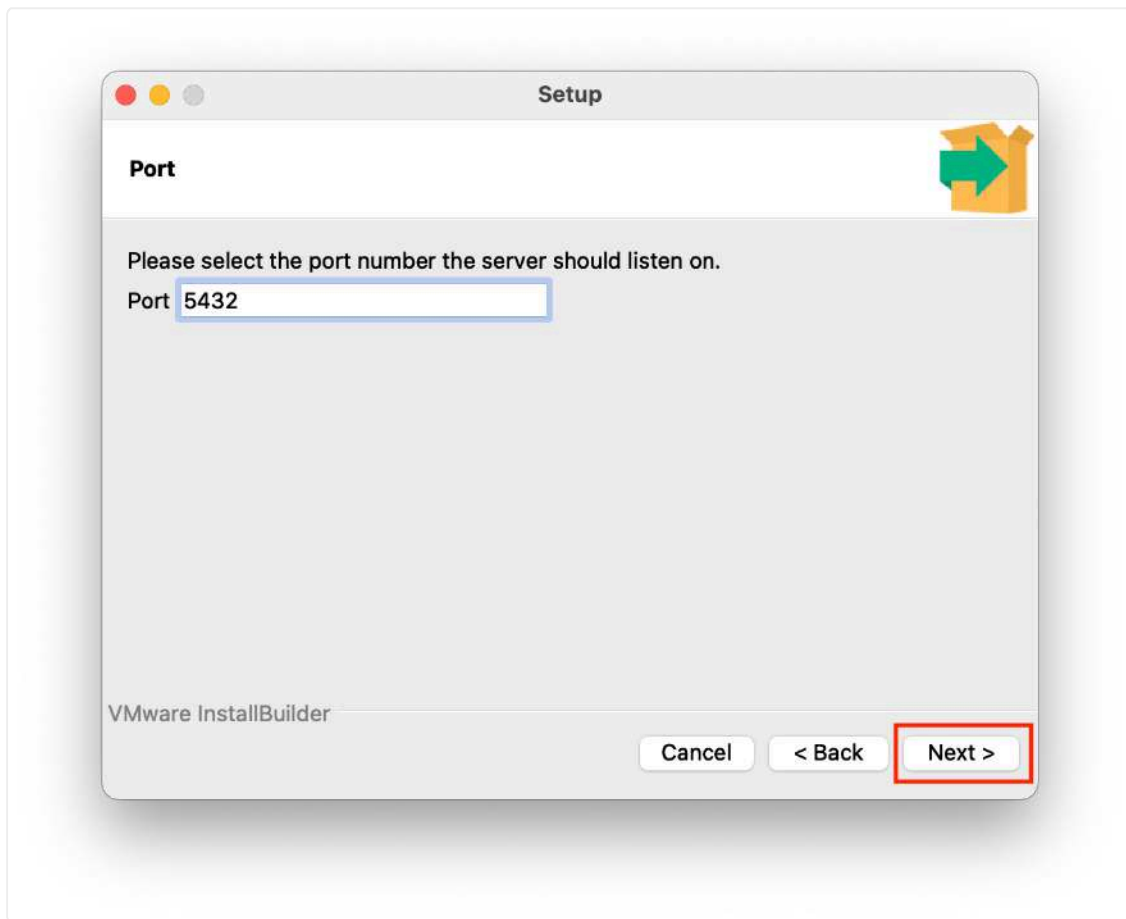
Password 화면에서 데이터베이스 슈퍼유저의 비밀번호를 입력 후 Next 버튼을 클릭한다.



비밀번호 입력

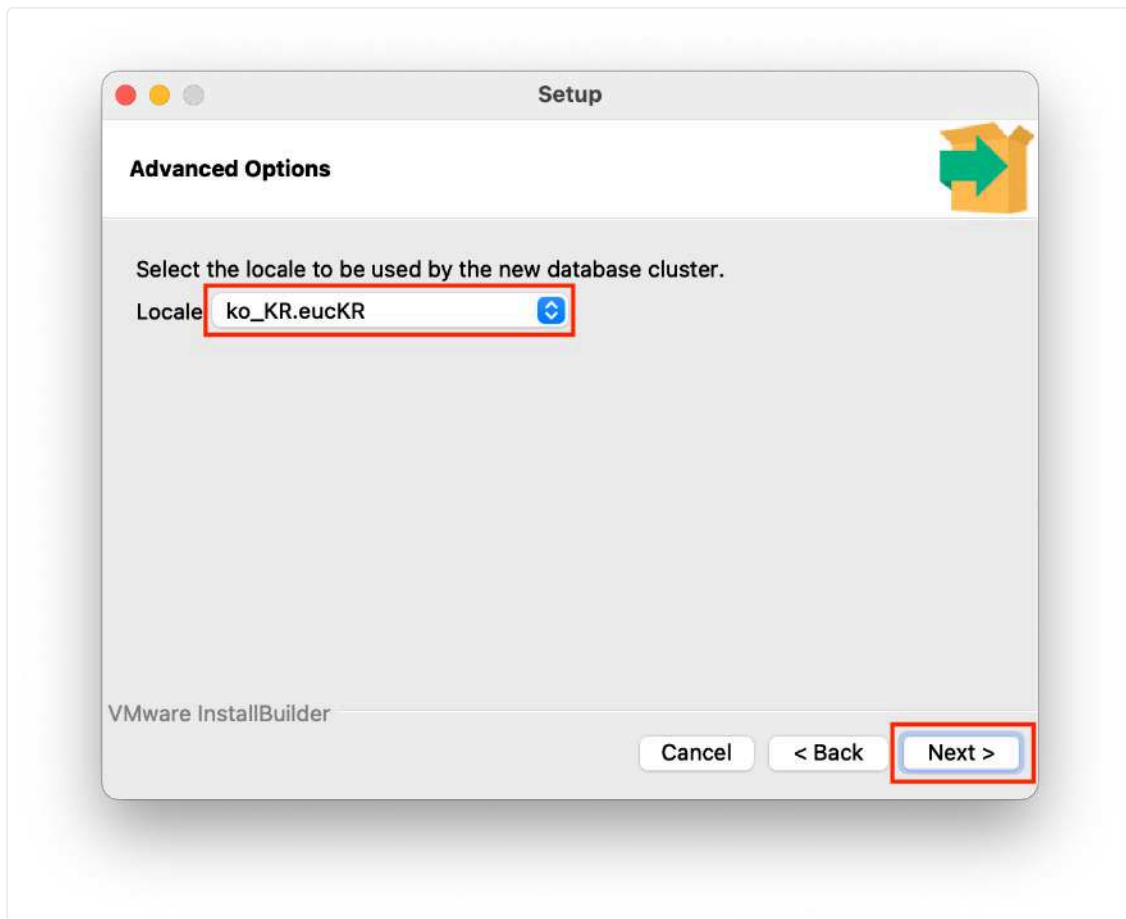
Port 입력 화면에서 포트번호를 5432로 입력 후 Next 버튼을 클릭한다.

@노건호 앞의 포트 번호가 여기서 사용되는 건가요? 포트 번호의 의미를 설명해 주면 좋겠어요.



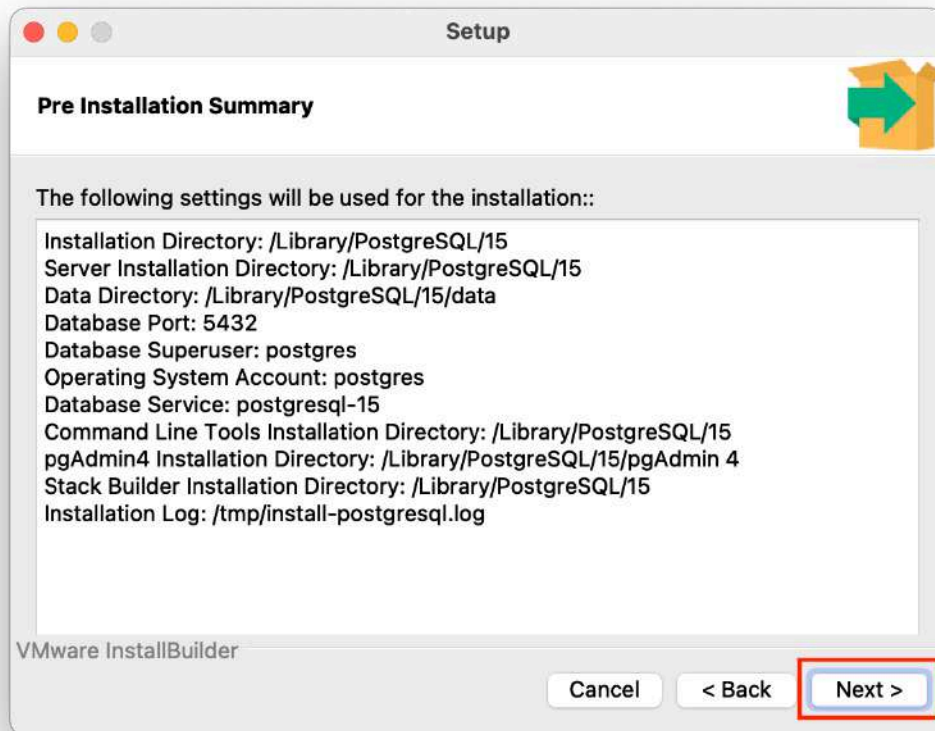
포트번호 입력

데이터베이스 클러스터 위치선택 화면에서 데이터베이스 클러스터가 설치될 위치를 ko_KR.eucKR로 설정 후 Next 버튼을 클릭한다.



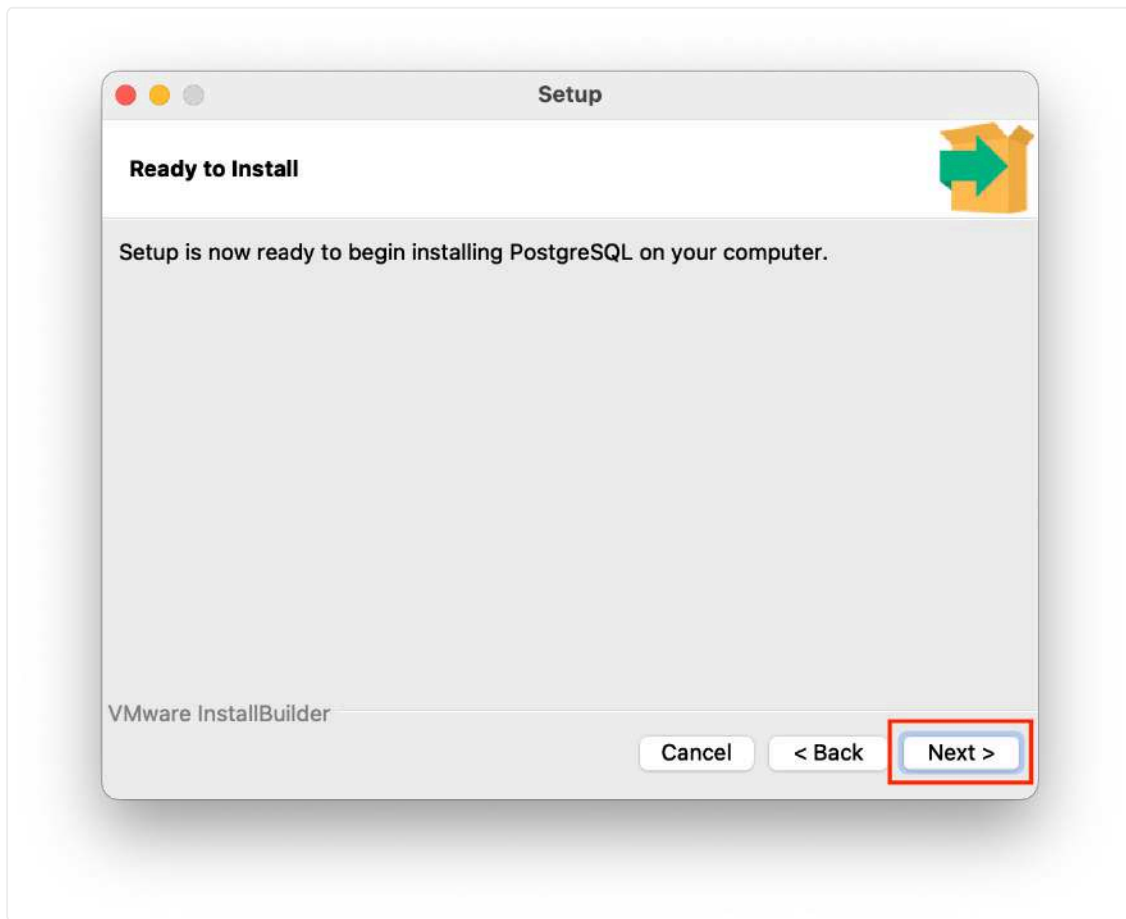
데이터베이스 위치 설정

설치 관련된 설정이 끝나면 설치정보 요약 화면이 나타난다. 설치정보가 올바르게 되었다면 Next 버튼을 클릭해 설치를 진행한다.



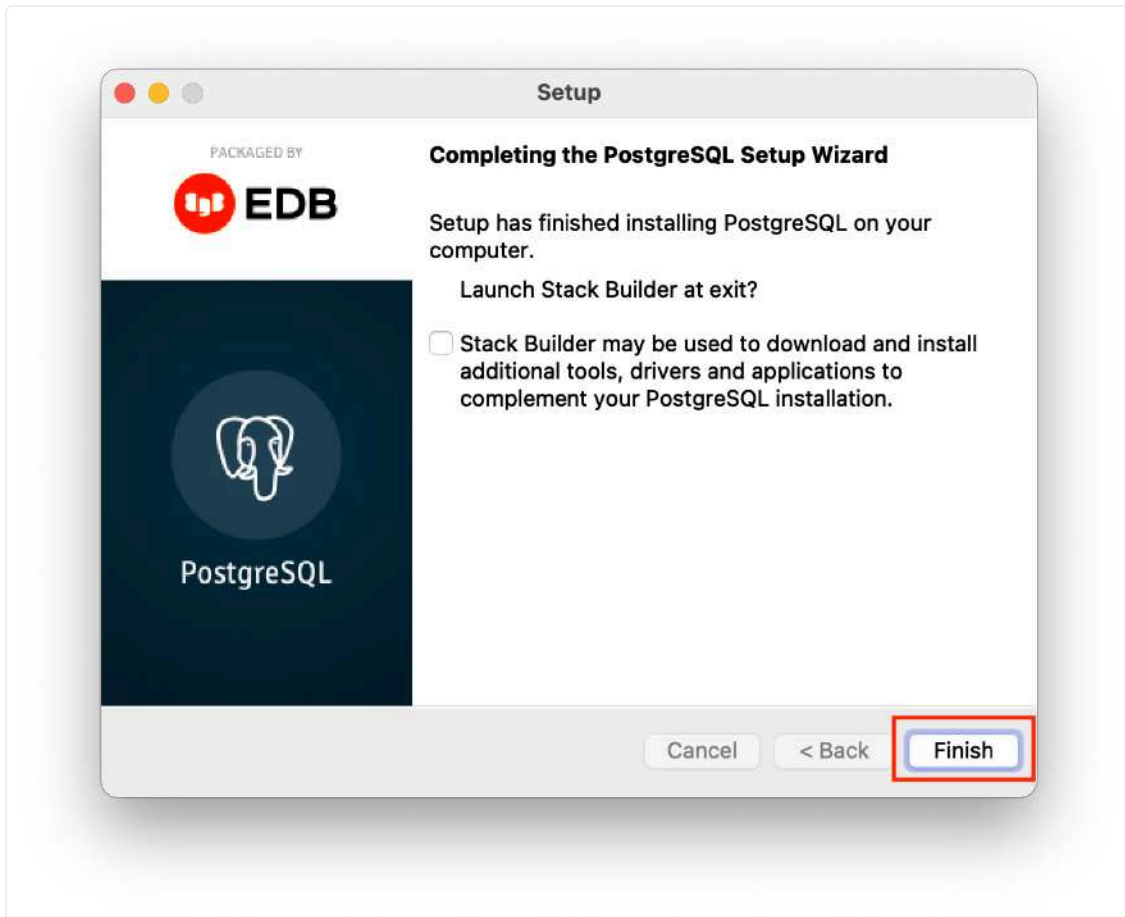
설치정보 요약

설치준비 화면이 나타나면 Next 버튼을 클릭해 설치를 진행한다.



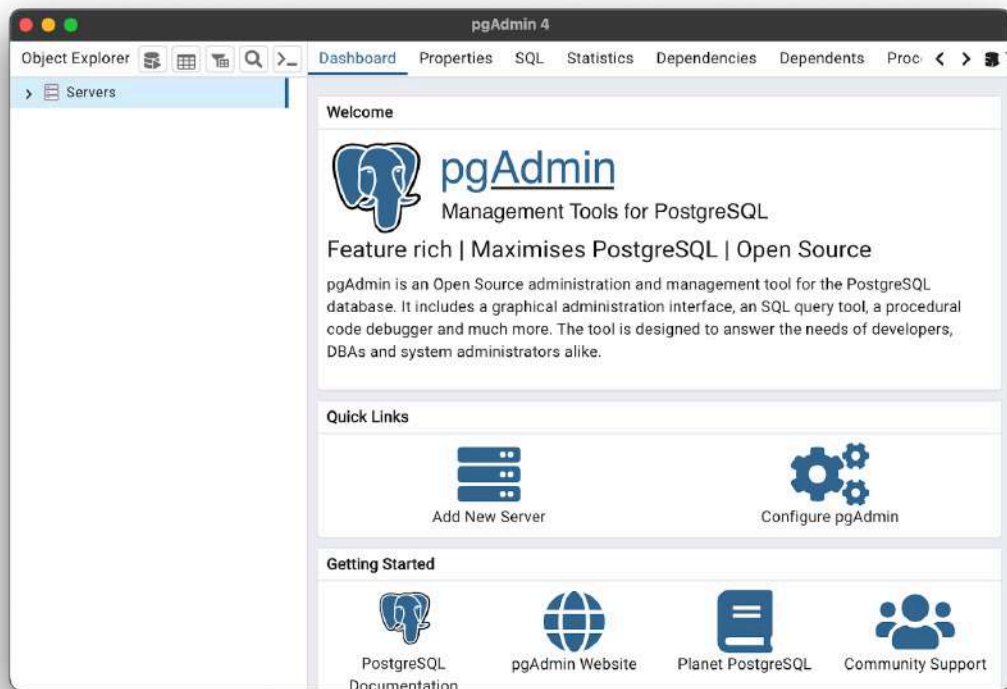
설치설정 완료

설치가 완료되면 아래와 같은 화면이 나타난다. Finish 버튼을 클릭해 설치를 완료한다.



설치완료 화면

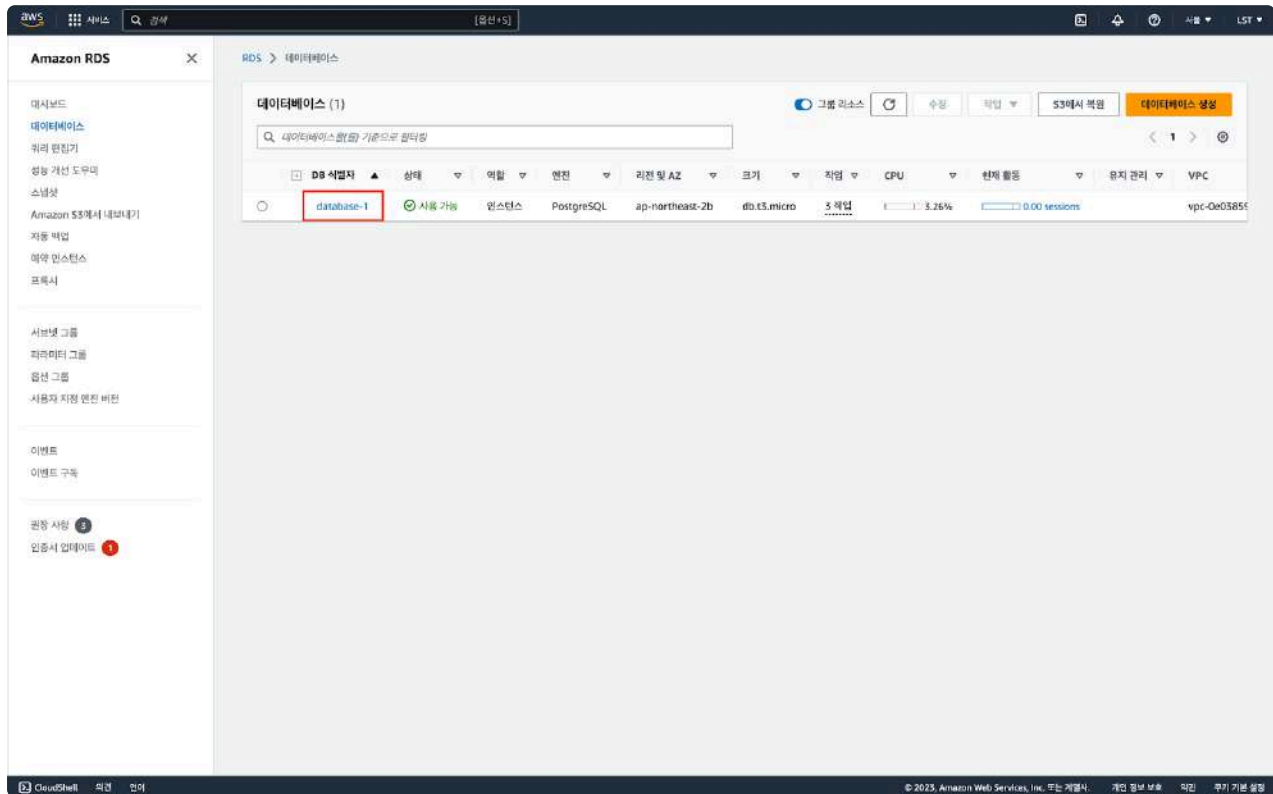
이후 설치가 완료되면 컴퓨터에 설치된 pgAdmin 4 를 클릭해 실행한다.



pgAdmin 콘솔

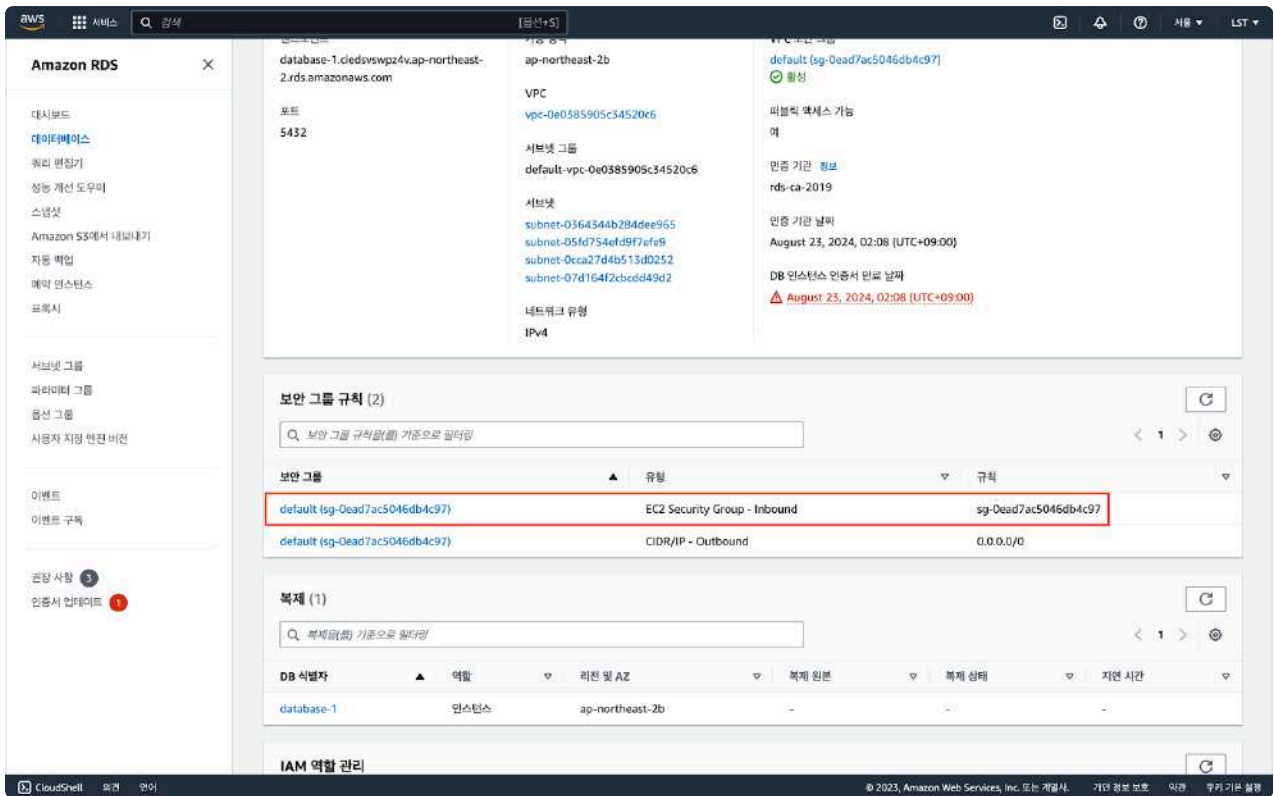
5.6. AWS 데이터베이스 연결

@노건호 앞의 작업에 대한 정리를 한번 하고(예를 들면, 지금까지 데이터 베이스를 사용할 준비 작업을 마쳤다. 이제 ***을 위해서 AWS와 연결하는 방법을 설명한다.와 같이) 넘어가면 좋겠네요.
AWS에 데이터베이스를 연결하기 위해 Inbound 관련 접근경로를 추가한다. RDS 콘솔로 이동해 기존에 생성했던 데이터베이스를 클릭한다.



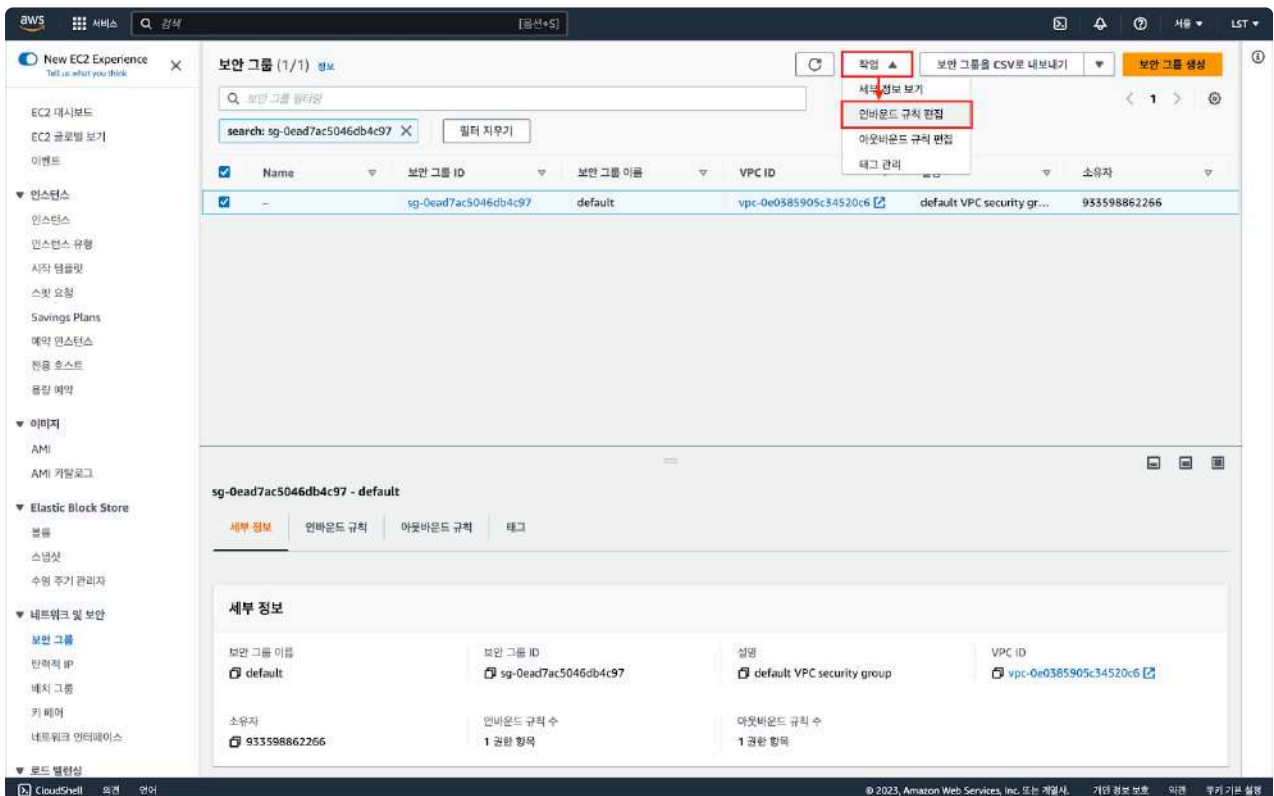
데이터베이스 콘솔

이후 스크롤을 아래로 내려 보안 그룹 규칙 영역으로 이동 후 유형이 Inbound인 보안 그룹을 클릭한다.



보안그룹 규칙

보안 그룹 화면에서 우측 상단의 작업 → 인바운드 규칙 편집을 클릭한다.

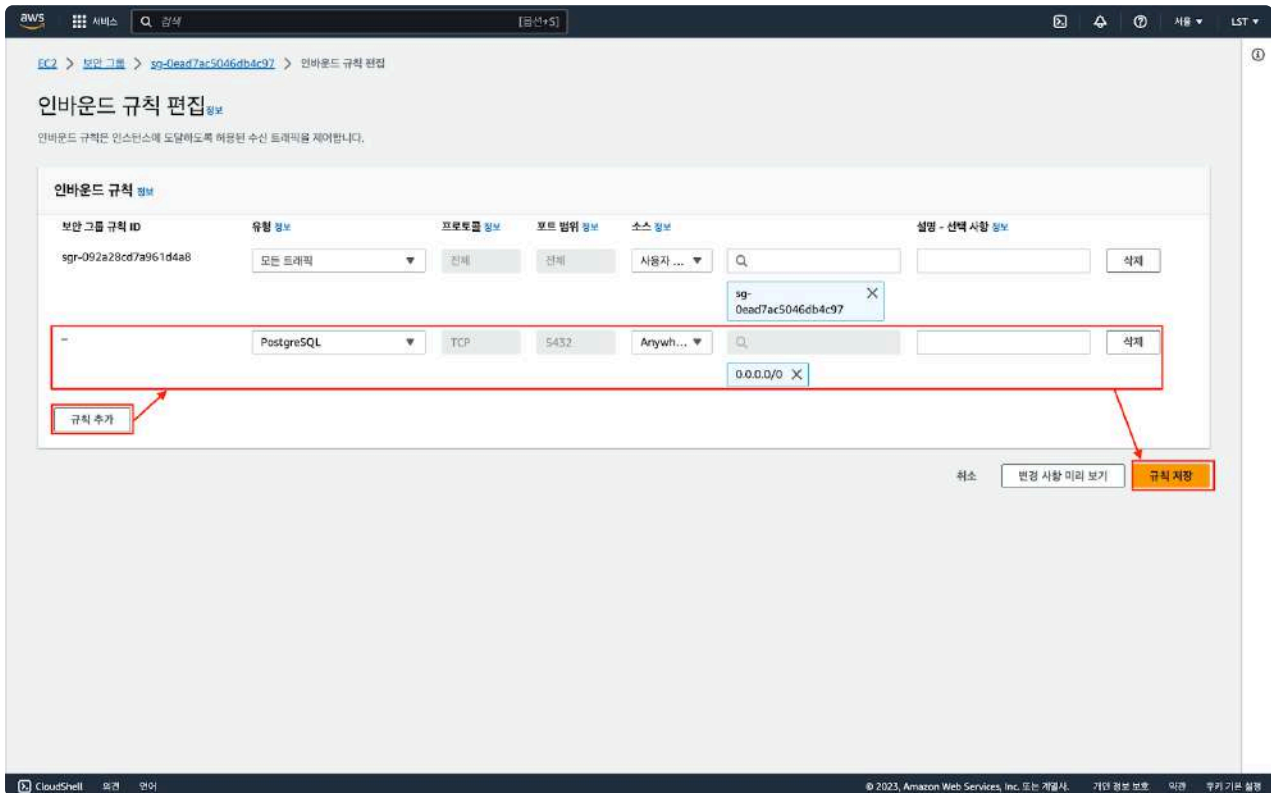


인바운드 규칙 편집 이동

왼쪽의 규칙 추가 버튼을 클릭하고, 아래 정보를 입력 후 규칙 저장 버튼을 클릭한다. 이 과정을 모두 거친 후 PostgreSQL에서 AWS 데이터베이스에 접근할 수 있다.

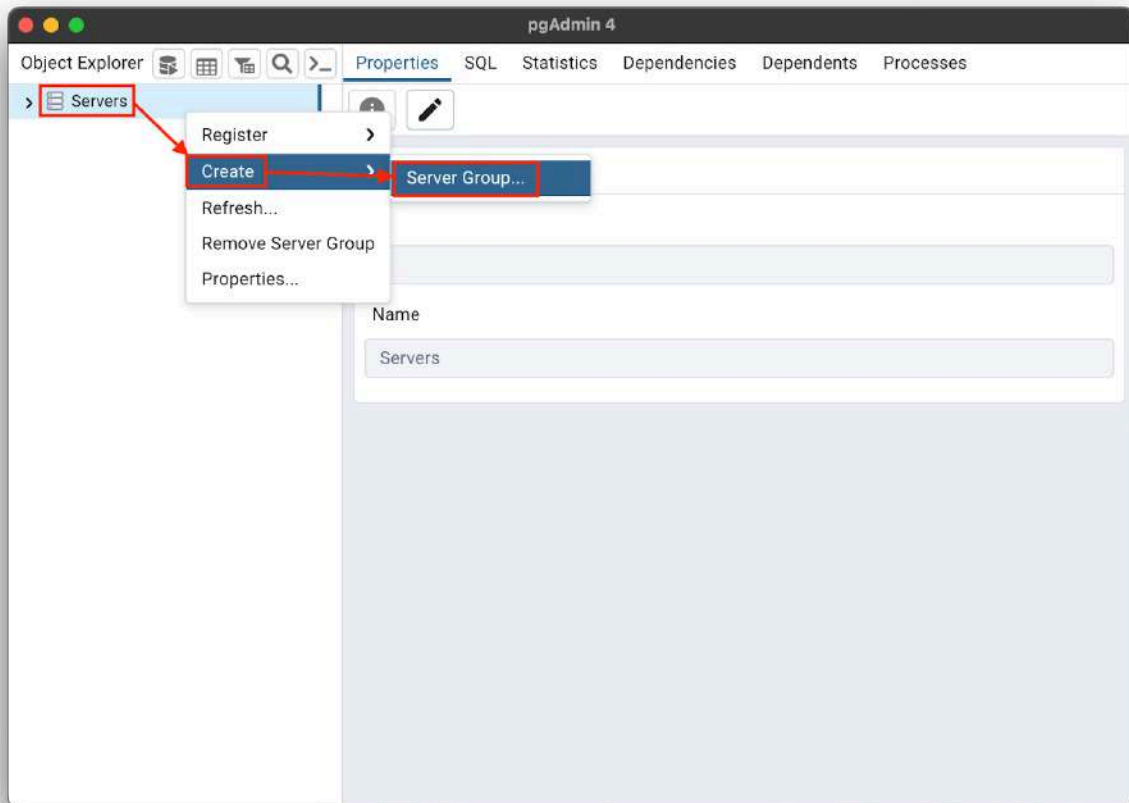
유형: PostgreSQL

소스: Anywhere-IPv4



인바운드 규칙 편집

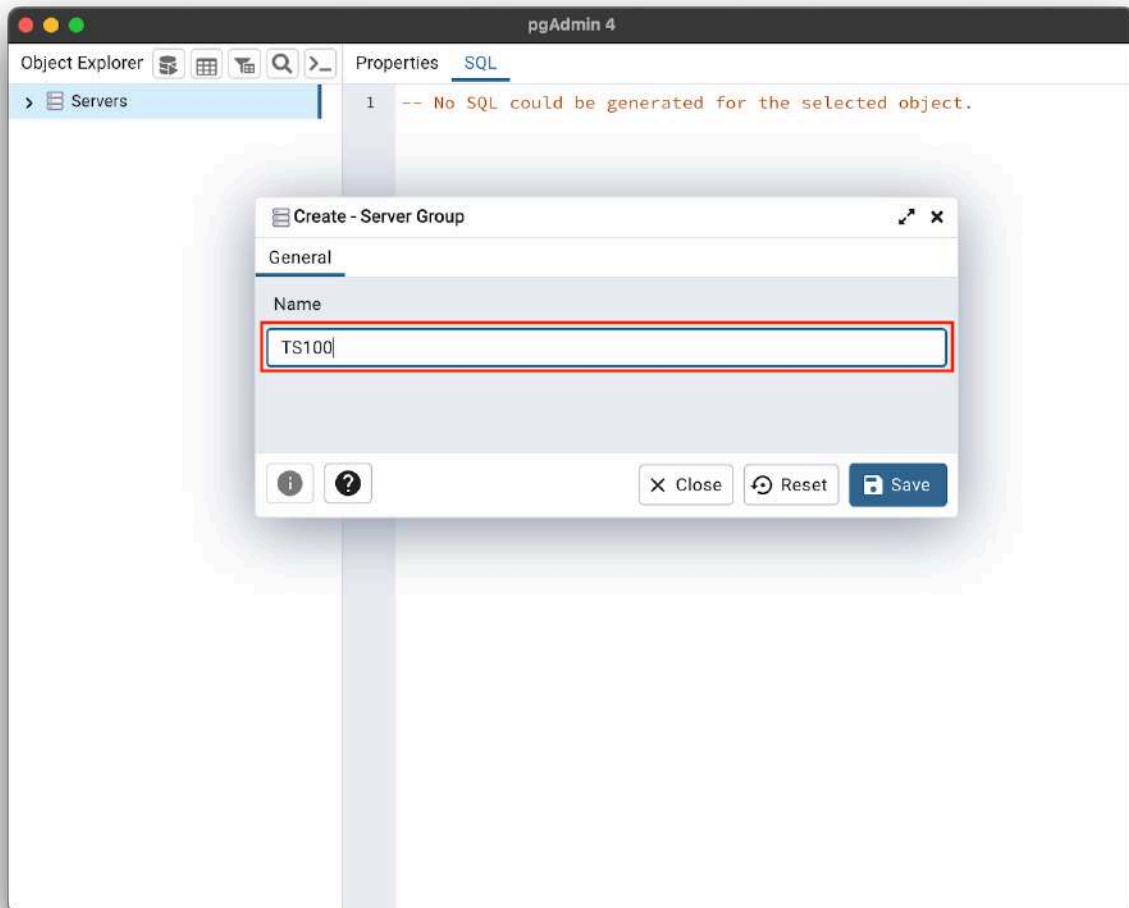
다시 pgAdmin 콘솔로 돌아와 Servers 우클릭 → Create → Server Group 선택해 새로운 서버 그룹을 생성한다.



Server Group 생성방법

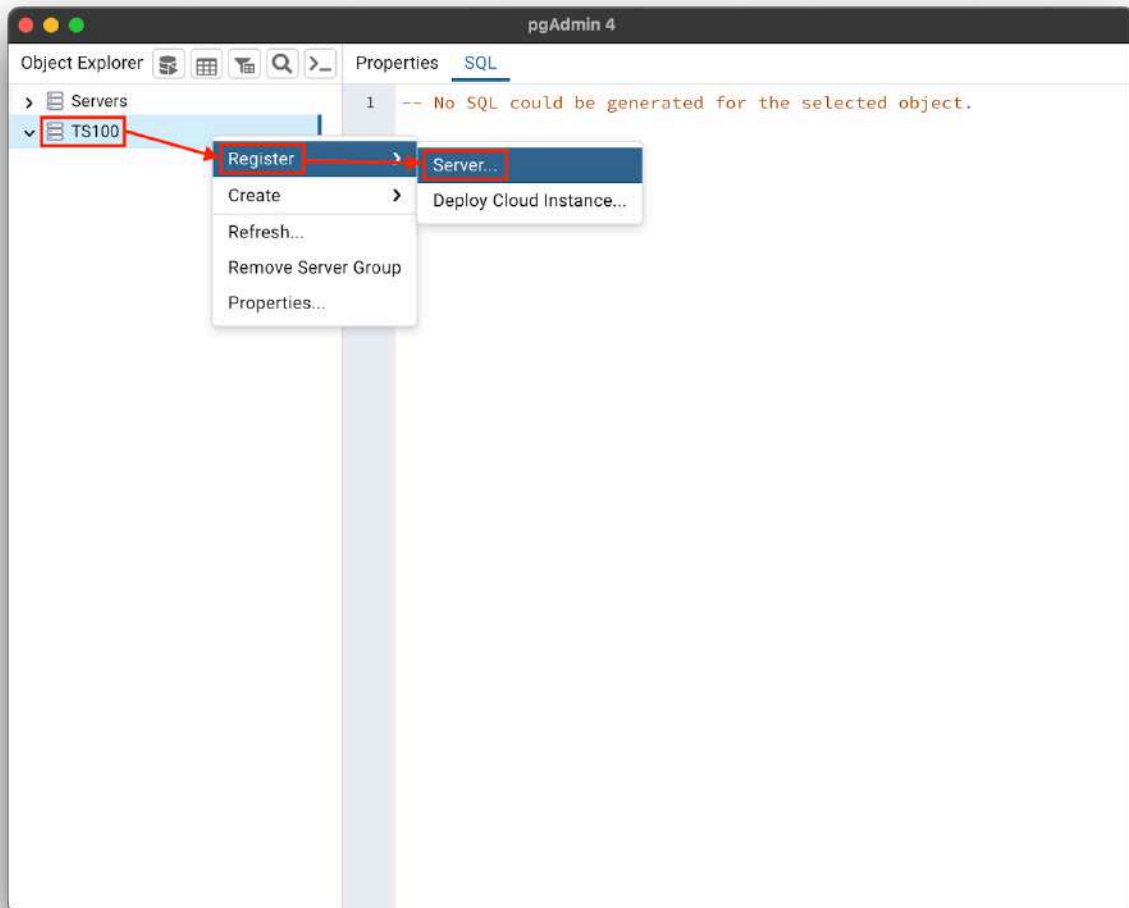
Server Group을 원하는 이름으로 입력 후 Save 버튼을 클릭한다. ~~본 문서는~~ TS100으로 입력한다.

본 문서 대신.
"여기서는" 혹은 조금 다른 표현이 적절해 보
임



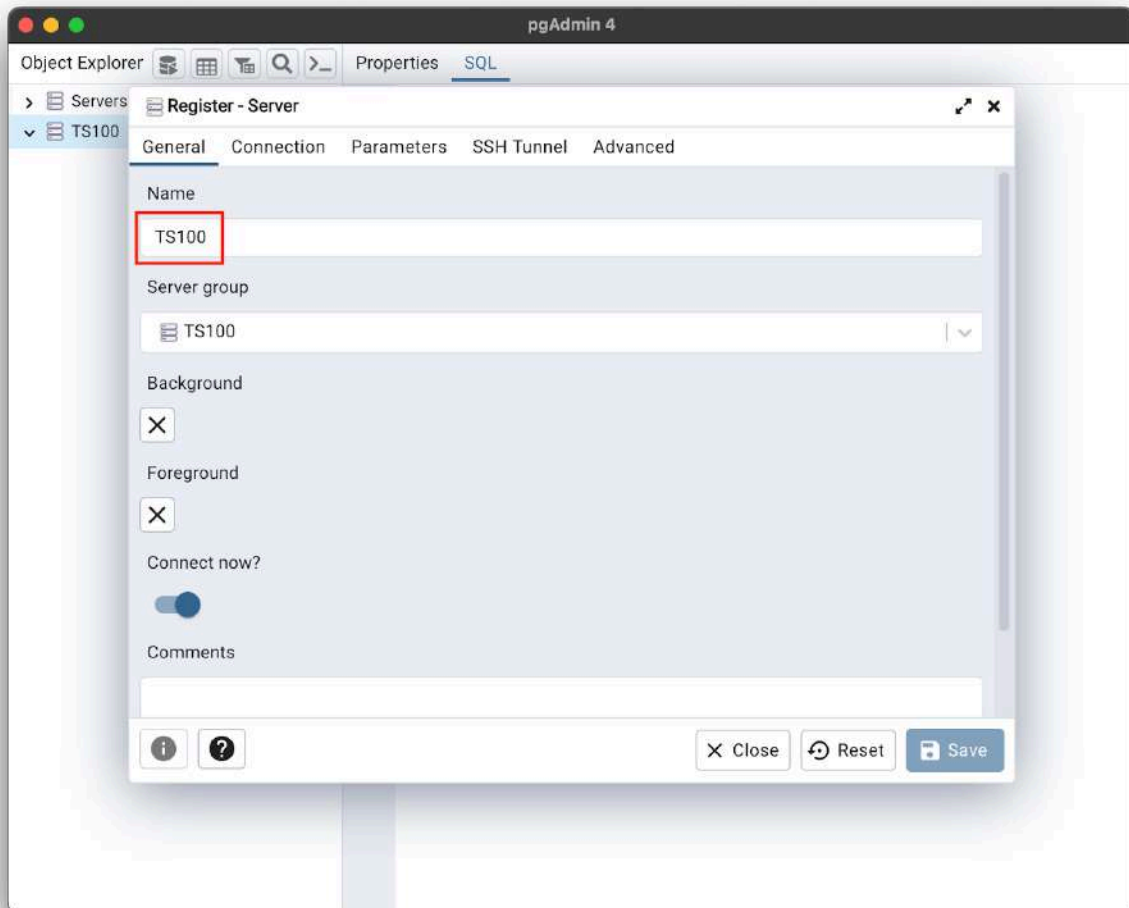
Server Group 생성 팝업

TS100 우클릭 → Register → Server 순서로 버튼을 클릭해 서버를 생성한다.



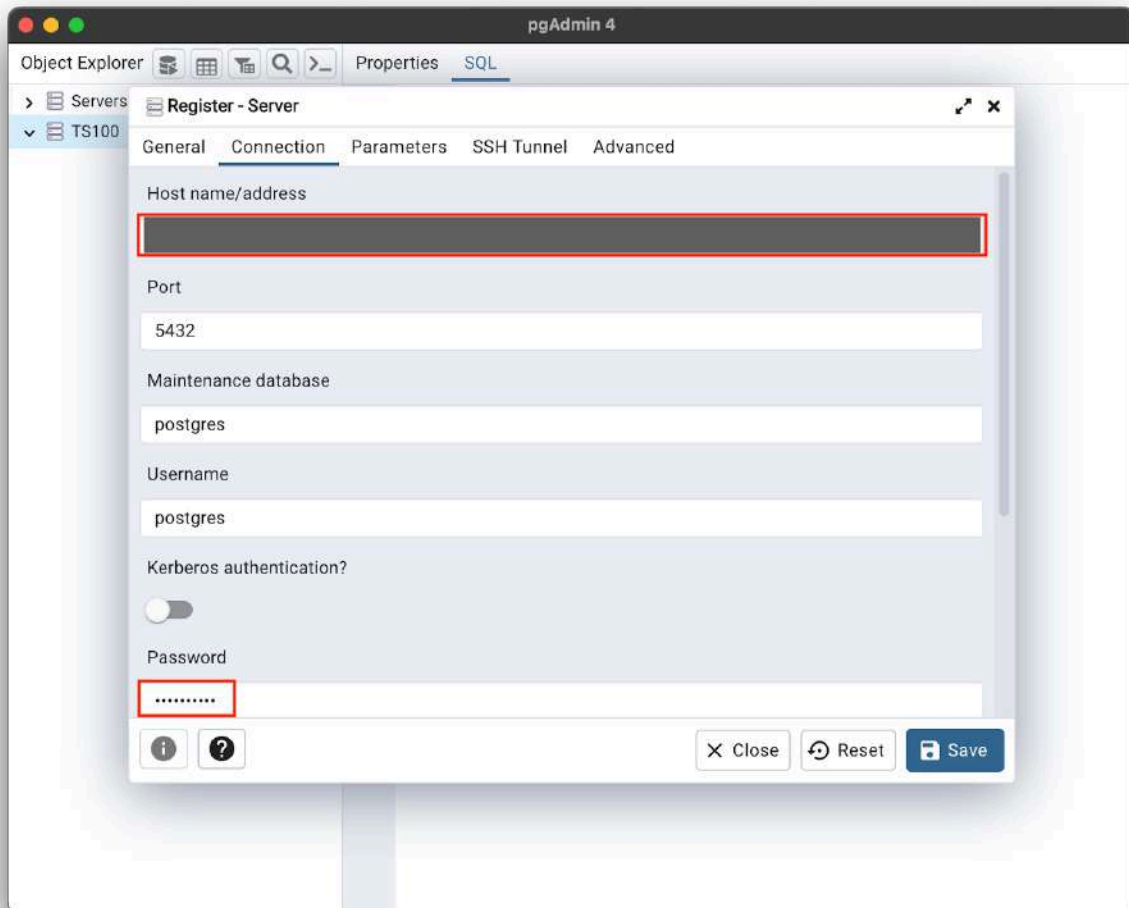
Server 연결 방법

서버 등록 팝업이 나타나면 Name을 입력한다. 본 문서는 TS100으로 입력한다.



서버 등록 - 이름 설정

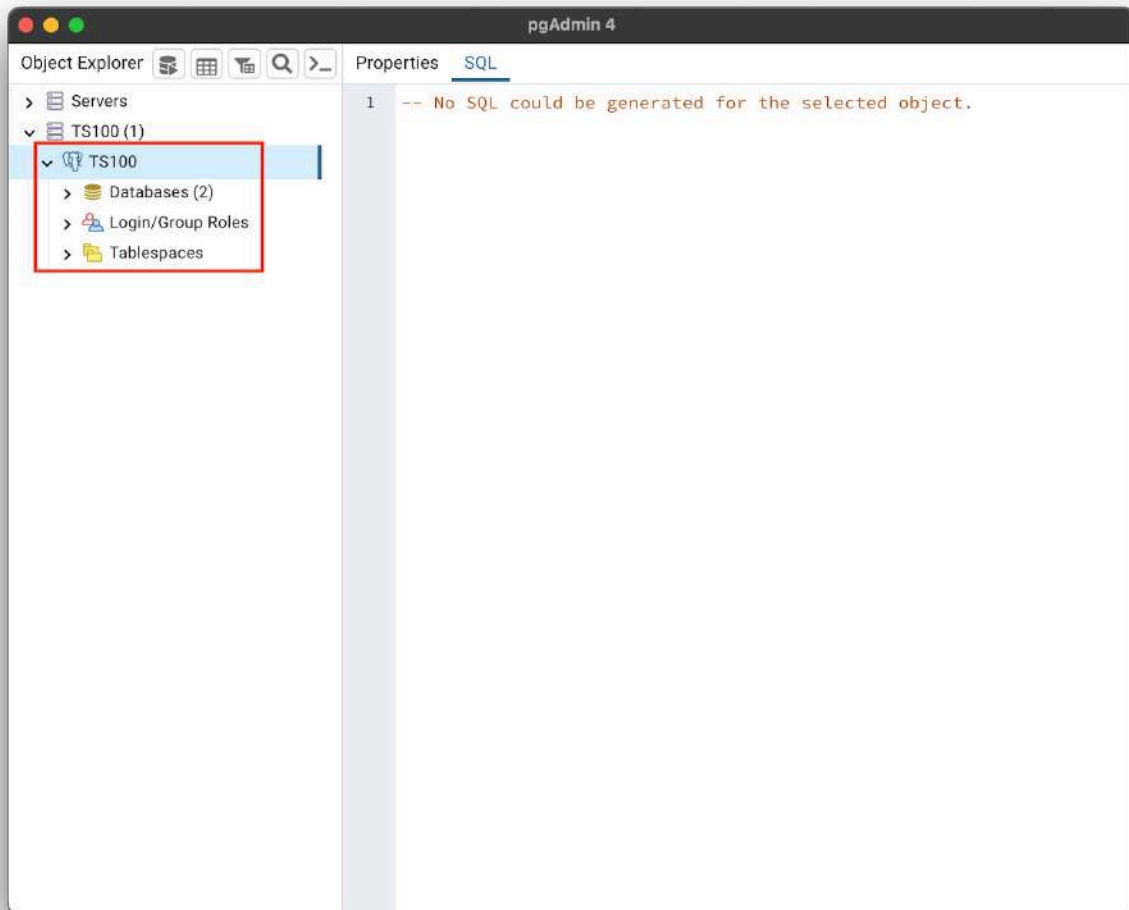
이후 Connection 탭으로 이동하여 Host name/address 영역에 AWS에서 만들었던 엔드포인트를 입력하고, password를 입력 후 Save 버튼을 클릭하여 RDS와 연결한다.



서버 등록 - 연결정보 입력

서버와 연결이 완료되면 ~~아래 그림과 같이~~ TS100이 추가된것을 확인할 수 있다.

Object Explorer 창(?)에



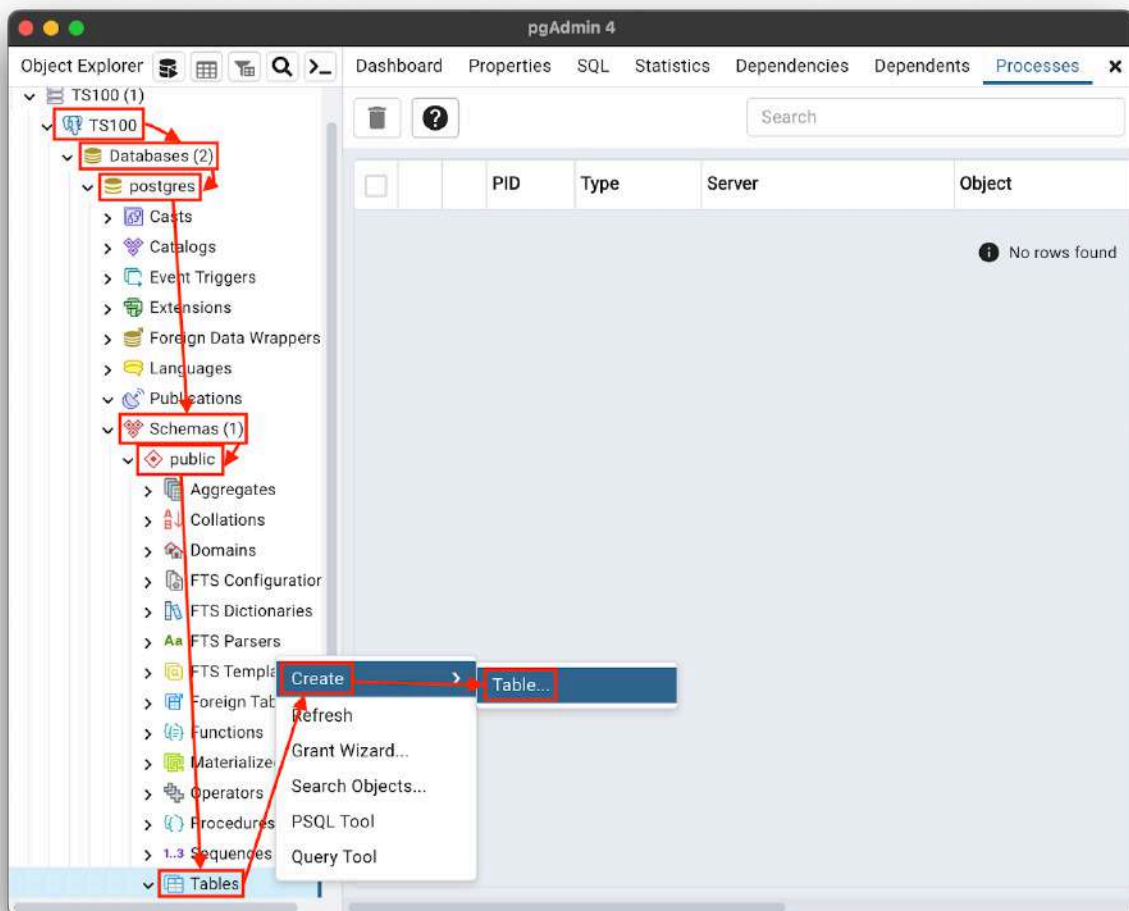
PostgreSQL AWS 데이터베이스 연결

5.7. AWS 데이터베이스 테이블 생성

스마트폰에서 측정된 온도, 날짜 정보를 클라우드에 저장하기 위해 클라우드 데이터베이스인 PostgreSQL 데이터베이스에 ~~테이블 추가 과정을 진행한다.~~

아래 과정을 거쳐 테이블을 추가한다.

TS100 → Databases → postgres → Schemas → public → Tables 영역 이동 후 Tables 우클릭 → Create → Table 을 순서대로 클릭한다.



PostgreSQL 테이블 생성 방법

팝업 화면에서 상단 General 탭 선택 후 테이블 이름을 입력한다. 본 문서는 TemperatureInformation 으로 입력한다.

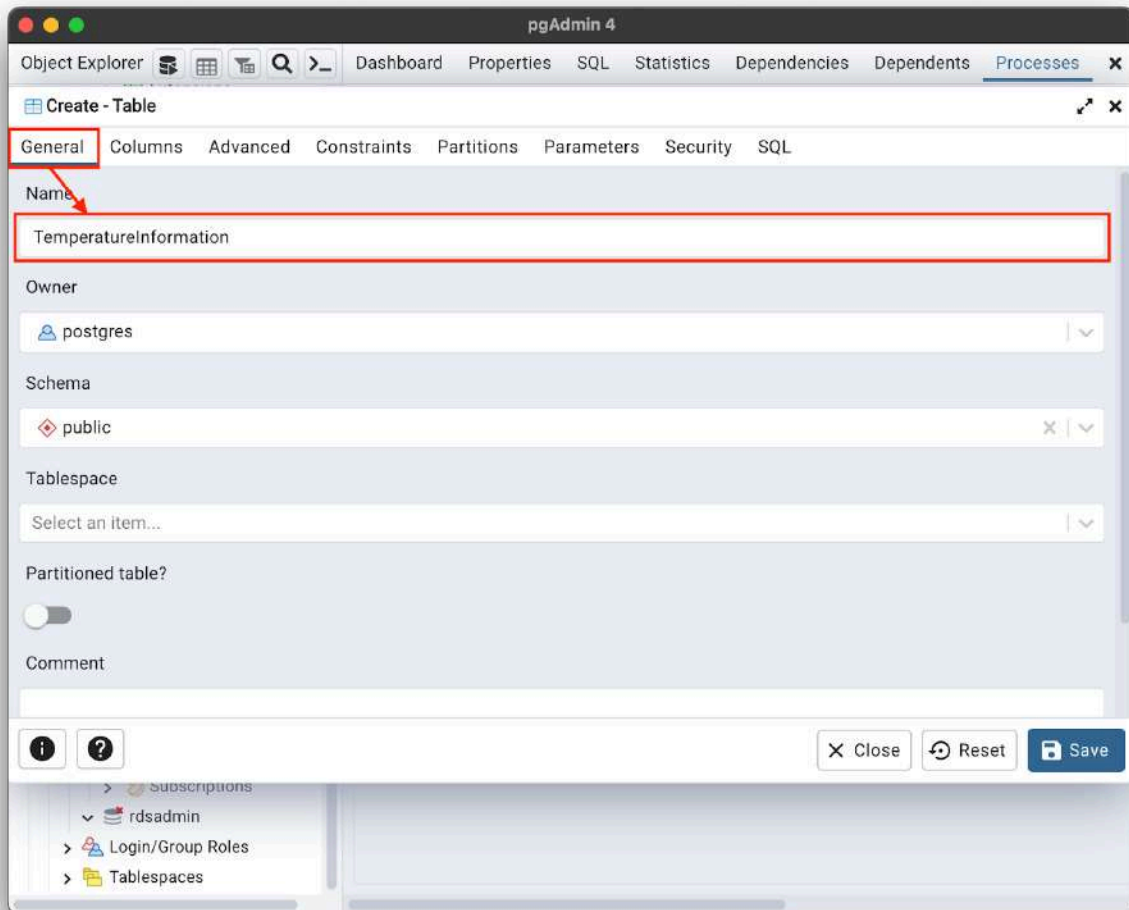


Table 이름 입력

상단의 Columns 클릭 후 우측의 + 버튼을 클릭해 테이블 정보를 추가 후 Save 버튼을 클릭해 테이블을 추가한다. 데이터베이스에 추가하는 정보는 아래와 같다.

Name	Data type	Not NULL?
name	text	YES
temperature	double precision	YES
date	timestamp without time zone	YES

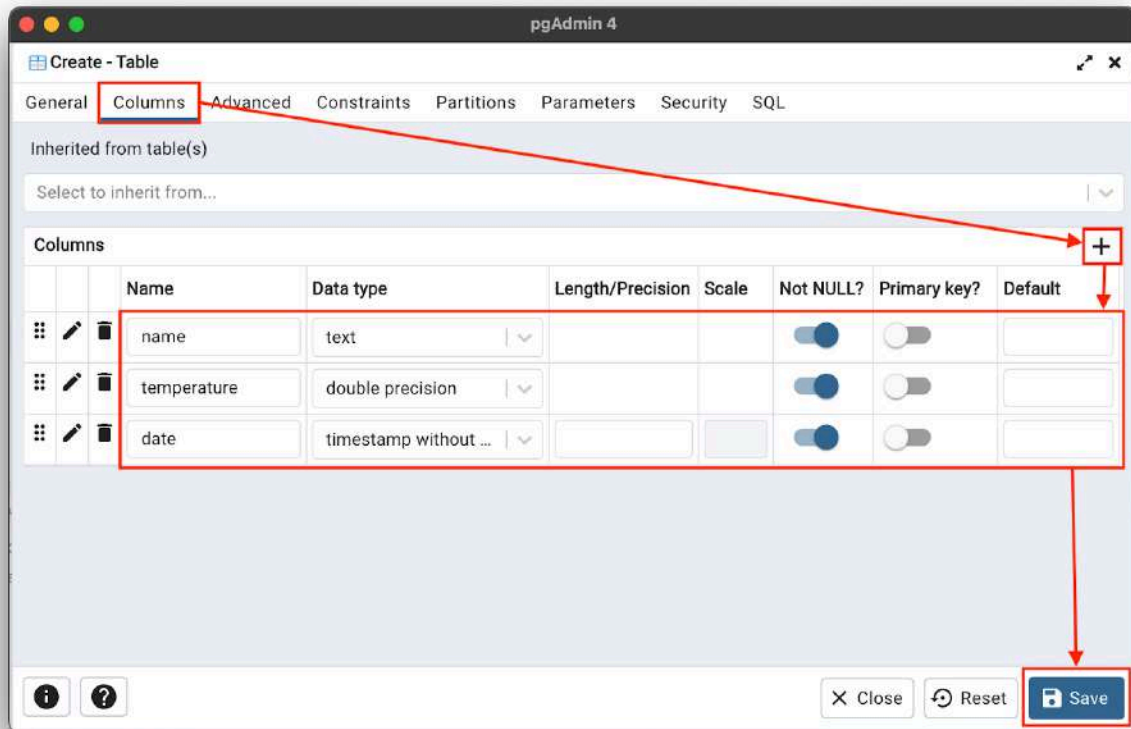
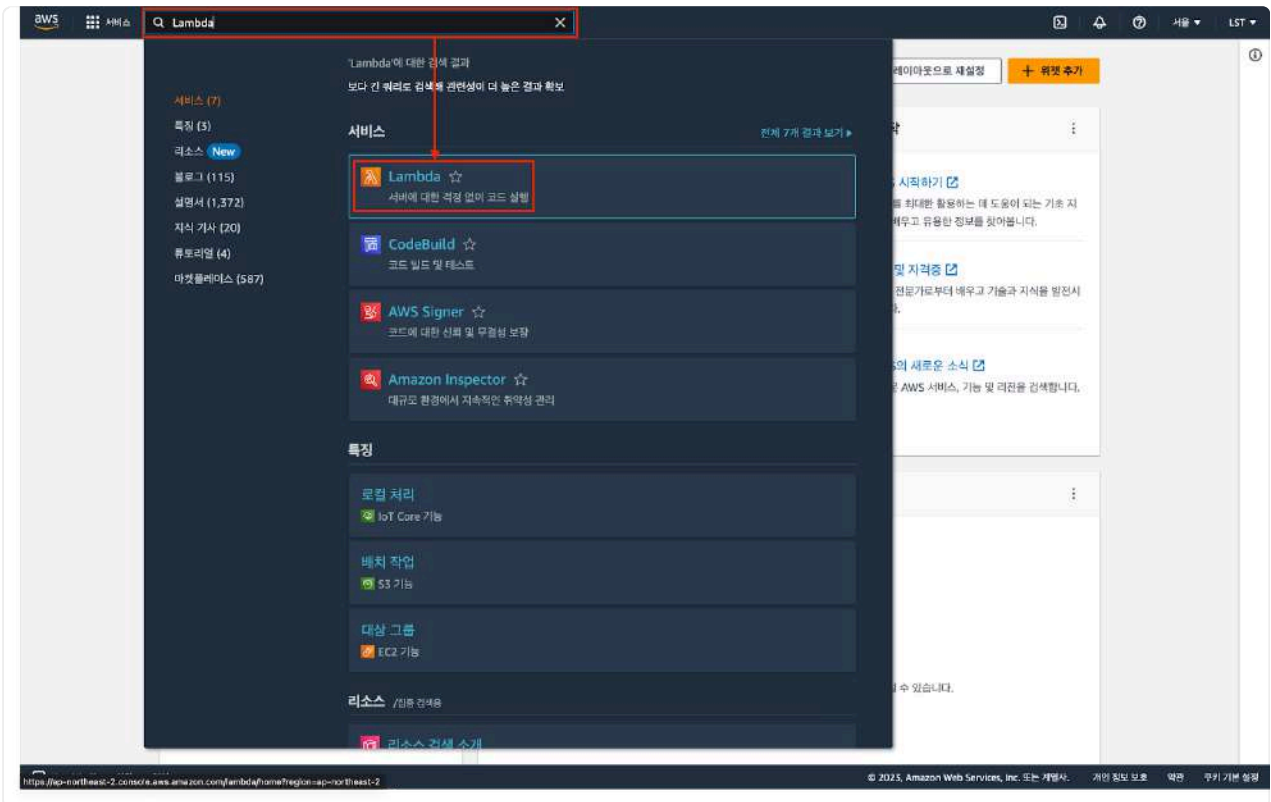


Table Columns 설정

5.8. AWS 서버 생성

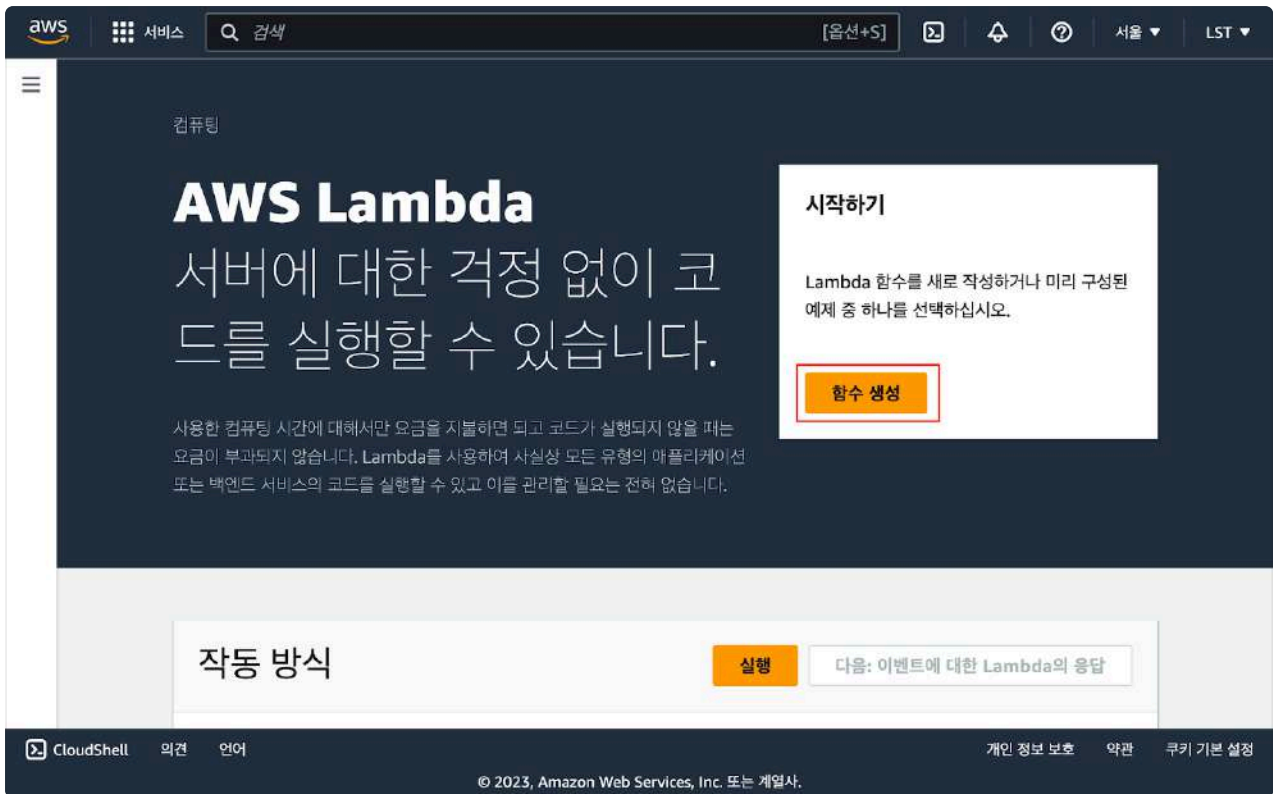
본 단락은 AWS에서 제공하는 서버리스 프레임워크인 Lambda를 사용해 클라우드 서버 구현을 진행한다.

AWS 콘솔에서 상단의 검색 영역에 Lambda 입력 후 Lambda를 클릭해 화면을 전환한다.



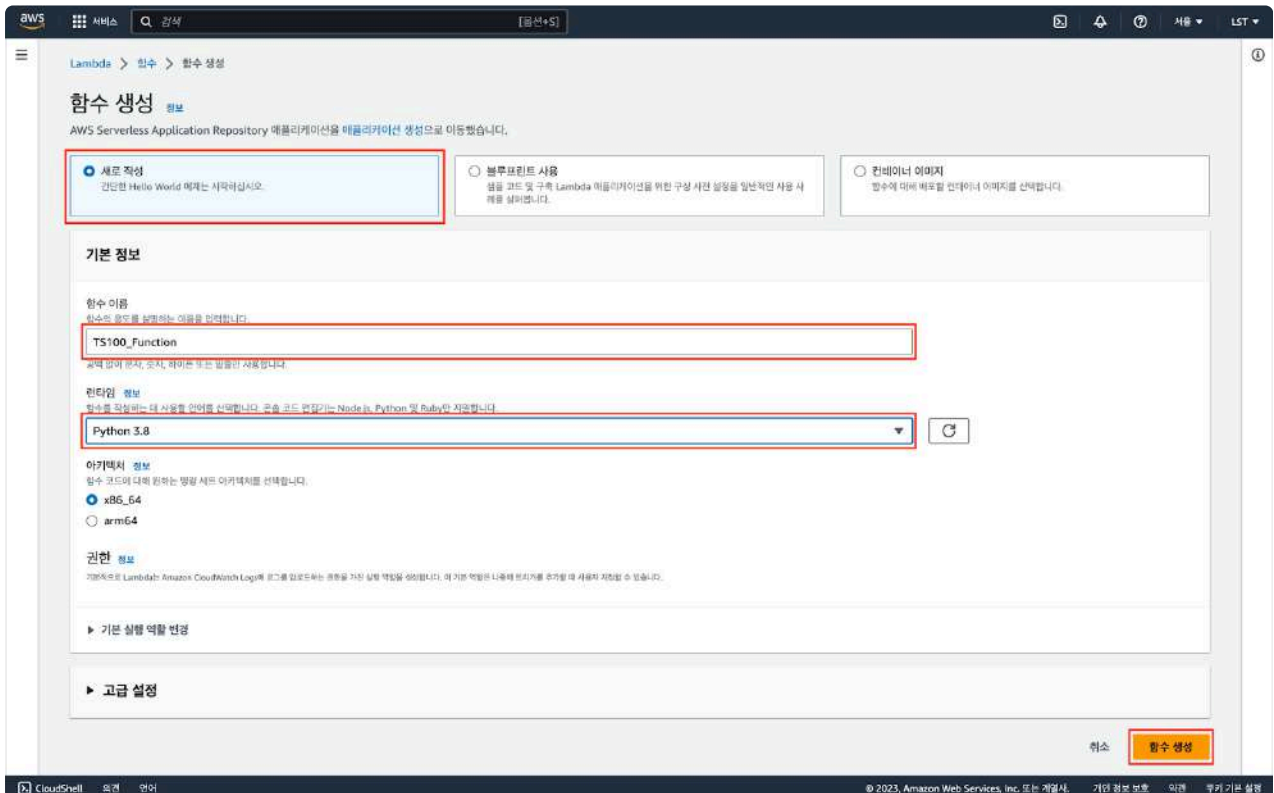
AWS Lambda 이동 방법

Lambda 콘솔화면으로 이동 후 우측의 함수 생성 버튼을 클릭해 함수를 생성한다.



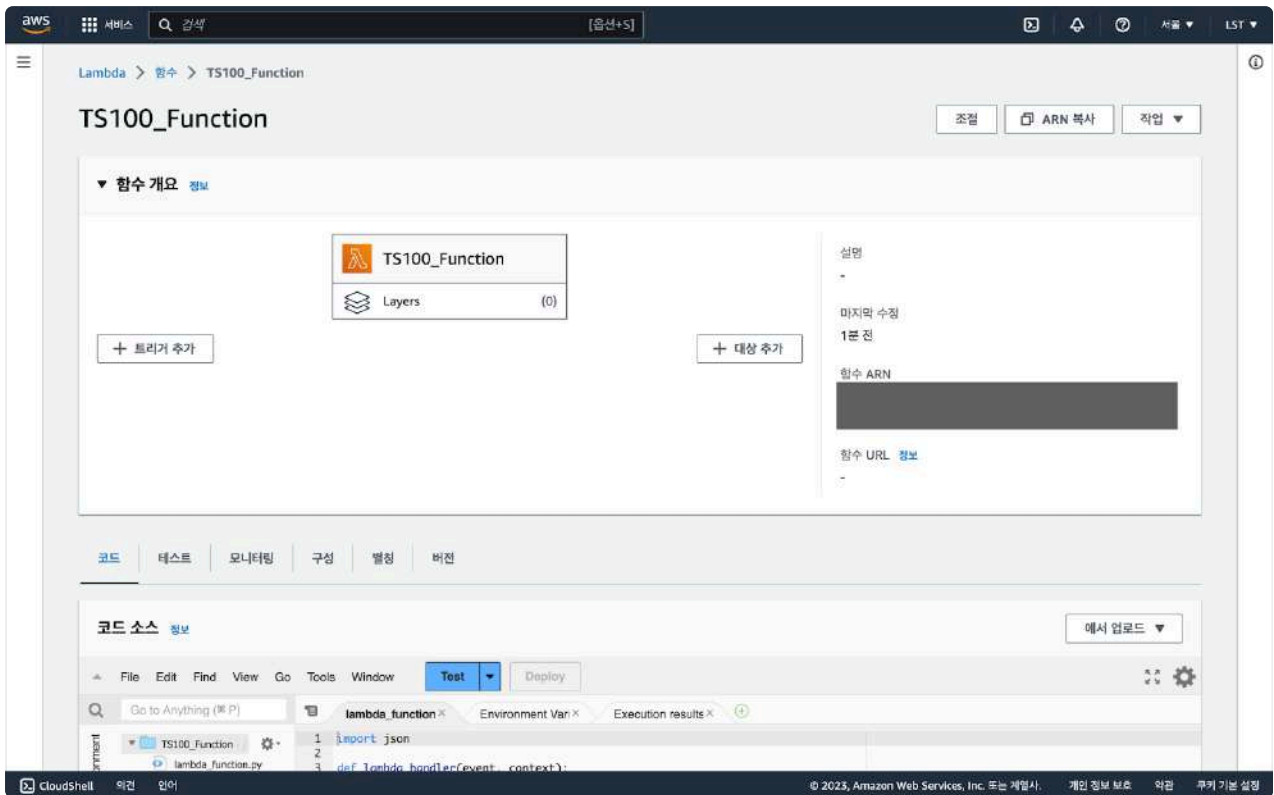
Lambda 함수 생성

함수 생성 화면에서 상단의 새로 작성을 클릭한다. 아래 기본정보 중 함수 이름을 작성한다. 본 문서에서는 TS100_Function으로 입력한다. 이후 런타임을 Python으로 설정한다. 본 문서에서는 3.8 버전을 선택한다. 모든 정보 입력 후 함수 생성 버튼을 클릭하여 함수 생성을 완료한다.



함수 정보 입력

위 과정을 거친 후 생성된 함수 콘솔 화면은 아래 그림과 같다.

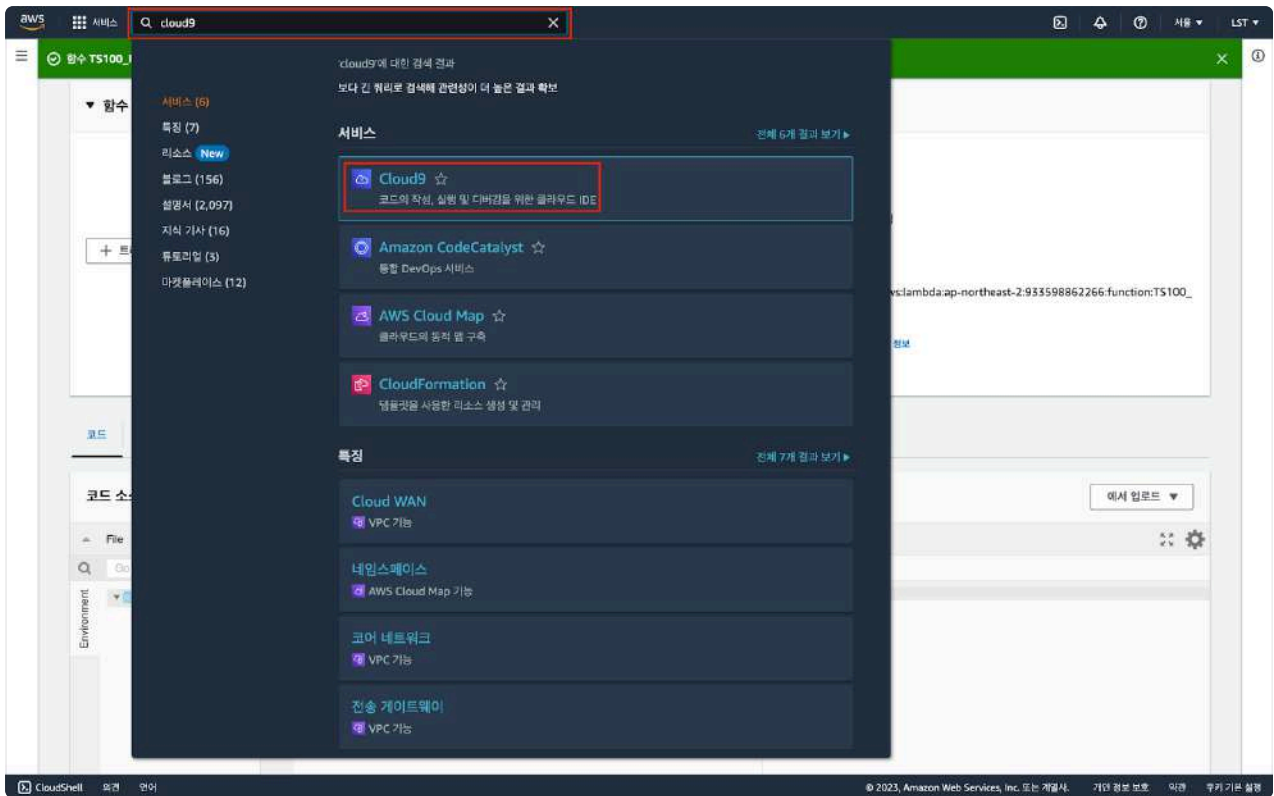


Lambda 함수 콘솔

psycopg2 라이브러리 설치

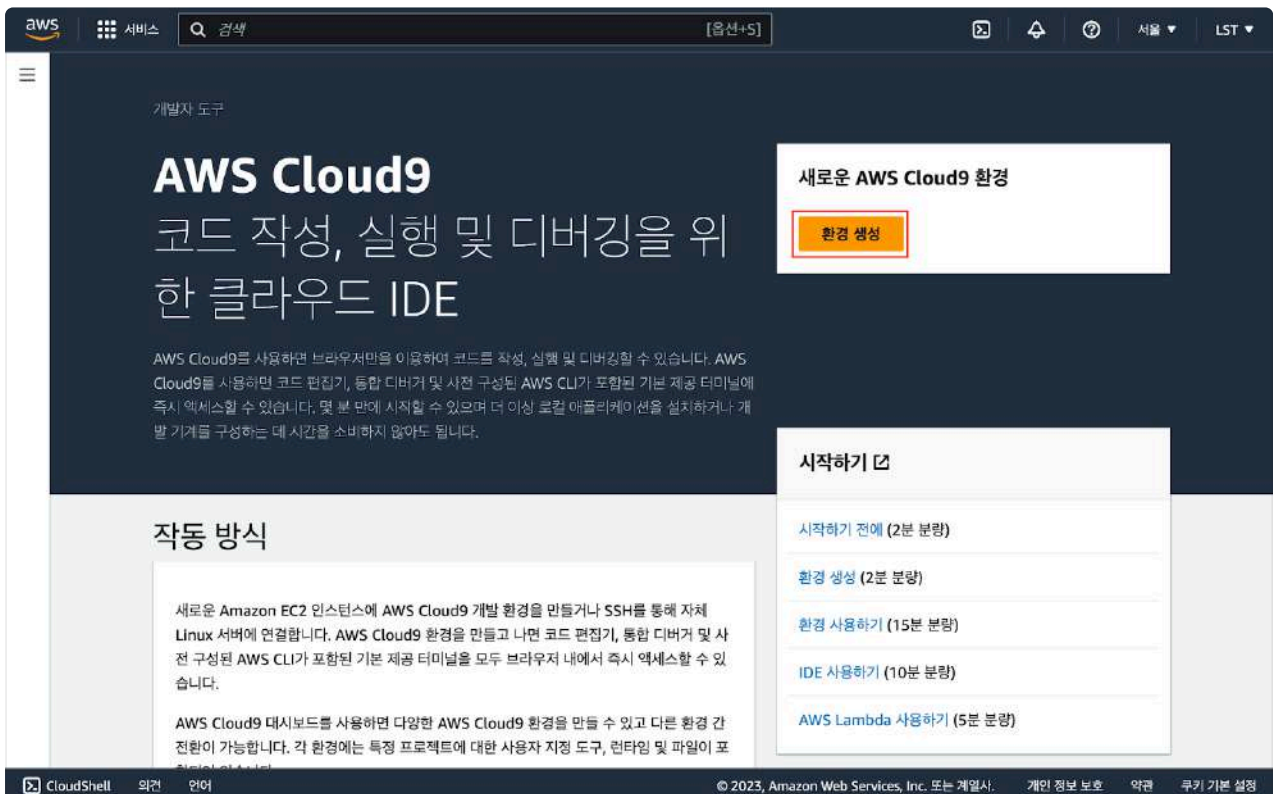
Lambda 함수와 PostgreSQL을 연결하기 위해 파이썬 라이브러리인 psycopg2를 사용한다. psycopg2는 파이썬 외부 라이브러리기 때문에 직접 라이브러리를 설치해야 한다.

검색 영역에서 cloud9 입력 후 Cloud9을 클릭하여 화면을 이동한다.



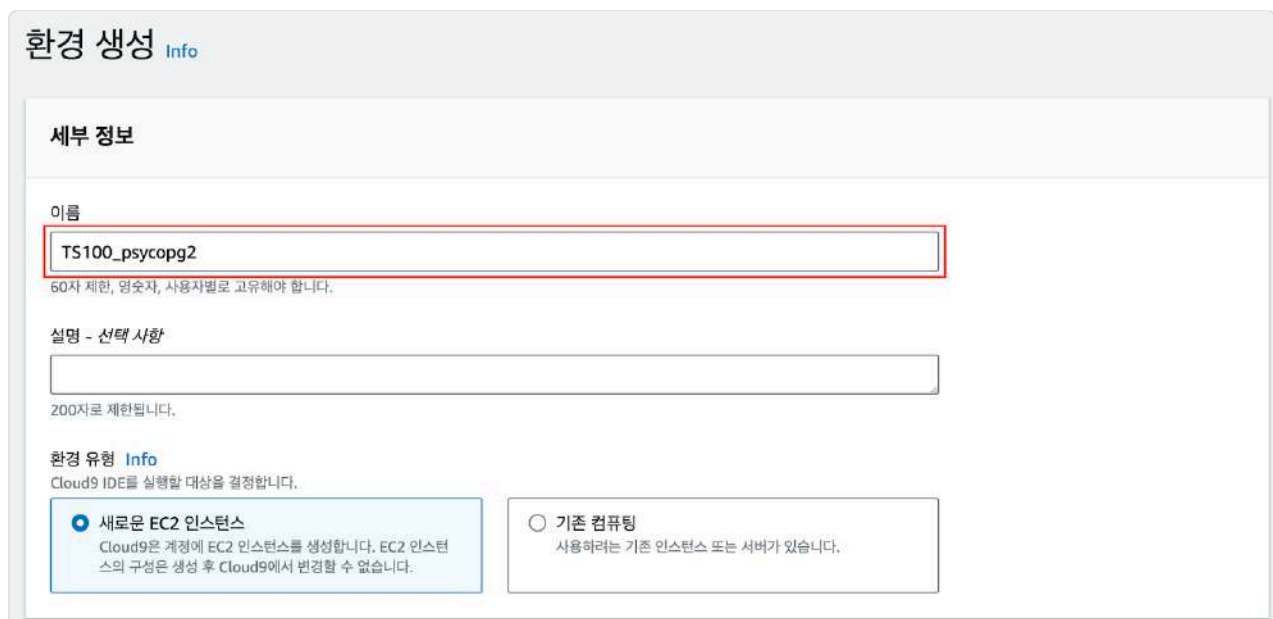
Cloud9 검색 및 이동

Cloud9 콘솔로 이동 후 환경 생성 버튼을 클릭해 환경 생성 화면으로 이동한다.



Cloud9 환경 생성 방법

세부 정보 영역에서 이름을 입력한다. 본 문서는 TS100_psycopg2로 입력한다.



The image shows a web form titled "환경 생성" (Environment Creation) with a sub-header "세부 정보" (Detailed Information). It contains three main sections: 1. "이름" (Name) with a text input field containing "TS100_psycopg2" and a note "60자 제한, 영숫자, 사용자별로 고유해야 합니다." (60 character limit, alphanumeric, must be unique per user). 2. "설명 - 선택 사항" (Description - Optional) with a text input field and a note "200자로 제한됩니다." (limited to 200 characters). 3. "환경 유형" (Environment Type) with two radio button options: "새로운 EC2 인스턴스" (New EC2 Instance) which is selected, and "기존 컴퓨팅" (Existing Computing). The "새로운 EC2 인스턴스" option has a sub-note: "Cloud9은 계정에 EC2 인스턴스를 생성합니다. EC2 인스턴스의 구성은 생성 후 Cloud9에서 변경할 수 없습니다." (Cloud9 creates an EC2 instance in your account. The configuration of the EC2 instance cannot be changed in Cloud9 after creation).

Cloud9 환경 생성 세부정보 입력

새로운 EC2 인스턴스 영역은 별도로 수정하지 않고 진행한다.

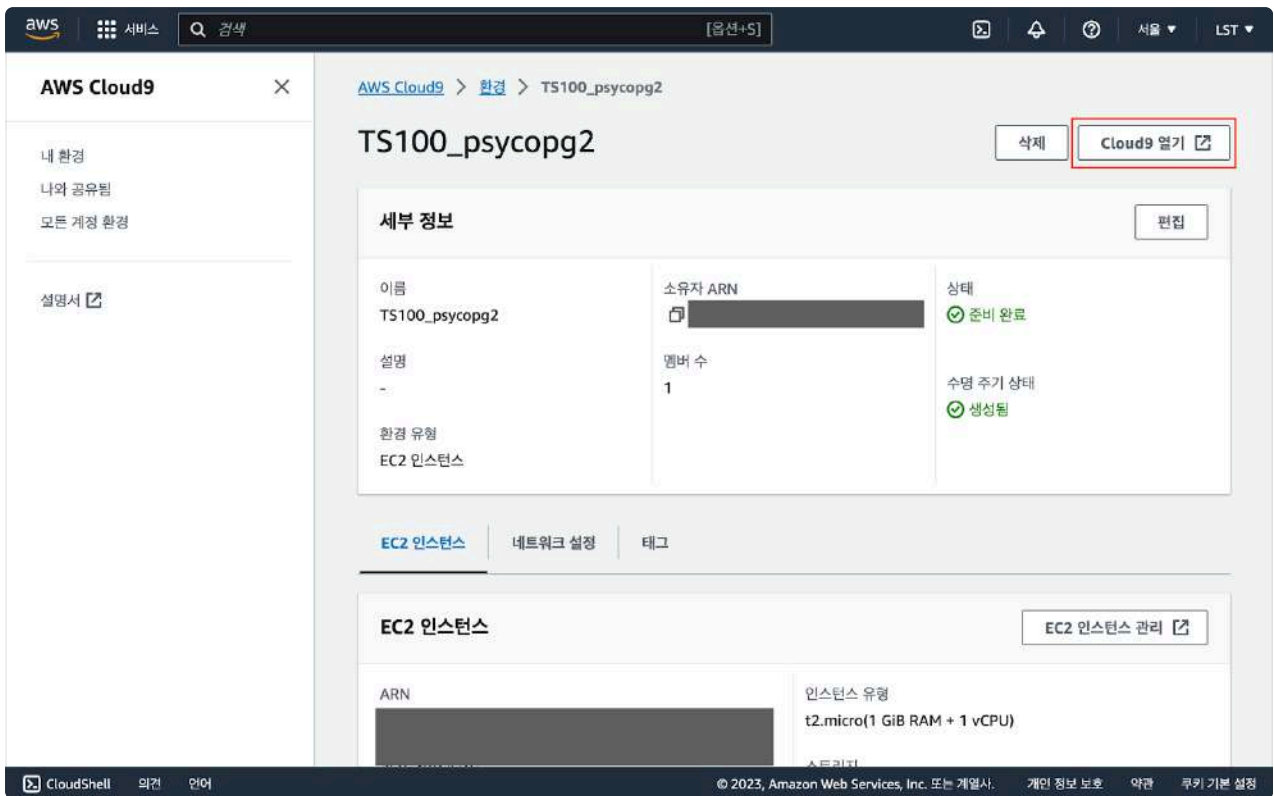


The image shows a web form titled "새로운 EC2 인스턴스" (New EC2 Instance). It contains three main sections: 1. "인스턴스 유형" (Instance Type) with three radio button options: "t2.micro(1 GiB RAM + 1 vCPU)" (selected), "t3.small(2 GiB RAM + 2 vCPU)", and "m5.large(8 GiB RAM + 2 vCPU)". Each option has a brief description of its use case. 2. "플랫폼" (Platform) with a dropdown menu showing "Amazon Linux 2". 3. "시간 제한" (Time Limit) with a dropdown menu showing "30분" (30 minutes).

Cloud9 새로운 EC2 인스턴스 영역

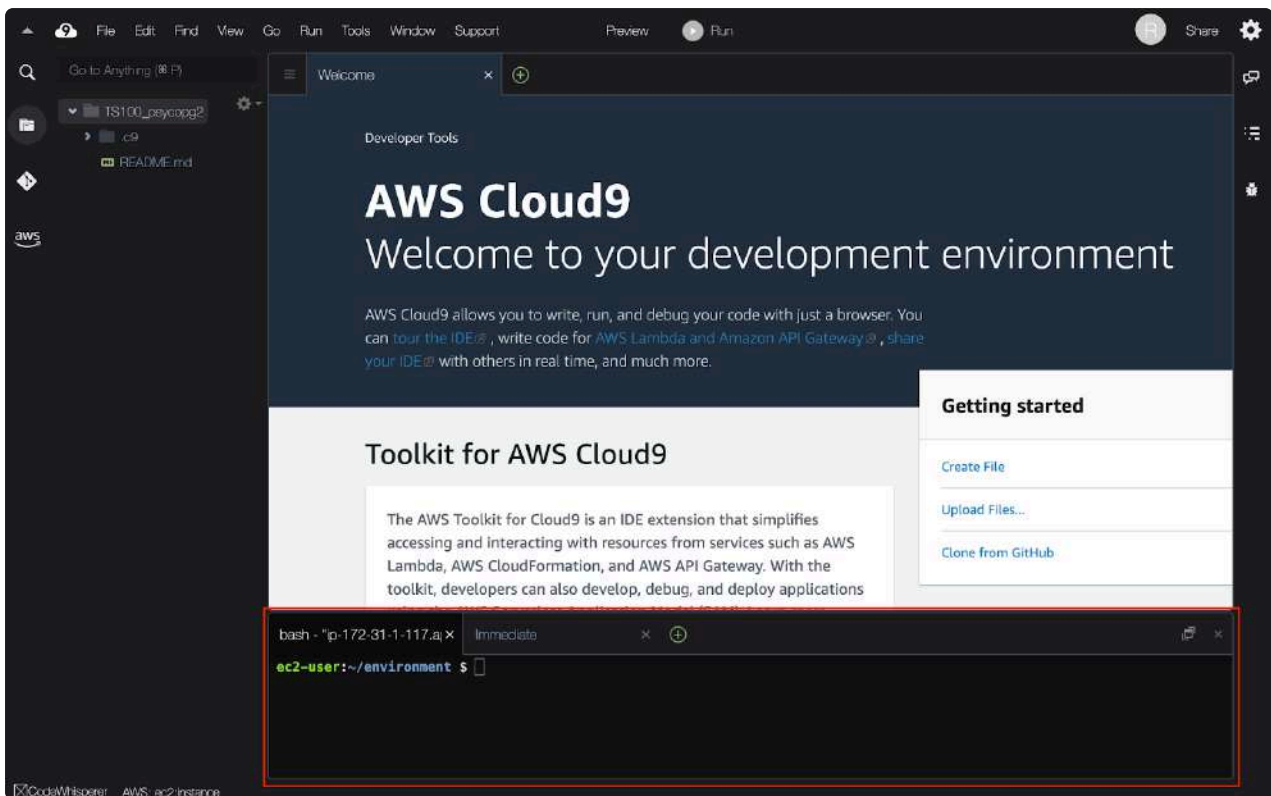
이후 네트워크 설정 영역도 수정 없이 우측 하단의 생성 버튼을 클릭해 새로운 Cloud9 환경을 생성한다.

아래 화면에서 Cloud9 열기 버튼을 클릭하여 클라우드를 실행한다.



Cloud9 실행

Cloud9를 실행하면 아래와 같은 화면이 나타난다. 본 문서에서는 하단 영역인 bash 영역을 사용한다.



Cloud9 콘솔

bash 영역에 아래 명령어를 입력해 Python3.8을 설치한다.

```
sudo amazon-linux-extras install python3.8
```

파이썬을 설치하던 중 아래와 같은 화면이 나타난다. 소문자 y 입력 후 엔터를 입력하면 파이썬이 설치된다.

```
Total download size: 13 M
Installed size: 54 M
Is this ok [y/d/N]:
```

Python 설치 선택 화면

이후 오픈소스 모듈을 간편하게 설치하기 위해 아래 명령어를 차례대로 입력해 pip를 설치한다.

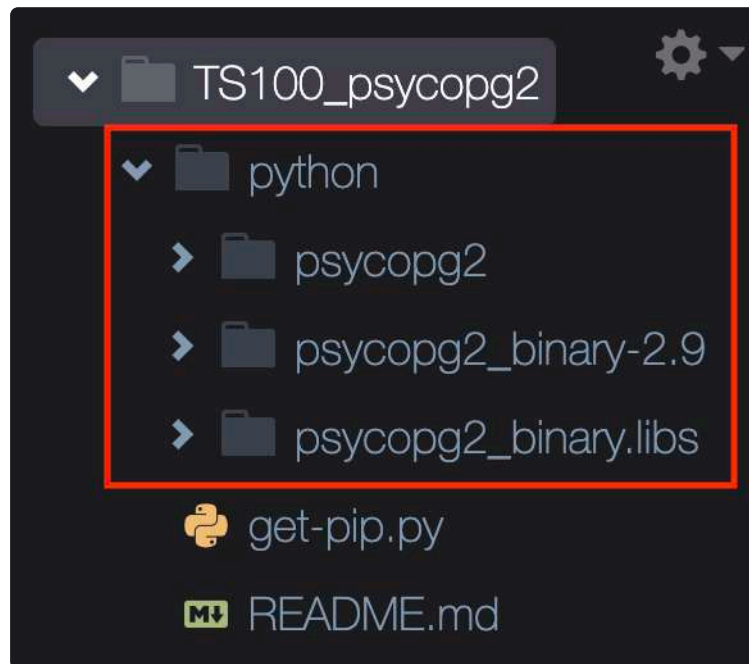
```
curl -O https://bootstrap.pypa.io/get-pip.py
python3.8 get-pip.py --user
```

아래 명령어를 입력해 python 폴더를 생성한다.

```
mkdir python
```

이후 아래 명령어를 입력해 psycopg2 모듈을 다운로드한다. 모듈이 설치완료되면 아래 그림과 같이 python 폴더 내 3개의 폴더가 생성된것을 확인할 수 있다.

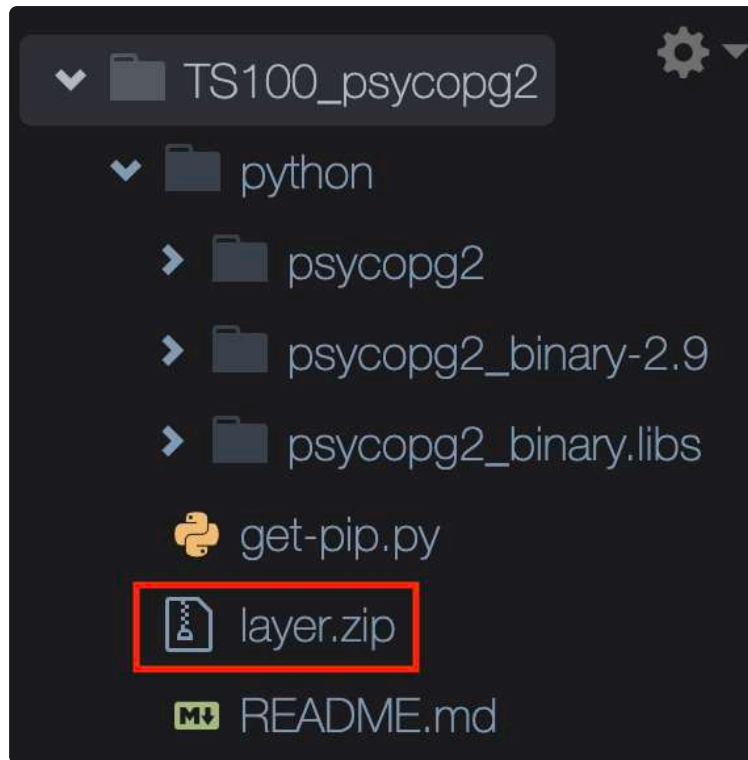
```
sudo python3.8 -m pip install psycopg2-binary -t python/
```



psycopg2 설치 후 폴더구조

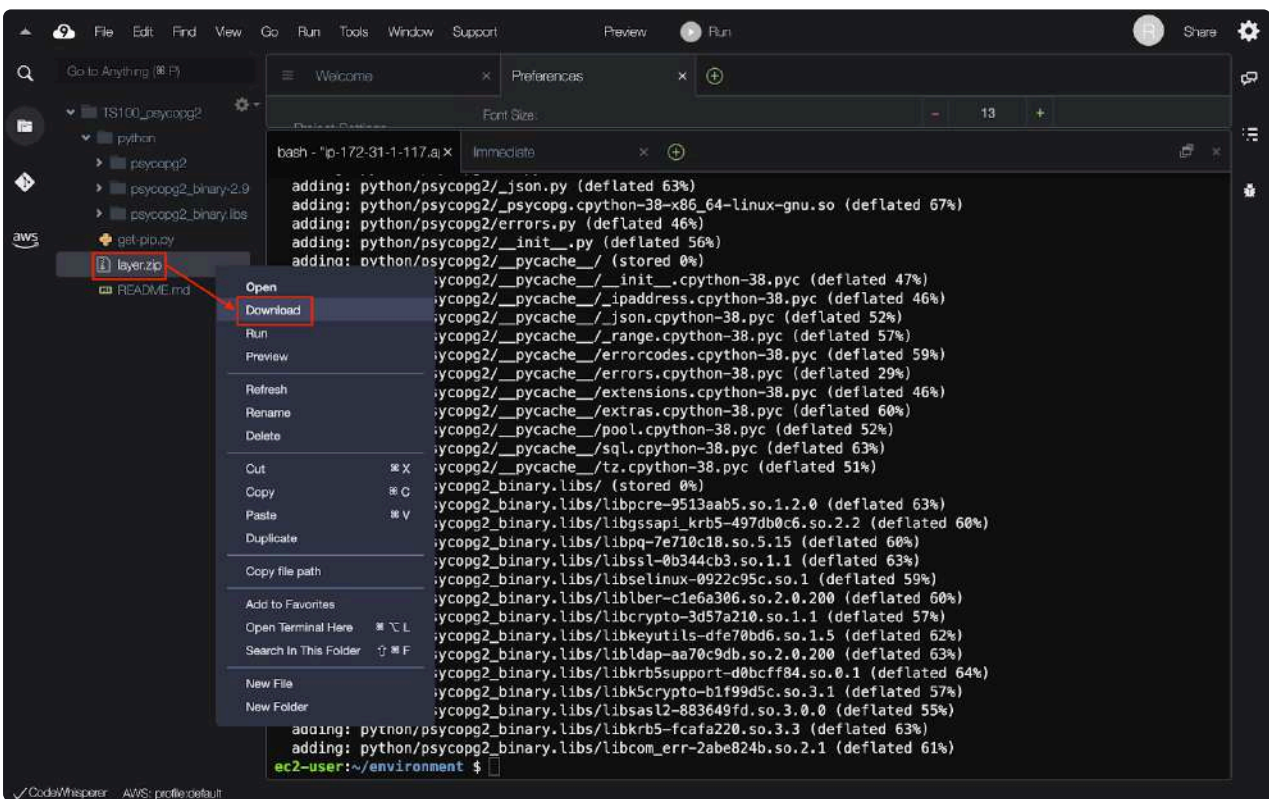
아래 명령어를 입력해 zip파일을 생성한다.

```
zip -r layer.zip python
```



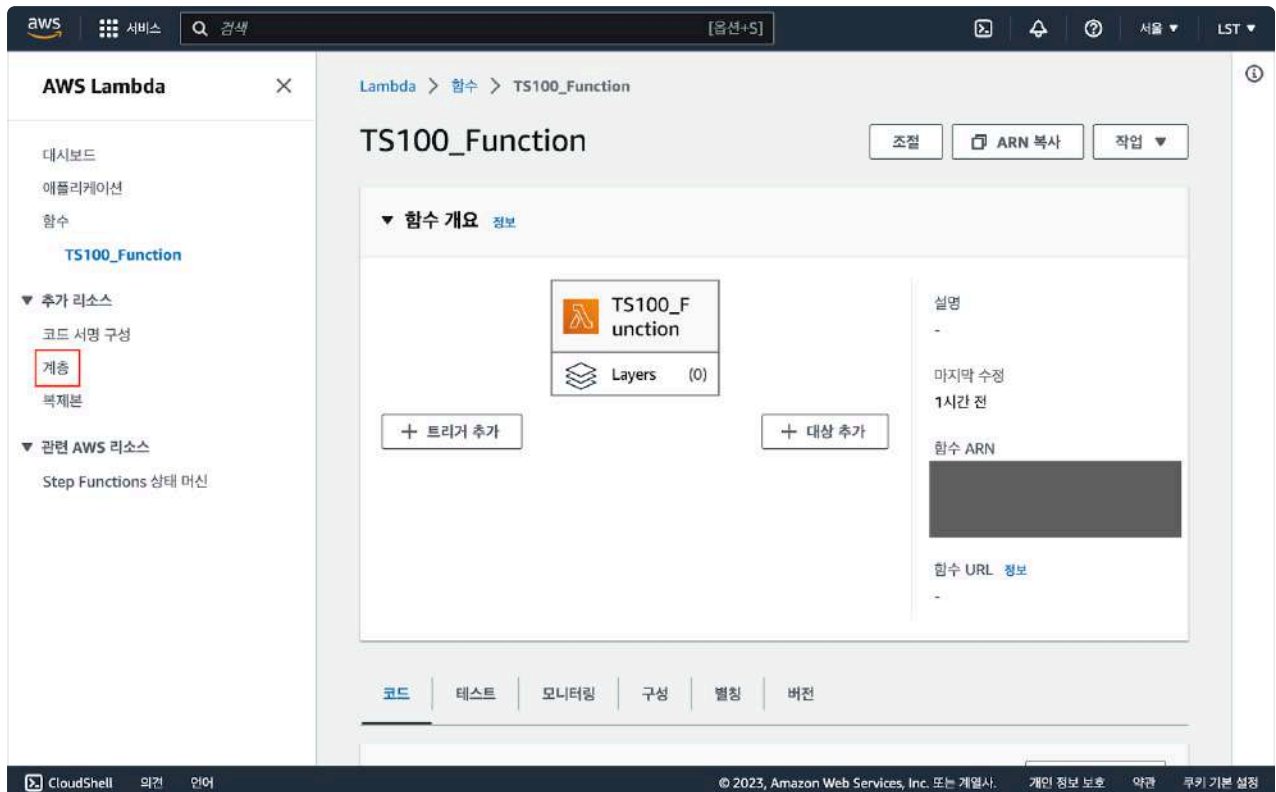
layer.zip 추가 후 폴더구조

생성된 layer.zip 파일을 우클릭해 Download 버튼을 클릭하여 다운로드를 진행한다.



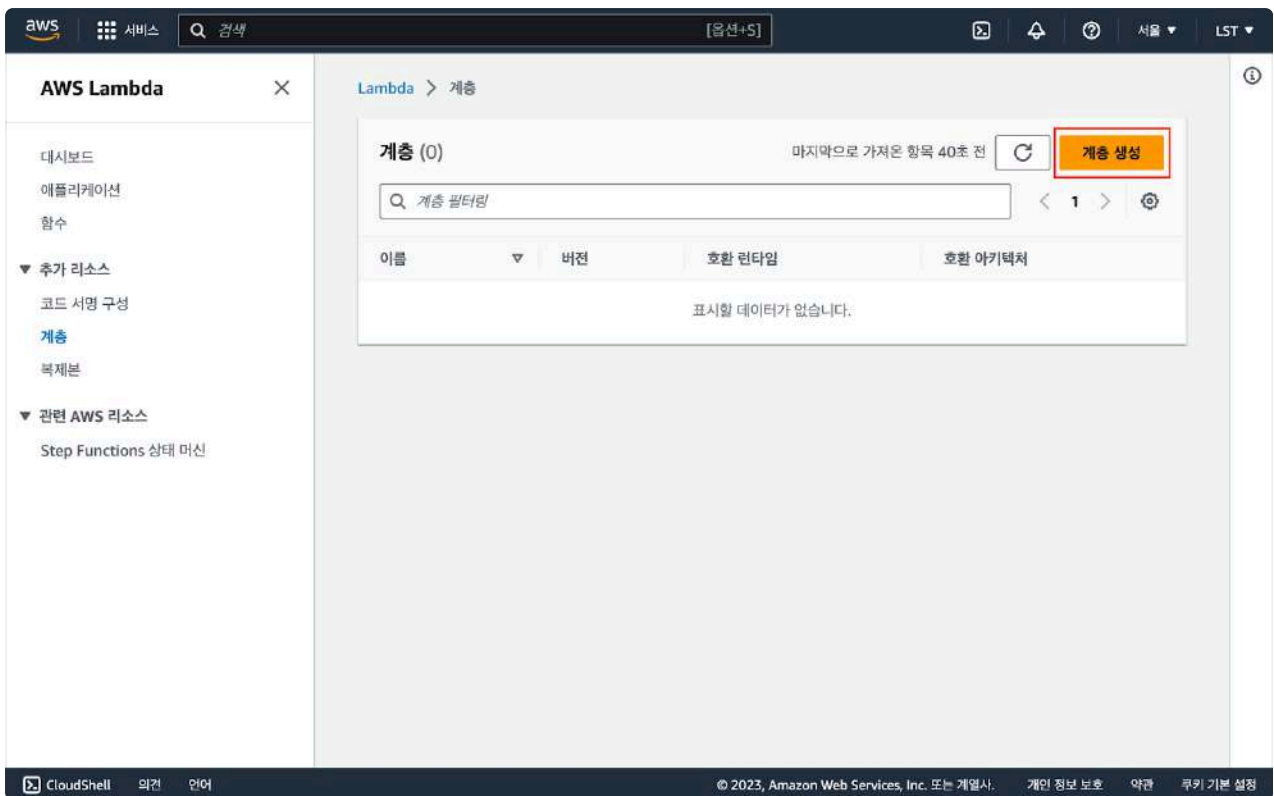
layer.zip 다운로드 방법

다시 AWS Lambda 콘솔로 이동해 TS100_Function을 클릭한다. 좌측에 있는 계층 버튼을 클릭한다.



계층 설정 이동 방법

계층 콘솔에서 우측 상단의 계층 생성 버튼을 클릭한다.



계층 생성 방법

계층 생성 화면에서 계층 구성 영역의 정보를 입력한다. 본 문서는 `psycopg2Layer` 이름을 입력한다. 이후 이전에 다운로드 받았던 `layer.zip` 파일을 업로드한다. 호환 런타임을 `Python3.8`로 설정 후 생성 버튼을 클릭해 새로운 레이어를 생성한다.

계층 생성

계층 구성

이름

psycopg2Layer

설명 - 선택 사항

설명

☒ .zip 파일 업로드

☐ Amazon S3에서 파일 업로드

 업로드

layer.zip

3.07 MB

10MB가 넘는 파일의 경우, Amazon S3를 사용한 업로드를 고려하십시오.

호환 아키텍처 - 선택 사항 정보

계층에 대해 호환 명령 세트 아키텍처를 선택합니다.

☐ x86_64

☐ arm64

호환 런타임 - 선택 사항 정보

최대 15개의 런타임을 선택합니다.

실행 시간

Python 3.8

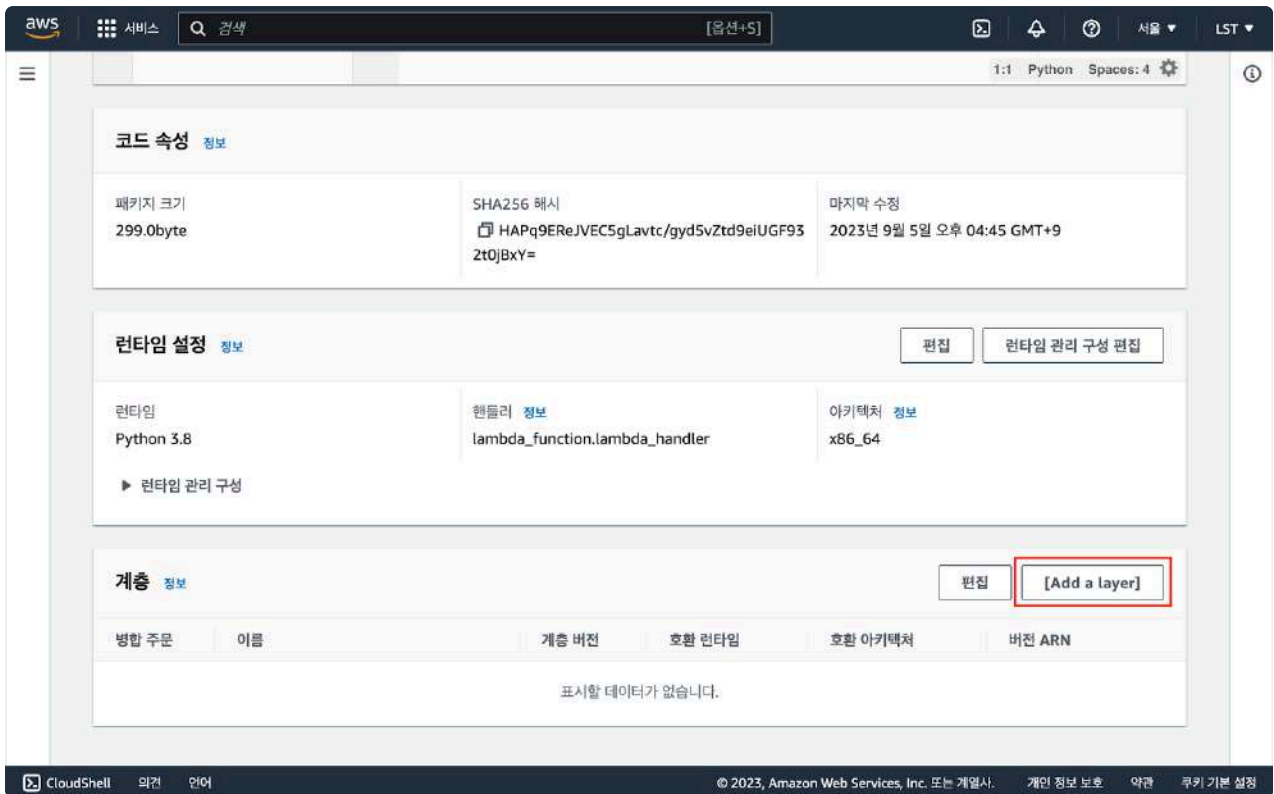
라이선스 - 선택 사항 정보

취소

생성

AWS Lambda 계층 생성

다시 TS100_Function 콘솔로 이동해 가장 아래 영역까지 스크롤 하면 계층 영역이 나타나는것을 확인할 수 있다. [Add a layer] 버튼을 클릭하여 레이어 추가 화면으로 이동한다.



AWS Lambda Layer 추가 방법

계층 추가 화면에서 계층 소스를 사용자 지정 계층을 선택한다. 사용자 지정 계층을 psycopg2Layer로 설정하고, 버전을 1로 설정 후 우측 하단의 추가 버튼을 클릭해 계층을 추가한다.

계층 추가

함수 런타임 설정

런타임

Python 3.8

아키텍처

x86_64

계층 선택

계층 소스 정보

호환 런타임 및 명령 세트 아키텍처가 있는 계층에서 선택하거나 계층 버전의 Amazon 리소스 이름(ARN)을 지정합니다. 새로운 계층을 생성할 수도 있습니다.

☐

AWS 계층

AWS에서 제공하는 계층 목록에서 계층을 선택합니다.

☒

사용자 지정 계층

AWS 계정 또는 조직에서 생성한 계층 목록에서 계층을 선택합니다.

☐

ARN 지정

ARN을 제공하여 계층을 지정합니다.

사용자 지정 계층

함수의 런타임과 호환되는 AWS 계정 또는 조직 생성 계층입니다.

psycpg2Layer

버전

1

취소

추가

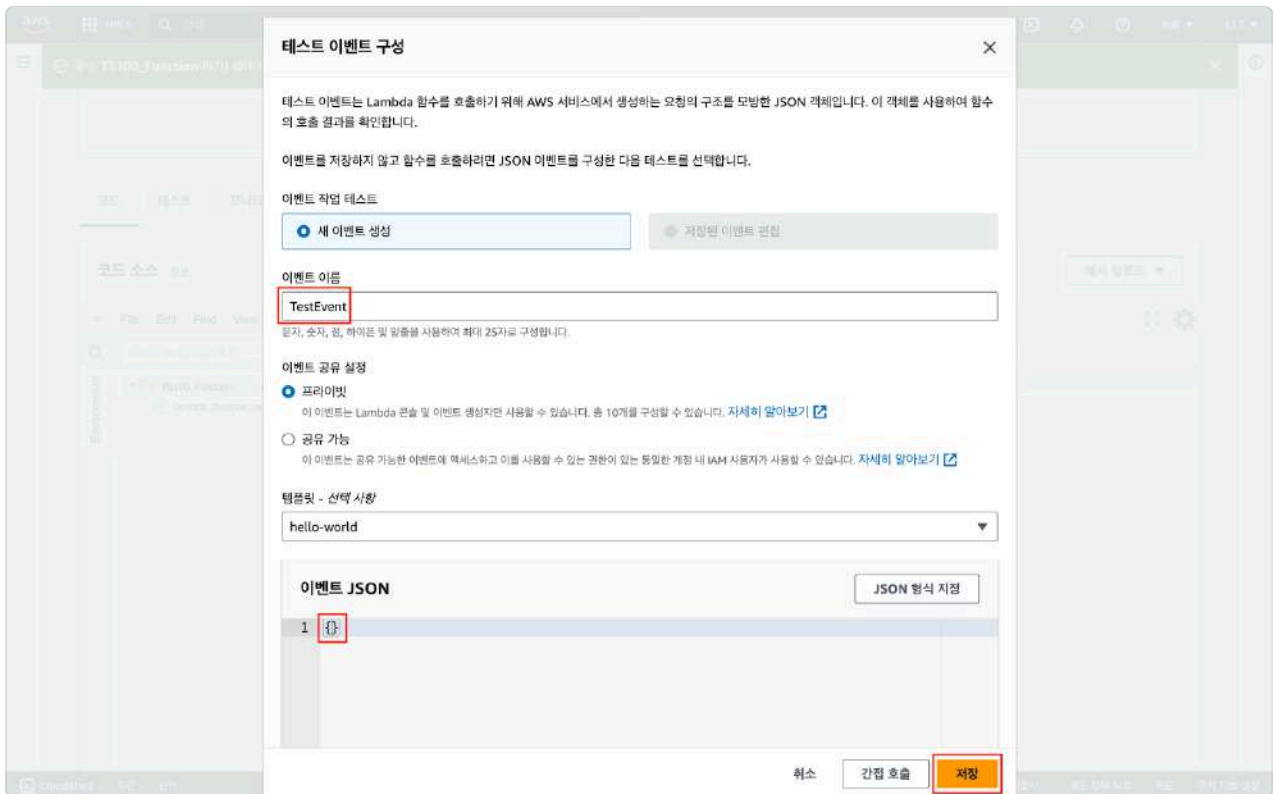
계층 선택 방법 및 생성

이후 코드 영역을 아래와 같이 작성 후 중앙의 Deploy 버튼 클릭해 함수를 업데이트한다. 함수가 업데이트되면 Test 버튼을 클릭한다.

```
import psycpg2 # 추가된 부분
import json

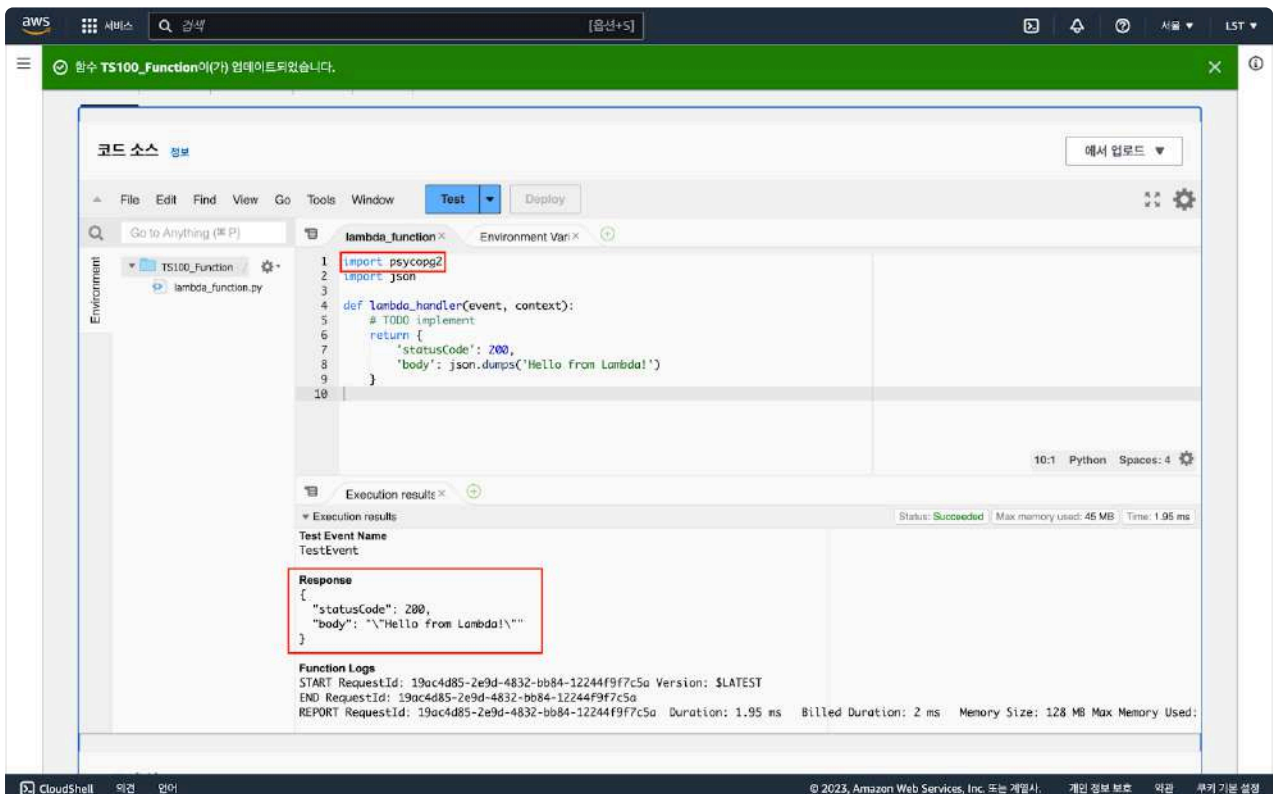
def lambda_handler(event, context):
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```

처음 Test 버튼을 클릭하면 팝업화면이 나타난다. 아래 그림과 같이 입력 후 저장 버튼을 클릭한다.



테스트 이벤트 구성 팝업

오류가 발생하지 않고 psycopg 라이브러리를 import 할 수 있는것을 확인할 수 있다.



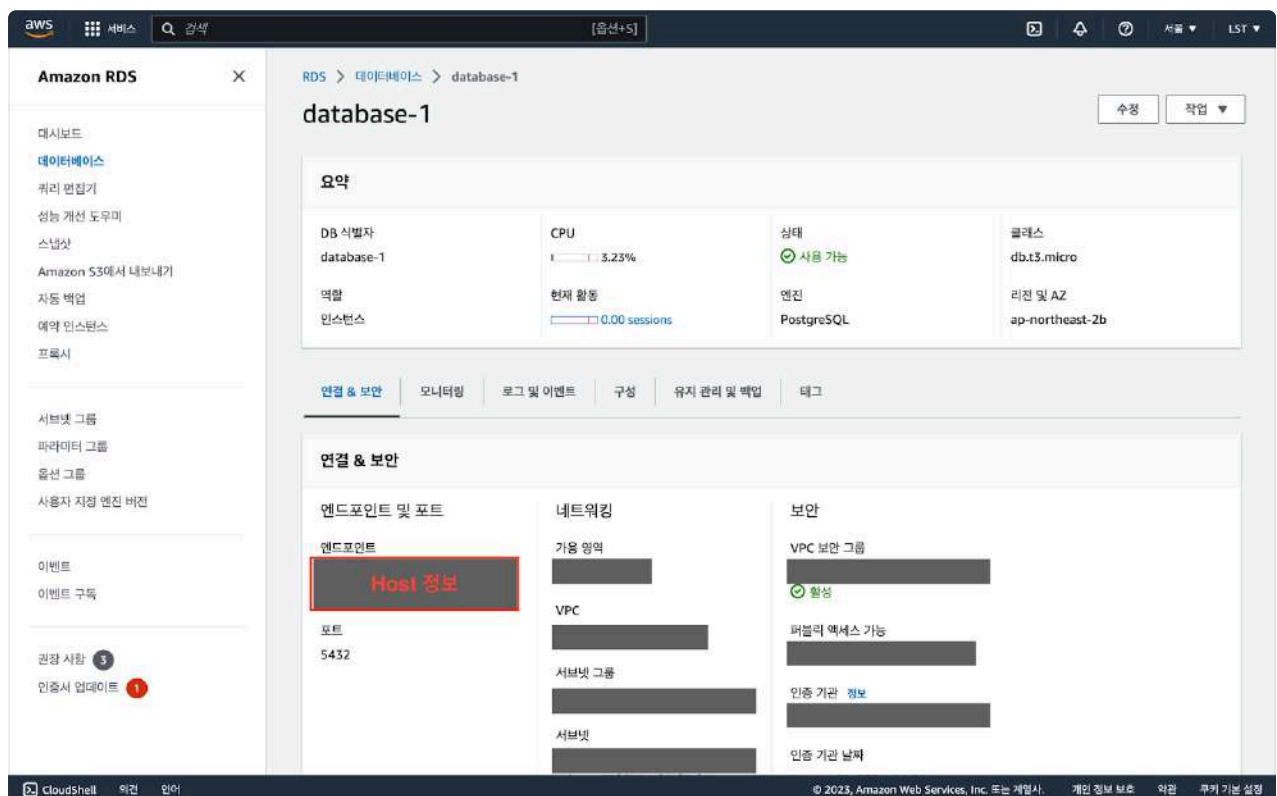
psycopg2 라이브러리 import 성공

데이터베이스 연결

psycopg2 라이브러리를 이용해 AWS에 업로드되어있는 PostgreSQL을 연결한다.

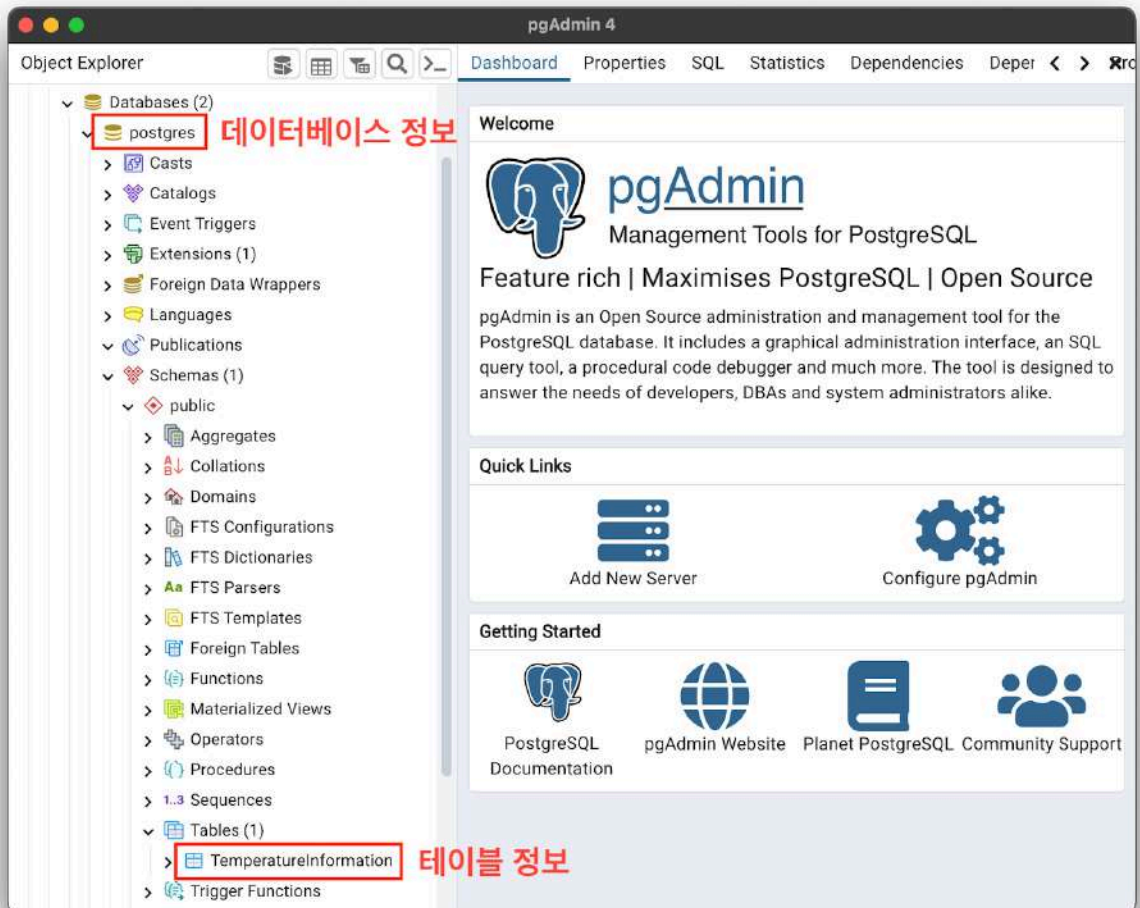
우선 PostgreSQL을 연결하기 위해 호스트, 데이터베이스, 유저, 비밀번호 정보가 필요하다.

호스트 정보는 RDS 콘솔 → 생성한 데이터베이스 → 연결 & 보안 → 엔드포인트 부분을 확인한다.



Host 정보 위치 확인

데이터베이스, 테이블 정보는 pgAdmin 화면에서 확인할 수 있다.



데이터베이스, 테이블 정보 확인

비밀번호 정보는 AWS내 RDS 생성 시 입력했던 마스터 비밀번호를 사용한다.

다시 Lambda 함수 콘솔 화면으로 돌아와 코드를 아래와 같이 수정한다.

```

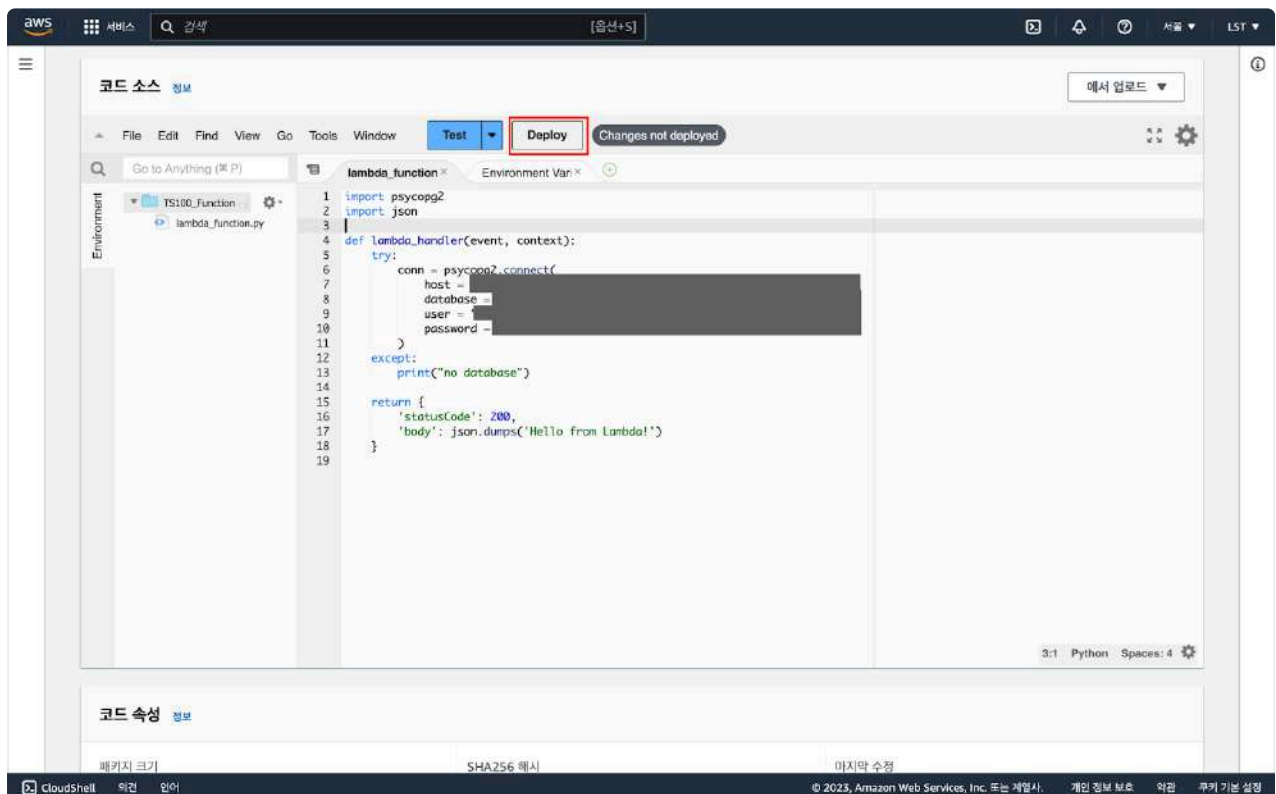
import psycopg2
import json

def lambda_handler(event, context):
    try:
        conn = psycopg2.connect(
            host = "호스트 주소",
            database = "데이터베이스 이름",
            user = "유저 이름",
            password = "마스터 비밀번호"
        )
    except:
        print("no database")

    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }

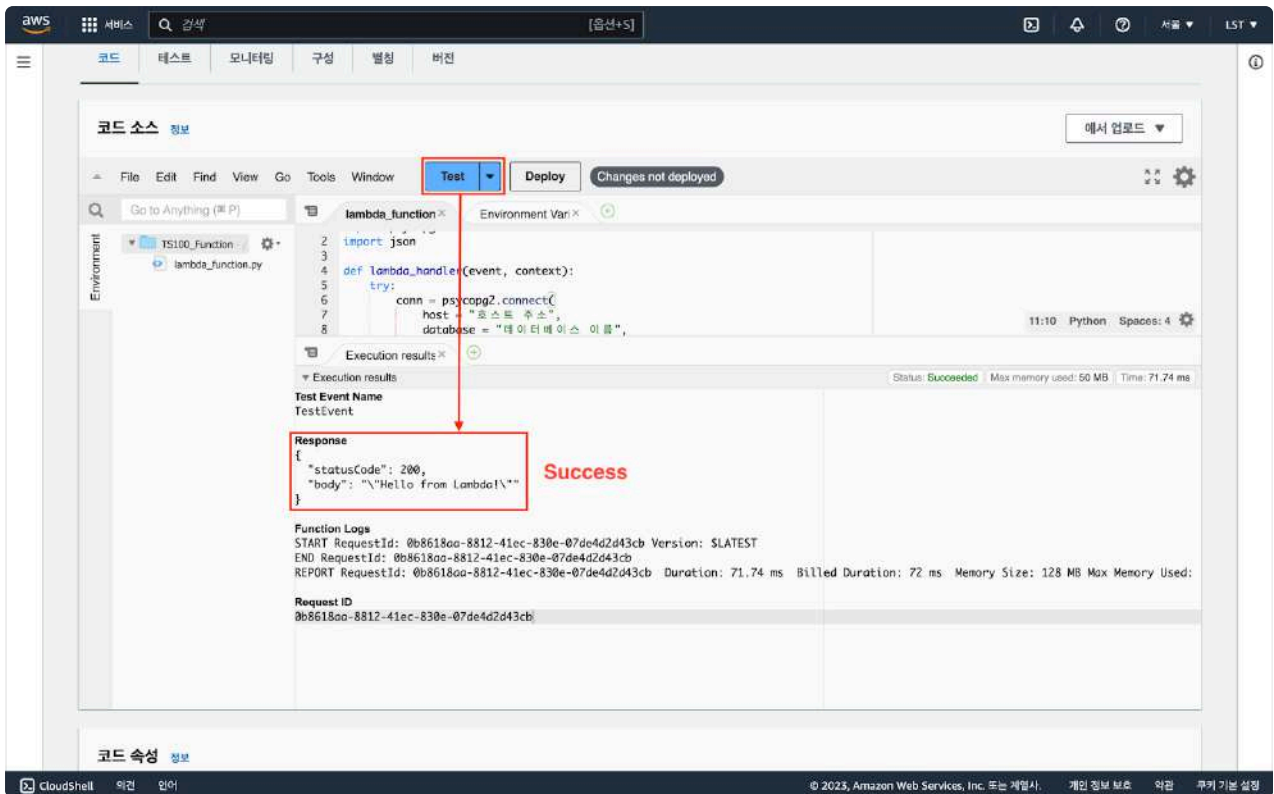
```

이후 중앙의 Deploy 버튼을 클릭해 코드를 배포한다. 코드가 반영된다면 상단에 “함수가 업데이트 되었습니다”라는 알림이 나타난다.



Lambda 함수 배포

함수 배포 후 Test 버튼을 클릭하면 아래 그림과 같이 함수실행이 성공한것을 확인할 수 있다.



Lambda 데이터베이스 연결 성공

데이터베이스 저장

Lambda를 통해 연결된 데이터베이스 온도정보를 저장하는 방법을 설명한다. PostgreSQL의 데이터베이스 내용은 아래와 같다.

```
name: text(텍스트)
temperature: double percision(정수)
date: timestamp without timezone(년-월-일 시:분:초)
```

PostgreSQL에 온도정보를 저장하기 위해 아래와 같은 SQL문을 작성한다.

```
INSERT into "TemperatureInformation" (name, temperature, date) values
('TS100-12345', 34.5, '2023-09-07 09:00:00');
```

Lambda 함수 콘솔 화면으로 이동해 lamdba_function코드를 아래와 같이 수정한다.
TemperatureInformation 데이터베이스에 이름 TS100-12345온도 34.5, 타임스탬프 2023-09-07 09:00:00 데이터를 삽입하는 코드이다.

```

import psycopg2
import json

def lambda_handler(event, context):
    try:
        conn = psycopg2.connect(
            host = "호스트 주소",
            database = "데이터베이스 이름",
            user = "유저 이름",
            password = "마스터 비밀번호"
        )

        # 추가된 부분
        with conn.cursor() as curs:
            try:
                curs.execute("INSERT into \"TemperatureInformation\" (name, temperature) values ('%s', '%s')")
                curs.execute("commit") # 트랜잭션 테이블에 반영

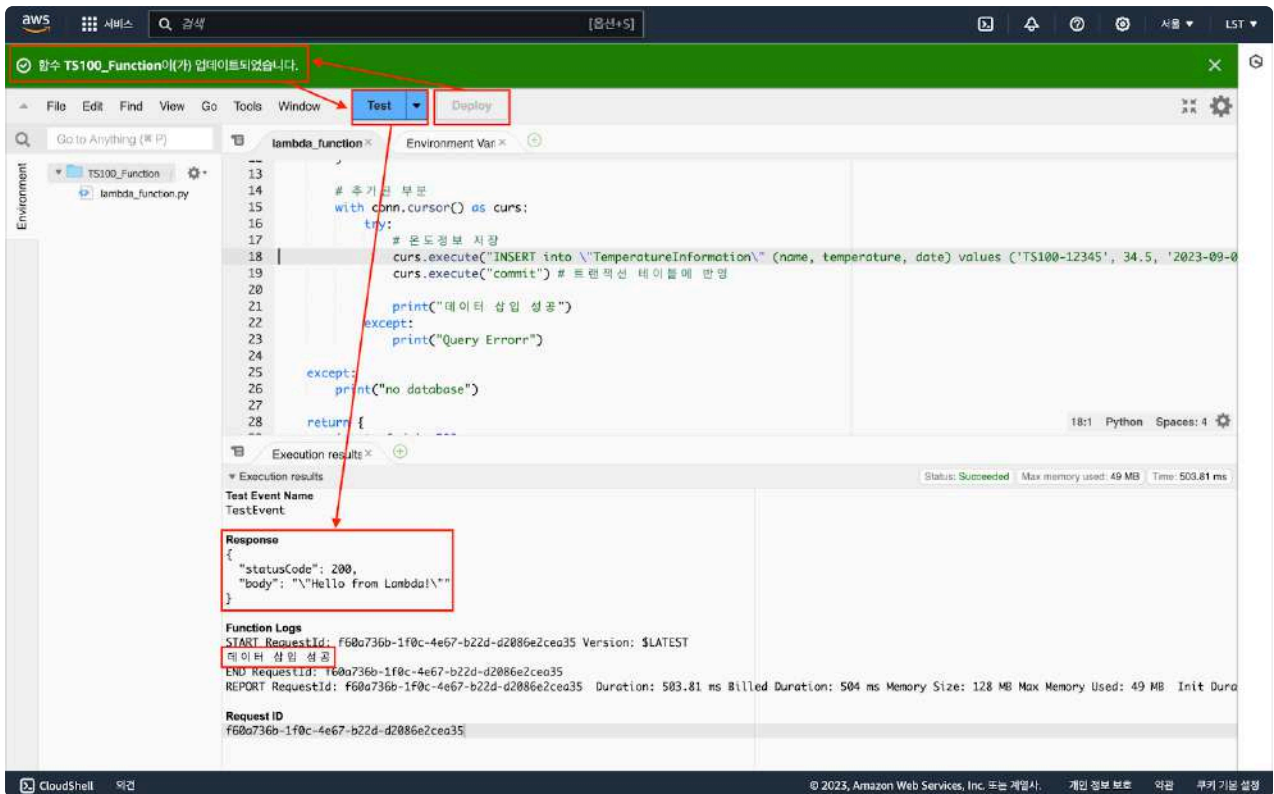
                print("데이터 삽입 성공")
            except:
                print("Query Errorr")

        except:
            print("no database")

    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }

```

이후 화면 중앙의 Deploy 버튼 클릭 후 함수 배포가 완료되면 Test 버튼을 클릭해 데이터베이스에 온도정보가 저장된다.

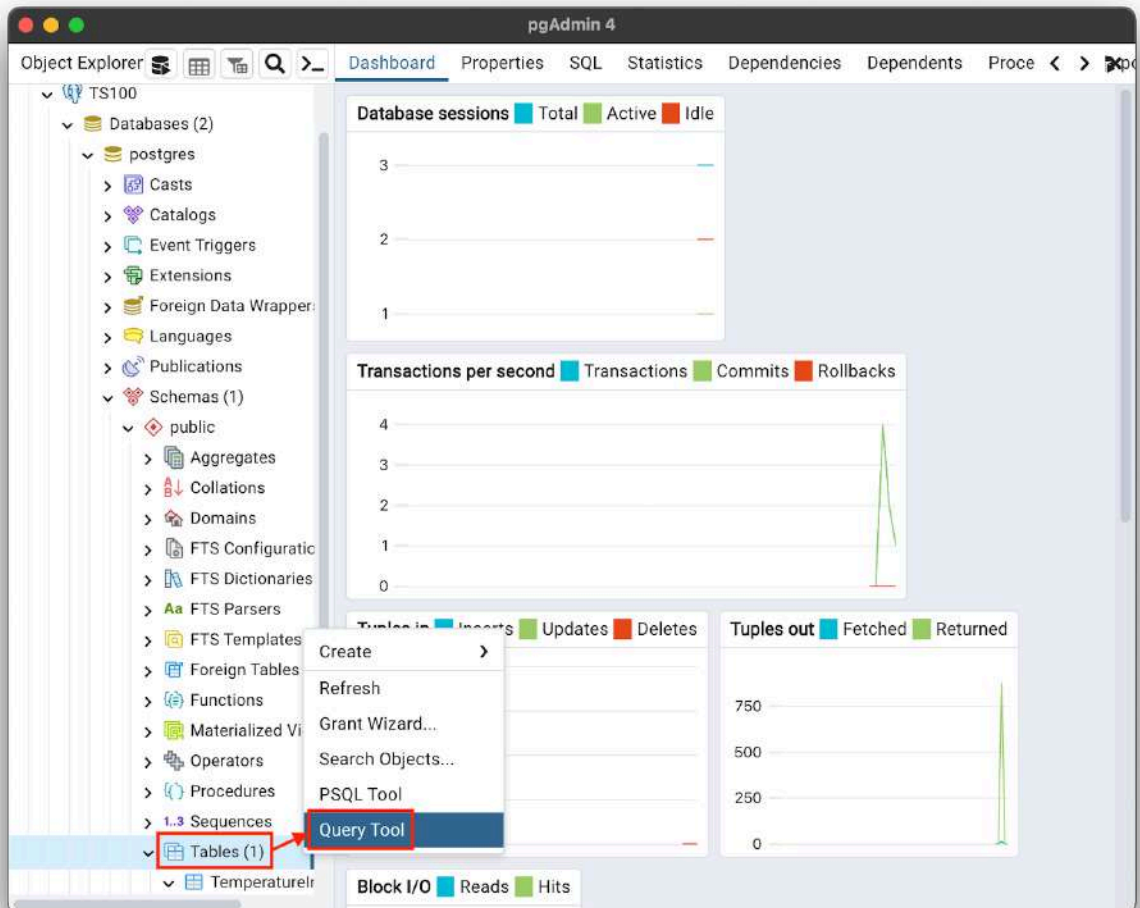


데이터베이스 정보 추가

클라우드 데이터베이스 온도 확인

데이터베이스에 저장된 온도정보를 확인하는 방법에 대해 설명한다.

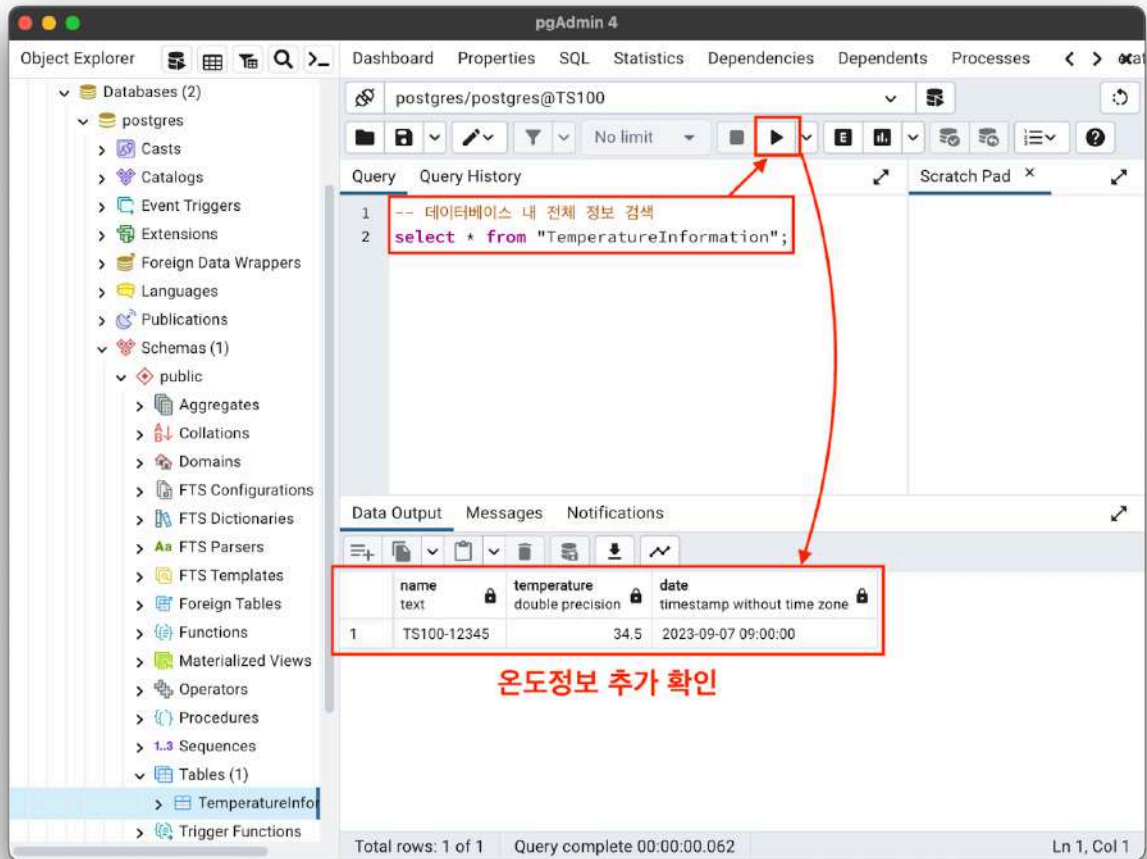
pgAdmin으로 온도정보 확인을 위해 pgAdmin 실행 후 Tables 우클릭 → Query Tool을 클릭해 Query 실행 화면으로 전환한다.



Query Tool 화면 이동방법

이후 Query 영역에 아래 SQL문 작성 후 실행버튼을 클릭하면 데이터베이스에 정보가 추가되어있는 것을 확인할 수 있다.

```
-- 데이터베이스 내 전체 정보 검색  
select * from "TemperatureInformation";
```



클라우드 데이터베이스 내 온도정보 추가 확인

Lambda 함수에서 저장된 온도정보를 수신하는 코드를 작성한다. Lambda 함수를 아래와 같이 수정 후 Deploy → Test 버튼을 클릭하면 아래 그림과 같이 클라우드 데이터베이스에 저장된 정보를 확인할 수 있다.

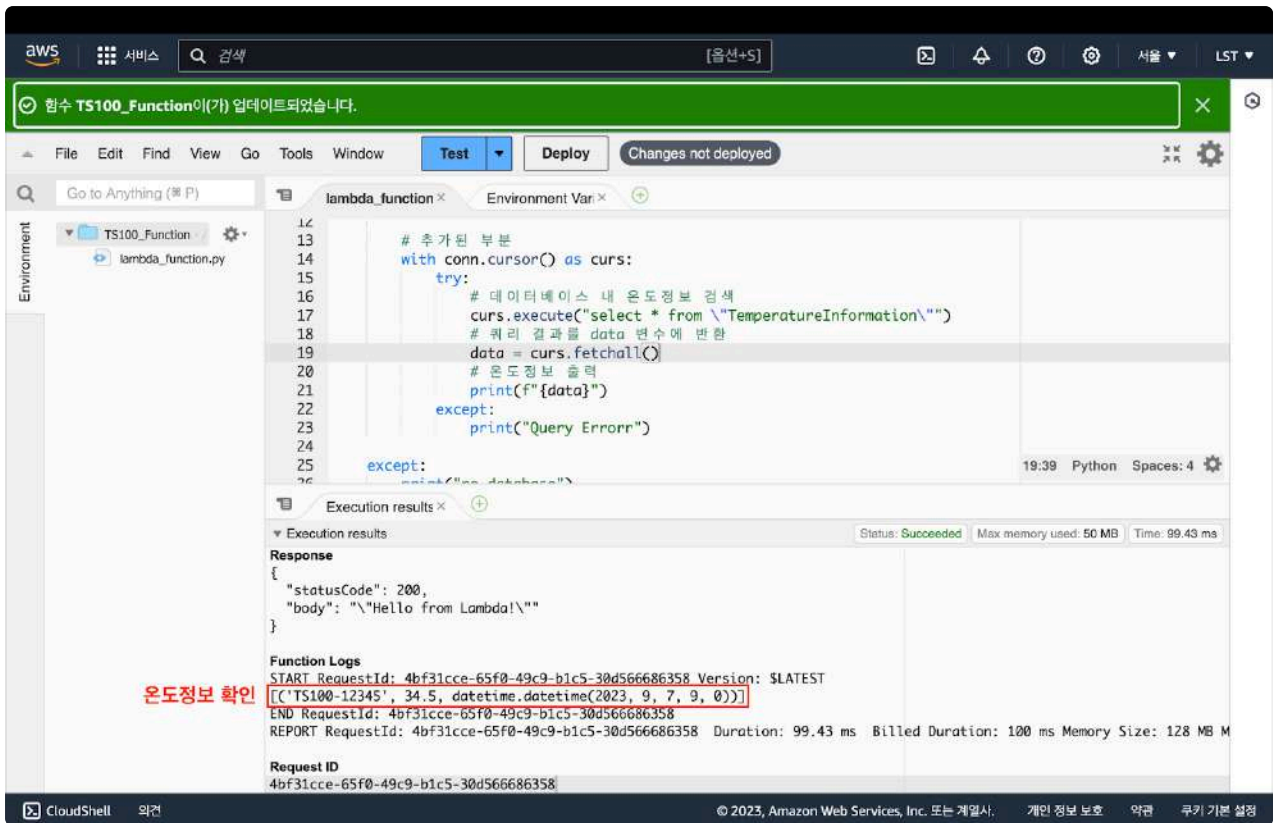
```
import psycopg2
import json

def lambda_handler(event, context):
    try:
        conn = psycopg2.connect(
            host = "호스트 주소",
            database = "데이터베이스 이름",
            user = "유저 이름",
            password = "마스터 비밀번호"
        )

        # 추가된 부분
        with conn.cursor() as curs:
            try:
                # 데이터베이스 내 온도정보 검색
                curs.execute("select * from \"TemperatureInformation\"")
                # 쿼리 결과를 data 변수에 반환
                data = curs.fetchall()
                # 온도정보 출력
                print(f"{data}")
            except:
                print("Query Errorr")

    except:
        print("no database")

    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```



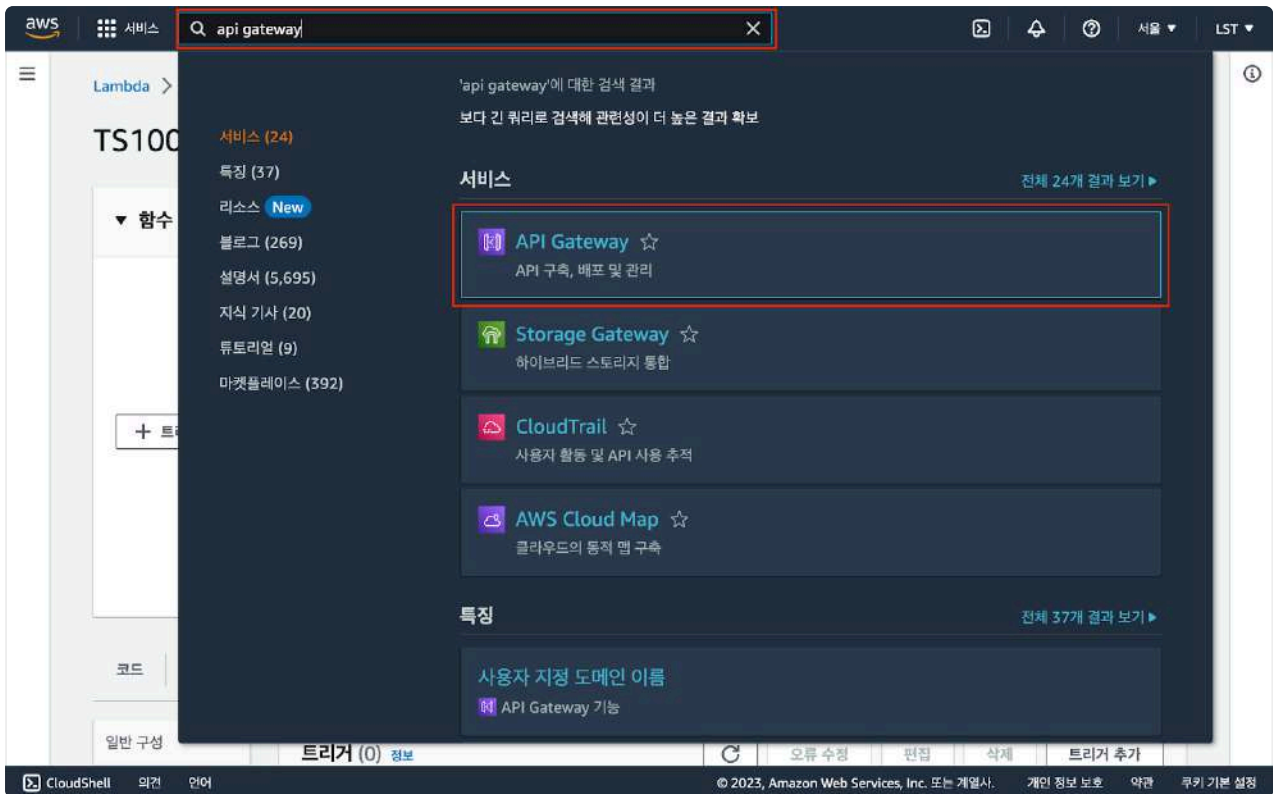
Lambda 함수로 온도 정보확인

API Gateway 연결

스마트폰에서 측정된 온도정보를 서버에 전달하기 위해서 Lambda와 API Gateway를 연결한다. API Gateway란 개발자가 API를 손쉽게 생성, 게시, 유지 관리, 모니터링을 제공하는 서비스이다.

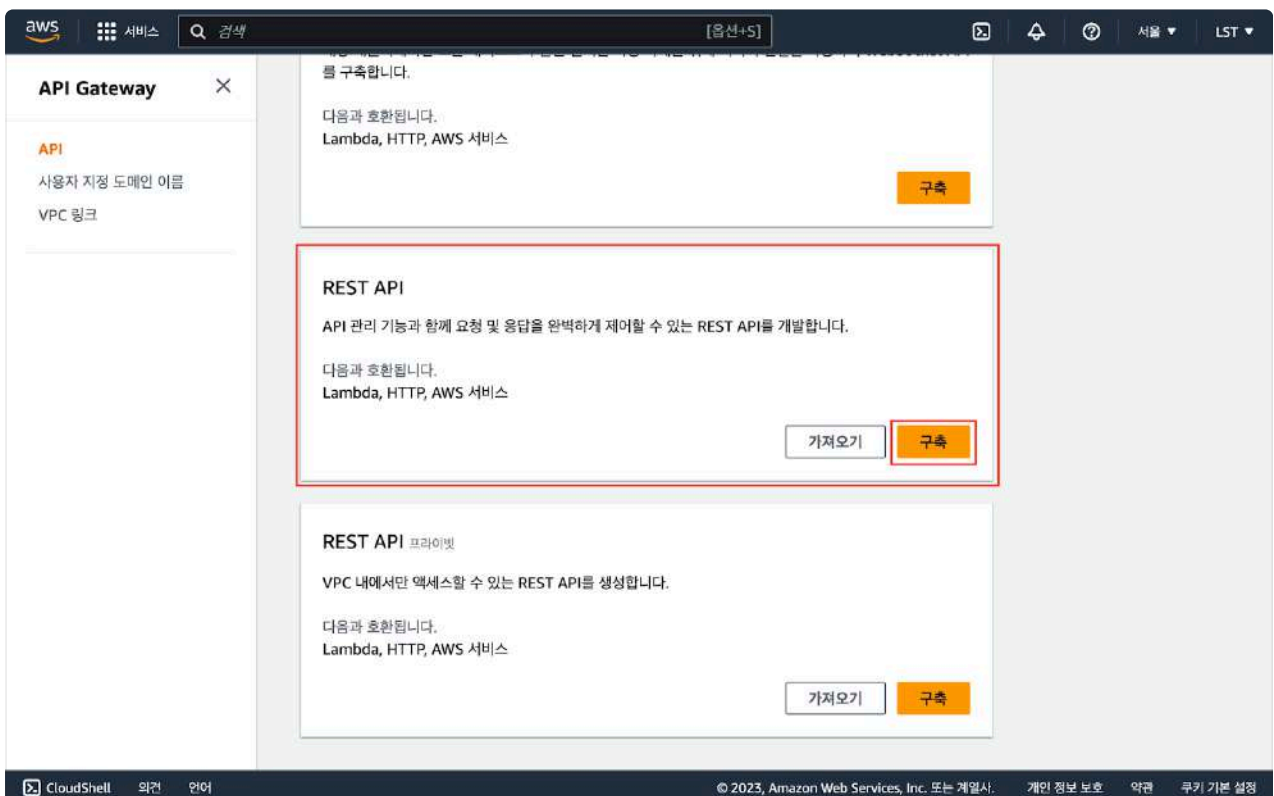
API Gateway 서버 연결

AWS 콘솔에서 검색영역에 API Gateway 입력 후 API Gateway를 클릭한다.



API Gateway 이동 방법

API Gateway 콘솔에서 아래로 스크롤 후 REST API 영역의 구축 버튼을 클릭한다.



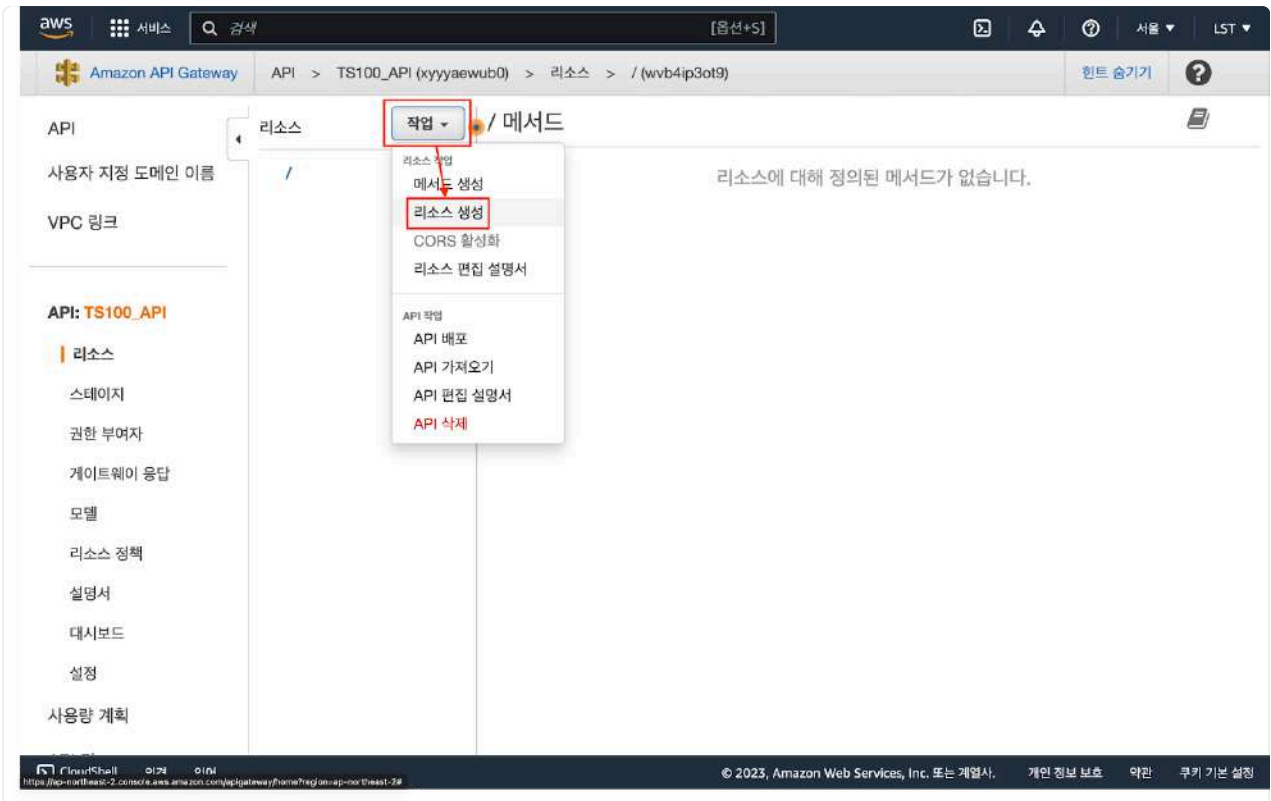
REST API 생성

REST API 생성화면에서 프로토콜은 REST, 새 API를 차례대로 선택 후 API 이름을 입력한다. API 이름 입력을 완료했다면, 우측 하단의 API 생성 버튼을 클릭해 API를 생성한다.

The screenshot shows the AWS API Gateway console interface for creating a new API. The top navigation bar includes the AWS logo, '서비스' (Services), a search bar, and user information. The main header shows 'Amazon API Gateway' and the breadcrumb 'API > 생성' (Create). The '프로토콜 선택' (Protocol Selection) section is active, with the instruction 'REST API 또는 WebSocket API의 생성 여부를 선택합니다.' (Select whether to create a REST API or a WebSocket API). The 'REST' radio button is selected and highlighted with a red box. Below this, the '새 API 생성' (Create New API) section is visible, with the instruction 'Amazon API Gateway에서 REST API는 HTTPS 엔드포인트를 통해 호출할 수 있는 리소스와 메서드의 컬렉션을 참조합니다.' (In Amazon API Gateway, a REST API is a collection of resources and methods that can be invoked over HTTPS endpoints). The '새 API' (New API) radio button is selected and highlighted with a red box. The '설정' (Settings) section follows, with the instruction 'API에 대해 기억하기 쉬운 이름과 설명을 선택합니다.' (Select an easy-to-remember name and description for the API). The 'API 이름*' (API Name*) field is filled with 'TS100_API' and is highlighted with a red box. The '설명' (Description) field is empty. The '엔드포인트 유형' (Endpoint Type) dropdown is set to '지역' (Regional). At the bottom right, the 'API 생성' (Create API) button is highlighted with a red box. The footer includes links for 'CloudShell', '의견' (Feedback), '언어' (Language), and copyright information for Amazon Web Services, Inc. (© 2023).

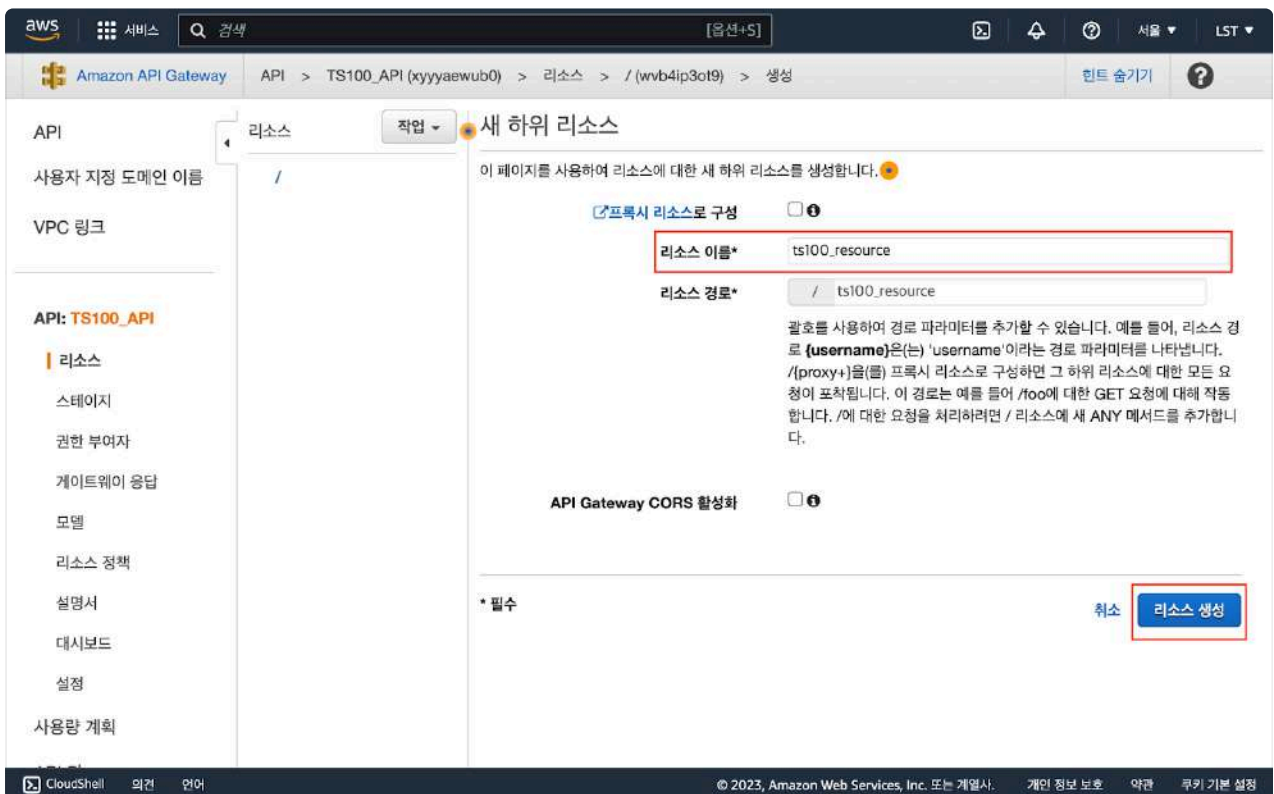
REST API 생성 방법

API Gateway 콘솔에서 상단의 작업 → 리소스 생성 버튼을 차례대로 클릭해 리소스를 생성한다.



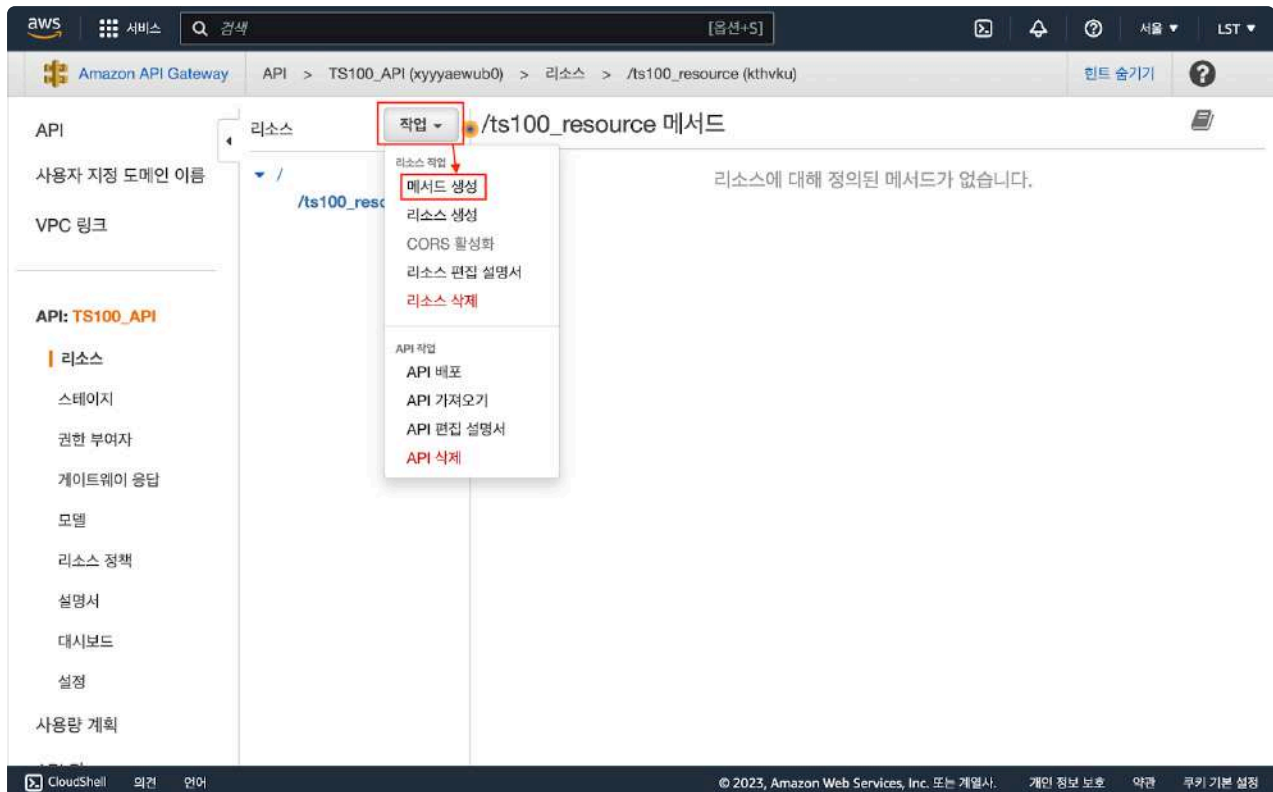
리소스 생성 방법

새 하위 리소스 영역이 나타나면 리소스 이름 입력 후 우측 하단의 리소스 생성 버튼을 클릭해 리소스를 생성한다.



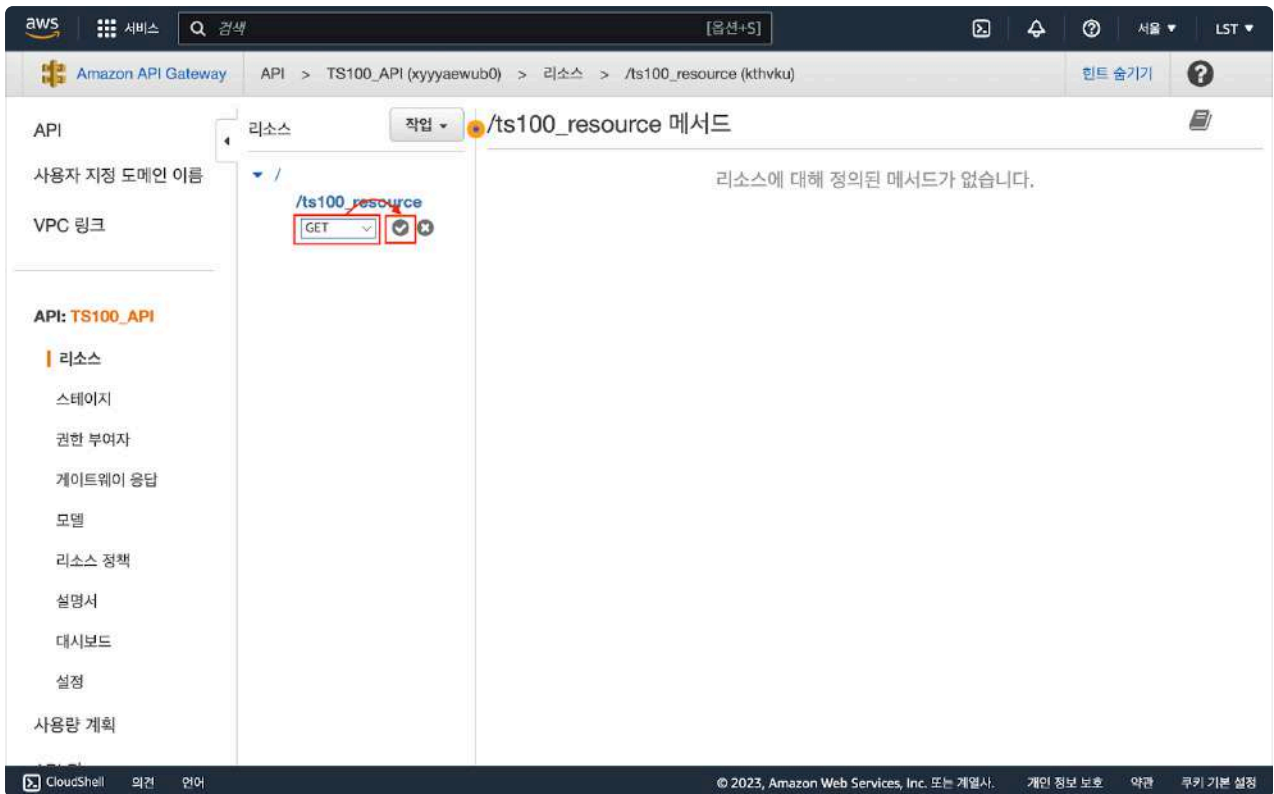
새 리소스 생성 방법

ts100_resource 리소스가 생긴것 확인 후 작업 → 메서드 생성 버튼을 차례대로 클릭해 메서드를 생성한다.



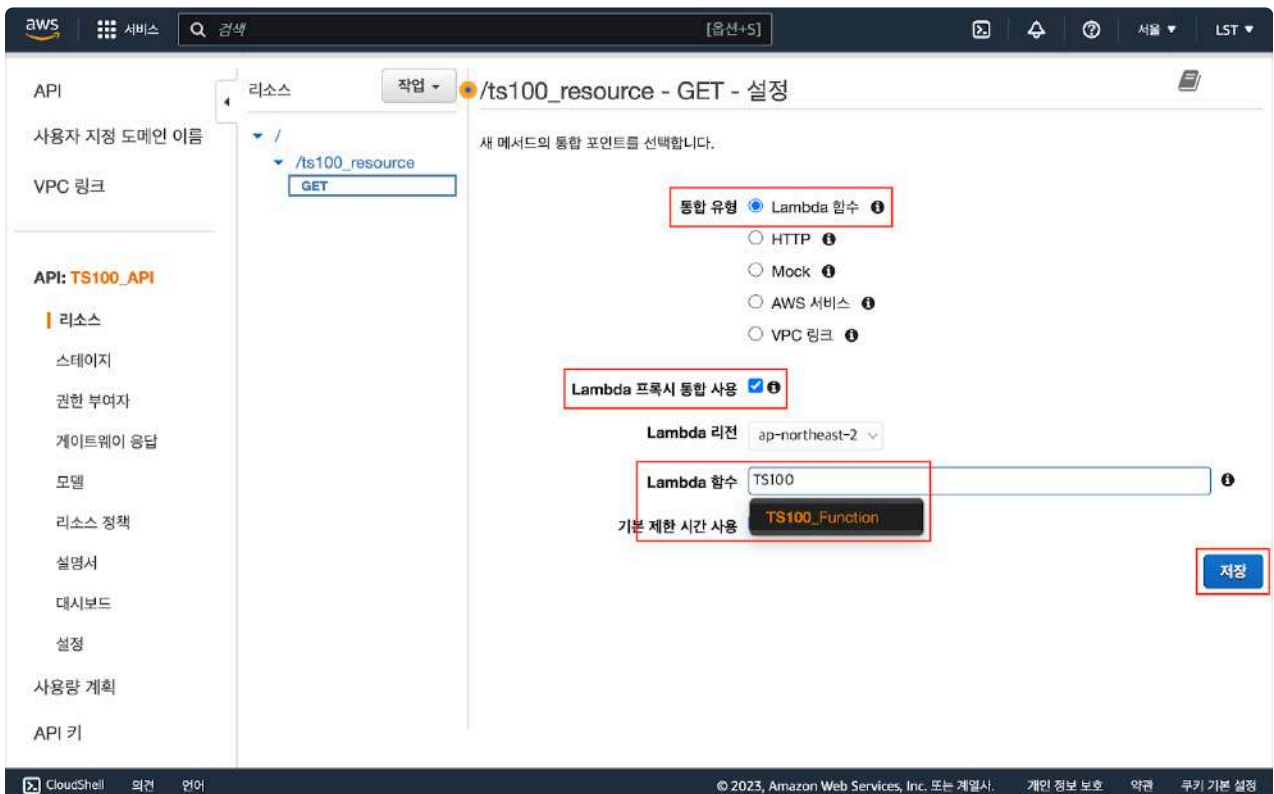
리소스 내 메서드 생성

ts100_resource 아래영역에 나와있는 메서드를 GET으로 설정하고 우측의 체크버튼을 클릭한다.



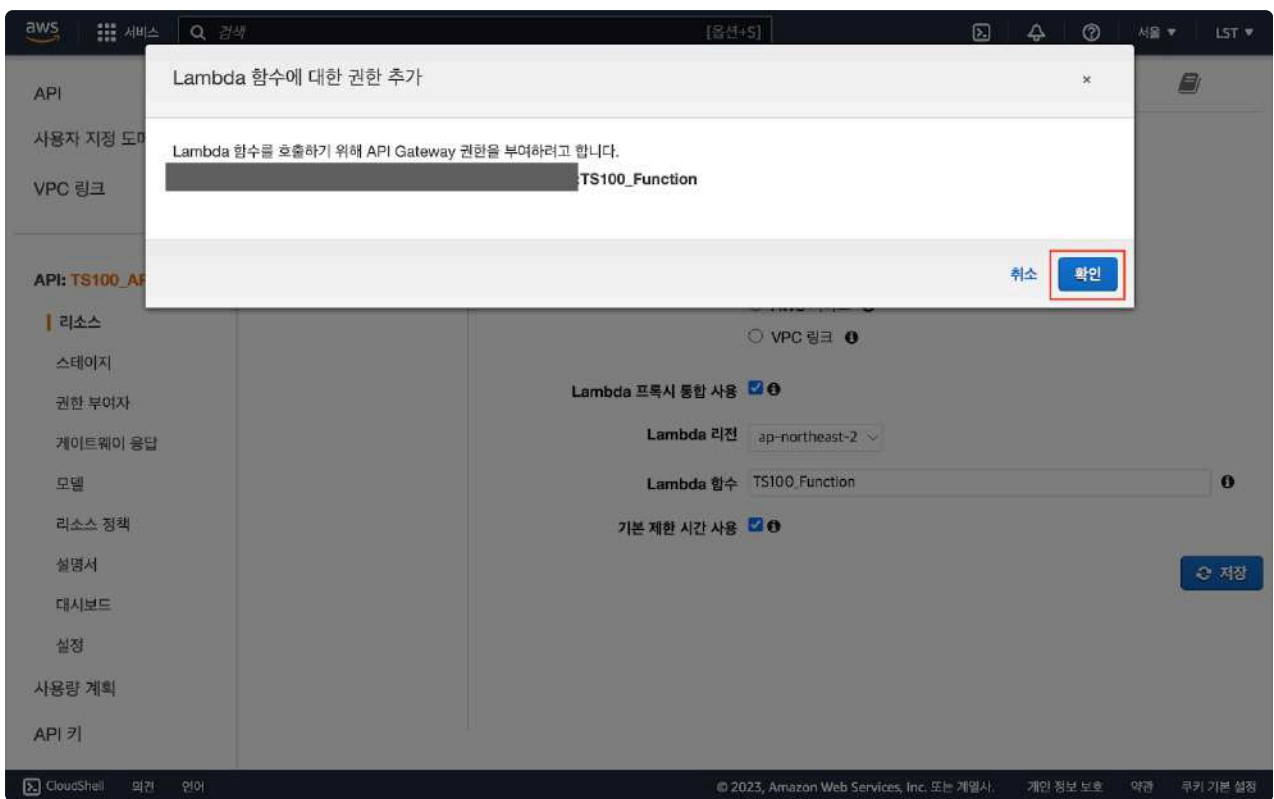
메서드 타입 설정

우측에 나타난 설정 영역을 아래 그림과 같이 설정 후 저장 버튼을 클릭해 메서드를 생성한다. 이때, API Gateway와 Lambda 함수가 연결된다.



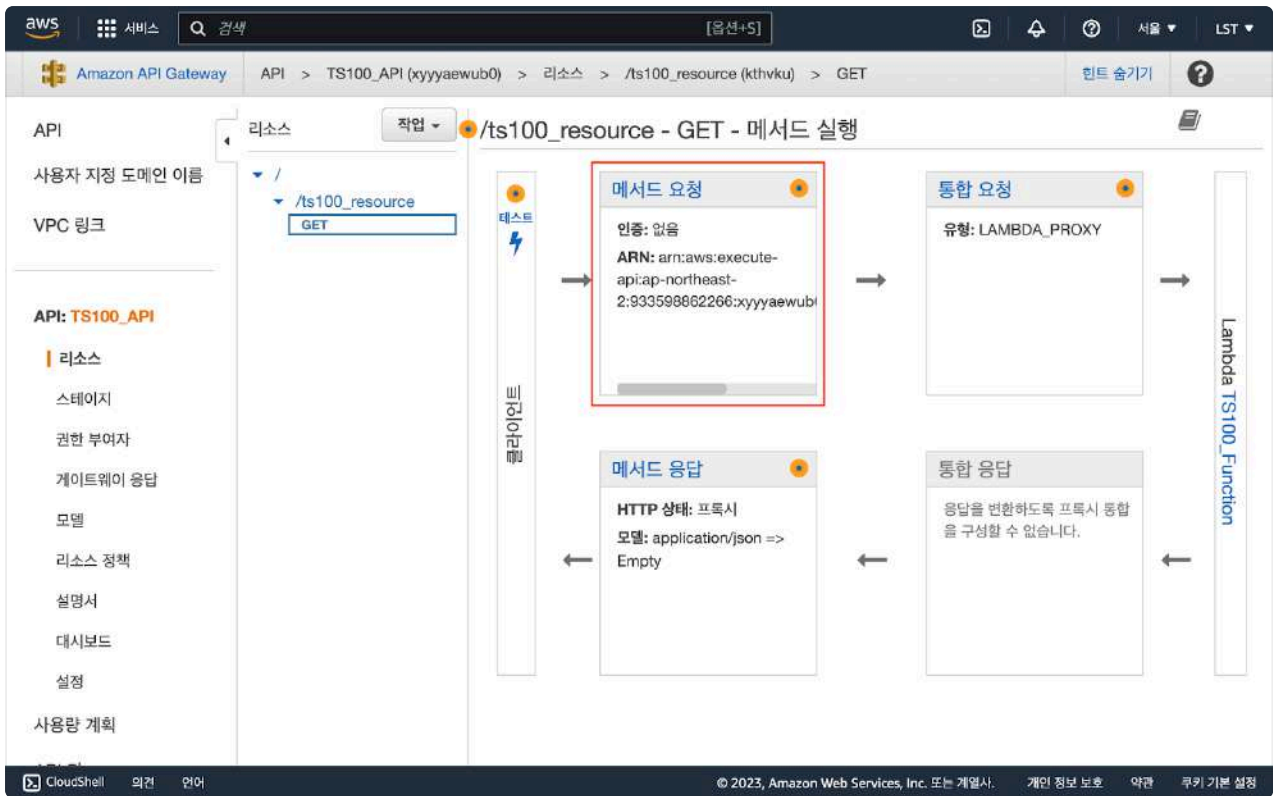
메서드 속성 설정 방법

Lambda 함수에 대한 권한 추가 팝업이 발생하면 확인 버튼을 클릭한다.



Lambda 함수 권한 추가 팝업

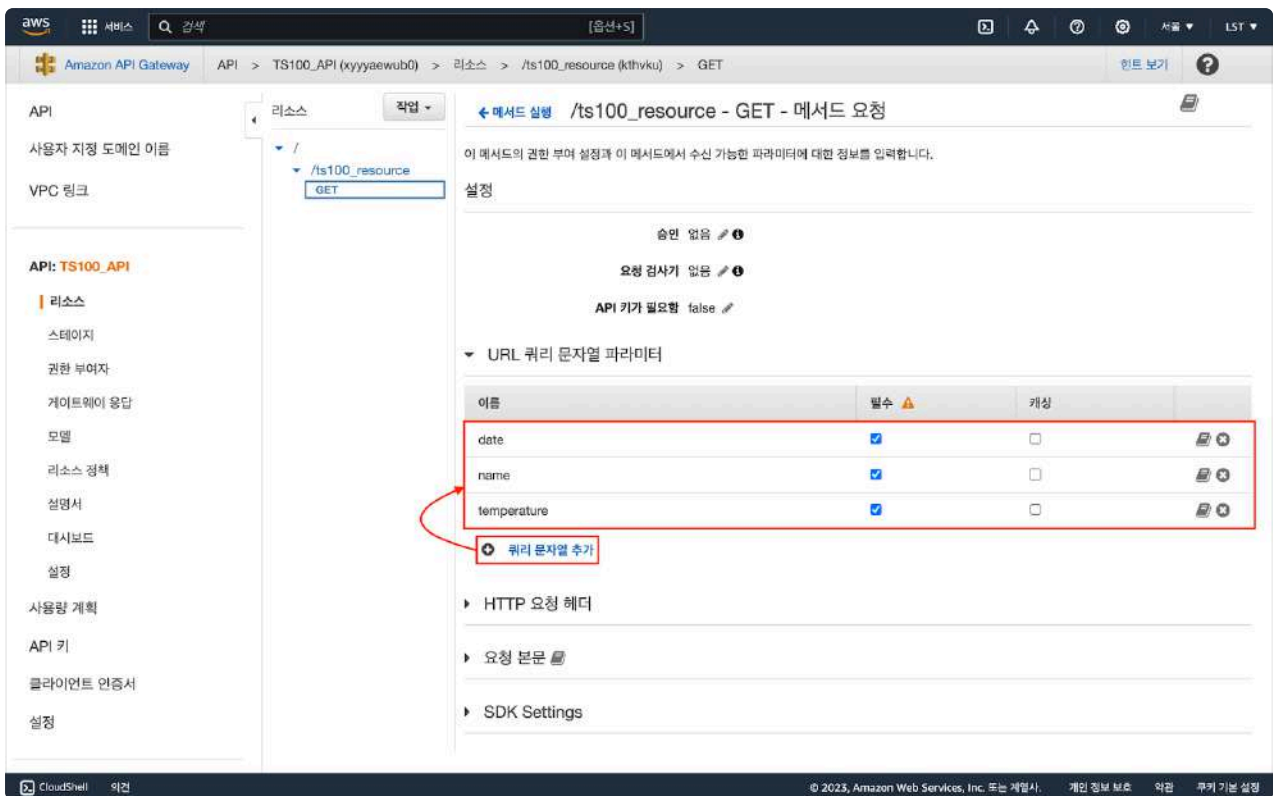
함수 생성을 완료하면 아래 화면이 나타난다. 메서드 요청 버튼을 클릭해 메서드 속성 관련 설정 화면으로 이동한다.



메서드 설정 이동 방법

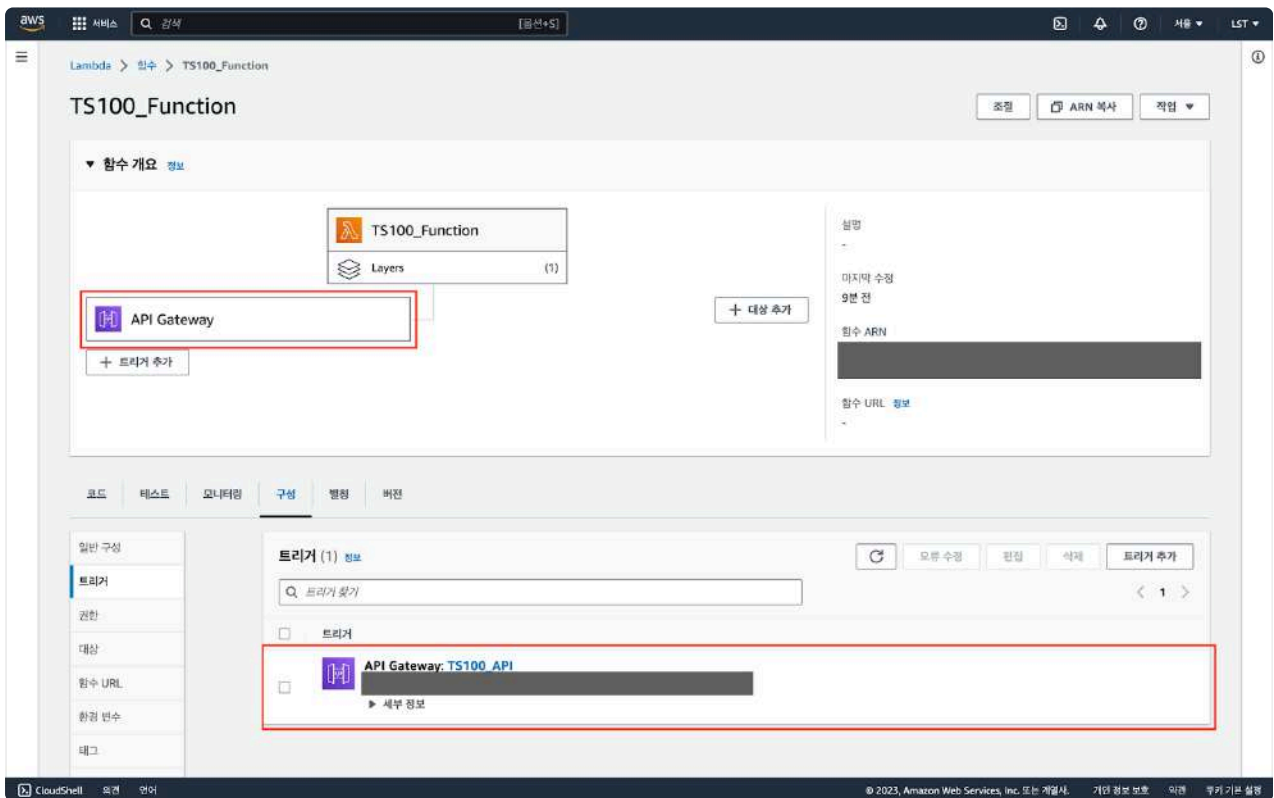
메서드 콘솔 설정 화면으로 이동 후 하단의 쿼리 문자열 추가버튼 클릭 후 아래 표와 같이 파라미터를 추가한다.

이름	필수	캐싱
name	YES	NO
temperature	YES	NO
date	YES	NO



메서드 문자열 파라미터 추가

다시 Lambda 콘솔로 돌아오면 TS100_Function 함수와 API Gateway가 연결된것을 확인할 수 있다.



API Gateway 연결 후 Lambda 함수 콘솔

이후 코드영역을 아래와 같이 수정한다. 파라미터를 올바르게 수신했다면 바나나 체온계의 기기 이름, 온도, 날짜정보를 리턴한다.

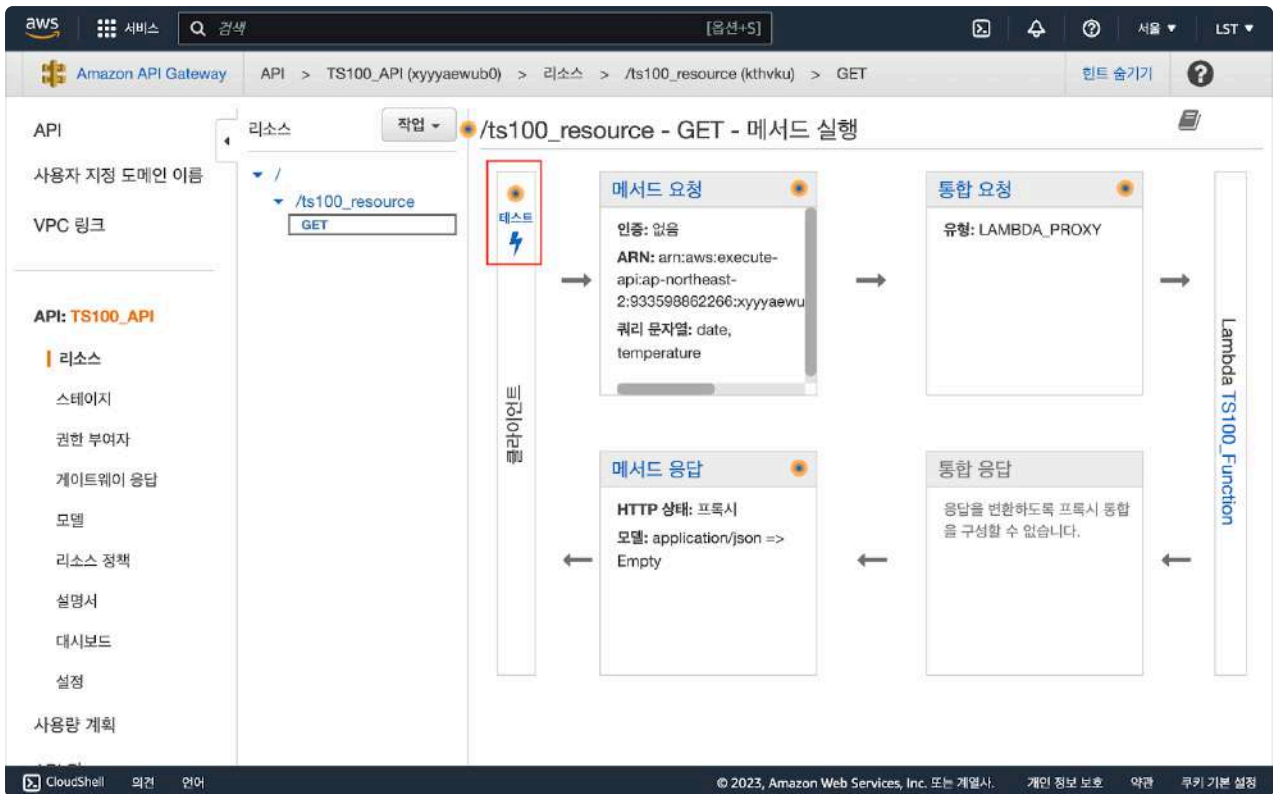
```
import psychopg2
import json

def lambda_handler(event, context):
    # 추가된 부분
    query_parameters = event.get('queryStringParameters', {})

    # 바나나 체온계 이름, 온도, 날짜정보 수신
    name = query_parameters.get('name', None)
    temperature = query_parameters.get('temperature', None)
    date = query_parameters.get('date', None)

    # 파라미터가 올바르게 수신되었다면
    if name is not None and temperature is not None and date is not None:
        return {
            'statusCode': 200,
            'body': json.dumps([name, temperature, date])
        }
    else:
        return {
            'statusCode': 400,
            'body': json.dumps('Parameter Error')
        }
```

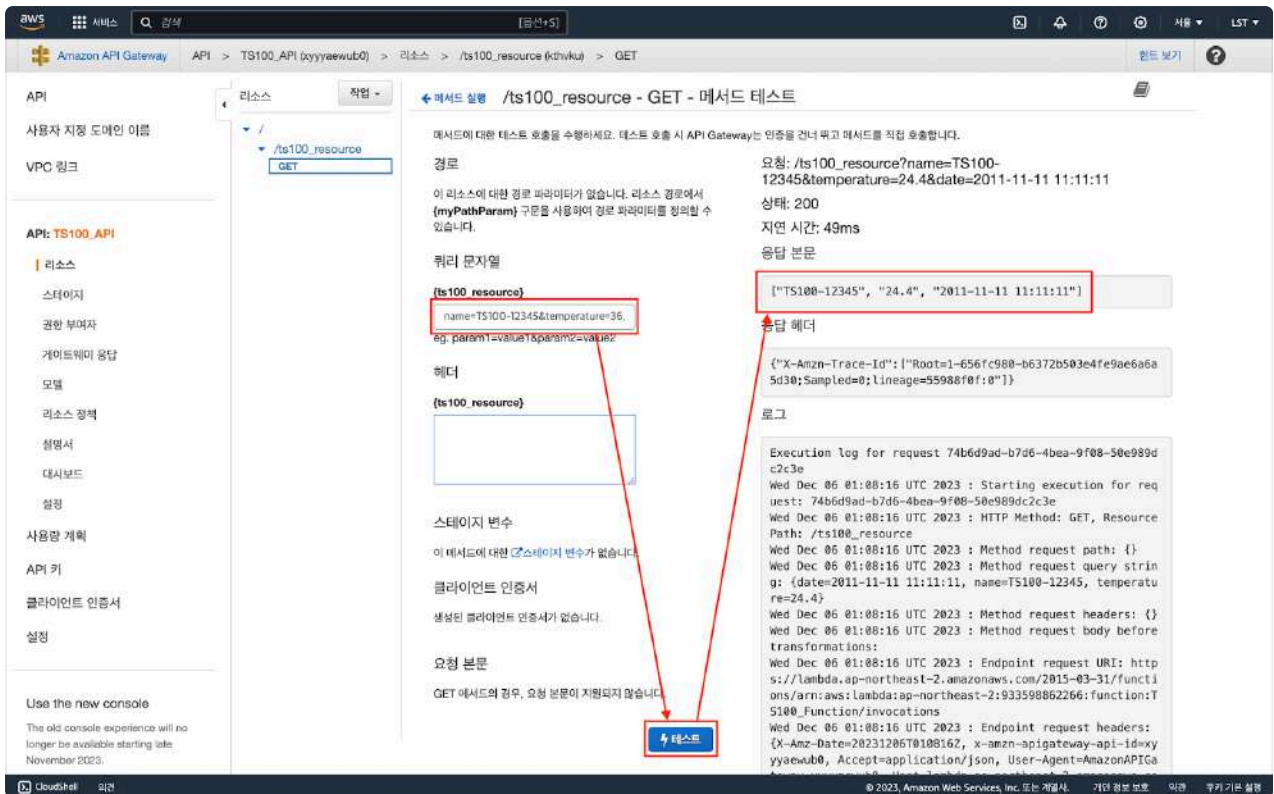
Lambda 함수가 온도정보를 올바르게 수신하는지 확인하기 위해 API Gateway 콘솔로 이동 후 테스트 버튼을 클릭해 테스트 화면으로 이동한다.



API 테스트 영역으로 이동

쿼리 문자열을 아래와 같이 입력 후 테스트 버튼을 클릭하면 응답 본문에 온도, 날짜정보가 출력되는것을 확인할 수 있습니다.

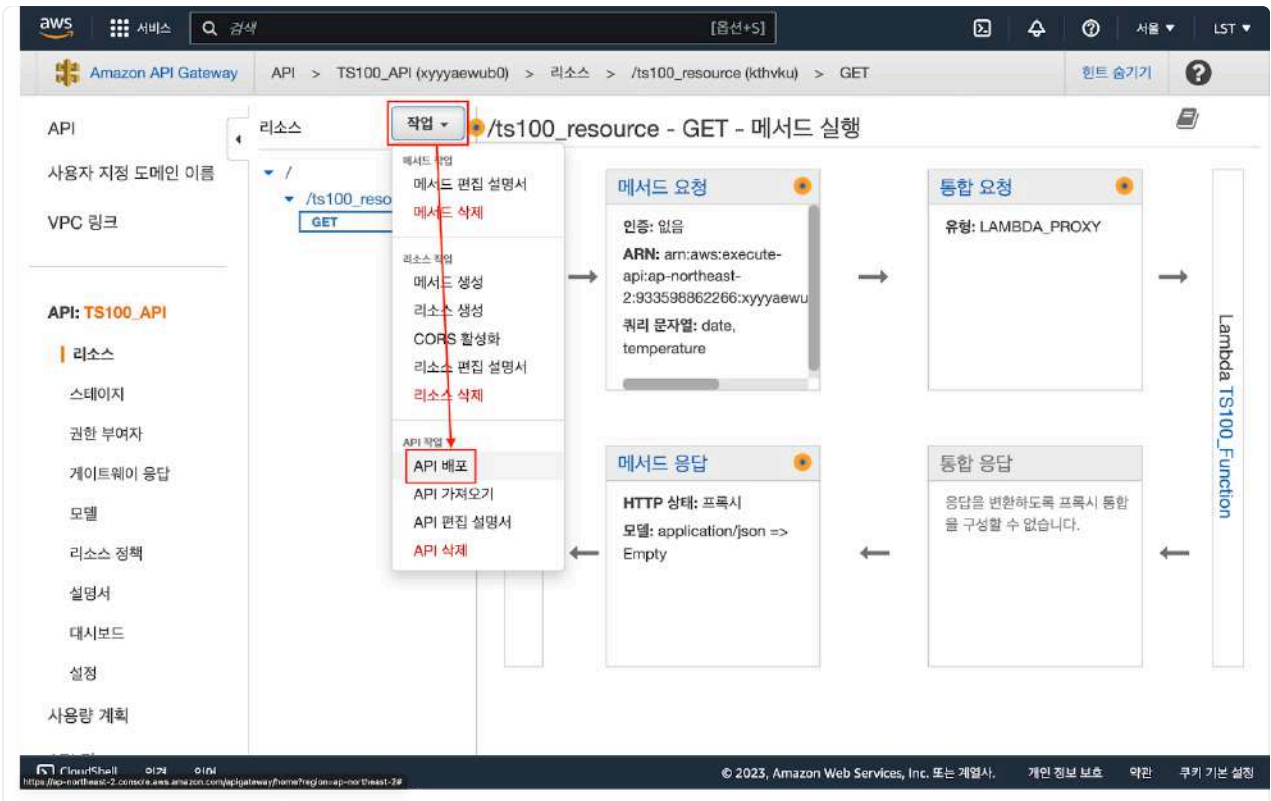
```
name=TS100-12345&temperature=36.5&date=2011-11-11 11:11:11
```



API 메서드 테스트

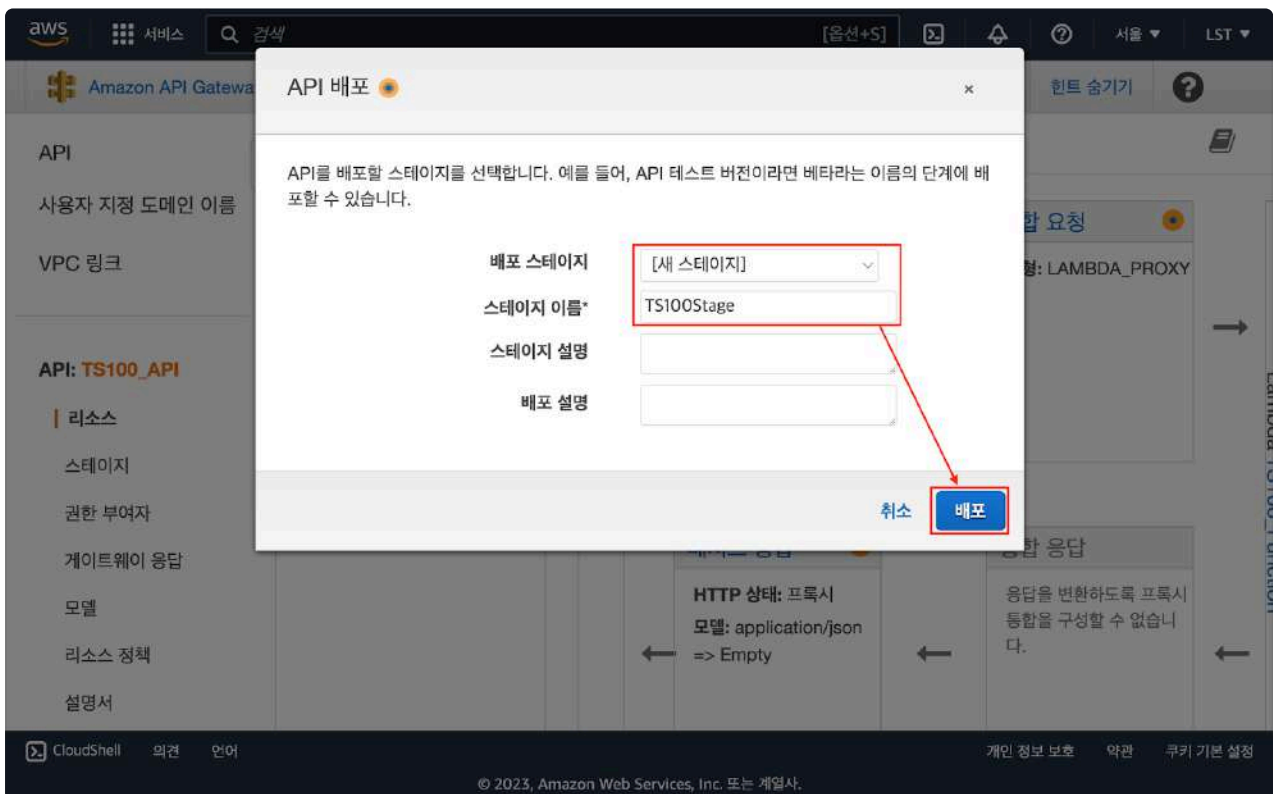
API 배포

API 메서드 테스트를 진행했으니, API를 배포해 직접 URL로 API를 호출을 수행한다. API 리소스 콘솔에서 작업 → API 배포 버튼을 클릭한다.



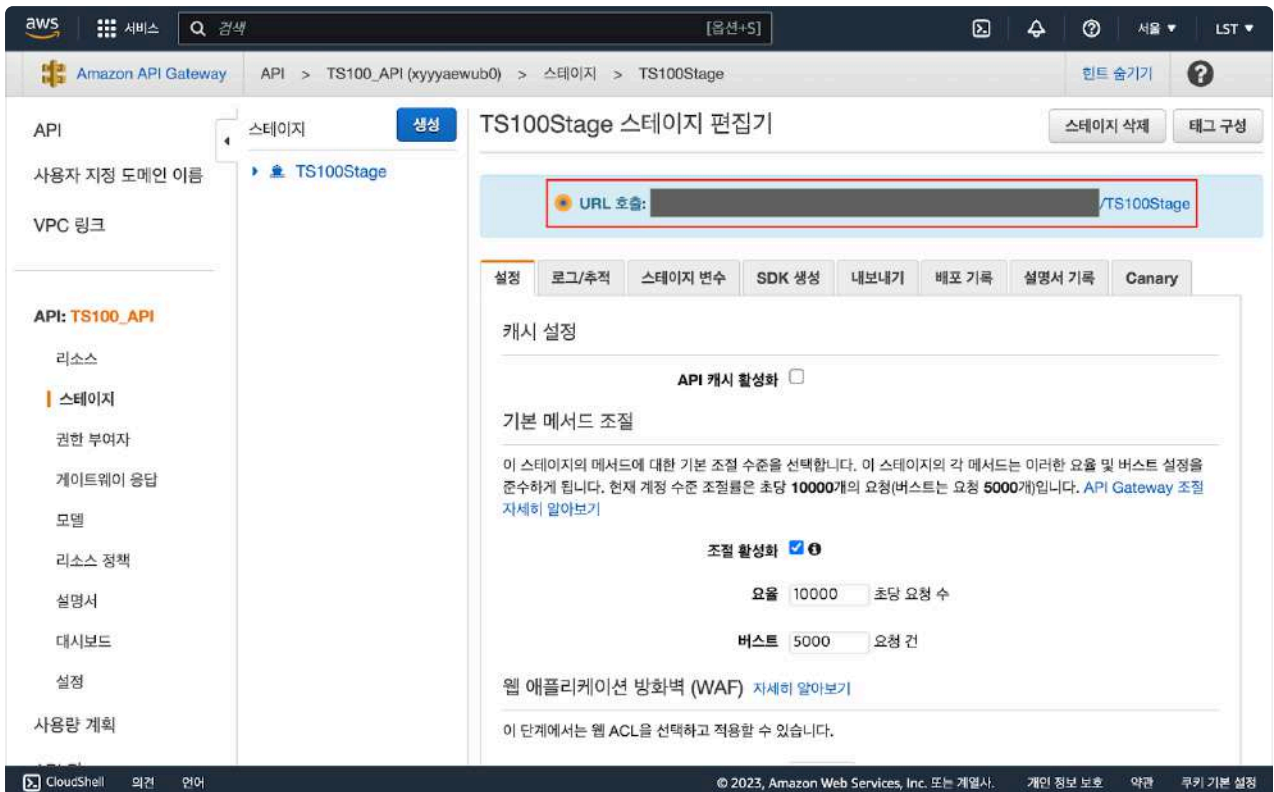
API 배포

API 배포 팝업이 나타나면, 배포 스테이지를 [새 스테이지]로 설정하고, 스테이지 이름 입력 후 배포 버튼을 클릭해 API를 배포한다.



API 배포 팝업

API배포를 수행하게 되면 API 스테이지 화면으로 이동되고, 화면 상단에 API URL이 나타나게 됩니다.

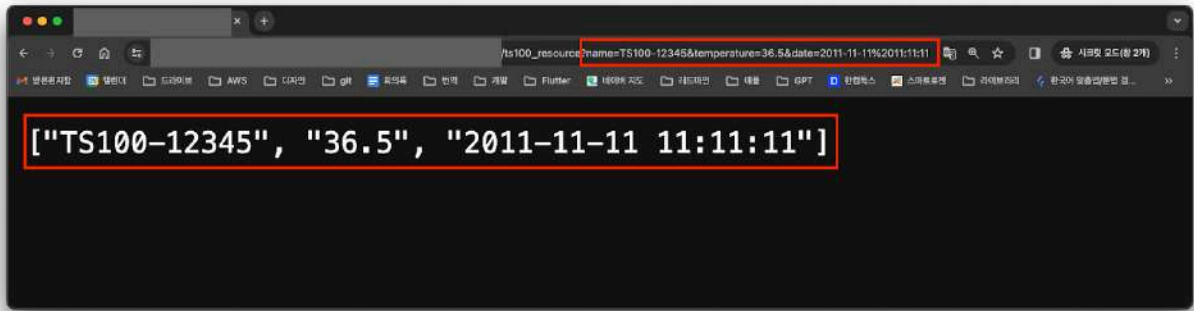


API 스테이지

API URL 호출

이제 API를 생성했으니, URL로 API를 호출해 보겠습니다. 웹 화면에서 아래와 같이 URL작성 후 접속하면 API에 작성했던 온도 정보가 화면에 출력되는것을 확인할 수 있습니다.

```
API 스테이지 URL + /ts100_resource?name=TS100-12345&temperature=36.5&date=2011-11-11 11:11:11
```



API 호출 테스트

5.9. 클라우드 데이터베이스에 온도 저장

이전 과정을 통해 API Gateway, Lambda, 데이터베이스를 연결하였다. 본 단락에서 스마트폰에서 측정된 온도정보를 클라우드 데이터베이스에 저장하는 기능을 구현한다.

서버 코드 변경

Lambda 함수를 API가 호출되면 데이터베이스에 온도정보를 저장할 수 있도록 아래 코드로 수정한다.

```

import psycopg2
import json
from datetime import datetime

def lambda_handler(event, context):
    query_parameters = event.get('queryStringParameters', {})

    # 이름, 온도, 날짜정보 수신
    name = query_parameters.get('name', None)
    temperature = query_parameters.get('temperature', None)
    date = query_parameters.get('date', None)

    # 파라미터가 없다면 오류 반환
    if name is None or temperature is None or date is None:
        return {
            'statusCode': 400,
            'body': json.dumps(f'parameter error {name}, {temperature}, {
}

    # 문자열을 적절한 형태로 변환
    temperature = float(temperature)
    date = datetime.strptime(date, "%Y-%m-%d %H:%M:%S")

    try:
        # 데이터베이스 연결
        conn = psycopg2.connect(
            host = "호스트 주소",
            database = "데이터베이스 이름",
            user = "유저 이름",
            password = "마스터 비밀번호"
        )
    except:
        return {
            'statusCode': 401,
            'body': json.dumps('Database Connect Error')
        }

    with conn.cursor() as curs:
        try:
            # 온도정보 저장
            curs.execute("""INSERT INTO "TemperatureInformation" (name, t
            # 트랜잭션 테이블에 반영
            curs.execute("commit;")

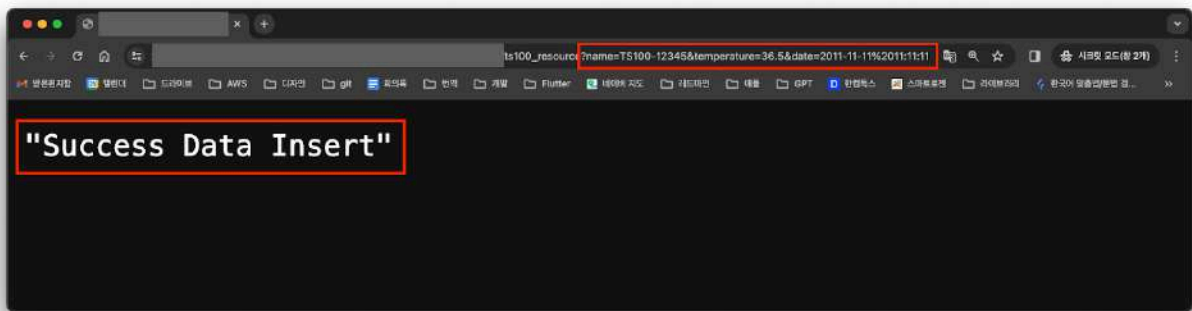
        except:
            return {
                'statusCode': 402,
                'body': json.dumps('Insert Error')
            }

```

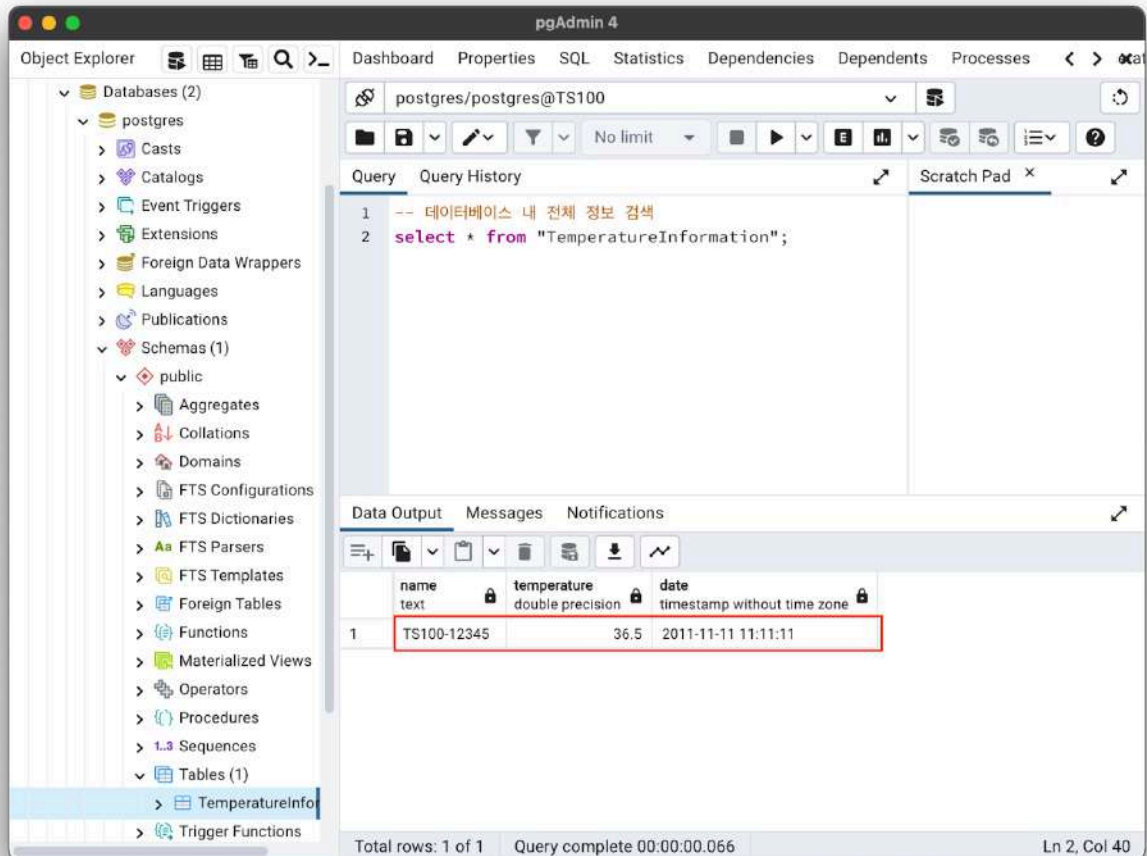
```
}  
  
return {  
  'statusCode': 200,  
  'body': json.dumps('Success Data Insert')  
}
```

이후 아래 URL을 사용해 API 호출을 테스트해보면 Success Data Insert가 화면에 나타나고, pgAdmin4에 온도정보가 올바르게 저장된것을 확인할 수 있다.

API 스테이지 URL + /ts100_resource?name=TS100-12345&temperature=36.5&date=2011-11-11 11:11:11



API로 데이터베이스 저장 성공



pgAdmin4 온도정보 추가 확인

온도 정보 클라우드에 저장

각 플랫폼별 클라우드 데이터베이스에 온도정보를 저장하는 방법에 대해 설명한다.

Raspberry Pi(Python)

클라우드에 온도정보를 저장하기 위해 API 호출 관련 모듈을 아래와 같이 추가한다.

```
# API 호출 관련 라이브러리
import requests
```

이후 API 호출 함수를 아래와 같이 작성한다.

```

def send_cloud(temperature, date):
    # String으로 변환
    convert_date = date.strftime("%Y-%m-%d %H:%M:%S")
    convert_temperature = str(temperature)

    # URL 생성
    base_url = "API 스테이지 URL"
    url_params = {'name': selected_device.name, 'date': convert_date, 'te
    my_url = f"{base_url}?name={url_params['name']}&date={url_params['dat

    print(my_url)
    # URL 인코딩
    encoded_url = requests.utils.requote_uri(my_url)

    # HTTP GET 요청
    response = requests.get(encoded_url)

    # 상태 확인
    if response.status_code in range(200, 300):
        print("Success")
    elif response.status_code in range(400, 500):
        print("Error")
    else:
        print("Default Error")

```

바나나 체온계로부터 온도정보를 수신했을 때 API호출하는 메서드를 추가 후 실행하면 온도정보가 클라우드에 저장되는 코드가 실행되는것을 확인할 수 있다.

```

# 온도정보 변환
def parse_temperature_information(data: bytearray):
    global cursor

    # 온도정보 변환
    temperature = temperature_calculate(data[1], data[2], data[3], data[4])

    array_slice = data[5:]
    date_array = list(map(int, array_slice))
    # 날짜정보 변환
    date = date_calculate(date_array)
    # 온도, 날짜정보 출력
    print(date)
    print(temperature)

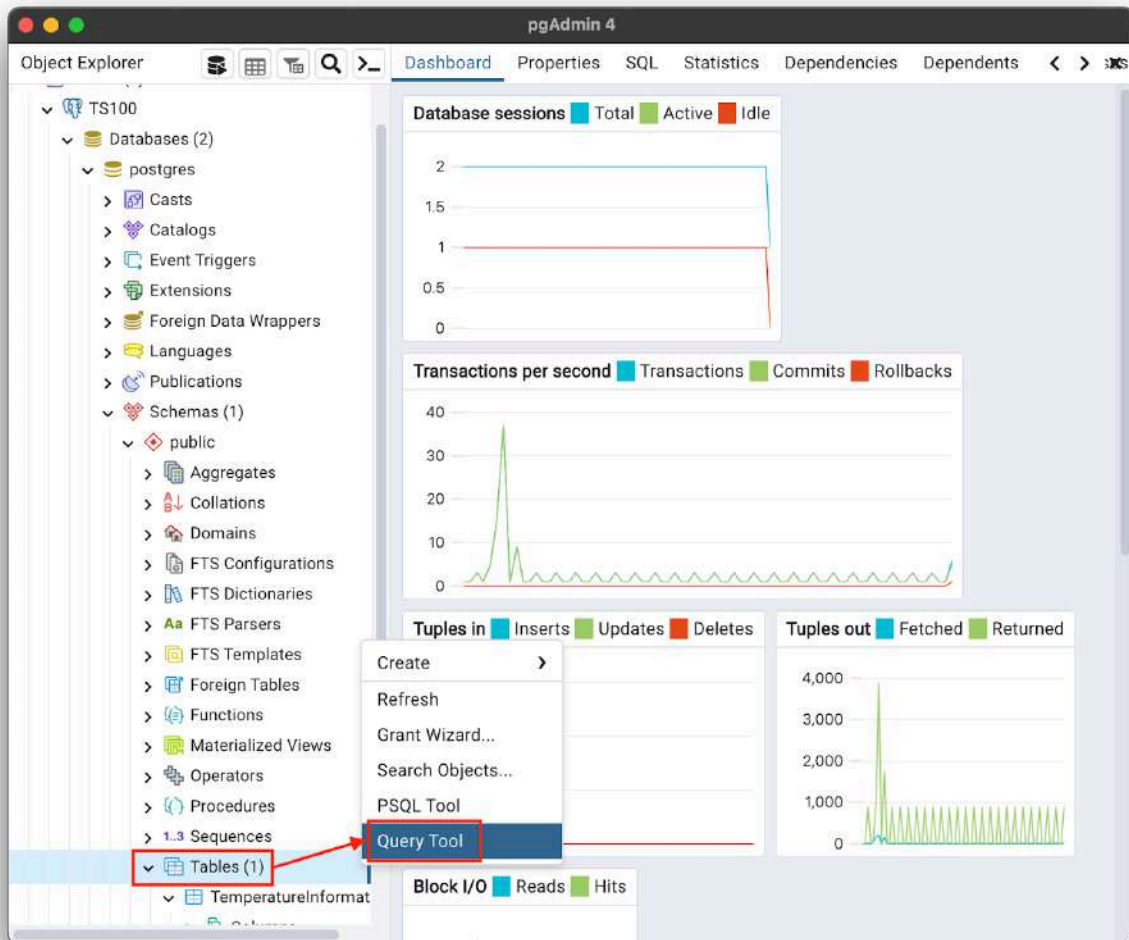
    # 데이터베이스에 저장
    cursor.execute("INSERT INTO TEMPERATURE (name, temperature, date) VALUES (%s, %s, %s)" % (data[0], temperature, date))
    conn.commit()

    # 클라우드에 저장(추가된 부분)
    send_cloud(temperature, date)

```

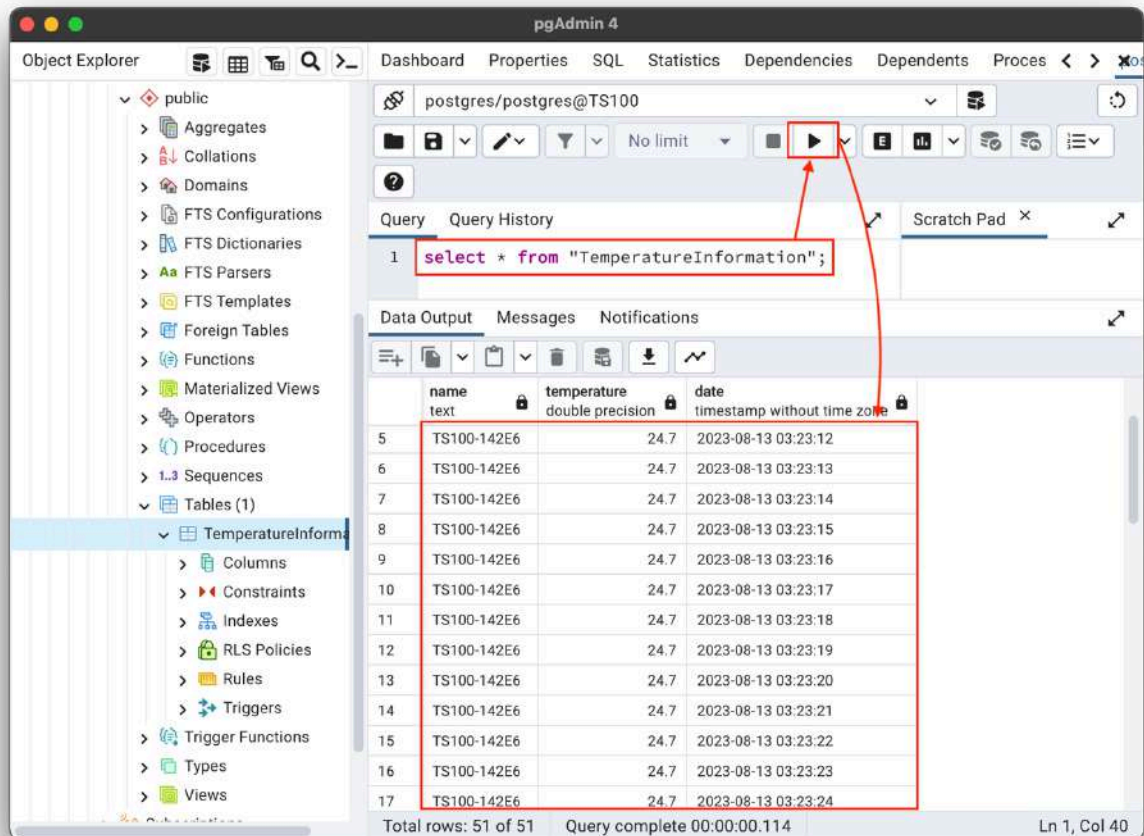
클라우드 데이터베이스 정보 확인

클라우드에 저장된 온도정보 확인을 위해 [pgAdmin4](#)를 실행해 Tables 우클릭 → Query Tool을 클릭해 Query Tool 화면으로 전환한다.



Query Tool 실행 방법

Query문을 아래와 같이 작성한 후 실행버튼을 클릭하면 1초단위로 온도정보가 클라우드 데이터베이스에 저장되어있는것을 확인할 수 있다.



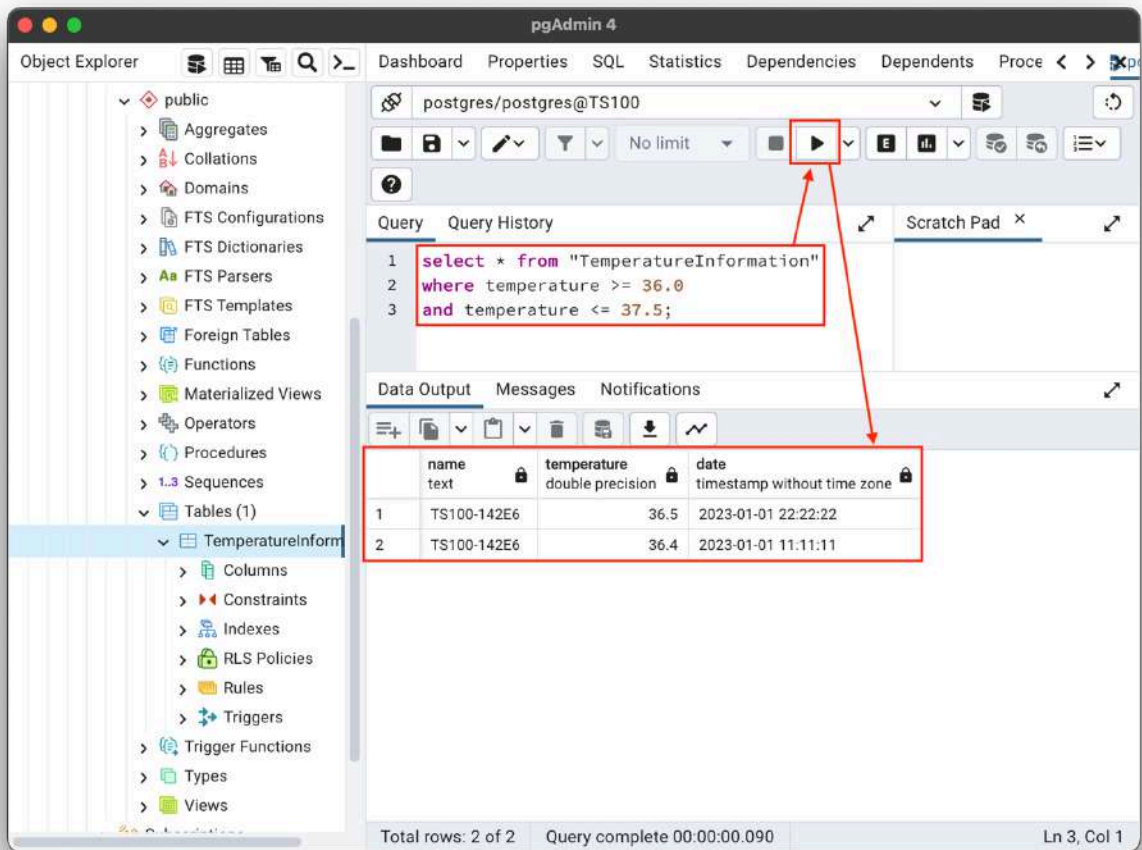
온도정보 클라우드 저장 확인

온도정보 필터링

특정 조건을 만족하는 온도정보를 출력하는 방법에 대해 설명한다. 체온의 정상 범위인 36 ~ 37.5°C 내에 있는 온도정보를 확인하기 위해 SQL문을 아래와 같이 작성한다.

```
select * from "TemperatureInformation" where temperature >= 36.0 and  
temperature <= 37.5;
```

SQL문을 작성하고 실행하면 아래와 같이 정상 온도범위의 온도정보만 출력되는것을 확인할 수 있다.



정상 온도범위 내 온도정보 출력 확인

6. 부록: BLE_UART

6.1. BLE_UART

본 단락은 Raspberry Pi를 이용해 BLE_UART의 데이터를 수신하는 방법에 대해 설명한다.

BLE_UART는 UART(유선)와 BLE를 사용하여 MCU(예: 아두이노)와 무선으로 BLE 통신이 가능하다. UART는 MCU와의 직접적인 연결을 통해 데이터 전송을 담당하며, BLE 기술을 이용해 정보를 교환한다.



BLE_UART

6.1.1. 프로젝트 생성

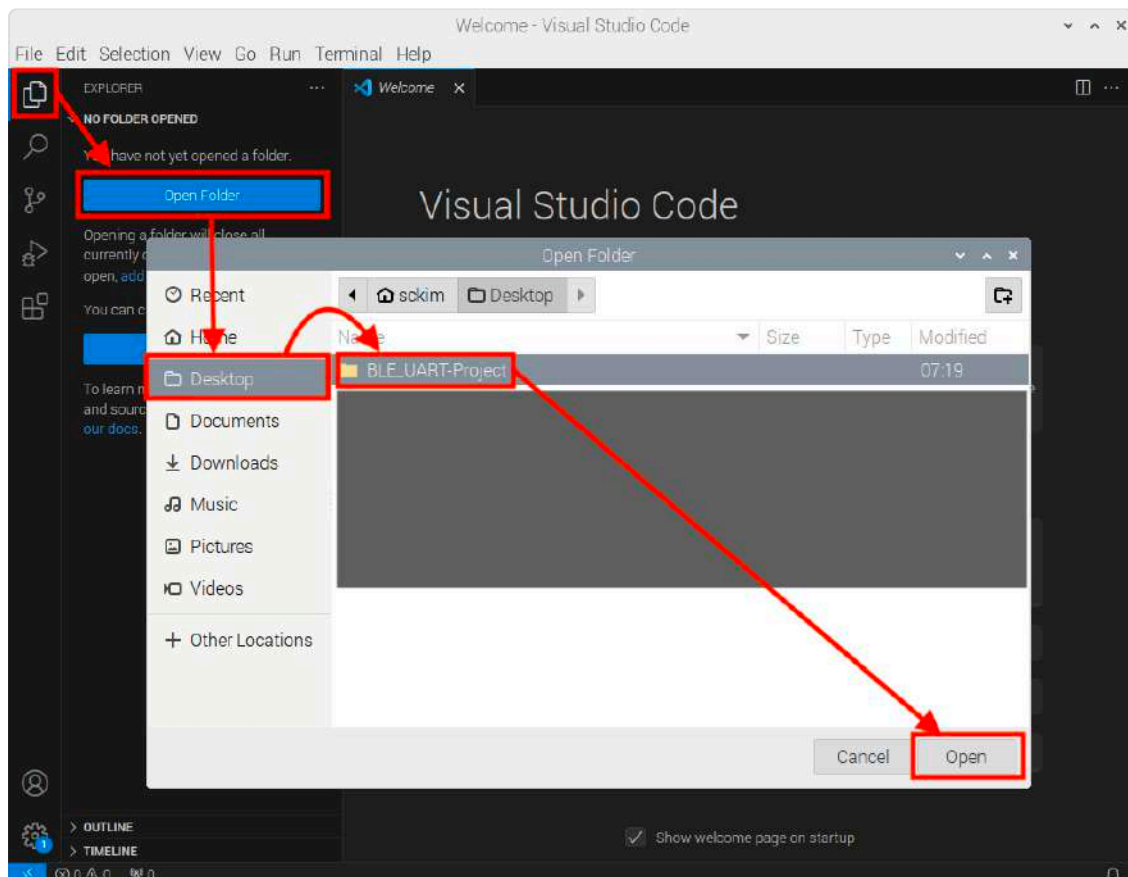
본 단락을 시작하기 전 Raspberry Pi를 사용해 [4.1.1. 개발환경 구축](#) 문서를 따라 개발환경 구축을 진행한다.

프로젝트에서 사용될 파일을 생성하기 위해 홈화면(라즈베리파이의 바탕 화면)에서 우클릭 -> New Folder 버튼을 클릭해 “BLE_UART-Project” 폴더를 생성한다.



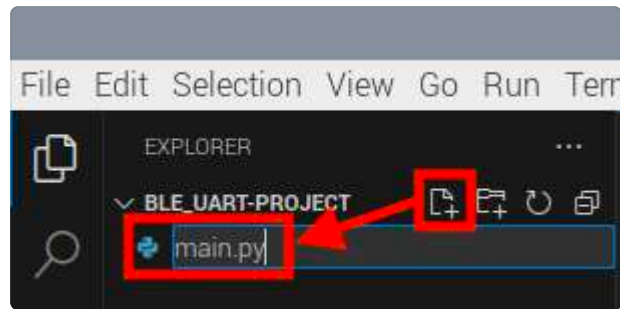
Python 프로젝트 폴더 생성

이후 VSCode를 열고 Open Folder 버튼을 클릭해 기존에 만들었던 BLE_UART-Project 폴더 선택 후 Open 버튼을 클릭해 파일을 연다.



Visual Studio Code 폴더 열기

이후 아래 그림과 같이 main.py 파일을 생성한다.



main.py 파일 생성 방법

6.1.2. BLE 연결

블루투스 기기 Scan

Python에서 BLE_UART를 Scan하기 위해서는 아래 모듈을 import 해야한다. main.py 파일에 아래와 같이 모듈을 import한다.

```
import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈
```

BLE 관련 모듈 import 후 main 모듈을 추가한다. if 내 프로그램이 메인 모듈이 된다.

```
if __name__ == '__main__':
    # 메인 프로그램 실행
```

이후 비동기 처리를 위해 현재 실행중인 이벤트 루프를 가져온다. 아래와 같이 코드를 추가한다.

```
if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
```

이후 전체 기기를 Scan하기 위해 scan 함수를 생성한다. scan하는 함수는 비동기 처리를 위해 async 함수로 구현된다.

```

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart를 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

```

이후 main 함수에 scan을 호출하는 코드를 추가한다. scan 함수가 종료될 때까지 기다려야하기 때문에 함수가 종료될 때까지 기다린다.

```

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan 함수 실행
    scan_devices = loop.run_until_complete(scan())

```

이후 코드를 실행하면 아래 그림과 같이 콘솔에 scan된 BLE_UART 기기를 확인할 수 있다.

```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943

```

Scan된 BLE_UART 콘솔 출력

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart을 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan 함수 실행
    scan_devices = loop.run_until_complete(scan())

```

블루투스 기기 선택

Scan된 BLE 기기 정보를 GUI로 보여주기 위해 tkinter를 import한다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # 추가된 부분(GUI 관련 라이브러리)

```

Scan된 device를 확인할 수 있는 show_scan_devices() 함수를 추가한다. Scan된 device가 화면에 버튼 형태로 나타나고, 버튼을 클릭했을 때 button_click() 함수가 호출되어 화면이 사라지는 형태로 구현되었다.

```
# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("BLE_UART Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device: button_click(device, window))
        button.pack()
        buttons.append(button)

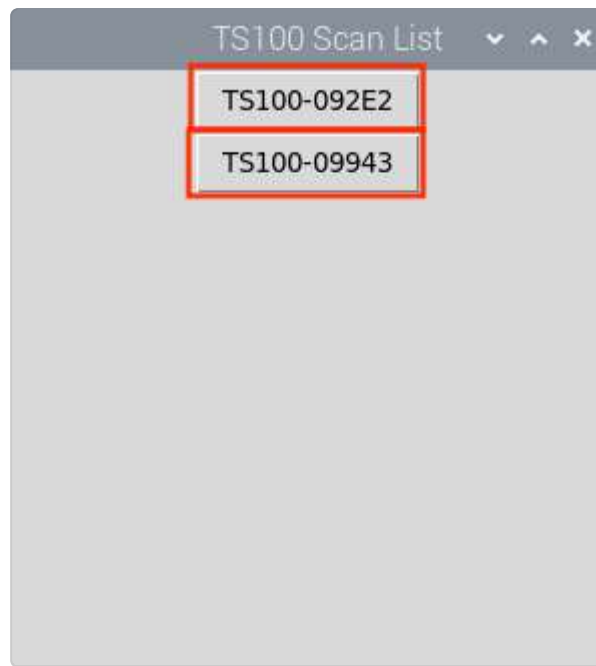
    # 윈도우 종료까지 실행
    window.mainloop()

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    print("Select Device is " + device.name)

    # GUI 제거
    window.quit()

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)
```

위 코드를 실행하면 아래 그림처럼 화면에 Scan된 기기를 확인할 수 있고, 연결하고자 하는 Device를 선택하면 화면이 종료된다.



Scan된 Device GUI 확인

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈


import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart를 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("BLE_UART Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    print("Select Device is " + device.name)

    # GUI 제거
    window.quit()

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

```

블루투스 기기 연결

Scan된 device를 담는 전역변수 selected_device를 추가한다.

```
selected_device = None
```

button_click() 함수에 연결하고자 하는 BLE_UART를 selected_device 변수에 담는 코드를 추가한다.

```

def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

```

블루투스 기기를 연결하기 위해 connect 함수를 추가한다. 비동기 처리를 위해 async 함수로 구현한다.

```

# 선택한 BLE_UART에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

```

main 함수에서 연결요청을 수행할 device를 선택했을 때 connect 과정을 수행하는 코드를 추가한다.

```

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

```

위 코드를 실행하면 아래와 같은 콘솔(Terminal)에 다음과 같이 출력된다.이 출력되는것을 확인할 수 있다.

```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-09943
TS100-092E2
connect start
connected

```

connect된 BLE_UART 콘솔 출력

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈


import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart를 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("BLE_UART Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 BLE_UART에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

selected_device = None

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

```

Service, Characteristic 요청

블루투스 연결이 완료 후, BLE_UART에서 제공하는 전체 Service와 Characteristic을 확인하기 위해 BLE_UART에 Service와 Characteristic을 요청한다.

Service 요청을 위해 아래 함수를 추가한다.

```
# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(device):
    # Service 확인
    for service in device.services:
        print(service)
    # Characteristic 확인
    for characteristic in service.characteristics:
        print(characteristic)

    print()
```

get_service_and_characteristic 함수를 호출하는 코드를 main함수에 추가한다.

```
if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    if selected_device is not None:
        connected_device = loop.run_until_complete(connect(selected_device))
        print("device is " + selected_device.name)

    # BLE_UART와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))
```

위 코드를 실행하면 아래 그림과 같이 전체 Service와 Characteristic을 확인할 수 있다.

```
00001801-0000-1000-8000-00805f9b34fb (Handle: 12): Generic Attribute Profile
00002a05-0000-1000-8000-00805f9b34fb (Handle: 13): Service Changed

00001809-0000-1000-8000-00805f9b34fb (Handle: 47): Health Thermometer
0000fbfb-0000-1000-8000-00805f9b34fb (Handle: 48): Vendor specific
00002a08-0000-1000-8000-00805f9b34fb (Handle: 70): Date Time
0000f9f9-0000-1000-8000-00805f9b34fb (Handle: 62): Vendor specific
0000fdfd-0000-1000-8000-00805f9b34fb (Handle: 66): Vendor specific
0000fafa-0000-1000-8000-00805f9b34fb (Handle: 51): Vendor specific
0000fcfc-0000-1000-8000-00805f9b34fb (Handle: 58): Vendor specific
00002a1c-0000-1000-8000-00805f9b34fb (Handle: 54): Temperature Measurement

0000180a-0000-1000-8000-00805f9b34fb (Handle: 20): Device Information
00002a50-0000-1000-8000-00805f9b34fb (Handle: 31): PnP ID
00002a24-0000-1000-8000-00805f9b34fb (Handle: 23): Model Number String
00002a29-0000-1000-8000-00805f9b34fb (Handle: 21): Manufacturer Name String
00002a27-0000-1000-8000-00805f9b34fb (Handle: 25): Hardware Revision String
00002a23-0000-1000-8000-00805f9b34fb (Handle: 29): System ID
00002a26-0000-1000-8000-00805f9b34fb (Handle: 27): Firmware Revision String

0000fef5-0000-1000-8000-00805f9b34fb (Handle: 33): Dialog Semiconductor GmbH
724249f0-5ec3-4b5f-8804-42345af08651 (Handle: 36): Unknown
9d84b9a3-000c-49d8-9183-855b673fda31 (Handle: 40): Unknown
5f78df94-798c-46f5-990a-b3eb6a065c88 (Handle: 44): Unknown
8082caa8-41a6-4021-91c6-56f9b954cc34 (Handle: 34): Unknown
457871e8-d516-4ca1-9116-57d0b17b9cb2 (Handle: 42): Unknown
6c53db25-47a1-45fe-a022-7c92fb334fd4 (Handle: 38): Unknown

0000180f-0000-1000-8000-00805f9b34fb (Handle: 16): Battery Service
00002a19-0000-1000-8000-00805f9b34fb (Handle: 17): Battery Level
```

BLE_UART 전체 Service, Characteristic

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart를 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("BLE_UART Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 BLE_UART에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인
    for service in connected_device.services:
        print(service)
        # Characteristic 확인
        for characteristic in service.characteristics:
            print(characteristic)

    print()

selected_device = None

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면

```

```

if selected_device is not None:
    # connect 요청 후 함수가 끝날때까지 기다림
    connected_device = loop.run_until_complete(connect(selected_device))
    # BLE_UART와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_

```

온도정보 가져오기

BLE_UART의 여러 characteristic 중 온도정보를 받아오는 기능을 구현한다. 온도 정보는 아래 특성에서 확인할 수 있다.

```

Service: Health Thermometer(00001809-0000-1000-8000-00805f9b34fb)
Characteristic: Temperature Measurement(00002a1c-0000-1000-8000-00805f9b34fb)

```

온도정보 수신

여러 Service 중 Health Thermometer와 관련된 service만 수신받기 위해 get_service_and_characteristic 함수를 아래와 같이 수정한다.

```

# Service
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                print(characteristic)

    print()

```

이후 온도정보 characteristic을 통해 notify를 수신하기 위해 함수를 아래와 같이 수정한다.

```

# Characteristic
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

```

연결된 기기가 종료될 때까지 이벤트 루프를 종료하지 않기 위해 wait_connect 함수를 추가하고 main을 수정한다.

```

# 이벤트루프 종료를 방지하기 위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan 함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

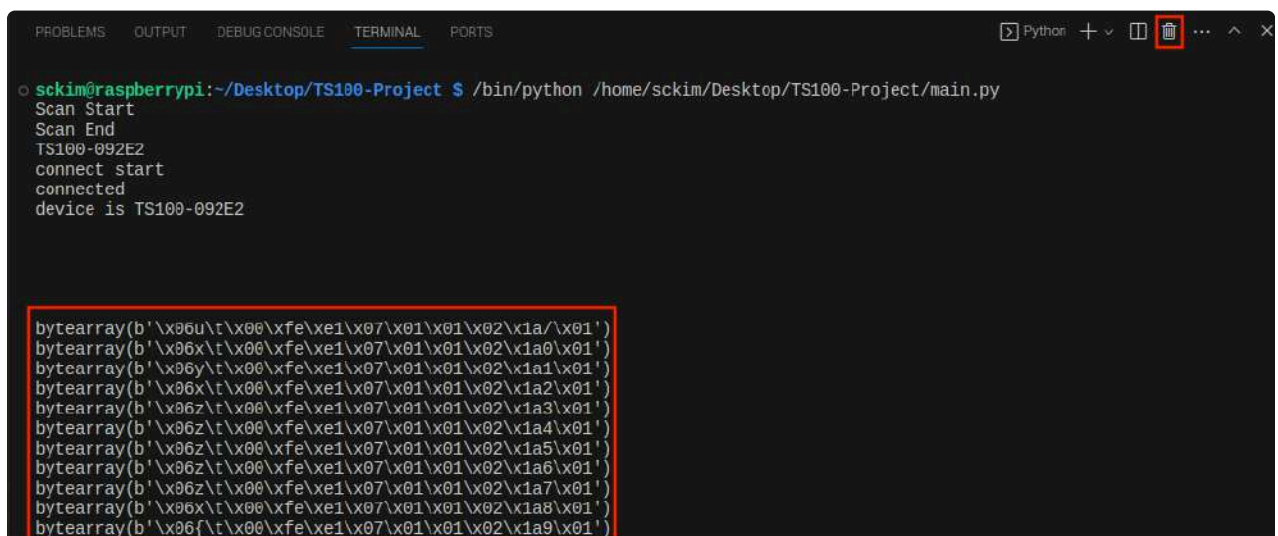
    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

    # BLE_UART와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))

    # 종료 방지 함수 호출
    loop.run_until_complete(wait_connect())

```

이후 코드를 실행하면 아래와 같이 온도정보가 콘솔창에 출력되는 것을 확인할 수 있다. 현재 wait_connect 함수는 별도의 종료를 하지 않으면 코드가 계속 실행되므로 우측 상단의 쓰레기통 버튼을 클릭해 코드 실행을 종료한다.



```

sckim@raspberrypi:~/Desktop/TS100-Project $ /bin/python /home/sckim/Desktop/TS100-Project/main.py
Scan Start
Scan End
TS100-092E2
connect start
connected
device is TS100-092E2

bytearray(b'\x06u\t\x00\xfe\xe1\x07\x01\x01\x02\x1a\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a0\x01')
bytearray(b'\x06y\t\x00\xfe\xe1\x07\x01\x01\x02\x1a1\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a2\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a3\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a4\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a5\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a6\x01')
bytearray(b'\x06z\t\x00\xfe\xe1\x07\x01\x01\x02\x1a7\x01')
bytearray(b'\x06x\t\x00\xfe\xe1\x07\x01\x01\x02\x1a8\x01')
bytearray(b'\x06{\t\x00\xfe\xe1\x07\x01\x01\x02\x1a9\x01')

```

온도정보 notify 수신 확인

지금까지의 내용이 반영된 코드이다.

```

import asyncio # 비동기화 모듈
from typing import Optional # 타입 어노테이션 관련 모듈

from bleak import BleakScanner, BleakClient # BLE 관련 모듈

import tkinter as tk # GUI 라이브러리

# 비동기 형태로 BLE 장치 검색
async def scan():
    ble_uart_device = []

    print('Scan Start')
    # scan start
    scan_devices = await BleakScanner.discover()
    print('Scan End')

    # 스캔된 device 탐색
    for device in scan_devices:
        # 이름이 없는 Device "Unknown Device로 변경"
        device_name = device.name if device.name else "Unknown Device"
        # 이름에 ble_uart를 포함하면 콘솔에 출력
        if "BLE_UART" in device_name:
            ble_uart_device.append(device)
            print(device_name)

    return ble_uart_device

# GUI로 화면에 나타내는 함수
def show_scan_devices(devices):
    if not devices:
        return False

    window = tk.Tk()
    # 화면 제목
    window.title("BLE_UART Scan List")

    buttons = []
    for device in devices:
        text = device.name
        # 버튼 이벤트 추가(button_click 함수 호출)
        button = tk.Button(window, text=text, command=lambda device=device)
        button.pack()
        buttons.append(button)

    # 윈도우 종료까지 실행
    window.mainloop()

```

```

# 버튼 클릭 시 호출되는 함수
def button_click(device, window):
    global selected_device # 전역변수 함수내 사용하도록 설정
    selected_device = device # 선택한 Device selected_device 변수에 담기

    # remove gui
    window.quit()

# 선택한 BLE_UART에 연결하는 함수
async def connect(candidate_device):
    print('connect start')

    selected_client = BleakClient(candidate_device)

    try:
        # 장치 연결 시작
        await selected_client.connect()
        print('connected')
    except Exception as e:
        # 연결 실패시 발생
        print('error: ', e, end='')
        return None

    return selected_client

# Service, Characteristic 확인하는 함수
async def get_service_and_characteristic(connected_device):
    # Service 확인(제대로 연결되었는지 테스트용)
    for service in connected_device.services:
        # Health Thermometer 관련 Service일때만 Characteristic 검색
        if service.uuid == health_thermometer_service:
            for characteristic in service.characteristics:
                # 온도정보 notify 설정
                if characteristic.uuid == temperature_characteristic:
                    await connected_device.start_notify(characteristic, n

# notify를 수신했을 때 호출되는 함수
def notify_callback(handle, data):
    print(data)

# 이벤트루프 종료를 방지하기위한 함수
async def wait_connect():
    while connected_device.is_connected:
        await asyncio.sleep(1)

selected_device = None

```

```
# Service, Characteristic
health_thermometer_service = '00001809-0000-1000-8000-00805f9b34fb'
temperature_characteristic = '00002a1c-0000-1000-8000-00805f9b34fb'

if __name__ == '__main__':
    # 비동기 이벤트 루프 생성
    loop = asyncio.get_event_loop()
    # 비동기 형태로 Scan함수 실행
    scan_devices = loop.run_until_complete(scan())
    # Scan된 기기 화면으로 출력
    show_scan_devices(scan_devices)

    connected_device = None
    # 선택한 기기가 None이 아니라면
    if selected_device is not None:
        # connect 요청 후 함수가 끝날때까지 기다림
        connected_device = loop.run_until_complete(connect(selected_device))

    # BLE_UART와 연결되어 있다면
    if connected_device is not None:
        # Service, Characteristic 출력하는 함수 호출
        loop.run_until_complete(get_service_and_characteristic(connected_device))

    # 종료 방지함수 호출
    loop.run_until_complete(wait_connect())
```